Sanad Aburass · Ibrahim Aljarah

Classical Machine Learning

A Practical Guide Using Python



Springer

Classical Machine Learning

Sanad Aburass • Ibrahim Aljarah

Classical Machine Learning

A Practical Guide Using Python

Springer

Sanad Aburass
Department of Computer Science
Luther College
Decorah, IA, USA

Ibrahim Aljarah
Department of Artificial Intelligence
University of Jordan
Amman, Jordan

ISBN 978-3-032-04398-6 ISBN 978-3-032-04399-3 (eBook)
https://doi.org/10.1007/978-3-032-04399-3

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Cover illustration created by the authors using Adobe Firefly

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.



To my father—the greatest man in my life and the cornerstone of my achievements.
To my mother—a remarkable woman whose warm heart and unconditional love and support have carried me through every stage of life.
To my wife—the love of my life and my soulmate—for all the sacrifices you've made to give me a family and a home.
To my children—the fuel that keeps me going, and the source of so much happiness and meaning in my life.
To my brother, sisters, and friends—thank you for your unwavering love, support, and laughter.
Thank you all!
—Sanad Aburass
To the ever-present spirits of my beloved father and mother, for the values you planted and the dreams you nurtured, your memory continues to light my path.
To my dearest wife, Nailah Al-Madi, for your unwavering love, patience, and strength—you are my anchor and inspiration.
To my beloved children—Nada, Sanad, Salma, and Awn—for every smile, question,

and hope you bring, you are the promise of tomorrow.
To my cherished brothers and sister, for your constant support, encouragement, and love—you are a lifelong blessing I deeply treasure.
And to all who seek knowledge, and to those who honor the pursuit of truth and science, this book is for you.

—Ibrahim Aljarah

Preface

The field of Artificial Intelligence has rapidly transformed in recent years, with machine learning becoming one of its most impactful and widely applied branches. From intelligent recommendation systems to self-driving cars, and from language translation to medical diagnosis, machine learning now touches nearly every aspect of modern life.

Yet, for those beginning their journey into AI, the field can feel daunting—particularly with the increasing complexity of deep learning and generative models. In the midst of this fast-paced evolution, it is easy to overlook the foundational ideas that make these breakthroughs possible. This book was born from the belief that a solid understanding of classical machine learning is not just helpful, but essential for truly grasping the advanced models shaping today's AI landscape.

This book is written to bridge this gap. Unlike many texts that either focus narrowly on theoretical mathematics or jump straight into advanced code with little explanation, this book offers a balanced, practical approach. Our goal is to explain classical models clearly and intuitively, while also providing hands-on Python implementations that bring these models to life. By grounding advanced techniques in first principles, we empower readers to not only use machine learning tools—but to understand them.

Throughout the book, we cover a wide range of foundational topics, from linear regression and logistic regression to decision trees, ensemble methods, clustering, dimensionality reduction, neural networks, and convolutional operations. We also introduce emerging ideas like Cubixel representation in image processing, providing a forward-looking perspective on evolving practices. Each chapter builds on the last, combining theory, math, and code in a way that is accessible to students, researchers, and professionals alike.

This book assumes a working knowledge of Linear Algebra and Calculus, as many algorithms rely on these mathematical underpinnings. A solid foundation in Python is also recommended since practical examples and implementations are written in Python with widely used libraries such as NumPy, pandas, scikit-learn, and TensorFlow.

Whether you're an aspiring machine learning engineer, a data scientist transitioning from another field, or an academic looking to refresh your knowledge, this book aims to be a practical companion on your learning journey. Our hope is that it not only demystifies machine learning, but inspires deeper curiosity and confidence to explore further.

We invite you to read with an open mind, try the examples, experiment with variations, and above all—enjoy the process of learning.

With the rise of modern AI systems capable of writing essays, solving mathematical problems, generating code, creating images, videos, and even audio, the field has become both exciting and, at times, overwhelming, especially for learners trying to understand how these models work.

This complexity is one of the key reasons why it is essential to begin with the fundamentals of AI and Machine Learning (ML). Understanding the basic concepts behind AI models is crucial to developing a deep and intuitive grasp of the more sophisticated systems we see today. These foundational models, which we refer to as Classical Machine Learning, form the building blocks upon which modern AI is constructed.

Classical machine learning models are formulated to identify patterns, make predictions, classify inputs, and derive insights from data. These include well-established algorithms such as Linear Regression, Logistic Regression, Naïve Bayes, Decision Trees, Random Forests, K-Means Clustering, Artificial Neural Networks (ANNs), as well as initial versions of Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs). These models laid the groundwork for the advancement of modern deep learning architectures and large-scale AI systems.

Grasping these classical models thoroughly is essential for effectively understanding or creating sophisticated AI algorithms. This book's primary aim is to deliver a clear, detailed, and accessible explanation of the fundamental machine learning algorithms that form the modern AI systems.

In the upcoming chapters, we will delve into each classical ML model in depth. We will discuss their theoretical, mathematical formulations, and practical coding implementations—providing the reader with a robust foundation for advancing and more understanding of the machine and deep learning topics.

We start our discussion with the nature of features—the properties or characteristics of data—and how they are represented across different data types. Whether dealing with text, tables, or images, each type of data requires specific techniques for preprocessing and feature extraction. These features are vital for machine learning models to capture the correlations needed to get accurate predictions.

Moreover, we explore the emerging concepts in image processing such as Cubixel, where images are represented in a three-dimensional structure rather than the traditional two-dimensional structure. This innovative approach aims to extract richer information from visual data and could lead to more advanced understanding in computer vision tasks.

In Chap. 2, we begin our journey with one of the simplest yet most powerful models: Linear Regression. This statistical method seeks to model the relationship between independent and dependent variables using a linear equation. You can think of it as the reverse process of constructing an equation based on observed values of x and y.

Ultimately, the learning process in machine learning revolves around discovering relationships between features and outputs. And this is precisely where we begin.

In Chap. 3, we discuss the Logistic Regression model in the context of binary classification tasks. We explain the gradient descent algorithm in detail. In the second part of the chapter, we cover classification model performance evaluation, including accuracy, precision, recall, and the F1 score. In the third part, we explain how to classify text using the Naïve Bayes algorithm.

In Chap. 4, we delve into Decision Trees and Random Forests, two powerful models used for both classification and regression tasks. Decision Trees provide a visual and intuitive way of making predictions by splitting data based on feature values. However, they are prone to overfitting, which is where Random Forests come in. By building an ensemble of Decision Trees and aggregating their outputs, Random Forests significantly improve accuracy and generalization. In this chapter, we explain how these models work, how they are trained, and how to interpret and tune them effectively.

In Chap. 5, we expand on the idea of combining multiple models through Ensemble Learning. You will learn how algorithms like AdaBoost and Gradient Boosting operate, which are used to build more robust predictive systems by aggregating the strengths of various base models.

In Chap. 6, we shift from supervised to unsupervised learning, where the goal is to find hidden structures in unlabeled data. We focus on Clustering techniques such as K-Means and DBSCAN Clustering, as well as Dimensionality Reduction methods like Principal Component Analysis (PCA). These techniques are vital for exploratory data analysis, pattern discovery, and data visualization.

In Chap. 7, we introduce ANNs, the foundation of deep learning. We begin by explaining how a neuron works, how multiple neurons form layers, and how data flows through the network. We walk through forward propagation, loss calculation, backpropagation, and training using gradient descent. This chapter lays the groundwork for understanding more complex neural network architectures.

Chapter 8 focuses on Convolutional Operations, which are at the heart of modern computer vision. We explore the concept of convolutional filters, feature maps, and how these components help in capturing spatial hierarchies in image data. Understanding convolution is essential before building and training full convolutional neural networks (CNNs).

Building on this, Chap. 9 introduces CNNs—deep learning architectures specifically designed for image recognition and classification. In this chapter, we discuss in detail how backpropagation works in CNNs, particularly in updating the feature maps and filters. We walk through the complete training process, showing how the values of the filters are learned and adjusted during training.

In Chap. 10, we explore RNNs, which are designed to handle sequential data such as text, speech, and time series. We explain the internal structure of RNNs, how they maintain memory through hidden states, and how they are trained.

Finally, in Chap. 11, we discuss Transfer Learning, a technique that allows us to take pre-trained models and adapt them to new tasks with relatively little data. This approach has become standard practice in both computer vision and natural language processing. We explain how transfer learning works, how to fine-tune models, and how it enables rapid experimentation and deployment in real-world applications.

Decorah, IA, USA — Sanad Aburass
Amman, Jordan — Ibrahim Aljarah

Acknowledgment

We would like to express our sincere gratitude to everyone who contributed to this work—from the previous authors, researchers, and professors who shared their knowledge of Machine Learning, making it possible for us to understand and build upon it, to the Springer team, whose support was essential in producing this book.

Specifically, we would like to thank the editor, Dr. Alexandru Ciolan, for his support and valuable insights throughout this project.

Sanad would like to express his sincere gratitude to Mr. Peter Vonderheide and Professor Clyde Ruby for the opportunity to teach the Machine Learning class at MIU, which directly inspired the writing of this book. He is also deeply grateful to Luther College and the head of the Computer Science Department, Dr. Roman Yasinovskyy, for providing a supportive atmosphere and a flexible, relaxed working environment that made it possible to complete this work during the academic year.

Ibrahim would like to extend his heartfelt thanks to Dr. Sanad Aburass, who initiated this great project and dedicated exceptional effort and commitment to bringing it to completion.

He also expresses his deep appreciation to the University of Jordan, represented by Prof. Natheer Obiedat, and to Arrowad Group and Kaizen Consulting Firm, represented by Dr. Saad Alkhalf, for their continued support, encouragement, and contributions to the success of this endeavor.

Competing Interests The authors have no competing interests to declare that are relevant to the content of this manuscript.

Contents

1 Data Preprocessing and Feature Engineering 1
1.1 What Is a Feature, and Why Are Features Important? 1
1.1.1 Examples of Features in Different Domains 2
1.1.2 Types of Features 3
1.2 Feature Extraction 5
1.2.1 Tabular Data 5
1.2.2 Imagery Data 15
1.3 Concluding Remarks 34

2 Regression and Linear Models 35
2.1 What Is Regression? 35
2.2 Regression as a Supervised Learning Method 36
2.3 Components of a Linear Regression Equation 37
2.3.1 Role of Coefficients in Regression 37
2.3.2 Constructing the Linear Equation 37
2.3.3 The Intercept Term β_0 in Regression 38
2.3.4 Adding the Intercept Term 38
2.3.5 Using Regression to Find Coefficients 38
2.4 The Normal Equation for Solving Linear Regression 39
2.4.1 The Normal Equation 39
2.4.2 Using the Normal Equation for Predictions 40
2.4.3 Applying the Normal Equation: An Example 40
2.4.4 Constructing the Linear Equation from Coefficients . . 41
2.5 The Limitations of Linear Regression 42
2.5.1 When the Relationship Is Not Perfectly Linear 43
2.5.2 The Limitations of a "Take It or Leave It" Approach . . 43
2.5.3 Beyond Linear Regression 43
2.6 Real-World Example: Predicting the Number of Meals in a University Cafeteria 44
2.7 Model Evaluation: Measuring the Accuracy of Predictions 47
2.7.1 The Importance of Model Evaluation 47

2.8 Polynomial Regression . . . 50
2.9 Regression in Real Life . . . 57
2.10 Concluding Remarks . . . 58

3 Classification: Logistic Regression to Naïve Bayes . . . 59
3.1 Logistic Regression . . . 59
3.1.1 Practical Example . . . 62
3.1.2 Evaluation of Classification Models . . . 76
3.2 Classifying Text with Naive Bayes . . . 83
3.2.1 Preprocessing Text for Naive Bayes Classification . . . 84
3.2.2 Naive Bayes Classification Steps . . . 85
3.3 Concluding Remarks . . . 90

4 Decision Trees and Random Forests . . . 93
4.1 Decision Trees in Classification . . . 93
4.1.1 Preprocessing . . . 95
4.1.2 Using Gini Impurity . . . 96
4.1.3 Using Information Gain . . . 106
4.1.4 Decision Trees in Regression . . . 112
4.2 Random Forests . . . 114
4.2.1 Using Subsamples to Train Trees . . . 115
4.2.2 How Random Forests Work . . . 115
4.3 Evaluating Decision Trees and Random Forests in Classification Tasks . . . 118
4.4 Concluding Remarks . . . 118

5 Ensemble Learning . . . 121
5.1 Weak Learners . . . 121
5.2 Adaptive Boosting (AdaBoost) . . . 123
5.2.1 Training the First Weak Learner . . . 124
5.2.2 Weighted Error for a Split . . . 124
5.2.3 Evaluating the Split at Midpoints . . . 125
5.2.4 Selecting the Best Split . . . 127
5.2.5 Comparing the Best Splits . . . 127
5.2.6 Updating the Weights . . . 128
5.2.7 The Second Iteration . . . 131
5.2.8 Select the Best Stump . . . 131
5.2.9 The Third Iteration . . . 134
5.2.10 The Prediction Phase . . . 134
5.2.11 AdaBoost in Regression and Multiclass Classification . 136
5.2.12 Why AdaBoost Stands Out . . . 138
5.2.13 AdaBoost Implantation . . . 138
5.3 Gradient Boosting . . . 139
5.3.1 Initial Prediction . . . 140
5.3.2 The Residual Gradient (Error) . . . 140
5.3.3 Iterative Steps in Gradient Boosting . . . 141
5.3.4 Implantation . . . 149

5.4 AdaBoost and Gradient Boosting with Images 150
5.5 Concluding Remarks 151

6 Unsupervised Learning: Clustering and Dimensionality Reduction . . 153
6.1 K-Means Clustering 153
6.1.1 Clustering Coffee Shop Customers 154
6.1.2 Drawbacks of K-Means 159
6.1.3 K-Means Implantation 160
6.2 Density-Based Spatial Clustering of Applications with Noise (DBSCAN) 163
6.2.1 Step 1: Calculate the Distance Matrix 166
6.2.2 Step 2: Determine Core Points and Initialize Clusters . . 167
6.2.3 Step 3: Process Remaining Points 167
6.2.4 Implementation 172
6.3 Principal Component Analysis (PCA) 172
6.3.1 Steps of PCA 173
6.3.2 Example: Calculating Eigenvectors and Eigenvalues . . 179
6.3.3 Implementation 186
6.4 Concluding Remarks 187

7 Introduction to Artificial Neural Networks 189
7.1 Revisiting Logistic Regression 189
7.2 From Logistic Regression to Artificial Neural Networks 191
7.3 Extending the Neural Network for a Single Output 192
7.4 Calculating the Loss for the Network 193
7.5 Backpropagation 194
7.5.1 Step 1: Derivative of the Loss Function with Respect to the Predicted Output 195
7.5.2 Step 2: Sigmoid Activation Function Derivative 195
7.5.3 Step 3: Calculating the Gradient of the Loss with Respect to z 196
7.6 Propagating the Error Backward 197
7.7 Updating the Weights 200
7.7.1 Step 1: Updating Weights Connected to Neuron2 from the Input Layer 200
7.7.2 Step 2: Updating Weights Connected to Neuron3 201
7.8 Understanding the Code: Building a Simple Neural Network . . . 204
7.8.1 The Basics: Activation and Loss Functions 204
7.8.2 Defining Our Simple Neural Network 205
7.8.3 Forward Pass: Computing the Output 206
7.8.4 Backward Pass: Adjusting the Weights 206
7.8.5 Training the Network 207
7.8.6 Making Predictions 207
7.8.7 Testing the Network with Real Data 208

7.9 Activation Functions in Artificial Neural Networks 208
7.9.1 Common Activation Functions in Artificial Neural Networks 209
7.9.2 Explanation of Activation Functions 209
7.9.3 Why Use Softmax for Multiclass Classification? 211
7.9.4 Understanding the Softmax Output 211
7.9.5 Example Calculation 212
7.10 Loss Function for Multiclass Classification with Softmax: Cross-Entropy Loss 213
7.10.1 What Is Cross-Entropy Loss? 213
7.10.2 The Derivative of Cross-Entropy Loss with Softmax . . . 215
7.11 Retouching on Gradient Descent 216
7.12 Implementation 218
7.12.1 Step 1: Loading the Data 220
7.12.2 Step 2: Inspecting the Data (Optional) 220
7.12.3 Step 3: Preprocessing the Data 220
7.12.4 Step 4: Normalizing the Features 221
7.12.5 Step 5: Splitting the Data 222
7.12.6 Step 6: Building the Neural Network Model 222
7.12.7 Step 7: Compiling the Model 224
7.12.8 Step 8: Training the Model 224
7.12.9 Step 9: Evaluating the Model 224
7.12.10 Step 10: Making Predictions (Optional) 225
7.13 Using Artificial Neural Networks for Regression Problems 225
7.13.1 Choosing the Right Activation Function for Regression 226
7.13.2 Loss Function for Regression 226
7.14 Concluding Remarks 227

8 Convolutional Operations 229
8.1 Extracting Features from Images 229
8.2 Implementing Convolutional Operations Using a Convolutional Layer 236
8.3 Cubixel Kernel for Feature Extraction 238
8.3.1 Understanding Volume (V) 238
8.3.2 Understanding Aspect Ratio (A) 239
8.4 Concluding Remarks 247

9 Convolutional Neural Networks 249
9.1 Convolutional Neural Networks 249
9.2 Conv2D Filters and Backpropagation 255
9.2.1 From the Flatten Layer to the Convolutional Layers . . . 256
9.2.2 Processing of the Reshaped Gradient Tensor by the Derivative of ReLU 257
9.2.3 Computing the Gradient of the Convolutional Layer . . . 258

9.2.4 The Gradient of the Filter Weights . . . 260
9.2.5 Incorporating Sliding Window in Gradient Computation . . . 261
9.2.6 Gradient Flow Mechanism for Layer Shape Adjustment . . . 263
9.2.7 Updating the Filters Weights . . . 265
9.3 The Pooling Layer . . . 266
9.3.1 Backpropagation Through the Pooling Layers . . . 270
9.4 The Dropout Layer . . . 271
9.5 Concluding Remarks . . . 272

10 Recurrent Neural Networks . . . 273
10.1 Recurrent Neural Networks . . . 273
10.1.1 Step 1: Set the Hidden State Size and the Sequence Size . . . 274
10.1.2 Step 2: Tokenization . . . 275
10.1.3 Step 3: Initialize the Weights . . . 275
10.1.4 Step 4: One-Hot Encode the Input . . . 277
10.1.5 Step 5: Prepare the Target . . . 277
10.1.6 Step 6: Initialize the Hidden State Vector . . . 278
10.1.7 Step 7: The Forward Pass . . . 279
10.1.8 Step 8: Backpropagation Through Time . . . 285
10.1.9 Step 9: Update the Weights Based on the Learning Rate . . . 287
10.2 Implementation . . . 287
10.3 Concluding Remarks . . . 290

11 Transfer Learning . . . 291
11.1 Transfer Learning . . . 291
11.2 Transfer Learning with Freeze Using VGGNet . . . 292
11.3 Transfer Learning Without Freeze Using DenseNet . . . 296
11.4 Transfer Learning with a Custom Cubixel Layer Using Inception Model on CIFAR-10 Dataset . . . 299
11.4.1 Cubixel Layer . . . 299
11.4.2 Code Implementation . . . 299
11.4.3 Integration of Cubixel Layer with Transfer Learning . . . 302
11.5 Concluding Remarks . . . 303

References . . . 305

Chapter 1
Data Preprocessing and Feature Engineering



Abstract This chapter highlights the essential role of feature extraction in transforming raw data into meaningful inputs for machine learning models. Effective features act as the connection between unstructured or tabular data and a model's capacity to learn and generalize. We examine a range of feature types—from simple numerical attributes to complex text and image-based representations—and demonstrate how they shape a model's understanding of its environment. Techniques such as binary conversion, one-hot encoding, and feature aggregation are discussed, emphasizing how data representation influences model performance. Using practical examples like the Telco Customer Churn dataset, we show how thoughtful feature extraction turns complex datasets into structured, model-ready formats. Ultimately, this process ensures that models receive not just data, but the most relevant data, enhancing predictive accuracy and insight.

1.1 What Is a Feature, and Why Are Features Important?

When we talk about machine learning, it's easy to get swept up by the idea of algorithms, predictions, and data processing. But the real magic often lies in the details we give the model to work with—details we call *features*. Think of features as the building blocks of information that guide a model, similar to how we use clues and characteristics to make sense of the world around us (Dong and Liu 2018; Zheng and Casari 2018).

Imagine you're meeting someone for the first time. What are the things you notice about them? Their height, the color of their hair, their smile, the tone of their voice—all these are features that help you understand who this person might be. In the same way, a feature in machine learning is a measurable piece of data that provides insights into whatever we're trying to understand or predict.

Each feature is like a descriptor that tells the model a little more about the subject, allowing it to make an educated guess about what's going on or what might happen next. Just like human traits, features in machine learning can vary widely in type and meaning depending on the context. They might be physical measurements, times and

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_1

dates, or even broader characteristics that represent a pattern or trend (Brownlee 2020).

Think back to how a detective works on a case. Each clue they gather builds a clearer picture of the situation, increasing their chance of solving the mystery. In machine learning, features are our clues. They are the pieces of information that, collectively, allow the model to understand and learn the patterns in data.

Take, for instance, a recommendation engine on a streaming service. To predict what you might want to watch next, it doesn't just need to know what movies you like but why you like them. Are you drawn to action-packed scenes, historical dramas, or films with certain actors? The features here—genre, actor, director, or even the movie's release year—are what allow the recommendation model to see patterns in your choices and make smart guesses about what else might interest you (Duboue 2020; Nargesian et al. 2017).

Without meaningful features, a machine learning model is almost like someone trying to solve a puzzle blindfolded. It can't see what's in front of it or make good guesses about what might happen. So, features don't just give the model the data it needs—they give it the right kind of data to make sense of the world we're asking it to understand (Liu and Motoda 2012; Pyle 1999).

1.1.1 Examples of Features in Different Domains

Let's dive into some specific examples to see how features work in action across different types of data.

Images: In the world of computer vision, where machines learn to "see," features are the elements that capture visual information. Imagine a model learning to identify animals in photos. Features might include color patterns, shapes, or textures—like the distinctive black-and-white stripes of a zebra or the rounded, fluffy shape of a panda. Each of these features helps the model distinguish one animal from another, even in a cluttered image (Scott and Matwin 1999; Turner et al. 1999).

Text: In natural language processing, machines learn to understand and work with language. Here, features could be individual words, phrases, or even the length of sentences. Think of a model that sorts customer reviews into positive or negative categories. Features might include specific words like "excellent" or "terrible," the number of words in each review, or even more complex characteristics like the sentiment behind certain phrases.

Tabular Data: When we deal with structured, table-like data, each column is often a feature. Imagine you're working with a dataset of house prices. Features might include the number of bedrooms, the square footage, the neighborhood, and the age of the home. These individual characteristics collectively help a model predict the price of a home by understanding how each feature influences its value.

Each of these examples illustrates how features serve as the key "hints" that lead a model to make educated predictions. Features are the bridge between raw data and useful insights, giving shape and structure to machine learning in much the same way that physical, visible traits help us understand and recognize people, places, and things in our everyday lives.

In the next sections, we'll explore more about how these features are crafted, chosen, and tuned—the art and science of making sure that the model has all the clues it needs to solve the mysteries we set in front of it.

1.1.2 Types of Features

Features are as diverse as the questions they help answer. Some features are straightforward and easy to measure, like a person's age or the height of a building. Others are more complex, like a person's favorite genre of music or the tone of a written sentence. Here, we'll break down the different types of features, showing how each one tells its own story to help a machine learning model "see" and understand data (Dasu and Johnson 2003; Fernandes et al. 2023).

1.1.2.1 Binary Features

Binary features are the simplest type of feature—they're either one thing or another, like an on/off switch. For example, a feature indicating whether a customer has signed up for a newsletter might be binary, with options like "yes" or "no," or coded as 1 and 0. Think of it as a simple yes/no question, like asking if someone likes coffee. The answer will be either "yes" (coffee lover) or "no" (not a coffee fan), and that single bit of information is all the model needs for that specific feature.

Binary features are used when only two options are possible, helping the model narrow down possibilities in simple, straightforward ways.

1.1.2.2 Numerical vs. Categorical Features

Let's start with the two main categories: numerical and categorical features.

Numerical features	Categorical features
Imagine you're planning a road trip and need to keep track of things like distance, speed, and gas level. All of these can be measured with numbers—and that's what makes them numerical features. In machine learning, numerical features are values that can be counted, measured, or ordered, like age, height, weight, or	Now, imagine you're choosing a hotel. You might categorize hotels by type—boutique, luxury, or budget. You don't "measure" these; you assign them to categories. That's exactly what categorical features are for: they describe qualities that belong to set groups, like gender, color, or country. A model analyzing car

(continued)

Numerical features	Categorical features
temperature. Think of a model predicting house prices: the number of bedrooms or the square footage would be numerical features that give the model a sense of "how much" or "how many" about each house.	preferences might use categorical features to note the car brand or fuel type, classifying each car without needing any measurements. Categorical features give the model an "either/or" type of understanding, helping it see things as distinct categories rather than scales.

1.1.2.3 Ordinal and Nominal Features

Some categorical features need a bit more nuance, and that's where we distinguish between ordinal and nominal features.

Ordinal features	Nominal features
Ordinal features are like levels in a game—they represent a ranking or order. Imagine a survey where customers rate a service from "very bad" to "excellent." These responses have a clear order, with "excellent" being better than "good" and "good" being better than "fair." Ordinal features let the model understand not only that these categories exist but that they're ranked in a certain way. In this way, ordinal features add a sense of hierarchy, helping the model "know" if something is higher or lower on a particular scale.	Now imagine a survey question asking for someone's favorite ice cream flavor: chocolate, vanilla, or strawberry. There's no inherent order here; we don't rank vanilla as "better" or "worse" than chocolate. That's what makes it a nominal feature—it's a label without any ranking. Nominal features are simply named categories, helping the model recognize distinct groupings without any sense of order between them.

1.1.2.4 Textual and Image Features in NLP and Computer Vision

When we go to fields like natural language processing (NLP) and computer vision, features take on even more intriguing forms.

Textual features in NLP	Image features in computer vision
Imagine trying to understand the sentiment behind a product review. Textual features in NLP could be individual words, phrases, or even the length of a sentence. If a review contains words like "fantastic" or "amazing," those words can be features that signal a positive sentiment. In text analysis, features could also include grammatical patterns, punctuation marks, or even complex embeddings that capture the meaning of words in context. Each word or phrase, in this case, is like a hint that helps the model understand what the text is really saying.	Think of looking at a photo and identifying objects within it. In computer vision, features can capture patterns like color gradients, edges, or shapes within an image. For example, a model recognizing dogs in photos might use image features to look for shapes resembling ears, tails, or paws. Imagine scanning for certain textures—fluffy, smooth, or spotted—to classify objects. These features give the model the visual clues it needs to detect and recognize objects in complex images, even when they're surrounded by noise or distractions.

Table 1.1 Examples of different types of features in machine learning

Feature type	Example 1	Example 2	Example 3
Numerical features	Age of a person	Annual income (in dollars)	Temperature (°C or °F)
Categorical features	Car brand (e.g., Toyota)	Payment method (e.g., cash)	Country of residence
Binary features	Signed up for newsletter (yes/no)	Owns a car (yes/no)	Is married (yes/no)
Ordinal features	Customer satisfaction (e.g., 1–5 rating)	Education level (e.g., high school, college)	Movie rating (e.g., PG, PG-13, R)
Nominal features	Eye color (e.g., blue, brown)	Type of cuisine (e.g., Italian)	Favorite genre (e.g., jazz, pop)
Textual features (NLP)	Word frequency in text	Sentiment (positive/negative)	Keyword presence (e.g., "awesome")
Image features (computer vision)	Color histogram	Edge detection patterns	Shape outline (e.g., circle, square)

To better understand the diversity of feature types, refer to Table 1.1, which provides examples for each type of feature discussed. These examples illustrate how different kinds of data inform machine learning models. For instance, numerical features such as age or temperature represent measurable quantities, offering precise, continuous values that allow for statistical analysis. Categorical and binary features, on the other hand, focus on classification—whether it's grouping data by car brand or determining a simple yes/no response, as in whether a person owns a car. Ordinal features add a level of order to categories, such as a satisfaction rating, where a "5" implies higher satisfaction than a "3." Nominal features, such as eye color, do not have any inherent order but allow for straightforward categorization. Finally, textual and image features provide specialized data for language and visual processing, such as word frequencies in natural language processing (NLP) or color histograms in computer vision. Each feature type brings unique value to a model, capturing diverse aspects of information that contribute to a more nuanced understanding of the data.

Each of these types of features—from simple binary decisions to intricate visual patterns—serves as a valuable piece of the puzzle. The mix of feature types helps models learn and understand data with greater depth and accuracy, much like the way our senses pick up different cues to help us interpret the world.

1.2 Feature Extraction

1.2.1 Tabular Data

Feature extraction is a key step in transforming raw data into meaningful insights for machine learning models. Let's walk through techniques to extract features from a popular dataset: the *Telco Customer Churn dataset* from IBM, available on Kaggle.

This fictional dataset contains data about 7043 customers of a telecommunications company in California, with features ranging from personal information like age and gender to service usage patterns and churn behavior. By examining how to extract relevant features, we'll see how these variables reveal patterns and insights about why customers may leave the company (Li et al. 2018; Whang et al. 2023).

1.2.1.1 Problem Overview: Predicting Customer Churn

One of the biggest challenges for subscription-based businesses, like our fictional telco company, is customer churn—the loss of customers who decide to leave for a competitor or discontinue their service. Churn prediction helps identify which customers are likely to leave so that the company can take preventive measures, such as improving service offerings or providing incentives. In this chapter, we'll use the Telco Customer Churn dataset from IBM to explore techniques for extracting features that can help predict churn, using various characteristics of the customers and their service usage to determine who is most at risk of leaving.

The Telco Customer Churn dataset provides detailed information on 7043 customers, each represented by 33 variables. These variables contain both customer demographic details and data about the services they use. To understand the available data, refer to Table 1.2, which provides a detailed description of each column in the Telco Customer Churn dataset, including customer demographics, service details, and billing information. This table gives an overview of the raw data we'll transform into features for predicting customer churn.

Each of these variables provides a unique piece of information about the customer and their relationship with the company, all of which can potentially serve as features for the churn prediction model. In Fig. 1.1a, b, you can see a sample of the dataset, which gives a snapshot of how this information is organized and represented.

In the Telco Customer Churn dataset, many of the fields are text-based or categorical, such as `Internet Service` or `Contract`, which can't be directly used by machine learning models that work best with numerical data. Feature extraction helps convert these text fields into numerical representations, making it easier for models to detect patterns and make predictions.

Imagine you're learning about customers who might leave a company. For you, reading about someone's contract type or payment method gives you an intuitive sense of their preferences and habits. However, a machine learning model lacks intuition, so feature extraction techniques turn this information into numbers and categories that allow the model to recognize trends. Let's walk through the steps of transforming some of these columns, writing Python code along the way, and explaining each step with relatable examples.

Let's start by examining a few different techniques for feature extraction in Python using the Telco Customer Churn dataset.

Table 1.2 Description of variables in the Telco Customer Churn dataset

Customer information	Geographic and demographic details	Service subscription details	Billing and contract information	Customer value	Churn indicators
• **CustomerID**: A unique identifier for each customer • **Count**: A count of customers, primarily used for dashboards or reporting	• **Country**: The country where the customer resides • **State and City**: The specific state and city of the customer's residence • **Zip Code**: The postal code of the customer's address • **Lat Long**: The combined latitude and longitude for the customer's location, along with separate latitude and longitude columns • **Gender**: Specifies if the customer is male or female • **Senior Citizen**: Indicates if the customer is 65 years or older • **Partner**: Specifies if the customer has a spouse or partner • **Dependents**: Indicates if the customer lives with dependents (children, parents, or others)	• **Tenure Months**: The length of time (in months) that the customer has been with the company • **Phone Service**: Whether the customer has a home phone service • **Multiple Lines**: Whether the customer has more than one telephone line • **Internet Service**: The type of internet service subscribed to by the customer (DSL, Fiber Optic, or No internet) • **Online Security, Online Backup, Device Protection, Tech Support**: Additional service options, such as security, backup, and support services • **Streaming TV and Streaming Movies**: Whether the customer uses the company's internet service for streaming	• **Contract**: The type of contract the customer has (Month-to-Month, 1 year, 2 year) • **Paperless Billing**: Indicates if the customer has opted for paperless billing • **Payment Method**: Specifies the payment method (e.g., Credit Card, Bank Withdrawal, or Mailed Check) • **Monthly Charge and Total Charges**: The current monthly charge and the total cumulative charges for the customer	• **CLTV** (Customer Lifetime Value): A calculated metric predicting the future value of each customer to the company	• **Churn Label and Churn Value**: These fields indicate if a customer has left the company (churned) or stayed, with 1 for churned and 0 for stayed • **Churn Score**: A calculated score that reflects the likelihood of churn based on various factors • **Churn Reason**: The specific reason a customer gave for leaving

CustomerID	Count	Country	State	City	Zip Code	Lat Long	Latitude	Longitude	Gender	Senior Citizen	Partner	Dependents
3668-QPYBK	1	United States	California	Los Angeles	90003	33.964131, -118.272783	33.964131	-118.272783	Male	No	No	No
9237-HQITU	1	United States	California	Los Angeles	90005	34.059281, -118.30742	34.059281	-118.30742	Female	No	No	Yes
9305-CDSKC	1	United States	California	Los Angeles	90006	34.048013, -118.293953	34.048013	-118.293953	Female	No	No	Yes
7892-POOKP	1	United States	California	Los Angeles	90010	34.062125, -118.315709	34.062125	-118.315709	Female	No	Yes	Yes
0280-XJGEX	1	United States	California	Los Angeles	90015	34.039224, -118.266293	34.039224	-118.266293	Male	No	No	Yes
7590-VHVEG	1	United States	California	Los Angeles	90001	33.973616, -118.24902	33.973616	-118.24902	Female	No	Yes	No
5575-GNVDE	1	United States	California	Los Angeles	90002	33.949255, -118.246978	33.949255	-118.246978	Male	No	No	No
7795-CFOCW	1	United States	California	Los Angeles	90004	34.076259, -118.310715	34.076259	-118.310715	Male	No	No	No
1452-KIOVK	1	United States	California	Los Angeles	90007	34.027337, -118.28515	34.027337	-118.28515	Male	No	No	Yes
6713-OKOMC	1	United States	California	Los Angeles	90008	34.008293, -118.346766	34.008293	-118.346766	Female	No	No	No

(a)

Tenure Months	Phone Service	Multiple Lines	Internet Service	Online Security	Online Backup	Device Protection	Tech Support	Streaming TV	Streaming Movies
2	Yes	No	DSL	Yes	Yes	No	No	No	No
2	Yes	No	Fiber optic	No	No	No	No	No	No
8	Yes	Yes	Fiber optic	No	No	Yes	No	Yes	Yes
28	Yes	Yes	Fiber optic	No	No	Yes	Yes	Yes	Yes
49	Yes	Yes	Fiber optic	No	Yes	Yes	No	Yes	Yes
1	No	No phone service	DSL	No	Yes	No	No	No	No
34	Yes	No	DSL	Yes	No	Yes	No	No	No
45	No	No phone service	DSL	Yes	No	Yes	Yes	No	No
22	Yes	Yes	Fiber optic	No	Yes	No	No	Yes	No
10	No	No phone service	DSL	Yes	No	No	No	No	No

(b)

Contract	Paperless Billing	Payment Method	Monthly Charges	Total Charges	Churn Label	Churn Value	Churn Score	CLTV	Churn Reason
Month-to-month	Yes	Mailed check	53.85	108.15	Yes	1	86	3239	Competitor made better offer
Month-to-month	Yes	Electronic check	70.7	151.65	Yes	1	67	2701	Moved
Month-to-month	Yes	Electronic check	99.65	820.5	Yes	1	86	5372	Moved
Month-to-month	Yes	Electronic check	104.8	3046.05	Yes	1	84	5003	Moved
Month-to-month	Yes	Bank transfer (automatic)	103.7	5036.3	Yes	1	89	5340	Competitor had better devices
Month-to-month	Yes	Electronic check	29.85	29.85	No	0	27	3964	
One year	No	Mailed check	56.95	1889.5	No	0	61	3441	
One year	No	Bank transfer (automatic)	42.3	1840.75	No	0	66	4307	
Month-to-month	Yes	Credit card (automatic)	89.1	1949.4	No	0	43	4459	
Month-to-month	No	Mailed check	29.75	301.9	No	0	51	2013	

(c)

Fig. 1.1 Sample data from the Telco Customer Churn dataset. (**a**) Displays the first half of the columns. (**b**) Displays the second half of the columns

1. **Converting Binary Text Data to Numerical Values**

   Binary data includes fields that have only two possible values, such as `Yes` or `No` for `Partner` or `Dependents`. We'll use Python's `map` function to convert these values to `1` (Yes) and `0` (No).

```
# Converting binary text columns to numerical values
import pandas as pd
# Load the dataset (replace 'path_to_file.csv' with the actual path
to your dataset)
df = pd.read_csv('path_to_file.csv')
# Converting 'Yes' to 1 and 'No' to 0 in relevant columns
binary_columns = ['Senior Citizen', 'Partner', 'Dependents',
'Phone Service',
          'Multiple Lines', 'Online Security', 'Online Backup',
          'Device Protection', 'Tech Support', 'Streaming TV',
          'Streaming Movies', 'Paperless Billing']
```

(continued)

```
# Apply mapping to each binary column
for column in binary_columns:
  df[column] = df[column].map({'Yes': 1, 'No': 0})
# Display the first few rows to confirm changes
print(df[binary_columns].head())
```

Here, we used the `map` function to convert each `Yes`/`No` value in `binary_columns` to `1` or `0`. Imagine this like flipping a switch: if a customer has a feature, it's "on" (1), and if they don't, it's "off" (0). This approach simplifies the model's task of identifying patterns, such as whether having a partner influences churn.

2. **One-Hot Encoding for Multi-category Text Data**

Some columns, like `Internet Service`, have multiple categories that don't have any particular order. For example, a customer could have DSL, Fiber Optic, or no internet service. To make this categorical data useful for machine learning, we need a way to represent each category as separate, unique columns. This process is called *one-hot encoding*.

One-hot encoding works by creating a new binary column for each category. If a customer has DSL internet, then only the `Internet_DSL` column would have a `1`, while the columns for other categories like `Internet_FiberOptic` and `Internet_No` would have a `0`. This way, each category is clearly distinguished and represented numerically.

Imagine a column called `Color` with three categories: Red, Blue, and Green. One-hot encoding would convert this column into three binary columns:

- `Color_Red` would be `1` if the color is Red, otherwise `0`.
- `Color_Blue` would be `1` if the color is Blue, otherwise `0`.
- `Color_Green` would be `1` if the color is Green, otherwise `0`.

So, if we had a row where the color is Red, the one-hot encoding would look like this:

Color	Color Red	Color Blue	Color Green
Red	1	0	0

And if we had a row where the color is Green, the one-hot encoding would look like this:

Color	Color Red	Color Blue	Color Green
Green	0	0	1

And if we had a row where the color is Blue, the one-hot encoding would look like this:

Color	Color Red	Color Blue	Color Green
Blue	0	1	0

In the Telco Customer Churn dataset, we'll apply one-hot encoding to columns with multiple categories, such as `Internet Service`, `Contract`, and `Payment Method`.

```
    # Applying one-hot encoding to categorical columns
    categorical_columns = ['Internet Service', 'Contract',
    'Payment Method']
# Perform one-hot encoding using pandas
df = pd.get_dummies(df, columns=categorical_columns,
drop_first=True)
# Display the new columns created by one-hot encoding
print(df.head())
```

- **`pd.get_dummies`**: This function from the Pandas library automatically applies one-hot encoding to the specified columns.
- **`columns=categorical_columns`**: We specify the columns we want to one-hot encode, which are `Internet Service`, `Contract`, and `Payment Method`.
- **`drop_first=True`**: This parameter removes the first category in each column to avoid redundancy. For example, if a column has three categories (e.g., DSL, Fiber Optic, No), dropping the first category means we'll only have columns for Fiber Optic and No, with `DSL` implied when both are `0`.

  One-hot encoding lets the model treat each category as a unique feature without any unintended ranking or order, making the data easier to interpret and analyze. This approach is particularly helpful when working with multiple-choice-type features, as each category is clearly separated into its own column, making patterns easier for the model to identify.

3. **Binning Numerical Data for Grouped Insights**

   The `Tenure Months` column tells us how long each customer has been with the company. Instead of using the raw values, we can group them into categories like "New," "Regular," and "Loyal" customers. This gives the model a clearer picture of tenure without needing to process specific month counts.

```
  # Binning 'Tenure Months' into categories
  bins = [0, 6, 24, df['Tenure Months'].max()]
  labels = ['New', 'Regular', 'Loyal']
  df['Tenure Category'] = pd.cut(df['Tenure Months'],
  bins=bins, labels=labels, right=False)
# Display the first few rows to confirm changes
print(df[['Tenure Months', 'Tenure Category']].head())
```

Using `pd.cut`, we grouped customers based on their tenure length, assigning each to a category. This is like describing a friend's job experience as "beginner," "intermediate," or "advanced" instead of counting the exact number of years. By grouping tenure, the model can more easily understand if customers with longer tenures are less likely to churn, for instance.

4. **Creating Aggregated Features**

   For customers using multiple add-on services like `Online Security`, `Online Backup`, and `Tech Support`, we can create an aggregated feature that counts the total number of additional services subscribed to. This aggregated view allows the model to understand a customer's engagement level with the company's offerings.

```
    # Aggregating add-on services
    addon_columns = ['Online Security', 'Online Backup', 'Device
    Protection',
            'Tech Support', 'Streaming TV', 'Streaming Movies']
# Summing up the add-on services each customer has
df['Total Addon Services'] = df[addon_columns].sum(axis=1)
# Display the first few rows to confirm the new column
print(df[['Total Addon Services']].head())
```

   The `sum(axis=1)` function sums across the specified columns, giving each customer a count of their additional services. Imagine asking someone how many hobbies they have rather than listing each one individually; this approach gives a quick, clear sense of a customer's engagement level, which can help the model identify who might be more invested and, therefore, less likely to churn.

5. **Extracting Sentiment from Textual Data (Optional)**

   For columns like `Churn Reason`, we can use natural language processing (NLP) to analyze the sentiment of each reason. This is more advanced but can reveal if certain reasons correlate with higher churn risk.

```
    from textblob import TextBlob
# Function to get sentiment polarity of the churn reason
def get_sentiment(text):
  if pd.isnull(text):
    return 0 # No churn reason provided
  return TextBlob(text).sentiment.polarity
# Applying the sentiment analysis to the 'Churn Reason' column
df['Churn Sentiment'] = df['Churn Reason'].apply(get_sentiment)
# Display the first few rows to confirm the sentiment column
print(df[['Churn Reason', 'Churn Sentiment']].head())
```

   Here, `TextBlob` is used to assign a sentiment score to each churn reason, where positive values indicate a positive sentiment and negative values indicate a

negative one. This is like judging a movie review as "happy" or "frustrated." A negative sentiment in `Churn Reason` could indicate an unhappy customer, helping the model recognize when dissatisfaction might lead to churn.

With these feature extraction techniques, we've converted text and categorical data into numerical forms, aggregated certain features, and grouped others into meaningful categories. These transformations help the model make sense of the dataset and predict churn more effectively, revealing insights about the characteristics and behaviors that are most likely to lead to customer turnover.

1.2.1.2 Feature Selection in Tabular Data

Feature selection is like packing for a trip: we have lots of items, but not all of them are necessary. In our dataset, some features will have a big impact on predicting churn, while others might have little to no influence. By carefully selecting the most relevant features, we make our model more efficient, reduce complexity, and improve its performance.

There are several techniques for feature selection, each helping us find the features that are likely to make the most impact on our predictions. Let's explore three popular methods: correlation analysis, feature importance, and recursive feature elimination (RFE). We'll use Python code to illustrate each approach with the Telco Customer Churn dataset.

1. **Correlation Analysis**

   Correlation analysis is a straightforward way to identify features that are strongly related to the target variable, `Churn Value`. By calculating the correlation between each feature and `Churn Value`, we can see which features have a meaningful relationship with churn. Features with higher absolute correlation values (either positive or negative) are more likely to impact churn prediction (Domingos 2012; Murphy 2022; Sculley et al. 2015).

   In our Telco Customer Churn dataset, we start by preparing the data: we drop irrelevant columns, convert `Yes`/`No` values to `1`/`0` for binary features, and apply one-hot encoding to multi-category features. This process ensures that our data is in the right format for correlation analysis.

```
    # Import necessary libraries
    import pandas as pd
# Load the dataset (replace 'path_to_file.csv' with your dataset's
path)
df = pd.read_csv('Book1.csv')
# Drop irrelevant columns that are not useful for modeling
df = df.drop(['CustomerID', 'Count', 'Country', 'State', 'City',
```

(continued)

```
'Zip Code',
       'Lat Long', 'Latitude', 'Longitude', 'Churn Label', 'Churn
Reason'], axis=1)
# Encode binary categorical columns (Yes/No columns) as 1/0
binary_columns = ['Gender', 'Senior Citizen', 'Partner',
'Dependents', 'Phone Service',
         'Multiple Lines', 'Online Security', 'Online Backup',
         'Device Protection', 'Tech Support', 'Streaming TV',
         'Streaming Movies', 'Paperless Billing']
for column in binary_columns:
  df[column] = df[column].map({'Yes': 1, 'No': 0})
# Apply one-hot encoding to multi-category columns
categorical_columns = ['Internet Service', 'Contract', 'Payment
Method']
df = pd.get_dummies(df, columns=categorical_columns,
drop_first=True)
# Target variable (Churn Value) and feature matrix (X)
y = df['Churn Value']
X = df.drop(['Churn Value'], axis=1)
# --- Feature Selection Method 1: Correlation Analysis ---
# Calculate correlation matrix and filter correlations with the
target variable
correlation_matrix = df.corr()
correlation_with_churn = correlation_matrix['Churn Value'].
sort_values(ascending=False)
# Display features with high correlation to Churn Value
print("Top features correlated with churn:\n",
correlation_with_churn.head(10))
```

This code calculates the correlation of each feature with `Churn Value`, helping us find features with a strong impact on churn. Think of it like understanding which friends' opinions influence your decisions the most—some opinions carry more weight than others. Here, we're identifying the features that strongly correlate with churn, making them prime candidates for further analysis and prioritization in our model.

2. **Feature Importance Using Tree-Based Models**

   Tree-based models, like Random Forest or Gradient Boosting, provide a feature importance score that ranks features by how useful they are for making predictions. This method doesn't require manually calculating correlations, and it works for both numerical and categorical data.

```
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.model_selection import train_test_split
df = pd.read_csv('Book1.csv')
# Prepare data: drop irrelevant columns and split data
X = df.drop(['CustomerID', 'Churn Value', 'Churn Label', 'Churn
Reason'], axis=1)
y = df['Churn Value']
X = pd.get_dummies(X, drop_first=True)  # Apply one-hot encoding
for categorical features
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
# Initialize and train the RandomForestClassifier
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
# Get feature importance scores
feature_importances = pd.Series(rf_model.feature_importances_,
index=X.columns)
top_features = feature_importances.sort_values
(ascending=False).head(10)
# Display the top 10 most important features
print("Top 10 important features:\n", top_features)
```

In this code, we use a Random Forest classifier to automatically rank features by importance. After training, we extract the feature importance scores, which indicate how much each feature contributes to the model's predictions. Think of this as getting advice from an expert—the model tells us which features are most valuable in determining churn, allowing us to prioritize them. The `top_features` output lists the ten most critical factors based on importance scores.

3. **Recursive Feature Elimination (RFE)**

   Recursive Feature Elimination (RFE) is an iterative technique that starts with all features and systematically removes the least important ones. RFE uses a model (often linear regression or a tree-based model) to evaluate feature importance and eliminate the least significant features until the desired number of features remains (Chatzimparmpas et al. 2022).

```
    from sklearn.feature_selection import RFE
    from sklearn.linear_model import LogisticRegression
    df = pd.read_csv('Book1.csv')
    # Initialize the model and RFE with the number of features we want
    to keep
```

(continued)

```
    logreg_model = LogisticRegression(max_iter=1000,
    random_state=42)
    rfe = RFE(estimator=logreg_model, n_features_to_select=10)
# Fit RFE and transform the dataset
rfe.fit(X, y)
# Get selected features
selected_features = X.columns[rfe.support_]
print("Selected features by RFE:\n", selected_features)
```

Here, RFE uses logistic regression to rank and remove the least important features in each round until only 10 remain. Imagine this like a reality competition where each round eliminates one contestant based on their performance. With each iteration, RFE narrows down the list, leaving only the top features that are likely to be most impactful for predicting churn.

Summary of Feature Selection Methods

Each feature selection method offers a unique perspective:

- **Correlation Analysis**: Best for initial exploration, especially with numerical features.
- **Feature Importance (Random Forest)**: Offers insights for both numerical and categorical features and is effective for complex relationships.
- **RFE**: An iterative, automated method to systematically reduce feature count, helping us achieve a lean, efficient model.

By choosing the most impactful features, we simplify the dataset, making the model faster and often more accurate. This careful selection process highlights the features that truly matter, ensuring our model focuses on the signals, not the noise, to predict churn more effectively.

1.2.2 Imagery Data

1.2.2.1 Understanding Image Data

When we think of images, we often picture photographs, illustrations, or graphics that convey scenes, objects, or ideas. In the digital world, however, images are actually just a collection of tiny elements called pixels. Each pixel holds information about color and brightness, making it a building block of the entire image. Together, pixels create the details, textures, and shapes that our brains interpret as a picture (AbuRass et al. 2020; Han et al. 2022; Wickham 2014).

Imagine an image as a giant mosaic where each tile is a pixel. If you zoom in close enough, you'll see each pixel as a small square of solid color. When you step back, your brain blends these pixels, creating a seamless image. This is the magic of digital images: a vast collection of simple, colored squares coming together to form complex visuals. Each pixel in an image has a color value that represents the shade or tone it contributes to the whole picture. The way these colors are represented depends on the image format. Most images we encounter are RGB images, where each pixel has three values corresponding to the Red, Green, and Blue components of its color. By mixing different intensities of red, green, and blue, each pixel can display a wide range of colors, from pure red to soft lavender to deep black. Let's think of pixels like colored lights. Imagine you're at a concert where red, green, and blue spotlights mix to create various lighting effects. If the red light shines brightly and the green and blue lights are dim, you see a strong red color. If all three lights are at maximum brightness, they combine to create white light. Similarly, the RGB values in a pixel control its color. In grayscale images, each pixel only has a single value, representing the intensity of light, from black (no light) to white (maximum light). So, in a grayscale image, the range of shades we see is created by varying levels of brightness.

In Fig. 1.2, we can see the famous Lena image with a 32×32 region extracted around the eye area. This cropped region reveals the structure of the pixels, each represented as a colored square, making it clear how individual pixels come together to form the details of an image. When viewed up close, each pixel is a distinct square of color, showing how each small component contributes to the overall appearance of Lena's eye. In Fig. 1.3, we display the actual RGB values for each pixel in the extracted 32×32 region. This figure highlights the numerical values that correspond to the colors we see in Fig. 1.2, with each cell showing the specific red, green, and blue intensities that define the color of each pixel. Together, Fig. 1.2 and Table 1.3



Fig. 1.2 Lena image with a 32×32 region extracted around the eye area, showing individual pixels as colored squares

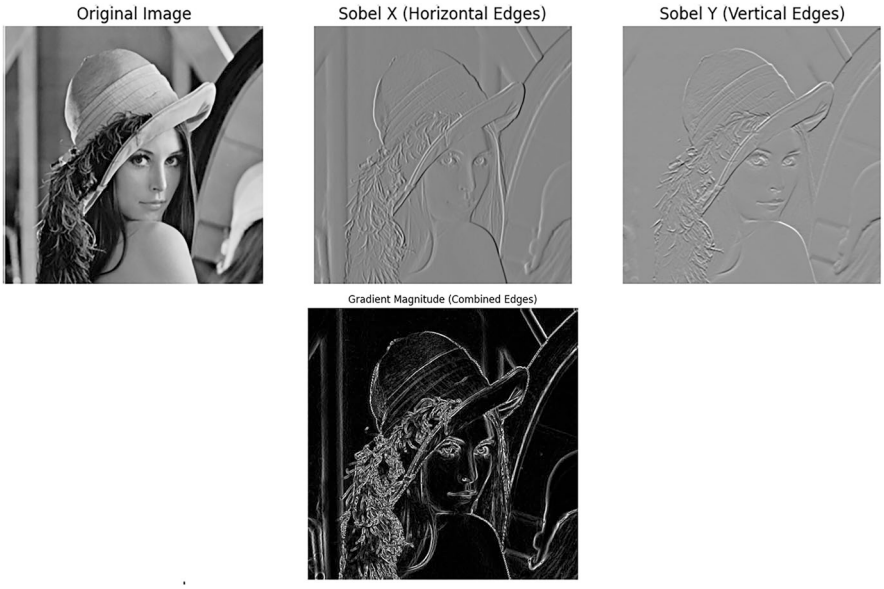


Fig. 1.3 Edge detection on the Lena image using the Sobel operator

illustrate how images are built from RGB values, transforming digital data into the visuals we recognize.

1.2.2.2 Edge Detection

Edge detection is a powerful technique in image processing used to find boundaries or outlines within an image. Think of edges as the lines that separate different parts of a scene, like the edges of a building against the sky, or the outline of a person in a photo. Detecting edges helps machine learning models understand shapes, objects, and boundaries, making it easier to interpret the content of images.

Edges are areas in an image where there's a significant change in intensity or color between adjacent pixels. Imagine looking at a black-and-white drawing of a house—the lines of the drawing are the edges, showing where the walls meet the roof or where a window starts. In the world of digital images, we use mathematical operations to identify these changes in intensity, translating them into edges (Dorgham 2008).

How Edge Detection Works

The process of edge detection involves identifying places in an image where the pixel values change abruptly. To find these changes, we use a mathematical tool

Table 1.3 RGB values for each pixel in the 16 × 16 area of the extracted 32 × 32 region, displaying the numerical color intensities that define each pixel's color

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	[214, 177, 170]	[215, 180, 169]	[215, 183, 181]	[223, 187, 180]	[219, 188, 168]	[214, 181, 167]	[211, 167, 136]	[200, 111, 100]	[198, 83, 85]	[204, 83, 96]	[207, 97, 107]	[214, 107, 103]	[214, 115, 122]	[220, 112, 119]	[224, 122, 121]	[225, 123, 118]
2	[214, 181, 175]	[217, 188, 180]	[220, 189, 176]	[219, 194, 171]	[218, 185, 173]	[206, 160, 135]	[196, 95, 86]	[196, 80, 93]	[202, 83, 94]	[205, 85, 91]	[211, 99, 109]	[217, 103, 105]	[218, 111, 105]	[223, 121, 121]	[224, 121, 117]	[221, 126, 122]
3	[217, 184, 176]	[218, 189, 180]	[219, 193, 174]	[217, 185, 163]	[206, 143, 109]	[183, 81, 80]	[192, 74, 84]	[193, 92, 96]	[202, 98, 109]	[213, 94, 97]	[210, 96, 100]	[210, 99, 105]	[218, 103, 99]	[218, 115, 114]	[217, 116, 112]	[217, 109, 109]
4	[223, 192, 169]	[222, 192, 173]	[218, 179, 161]	[201, 133, 93]	[184, 75, 79]	[187, 77, 88]	[198, 76, 81]	[201, 85, 87]	[209, 90, 95]	[210, 88, 94]	[201, 89, 92]	[208, 88, 89]	[210, 107, 110]	[213, 102, 104]	[215, 99, 100]	[211, 96, 102]
5	[223, 195, 176]	[213, 175, 147]	[193, 108, 81]	[176, 67, 74]	[182, 61, 67]	[187, 74, 80]	[196, 77, 82]	[200, 76, 77]	[199, 77, 83]	[204, 82, 91]	[201, 82, 80]	[198, 83, 90]	[203, 93, 86]	[196, 88, 88]	[201, 80, 78]	[203, 91, 86]
6	[209, 169, 147]	[179, 93, 74]	[154, 53, 69]	[168, 69, 82]	[184, 57, 62]	[181, 70, 78]	[192, 72, 77]	[188, 72, 78]	[185, 65, 70]	[173, 57, 77]	[170, 62, 66]	[177, 63, 73]	[182, 68, 69]	[180, 67, 76]	[184, 71, 75]	[184, 76, 71]
7	[165, 74, 70]	[153, 52, 64]	[154, 53, 70]	[175, 59, 73]	[175, 65, 68]	[173, 61, 74]	[180, 65, 72]	[183, 68, 74]	[176, 66, 76]	[169, 64, 73]	[142, 42, 61]	[151, 39, 61]	[148, 37, 58]	[141, 31, 57]	[152, 48, 60]	[139, 41, 58]
8	[150, 46, 69]	[158, 47, 61]	[155, 43, 66]	[174, 59, 68]	[164, 52, 67]	[156, 41, 57]	[138, 35, 60]	[141, 36, 60]	[157, 55, 70]	[152, 52, 71]	[130, 32, 60]	[124, 33, 63]	[125, 37, 65]	[123, 30, 63]	[117, 24, 56]	[120, 27, 60]
9	[145, 47, 75]	[162, 52, 66]	[149, 42, 65]	[152, 46, 65]	[148, 36, 60]	[141, 37, 65]	[118, 30, 59]	[116, 19, 57]	[123, 28, 55]	[128, 33, 59]	[115, 25, 59]	[110, 25, 60]	[114, 27, 62]	[114, 22, 55]	[107, 19, 62]	[111, 25, 61]
10	[144, 35, 64]	[143, 36, 61]	[142, 33, 57]	[143, 31, 55]	[129, 30, 63]	[126, 24, 52]	[115, 20, 59]	[108, 20, 57]	[111, 24, 59]	[110, 22, 57]	[110, 20, 55]	[108, 22, 59]	[109, 22, 58]	[108, 18, 56]	[109, 19, 57]	[107, 15, 52]
11	[144, 41, 65]	[135, 23, 57]	[126, 17, 53]	[121, 22, 51]	[115, 26, 63]	[118, 19, 58]	[110, 18, 54]	[110, 22, 60]	[110, 20, 62]	[106, 24, 53]	[112, 28, 69]	[105, 22, 56]	[105, 15, 56]	[108, 20, 55]	[109, 21, 55]	[103, 15, 53]
12	[136, 32, 62]	[120, 27, 58]	[118, 19, 59]	[111, 18, 55]	[104, 16, 53]	[103, 19, 57]	[109, 23, 60]	[109, 21, 67]	[108, 20, 57]	[104, 26, 60]	[111, 26, 60]	[103, 23, 60]	[104, 25, 65]	[103, 14, 54]	[99, 13, 49]	[101, 10, 54]
13	[132, 35, 61]	[111, 20, 57]	[104, 18, 53]	[102, 18, 59]	[104, 24, 61]	[100, 18, 60]	[106, 17, 53]	[106, 17, 56]	[106, 16, 56]	[106, 27, 67]	[102, 17, 56]	[109, 21, 57]	[104, 19, 54]	[104, 14, 57]	[95, 15, 57]	[92, 17, 67]
14	[124, 24, 56]	[101, 11, 46]	[106, 15, 56]	[101, 20, 61]	[102, 19, 57]	[101, 21, 59]	[102, 20, 54]	[107, 24, 57]	[114, 21, 56]	[102, 17, 55]	[99, 12, 55]	[106, 25, 63]	[106, 18, 61]	[105, 19, 63]	[91, 11, 59]	[94, 19, 60]
15	[115, 17, 50]	[101, 11, 55]	[96, 11, 52]	[98, 11, 54]	[99, 23, 67]	[105, 24, 63]	[115, 34, 60]	[133, 49, 71]	[131, 34, 68]	[96, 14, 52]	[104, 16, 56]	[112, 36, 62]	[118, 29, 65]	[97, 16, 58]	[86, 11, 62]	[82, 8, 53]
16	[105, 27, 60]	[92, 16, 60]	[94, 13, 54]	[101, 12, 59]	[106, 23, 55]	[119, 37, 67]	[139, 54, 76]	[148, 61, 84]	[144, 49, 78]	[101, 22, 62]	[113, 32, 67]	[131, 54, 79]	[132, 50, 87]	[102, 18, 61]	[86, 9, 55]	[80, 8, 56]

called a gradient, which measures how much a pixel value changes in different directions. Here's a step-by-step breakdown:

1. Convert the Image to Grayscale: Since edge detection relies on intensity changes, it's common to first convert the image to grayscale, reducing it to a single intensity value per pixel. This simplifies the process, as we only need to focus on brightness variations instead of colors.
2. Calculate the Gradient: The gradient is a mathematical way of measuring how quickly the intensity changes from one pixel to the next. It has two main components:
   - Gradient in the X direction (G_x)—measures how much intensity changes horizontally.
   - Gradient in the Y direction (G_y)—measures how much intensity changes vertically.

   Mathematically, the gradient at a pixel (x, y) in an image I can be represented as:

$$G_x = \frac{\partial I}{\partial x}, G_y = \frac{\partial I}{\partial y} \tag{1.1}$$

   where $\frac{\partial I}{\partial x}$ and $\frac{\partial I}{\partial y}$ are partial derivatives of the intensity I with respect to the x and y directions.
3. Compute the Edge Magnitude and Direction: The magnitude of the gradient gives the strength of the edge at each pixel, while the direction tells us the orientation of the edge. The gradient magnitude G at a pixel is calculated as:

$$G = \sqrt{G_x^2 + G_y^2} \tag{1.2}$$

   This value G represents the intensity of the edge, with higher values indicating stronger edges. The direction θ of the edge can be computed as:

$$\theta = \arctan \frac{G_y}{G_x} \tag{1.3}$$

   By calculating G and θ at each pixel, we can determine both where the edges are and which direction they're going.

Edge Detection Methods: Sobel Operator

One popular technique for edge detection is the Sobel operator, which uses convolution to approximate G_x and G_y. The Sobel operator applies two 3×3 filters (one for each direction), which detect intensity changes. These filters look like this:

- Sobel Filter for G_x:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- Sobel Filter for G_y:

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

These filters slide over the image, calculating G_x and G_y for each pixel by measuring the differences in intensity. After applying these filters, we compute the magnitude G and direction θ for each pixel to obtain a clear outline of the edges in the image.

Edge detection helps machine learning models and computer vision systems focus on the most important structural parts of an image. By identifying boundaries, models can more easily distinguish between different objects, recognize shapes, and even understand textures. Whether it's detecting lanes in self-driving cars, segmenting objects in medical images, or recognizing faces, edge detection provides a foundational step in interpreting images.

Here's how to implement edge detection in Python using the Sobel operator. This example demonstrates how to apply the Sobel filters in the X and Y directions to detect edges in an image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Load the image (replace 'lena.jpg' with your image file path)
image = cv2.imread('lena.jpg', cv2.IMREAD_GRAYSCALE)
# Apply Sobel operator in the X direction
sobel_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3) # Gx
# Apply Sobel operator in the Y direction
sobel_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3) # Gy
# Calculate the gradient magnitude
sobel_magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
sobel_magnitude = np.uint8(sobel_magnitude) # Convert to 8-bit
image for display
# Display the original image and edge-detected results
plt.figure(figsize=(12, 6))
```

(continued)

```
# Original Image
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis("off")
# Sobel X Image
plt.subplot(1, 3, 2)
plt.imshow(sobel_x, cmap='gray')
plt.title("Sobel X (Horizontal Edges)")
plt.axis("off")
# Sobel Y Image
plt.subplot(1, 3, 3)
plt.imshow(sobel_y, cmap='gray')
plt.title("Sobel Y (Vertical Edges)")
plt.axis("off")
plt.show()
# Display the gradient magnitude image
plt.figure(figsize=(6, 6))
plt.imshow(sobel_magnitude, cmap='gray')
plt.title("Gradient Magnitude (Combined Edges)")
plt.axis("off")
plt.show()
```

In this code, we start by loading the image in grayscale mode, which simplifies edge detection by focusing solely on intensity values rather than color. Grayscale images allow us to process brightness changes directly, which is essential for detecting edges effectively.

Next, we apply the Sobel operator in both the *X* and *Y* directions to capture edges along horizontal and vertical lines. The Sobel filter in the *X* direction highlights areas with significant horizontal intensity changes, identifying features such as the left or right sides of objects. Conversely, the Sobel filter in the *Y* direction detects vertical edges, focusing on areas where the intensity shifts up or down. These two filters together give a comprehensive view of edges in the image by isolating directional changes.

After obtaining the horizontal and vertical gradients, we calculate the gradient magnitude by combining the *X* and *Y* components. This combined gradient reveals the full strength of edges at each point, regardless of direction, by measuring the total rate of intensity change. Mathematically, this is achieved by taking the square root of the squared gradients in the *X* and *Y* directions, which highlights the overall structure and boundaries of objects in the image.

Finally, we display the results to visualize the different stages: the original grayscale image, the horizontal and vertical edge-detected images, and the complete gradient magnitude image. Each view serves a distinct purpose—the *X* and

Y gradients showcase edges in specific directions, while the gradient magnitude provides a unified look at all edges in the image. This step-by-step process demonstrates how the Sobel operator and gradient calculations work together to detect edges, revealing the underlying shapes and boundaries within the image. Figure 1.3 shows these outputs for the Lena image, with results from the Sobel X and Sobel Y filters, as well as the combined gradient, illustrating the effectiveness of edge detection across different orientations.

1.2.2.3 Cubixel-Based Feature Extraction

Cubixels

In traditional image analysis, we often think of images as 2D arrays made up of pixels, where each pixel is the smallest unit of information, holding color and intensity data. This pixel-based paradigm has been the backbone of computer vision tasks for decades. However, to uncover deeper, more spatially aware image features, researchers have introduced the concept of Cubixels. This innovative approach extends the idea of a pixel from a 2D plane into a 3D space, adding an extra layer of information to analyze. By working with Cubixels, we can better capture spatial relationships and interactions within an image, providing more robust features for machine learning and image processing tasks (Aburass 2024).

Imagine a pixel as a flat, colored square. While this square can tell us a lot about color and position, it doesn't capture any depth or 3D structure. A Cubixel, on the other hand, takes that colored square and adds a third dimension, turning it into a tiny cube. Each face of this cube can represent more complex data, giving us a richer understanding of the image's structure and composition. Refer to Fig. 1.4.

In a standard image, a pixel at location (x, y) holds color values defined in terms of RGB:

$$P(x, y) = (R, G, B) \tag{1.4}$$

where R, G, and B are the intensities of the red, green, and blue channels, respectively.

For Cubixels, we extend this concept by representing each pixel in three-dimensional space. Each Cubixel $C(x, y)$ at position (x, y) takes the original color intensities and uses them to define its width (w), length (l), and height (h). Mathematically, we can express a Cubixel as:

$$C(x, y) = (w, l, h) = (\alpha R, \alpha G, \alpha B) \tag{1.5}$$

where α is a scaling factor that adjusts the size of the Cubixel based on the context of the image.

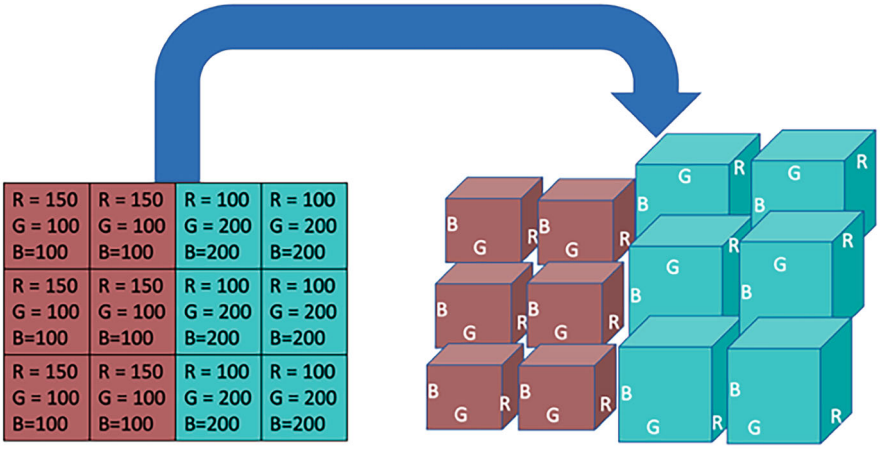


Fig. 1.4 Transformation from pixels to Cubixels

The size and orientation of each Cubixel, determined by *R*, *G*, and *B*, give depth and additional dimensionality to what was previously a flat pixel. Think of each Cubixel as a colored building block, where:

- Width (w) is influenced by the red channel.
- Length (l) is influenced by the green channel.
- Height (h) is influenced by the blue channel.

This cubic representation allows for a more nuanced analysis of an image's structure. By examining how these Cubixels interact and fit together in 3D space, we can extract features that reveal more complex spatial relationships within the image, paving the way for enhanced image recognition and machine learning applications.

Here's a Python code snippet to transform an image into a representation of Cubixels. For simplicity, this example will show how to create a 3D visualization where each pixel in the image is represented as a Cubixel with properties determined by its RGB values.

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
from PIL import Image
# Load the image (replace 'image_path.jpg' with your image file
path)
image = Image.open('/mnt/data/image.png')
# Resize the image to a smaller size for better visualization
image = image.resize((32, 32)) # Adjust size as needed for
```

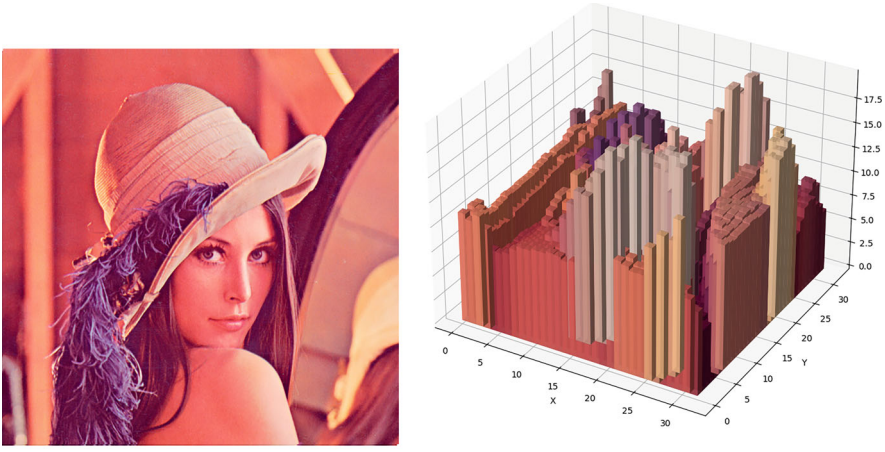
(continued)

```
visualization
image_array = np.array(image)
# Scaling factor to control the size of the Cubixels
alpha = 0.1
# Create a 3D plot
fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(111, projection='3d')
# Iterate over each pixel in the image and plot as Cubixels
for x in range(image_array.shape[1]): # width
  for y in range(image_array.shape[0]): # height
    # Get the RGB values of the current pixel
    R, G, B = image_array[y, x, 0], image_array[y, x, 1],
image_array[y, x, 2]
    # Define the dimensions of the Cubixel based on RGB values
    w, l, h = alpha * R, alpha * G, alpha * B
    # Plot the Cubixel as a small 3D rectangle with offset positions
    ax.bar3d(x, y, 0, 1, 1, h, color=np.array([R, G, B]) /
255, alpha=0.8)
# Set plot aesthetics
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Height')
ax.set_title('Cubixel Representation of the Image (No Overlap)')
plt.show()
```

In this code, we have modified the way Cubixels are visualized to prevent overlapping, ensuring that each one is positioned correctly in a 3D space. The "ax.bar3d" function has been adjusted to place each Cubixel at coordinates "(x, y, 0)" with a consistent base size of "1 × 1" grid unit and a height determined by the RGB values. This positioning ensures that each Cubixel is displayed side by side without intersecting with its neighbors, creating a clear and organized representation of the image.

The RGB values are still used to determine the dimensions of each Cubixel, with the width, length, and height scaled using a factor α. The color of each Cubixel corresponds to the original pixel color from the image, allowing us to see how the 2D image data translates into a 3D structure. By visualizing the image this way, we can see how each pixel in the image contributes to the overall 3D formation, providing a richer representation of the image data.

In Fig. 1.5, part (a) shows the original Lena image for reference, while part (b) illustrates the Cubixel-based transformation. This transformation highlights how image data can be represented in a 3D space, with each pixel turning into a small colored cube. The 3D visualization emphasizes the relationship between color



(a) (b)

Fig. 1.5 (**a**) The original Lena image, showing the familiar 2D representation. (**b**) The transformed image using Cubixel-based feature extraction

intensities and the resulting Cubixel dimensions, giving us a more profound understanding of image structure and potential feature extraction approaches.

The Volume of the Void (VoV) Concept

The Volume of the Void (VoV) is an innovative concept that helps quantify the "empty spaces" between Cubixels. This idea is crucial for understanding the spatial differences within an image. Think of it as assessing the gaps between Lego blocks when they are assembled: the space between the blocks reveals much about the overall structure and how pieces fit together. Similarly, in image analysis, the VoV measures the spatial discontinuity between adjacent Cubixels, providing insight into the image's texture and depth.

Mathematically, the VoV between two adjacent Cubixels C_i and C_j is defined as the absolute difference in their volumes:

$$VoV\left(C_i, C_j\right) = |\, V(C_i) - V\left(C_j\right) \,| \tag{1.6}$$

where $V(C)$ represents the volume of a Cubixel, calculated as the product of its width, length, and height (dimensions derived from the RGB color values). This approach means that if two adjacent Cubixels have very different volumes, it suggests a significant change in color intensity or depth, marking an edge or boundary in the image.

Expanding this to a grid of Cubixels, we can calculate the *cumulative VoV* across multiple neighbors. For a set of eight neighboring Cubixels around a central Cubixel C_0, the total VoV is:

$$VoV_{\text{total}} = \sum_{i=0}^{8} |V(C_0) - V(C_i)| \tag{1.7}$$

This sum gives a more comprehensive measure of how varied the image's local structure is, identifying regions with high disparity. Imagine analyzing the surface of a road. If it's smooth, the VoV between Cubixels representing it would be low because neighboring Cubixels would have similar volumes. However, if there are cracks or bumps, the VoV would be higher, capturing the changes in texture and depth. Here's the code to compute the Volume of the Void for an image represented by Cubixels:

```
import numpy as np
from PIL import Image
# Load and preprocess the image (convert to RGB and resize for
simplicity)
image = Image.open('image_path.jpg').resize((32, 32)) # Adjust
the size as needed
image_array = np.array(image)
# Define a function to calculate the volume of a Cubixel based on RGB
values
def calculate_volume(R, G, B, alpha=0.1):
  return alpha * R * alpha * G * alpha * B
# Calculate the VoV for the entire image
def calculate_vov(image_array, alpha=0.1):
  height, width, _ = image_array.shape
  total_vov = 0
  # Iterate over each pixel (Cubixel) in the image except for the
border
  for y in range(1, height - 1):
    for x in range(1, width - 1):
      # Get the RGB values of the current Cubixel
      R0, G0, B0 = image_array[y, x]
      V0 = calculate_volume(R0, G0, B0, alpha)
      # Sum the VoV with its 8 neighbors
      for dy in [-1, 0, 1]:
        for dx in [-1, 0, 1]:
          if dx != 0 or dy != 0: # Skip the center Cubixel
            Rn, Gn, Bn = image_array[y + dy, x + dx]
            Vn = calculate_volume(Rn, Gn, Bn, alpha)
            total_vov += abs(V0 - Vn)
  return total_vov
```

(continued)

```
# Compute the VoV for the given image
vov = calculate_vov(image_array)
print(f"The total Volume of the Void (VoV) for the image is: {vov}")
```

In this code, we start by loading the image using the `PIL` library and resizing it to a manageable size (e.g., 32 × 32 pixels) for easier computation. The image is converted into an RGB array so we can access the color values of each pixel. The function `calculate_volume` is then used to determine the volume of each Cubixel by taking the RGB values and scaling them using a factor α. This function represents the volume as $V(C) = (\alpha R) \times (\alpha G) \times (\alpha B)$, providing a 3D volume that reflects the pixel's color intensity.

The main function, `calculate_vov`, iterates over each Cubixel in the image, excluding the border pixels for simplicity. For each Cubixel at position (x, y), it computes the absolute difference in volume between it and its eight neighboring Cubixels. These differences are summed to calculate the cumulative VoV for the entire image. The nested loops ensure that each Cubixel is compared with its adjacent neighbors, capturing variations in color and intensity that reveal the spatial structure of the image.

Finally, the total VoV is printed, representing the sum of these volume differences across the image. A high VoV indicates significant color and depth variation between neighboring Cubixels, which can signify edges, textures, or other intricate features in the image. Conversely, a low VoV implies more uniformity and less variation, indicating smoother, more homogeneous areas. This approach provides a deeper understanding of an image's structural properties, crucial for advanced image analysis and feature extraction in machine learning and computer vision applications.

VoV Edge Detection for Feature Extraction

When it comes to feature extraction in image analysis, the VoV method offers a powerful way to identify edges using the Cubixel representation. Unlike traditional 2D edge detection techniques that only consider pixel intensity changes, VoV takes it a step further by assessing how the "depth" or volume of one Cubixel differs from its neighbors. This unique approach captures more nuanced details, making it a valuable tool for analyzing complex images where subtle depth or texture changes matter.

Imagine looking at a brick wall where some bricks stick out further than others. If you run your hand along the surface, you'll feel where one brick's edge meets another's. VoV edge detection works similarly but in the context of image data, where the "bricks" are Cubixels with volumes influenced by their RGB values. When two adjacent Cubixels have a noticeable difference in volume, the algorithm flags that boundary as an edge.

Mathematically, the process begins by calculating the volume of each Cubixel in the image. If C_p is a Cubixel at pixel p, its volume $V(C_p)$ can be expressed using the product of its color intensities, scaled by a factor α:

$$\mathrm{V}\left(C_p\right) = \left(\alpha\, R_p\right) \times \left(\alpha\, G_p\right) \times \left(\alpha\, B_p\right) \tag{1.8}$$

Here, R_p, G_p, and B_p are the red, green, and blue intensity values at pixel p. This volume provides a 3D representation that allows us to think about image features beyond just flat pixels.

The VoV for a Cubixel C_p is determined by comparing its volume to those of its neighboring Cubixels, N_p. For each pixel p, the maximum difference in volume between C_p and its neighbors is computed as:

$$VoV_p = \max_{n \in N_p} \left| V\left(C_p\right) - V(C_n) \right| \tag{1.9}$$

This step highlights significant differences in volume, much like noticing that one brick in the wall protrudes while its neighboring bricks are flush. A larger VoV value indicates a more pronounced edge.

To ensure that the algorithm only identifies meaningful edges, a threshold T is set based on the distribution of VoV values across the image. This threshold is determined using a percentile p_{value}, ensuring that only the most significant differences are highlighted:

$$T = \text{Percentile}\ (VoV, p_{\text{value}}) \tag{1.10}$$

This is akin to deciding that only the largest protrusions on the wall are worth noting, filtering out smaller, less important changes.

Once the threshold T is established, the algorithm generates an edge map by marking each pixel p as an edge if $\mathrm{VoV}_p > T$. The result is a binary image where high VoV regions are marked, revealing the edges in the original image. To improve the clarity and continuity of these edges, optional morphological operations like dilation or erosion can be applied. These operations help connect broken lines and remove small, noisy areas, much like smoothing out the rough edges of the bricks to make the overall structure clearer.

The VoVDetectEdges algorithm encapsulates this process, beginning with the calculation of Cubixel volumes and iterating over the image to assess the VoV for each pixel and its neighbors. The algorithm stores the largest VoV at each pixel location and applies the dynamically determined threshold to create the edge map. Finally, optional refinement steps enhance the map, making it more visually coherent and reliable for subsequent image processing tasks.

Require: `img`, `percentile_value`
Ensure: `edge_map`—Edge-detected image

```
void_volumes ← ZeroArray(height of img, width of img)
for each pixel at (x, y) in img do
  current_volume ← CalculateVolume(img[x, y])
  for each neighbor at (nx, ny) adjacent to (x, y) do
    neighbor_volume ← CalculateVolume(img[nx, ny])
    local_void ← ABS(current_volume - neighbor_volume)
    void_volumes[x, y] ← MAX(void_volumes[x, y], local_void)
  end for
end for
threshold ← CalculatePercentileThreshold(void_volumes,
percentile_value)
edge_map ← ApplyThresholdToVoidVolumes(void_volumes,
threshold)
edge_map ← MorphologicalRefinement(edge_map)
return edge_map
```

Algorithm 1 VoVDetectEdges

This method is particularly effective for images with intricate textures or depth variations, where conventional edge detection might struggle. By focusing on the volume differences between Cubixels, the VoV approach extracts edges that reflect real structural and textural changes, offering a more detailed and insightful analysis of the image. Here's the code to implement VoVDetectEdges Algorithm.

```
import numpy as np
from PIL import Image, ImageFilter
from scipy.ndimage import generate_binary_structure, binary_opening
import numpy as np
from PIL import Image, ImageFilter
import matplotlib.pyplot as plt
from scipy.ndimage import binary_opening, generate_binary_structure
import time
def calculate_void_volume(cubixel1, cubixel2):
    # Vectorized computation of volumes and their absolute differences
    volume1 = np.prod(cubixel1, axis=-1)
    volume2 = np.prod(cubixel2, axis=-1)
    void_volume = np.abs(volume1 - volume2)
    return void_volume

def detect_edges(image_path, percentile_value):
    # Load and preprocess the image
    img = Image.open(image_path)
    img.thumbnail((256, 256))
    img = img.filter(ImageFilter.MedianFilter(size=3))
    img_array = np.array(img, dtype=np.float32) / 255.0

    height, width, _ = img_array.shape
    void_volumes = np.zeros((height, width), dtype=np.float32)

    # Vectorized computation of void volumes
    for di in [-1, 0, 1]:
        for dj in [-1, 0, 1]:
            if di == 0 and dj == 0:
                continue
            # Compute volumes for neighboring pixels
            neighbor_pixels = np.roll(np.roll(img_array, di, axis=0), dj,
axis=1)
            local_void = calculate_void_volume(img_array, neighbor_pixels)
            void_volumes = np.maximum(void_volumes, local_void)

    # Calculate percentile-based threshold
    threshold = np.percentile(void_volumes, percentile_value)

    # Adjust the threshold if necessary
```

```
        if threshold > 0.1 * np.ptp(void_volumes):
            threshold = 0.1 * np.ptp(void_volumes)

        # Generate edge map
        edge_map = (void_volumes >= threshold).astype(np.uint8) * 255

        # Apply morphological opening
        structure = generate_binary_structure(2, 1)
        edge_map = binary_opening(edge_map,
structure=structure).astype(np.uint8) * 255

        return Image.fromarray(edge_map)

    # Example usage
    percentile_value = 80
    start = time.time()
    edge_map_image = detect_edges('img.jpeg', percentile_value)
    print(f"Time elapsed: {time.time() - start:.2f} seconds")
    # Display the edge map
    plt.imshow(edge_map_image, cmap='gray')
    plt.axis('off')
    plt.show()
```

This Python code provides an efficient way to perform VoV edge detection using Cubixels, which is a unique approach for identifying edges in an image based on volume differences between neighboring pixels. The code begins by loading an image and resizing it to a smaller, more manageable size of 256 × 256 pixels, simplifying the process and speeding up computation. To prepare the image for analysis, a median filter is applied. This step smooths the image by reducing noise, much like squinting your eyes to overlook minor details and better see the overall structure.

The essence of the code lies in the function "calculate_void_volume," which calculates the volume of a Cubixel using the product of its RGB values. This function then determines the absolute difference in volume between two Cubixels—essentially comparing the "size" of one pixel's color space to another's. Imagine comparing two blocks of different dimensions: the difference in their volumes tells you how much one stands out from the other. The greater this difference, the more likely it is that an edge is present between them.

The main function, "detect_edges," loops through each pixel in the image and compares it with its eight neighboring pixels. By shifting the image array using "np.roll," the function aligns the current pixel with its neighbors in all possible directions—up, down, left, right, and diagonally. This method ensures that the void volume is computed for every pixel and its surroundings, filling the "void_volumes" array with the largest detected volume difference at each position. The process is akin to running your hand along a textured surface and feeling where the bumps and dips are most pronounced.

To highlight only significant edges, a percentile-based threshold is calculated from the distribution of VoV values. This threshold acts as a filter, ensuring that only the most prominent volume differences are retained as edges. If the calculated threshold is too broad and risks capturing excessive minor details, an adjustment condition is applied to refine it. This is similar to deciding that only the most noticeable gaps between bricks are worth marking, ignoring smaller variations that don't contribute much to the overall structure.

The algorithm then generates an edge map by checking whether each pixel's VoV exceeds the threshold. If it does, that pixel is marked as part of an edge, resulting in a binary image where edges are represented in white (255) and non-edge areas remain black (0). To improve the visual clarity of this edge map, the code applies a morphological opening operation. This operation smooths out the map by connecting fragmented edges and removing small, isolated noise, like brushing away loose grains of sand to reveal the more defined lines beneath.

Imagine using this algorithm on an image of a complex surface, such as a fabric with intricate patterns. Each pixel becomes a Cubixel with a volume determined by its color. The algorithm moves through the image, comparing these blocks and marking areas where the volume difference between neighboring blocks is significant. The result is an edge map that highlights the boundaries within the image, showing you where the textures and depth differences are most apparent. The final edge map, visualized with "matplotlib," is ready for further analysis, such as segmentation or advanced feature extraction. This makes the VoV edge detection method particularly valuable for understanding the subtle, textured details in images that traditional 2D edge detectors might overlook. In Fig. 1.6, we present the original Lena image alongside the result of applying the VoV edge detection algorithm.



Fig. 1.6 The original Lena image (**a**) and the edge-detected image (**b**)

Comparing and Contracting

When comparing the pixel paradigm and the Cubixel paradigm specifically for feature extraction in machine learning, it's essential to focus on how each approach processes and extracts meaningful information from images and what implications this has for model performance. The pixel paradigm has long been the standard in feature extraction, leveraging individual pixel values and their relationships to nearby pixels to identify features such as edges, textures, and color patterns. Algorithms like Sobel and Canny edge detection, or methods that rely on pixel intensity histograms, are widely used to create feature maps that inform machine learning models. This paradigm works well for basic image processing and straightforward tasks like object detection and simple classification. Think of it as reading a printed page—each letter and space contributes to the overall meaning, but the interpretation remains largely flat.

However, the Cubixel paradigm revolutionizes feature extraction by incorporating an additional dimension: depth. Unlike traditional pixels, Cubixels have volumetric properties derived from their RGB values. This shift enables the calculation of advanced metrics, such as the VoV, which provides a richer set of features by capturing changes in both color and the spatial volume between neighboring Cubixels. This approach adds a new layer of detail, allowing machine learning models to discern subtle variations in texture and depth that would be overlooked in a 2D pixel analysis. Imagine analyzing a sculpture versus a photograph of it—the Cubixel paradigm lets you explore the form and depth of the sculpture, not just its appearance from one angle.

Feature extraction with Cubixels and VoV is particularly powerful in contexts where understanding the interplay between color and spatial structure is crucial. For example, in medical imaging, analyzing tissue patterns with depth and texture helps identify irregularities more accurately than flat pixel-based methods. The VoV edge detection algorithm discussed earlier exemplifies how Cubixel-based extraction highlights boundaries in an image by measuring significant volume differences between neighboring Cubixels, revealing fine details that traditional 2D methods might miss. This enables models to leverage more complex and informative features, leading to better performance on tasks that require detailed image analysis, such as image segmentation or fine-grained classification. The pixel paradigm, while simpler and computationally efficient, often falls short when tasked with capturing the true complexity of an image's spatial properties. Cubixel-based feature extraction, on the other hand, provides a more comprehensive view by integrating depth into the analysis. This allows for the creation of feature maps that are not just informed by color or basic intensity changes but are enriched with data about the three-dimensional relationships between parts of the image. For machine learning models, this means a more nuanced input space, which can translate to more robust predictions and better generalization, especially in domains requiring detailed visual understanding.

1.3 Concluding Remarks

In this chapter, we've unraveled the critical role of feature extraction in the journey from raw data to predictive insights. We've seen how well-crafted features serve as the bridge between unstructured or tabular data and a model's ability to learn and generalize. By exploring various types of features, from straightforward numerical ones to more nuanced textual and image-based features, we've underscored that features are much more than mere data points—they are the essence of what allows a model to "understand" its environment.

Techniques such as binary conversion, one-hot encoding, and feature aggregation remind us that the way data is represented impacts the success of any machine learning endeavor. The examples provided, such as those drawn from the Telco Customer Churn dataset, illustrate how thoughtful feature extraction transforms complex datasets into structured, model-ready information. This careful curation of features ensures that models are not just given data, but the *right* data, enhancing their ability to make accurate, relevant predictions.

As you move forward in building machine learning models, remember that the quality of your features often defines the limits of your model's capabilities. Investing time in understanding and implementing robust feature extraction methods pays off, making your models not only more effective but also more interpretable and efficient. In the next chapter, we'll delve deeper into feature selection and engineering to further refine this process and help you build even more powerful models.

Chapter 2
Regression and Linear Models



Abstract This chapter delves into regression analysis, a foundational technique in predictive modeling that helps uncover relationships between variables and generate actionable insights. Beginning with linear regression, we explore how simple models can approximate trends in data, before progressing to polynomial regression for capturing more complex, non-linear patterns. Through practical examples, we illustrate how model choice impacts performance, with a notable improvement in prediction accuracy—reflected in reduced Mean Squared Error—when shifting from linear to higher degree polynomial models. The chapter emphasizes that regression is not merely a statistical exercise but a vital tool across domains such as finance, healthcare, marketing, and agriculture. By understanding and applying regression techniques effectively, data scientists can better model real-world phenomena and support informed, data-driven decisions.

2.1 What Is Regression?

Let's begin by considering a simple linear equation:

$$y = 3 + 0.5 \times x \tag{2.1}$$

This equation defines a relationship between x (the independent variable) and y (the dependent variable). To illustrate how this equation works, we can plug in some values for x and calculate the corresponding y values:

x	$y = 3 + 0.5 \times x$
0	3
2	4
4	5
6	6
8	7

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_2

Now, consider the following question: *What if we are given the x values and the resulting y values and need to find the original equation that produced them?* This is where *regression* comes into play. *Regression* is a statistical method used to estimate the relationships between dependent variables (in this case, y) and independent variables (x) (Lewis-Beck and Lewis-Beck 2015; Ryan 2008; Sykes 1993). In simpler terms, regression helps us determine the underlying equation that best describes the relationship between the variables (Hastie et al. 2009; Headrick 2009; James et al. 2013).

If the data has a linear relationship, regression analysis will yield an equation similar to the original one (e.g., $y = 3 + 0.5 \times x$). However, if the variables do not exhibit a linear relationship, regression will provide the best possible estimate for the relationship between them, allowing for the most accurate prediction or understanding of the data.

2.2 Regression as a Supervised Learning Method

Regression is classified as a *supervised learning* method in machine learning. This means in order to build a regression model, we need access to historical data that includes both *independent variables* (also known as features) and *dependent variables* (also known as target values) (Peckov 2012; Stigler 1971). The independent variables are the inputs used to make predictions, while the dependent variable is the output that we aim to predict.

The essence of supervised learning is that the model learns from this labeled data to understand the relationship between the input features and the target values. This learned relationship is then used to make future predictions (Fahrmeir et al. 2022; Hahs-Vaughn 2024; Montgomery et al. 2021).

Why Do We Use Regression?

The primary purpose of regression is to predict *real values* based on the input features. For example, we may want to use regression to predict: *Temperature for the next day, Stock prices, House prices.*

These are all examples of continuous data, where the output is a real number rather than a discrete label.

How Does Regression Work?

After training a regression model using historical data, we obtain an equation that represents the relationship between the input features and the target variable. This equation allows us to input new feature values and receive a predicted output. For example, if our model learns the equation $y = 3 + 0.5 \times x$ during training, we can use it to predict new y values when provided with new x values.

This predictive capability is what makes regression a powerful tool in machine learning for tasks involving numerical data forecasting and analysis (Fox 2015).

2.3 Components of a Linear Regression Equation

To better understand regression, it's essential to break down the components of a *linear equation*. A linear equation in the context of regression can be expressed in the following form:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \ldots + \beta_n x_n \tag{2.2}$$

where:

y: is the *output* or the *dependent variable*, which is the target value we want to predict.

$x_1, x_2, x_3, \ldots, x_n$: are the *independent variables* or *features*. These are the input values provided to the model.

β_0: is the *intercept* term, representing the value of y when all independent variables are zero.

$\beta_1, \beta_2, \beta_3, \ldots, \beta_n$: are the *coefficients* (often called *betas*). These coefficients determine the weight or contribution of each corresponding feature x_i to the output y.

2.3.1 Role of Coefficients in Regression

The primary function of regression is to find the optimal values for these coefficients ($\beta_1, \beta_2, \beta_3, \ldots, \beta_n$) so that the resulting equation accurately models the relationship between the independent variables and the dependent variable. These coefficients help the model understand how much influence each feature x_i has on the target y. For instance:

A larger value of β_i implies that the feature x_i has a significant impact on the predicted y.

A smaller or zero value of β_i indicates a minimal or no impact from the feature x_i on y.

2.3.2 Constructing the Linear Equation

Given a set of features $x_1, x_2, x_3, \ldots, x_n$ provided as input, the main task of a regression model is to determine the best values for the coefficients $\beta_1, \beta_2, \beta_3, \ldots, \beta_n$. Once these coefficients are found, we can construct the linear equation that best describes the relationship between the dependent and independent variables. This equation can then be used for predicting the target variable y when new feature values are provided. By finding the right coefficients, the regression model effectively learns the nature of the relationship between input features and the output, allowing for accurate predictions.

2.3.3 The Intercept Term β_0 in Regression

As mentioned earlier, β_0 is known as the intercept term. This term is crucial because it allows the linear equation to have a meaningful prediction even when all feature values are zero. Without an intercept, the regression line would be forced to pass through the origin, which often does not reflect the true nature of the data.

2.3.4 Adding the Intercept Term

To include the intercept in the regression model, a new column must be added to the dataset. This column will have all values set to *one* and will be used to represent β_0. This ensures that when calculating the regression equation, the intercept is included alongside the feature coefficients.

Example:

Suppose we have two features:

- Wind Speed (x_1)
- Wind Direction (x_2)

Our goal is to predict Temperature (y) using these features. To include the intercept term in the model, we modify the feature matrix by adding a column of ones:

Feature matrix (with intercept)
1, $x_{1,1}$, $x_{2,1}$
1, $x_{1,2}$, $x_{2,2}$
1, $x_{1,3}$, $x_{2,3}$
…

The column of ones ensures that when we perform regression analysis, the intercept β_0 is accounted for in the resulting equation:

$$y = \beta_0 \times 1 + \beta_1 \times x_1 + \beta_2 \times x_2 \tag{2.3}$$

2.3.5 Using Regression to Find Coefficients

With this modification, we can now use regression algorithms to determine the best values for the coefficients β_0, β_1, and β_2. The model will learn these values based on the input features (including the added intercept column) and the target variable. This step is crucial in ensuring that the regression model captures the true relationship between the features and the dependent variable, providing meaningful predictions even when feature values are at their baseline (e.g., zero).

2.4 The Normal Equation for Solving Linear Regression

Regression is relatively straightforward compared to other machine learning methods. One of the core techniques used to find the coefficients (β) in linear regression is by solving the *normal equation*. This method involves matrix operations that yield a solution for the coefficients directly, without the need for iterative optimization algorithms.

2.4.1 The Normal Equation

Given:

X as the *feature matrix*, which includes all feature columns along with the intercept column (a column of ones).

y as the *target vector*, which contains the dependent variable values.

The normal equation is defined as:

$$\beta = \left(X^{\mathrm{T}} \cdot X\right)^{-1} \cdot \left(X^{\mathrm{T}} \cdot y\right) \tag{2.4}$$

where:

X^{T} is the *transpose* of the matrix X.
$(X^{\mathrm{T}} \cdot X)^{-1}$ is the *inverse* of the product of X^{T} and X.
β is the *vector of coefficients*, representing $\beta_0, \beta_1, \beta_2, \ldots, \beta_n$.

Explanation of the process

Transpose and Multiply: The feature matrix X is transposed and then multiplied by itself to create $X^{\mathrm{T}} \cdot X$.
Inverse Calculation: The inverse of $X^{\mathrm{T}} \cdot X$ is computed.
Final Multiplication: The inverse result is multiplied by $X^{\mathrm{T}} \cdot y$ to yield the β vector.

The outcome of this sequence of matrix operations is a vector β that contains all the coefficients ($\beta_0, \beta_1, \beta_2, \ldots, \beta_n$). These coefficients can then be used to construct the linear equation that best describes the relationship between the features and the target:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \ldots + \beta_n x_n \tag{2.5}$$

2.4.2 Using the Normal Equation for Predictions

Once the coefficients are calculated using the normal equation, we can use the resulting linear equation to predict new values. By inputting new feature values into this equation, the model can provide a predicted output y. The normal equation method is efficient for datasets with relatively small feature counts and sample sizes. For larger datasets, more computationally efficient optimization methods, such as *gradient descent*, are often used.

2.4.3 Applying the Normal Equation: An Example

To illustrate how the normal equation can be used to find the coefficients for a regression model, let's revisit the example dataset:

x_1	y
0	3
2	4
4	5
6	6
8	7

Assume that we don't know the original equation and want to use regression to find the coefficients that construct the linear relationship between x_1 and y.

Step 1: Create the Feature Matrix X

First, we create the feature matrix X by adding an intercept column (a column of ones) to the x_1 values:

$$\begin{bmatrix} 1 & 0 \\ 1 & 2 \\ 1 & 4 \\ 1 & 6 \\ 1 & 8 \end{bmatrix}$$

Step 2: Transpose the Matrix X

Next, we find the transpose of matrix X (X^T):

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 2 & 4 & 6 & 8 \end{bmatrix}$$

Step 3: Multiply X^T by X

Now, we multiply X^T by X to get:

$$X^T.X = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 2 & 4 & 6 & 8 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 1 & 2 \\ 1 & 4 \\ 1 & 6 \\ 1 & 8 \end{bmatrix} = \begin{bmatrix} 5 & 20 \\ 20 & 120 \end{bmatrix}$$

Step 4: Calculate the Inverse of $X^T \cdot X$

To solve for the coefficients, we need to find the inverse of $X^T \cdot X$:

$$X^T \cdot X = \left(\begin{bmatrix} 5 & 20 \\ 20 & 120 \end{bmatrix} \right)^{-1} = \begin{bmatrix} 0.6 & -0.1 \\ -0.1 & 0.025 \end{bmatrix}$$

Step 5: Multiply X^T by y

Next, we calculate $X^T \cdot y$. Using the original y values [3, 4, 5, 6, 7], we get:

$$X^T \cdot y = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 2 & 4 & 6 & 8 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{bmatrix} = \begin{bmatrix} 25 \\ 120 \end{bmatrix}$$

Step 6: Solve for β

Finally, we multiply the inverse of $(X^T \cdot X)$ by $(X^T \cdot y)$ to find β:

$$\beta = \left(X^T \cdot X\right) - 1 \cdot \left(X^T \cdot y\right)$$

$$\beta = \begin{bmatrix} 0.6 & -0.1 \\ -0.1 & 0.025 \end{bmatrix} \cdot \begin{bmatrix} 25 \\ 120 \end{bmatrix} = \begin{bmatrix} 3 \\ 0.5 \end{bmatrix}$$

2.4.4 Constructing the Linear Equation from Coefficients

After solving for the coefficients β_0 and β_1, we can construct the final linear equation that models the relationship between our independent variable x_1 and dependent variable y. The general form of a linear equation in regression is:

$$y = \beta_0 + \beta_1 \times x_1$$

With our calculated values:

$$\beta_0 = 3, \beta_1 = 0.5$$

we can plug these coefficients into the equation:

$$y = 3 + 0.5 \times x_1$$

Let's take a moment to appreciate this achievement—our calculated equation is:

$$y = 3 + 0.5 \times x_1$$

Wait a minute, that looks familiar! This is exactly the original equation we discussed at the beginning of this chapter:

$$y = 3 + 0.5x$$

It's not just close; it's spot on! The regression process has successfully brought us back to the original equation used to generate our data. This confirms that our regression method accurately captured the true relationship between the features and the target variable.

What This Means for Predictions
Now that we have our equation $y = 3 + 0.5 \times x$, we can use it to make predictions for new data points. For any given x value, the model will compute y using the learned coefficients. This illustrates the power of regression: with past data and statistical analysis, we can build equations that allow us to predict future outcomes with confidence.

2.5 The Limitations of Linear Regression

While our example showed a perfect reconstruction of the original equation, it's crucial not to get too excited. Real-world data is rarely as cooperative as our theoretical example where the input x_1 and the output y had a perfectly linear relationship.

2.5.1 When the Relationship Is Not Perfectly Linear

In many real-world scenarios, the relationship between the features and the target variable might not be perfectly linear. It may exhibit only a partial linear trend or might be entirely non-linear. This means that while linear regression can provide a good approximation when the data follows a somewhat linear pattern, it might struggle or fail when the data deviates significantly from linearity. For example, data that has quadratic, exponential, or more complex relationships won't be accurately modeled by a simple linear regression.

Why This Matters: When the underlying relationship between the variables isn't linear, a linear regression model will produce an approximation that may not capture the nuances of the data. This can lead to predictions that are either biased or simply inaccurate.

2.5.2 The Limitations of a "Take It or Leave It" Approach

Another limitation of linear regression is that it operates as a *take it or leave it* approach. Linear regression calculates the coefficients using the normal equation or other direct methods, providing an immediate result without the iterative process found in more complex machine learning algorithms.

What Does This Mean? Unlike models such as neural networks or other iterative machine learning methods, linear regression doesn't have an optimization phase involving *epochs* or *iterations*. Once the coefficients are calculated, that's it—the model is set, and there is no opportunity for further enhancement or fine-tuning. If the initial solution provided by linear regression doesn't meet your needs or lacks accuracy, you cannot adjust or optimize it directly within the framework of linear regression. You would need to explore alternative methods that offer:

- Iterative learning (e.g., gradient descent in more complex algorithms)
- Adjustable parameters to improve performance over multiple iterations
- Non-linear models that can capture more complex relationships in data

2.5.3 Beyond Linear Regression

If linear regression's "fixed solution" doesn't satisfy your requirements, or if the data suggests a non-linear relationship, it may be time to consider more advanced methods. Techniques such as *polynomial regression, decision trees*, or *neural networks* can handle more complex relationships and allow for optimization through iterative learning processes. These methods come with the ability to adapt, learn, and improve over time, offering more flexibility and often better predictive performance.

2.6 Real-World Example: Predicting the Number of Meals in a University Cafeteria

Let's explore a practical application of linear regression using data from a university cafeteria. The cafeteria management wants to predict the number of meals to prepare each day based on several factors. The goal is to avoid over-preparing meals (which leads to waste) and under-preparing meals (which results in students and guests not having enough to eat).

Problem Description
The dataset includes the following features:

Day: The day number (for context, but not used as a predictive feature).
No of Students: The number of students attending that day.
No of Guests: The number of additional guests present.
Temperature: The day's temperature in degrees Fahrenheit.
Wind Speed: The wind speed in mph.

The *target variable* is:

No of Meals: The number of meals to prepare.

By using these features, the cafeteria aims to create a model that predicts the number of meals needed. Table 2.1 shows the data.

Let's write a Python code snippet to load the data, create a linear regression model, and test it:

Table 2.1 Cafeteria data

Day	No of student	No of guests	Temperature	Wind speed	No of meals
1	500	20	70	5	163
2	520	19	75	10	152
3	550	30	90	15	131
4	600	23	61	3	251
5	570	22	70	2	217
6	610	43	77	9	220
7	530	21	90	8	133
8	670	10	90	3	222
9	490	12	77	2	145
10	510	20	65	1	192

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the dataset
data = pd.DataFrame({
    'Day': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'No of Students': [500, 520, 550, 600, 570, 610, 530, 670, 490, 510],
    'No of Guests': [20, 19, 30, 23, 22, 43, 21, 10, 12, 20],
    'Temperature': [70, 75, 90, 61, 70, 77, 90, 90, 77, 65],
    'Wind Speed': [5, 10, 15, 3, 2, 9, 8, 3, 2, 1],
    'No of Meals': [163, 152, 131, 251, 217, 220, 133, 222, 145, 192]
})

# Prepare the feature matrix X (with intercept) and target vector y
X = data[['No of Students', 'No of Guests', 'Temperature', 'Wind Speed']]
X = np.hstack([np.ones((X.shape[0], 1)), X])  # Add intercept column of
ones
y = data['No of Meals'].values.reshape(-1, 1)

# Calculate the normal equation: β = (X^T * X)^(-1) * X^T * y
XT_X = np.dot(X.T, X)
XT_y = np.dot(X.T, y)
beta = np.dot(np.linalg.inv(XT_X), XT_y)

# Print the coefficients (betas)
print("Calculated Coefficients (betas):")
print(beta.flatten())

# Create new test instances
new_instances = np.array([[1, 550, 25, 80, 5],  # Adding intercept of 1
                          [1, 600, 20, 72, 3],  # Adding intercept of 1
                          [1, 580, 15, 68, 7]])  # Adding intercept of 1

# Predict the number of meals for new instances
predictions = np.dot(new_instances, beta)
print("\nPredicted number of meals for new instances:")
print(predictions.flatten())

# Visualize actual vs. predicted values for the number of meals
plt.scatter(y, y_pred, color='blue', label='Predicted vs Actual')
plt.plot([y.min(), y.max()], [y.min(), y.max()], color='red', linestyle='-
-', label=' Regression Line')
plt.xlabel('Actual No of Meals')
plt.ylabel('Predicted No of Meals')
plt.title('Actual vs. Predicted No of Meals')
plt.legend()
plt.show()
```

In this code, we start by loading our dataset, which represents data from a university cafeteria. Each row in the dataset gives us information for 1 day, including the number of students present, the number of guests, the day's temperature, the

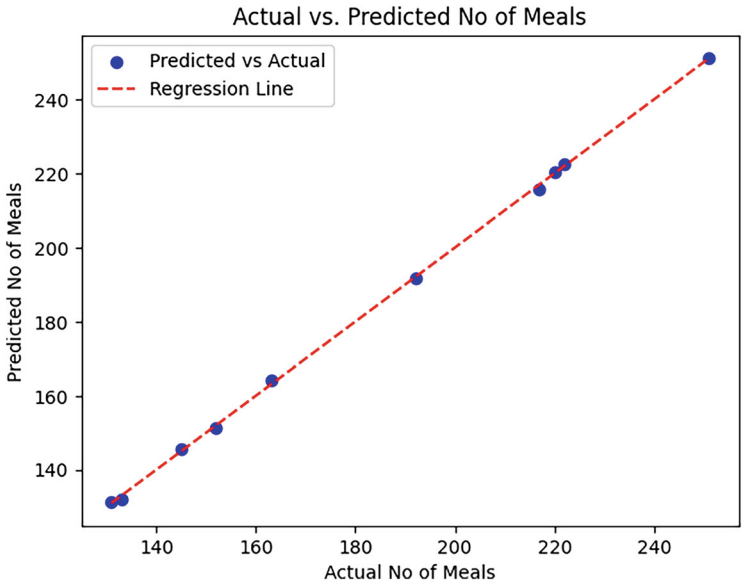


Fig. 2.1 Comparison of actual vs. predicted number of meals for university cafeteria

wind speed, and the number of meals prepared. Our goal is to use these features to build a model that predicts the number of meals needed, so the cafeteria can plan better and minimize waste or shortages.

To prepare the data for linear regression, we first need to set up our feature matrix X and target vector y. The feature matrix X includes columns for the number of students, guests, temperature, and wind speed. We add an intercept column (a column of ones) to X because it represents β_0, the intercept term in our linear regression equation. The target vector y contains the number of meals prepared each day, which we want to predict. Once we have the coefficients, we can use them to make predictions. For example, if we have new days with data about the number of students, guests, temperature, and wind speed, we can plug these values into our equation to predict the number of meals.

Finally, we plot the actual number of meals vs. the predicted number of meals to visualize how well our model performed. In this plot (Fig. 2.1), each blue point represents 1 day from our dataset. The red dashed line is the regression line, where predicted values would fall if the model were 100% accurate. Points that are close to this line indicate accurate predictions, while points far from the line show where the model's predictions deviate from reality.

In Fig. 2.1, you'll see that some points lie right on the red dashed line, meaning the model's predictions for those days were spot on. Other points are slightly above or below the line, indicating that the predictions were close but not exact. This is expected in real-life scenarios, where data rarely fits a model perfectly due to noise and natural variability. This plot helps us evaluate how well the model generalizes to the data. If most points cluster around the perfect fit line, we can be confident that our

model is doing a good job. If points are scattered far from the line, it signals that the model might not be capturing the relationship between the features and the target variable well, and more complex modeling techniques may be needed.

2.7 Model Evaluation: Measuring the Accuracy of Predictions

Imagine going to the cafeteria manager and proudly announcing that you have developed a prediction model capable of accurately determining the number of meals to prepare. The first question that would come up is: "How accurate is it?" Simply saying that the model is "accurate" is not enough; we need systematic ways to evaluate how good or bad our machine learning model truly is.

2.7.1 The Importance of Model Evaluation

Model evaluation is crucial in any machine learning project because it helps quantify the performance of a model in a clear, objective manner. Without evaluation metrics, there is no way to gauge whether a model's predictions are reliable or if it needs further improvement. When we present our model to stakeholders, such as the cafeteria manager, we need to provide tangible evidence of its predictive capabilities.

After training our model, we obtained the following coefficients:

$$\text{Coefficients} : [12.39955951, 0.60264828, 0.68403763, -2.13921063, -2.7014776]$$

These coefficients were applied to the features, producing the following comparison between the actual number of meals and the predicted number of meals (as visualized in Fig. 2.1 and now shown in tabular form in Table 2.2).

Looking at the table, we can see that the predicted values are remarkably close to the actual values. This suggests that the model is performing well, as most predictions are either exact or differ by only one or two meals. Such a small margin of error would likely be acceptable for the cafeteria manager, as it indicates that the model can effectively guide daily meal preparation and minimize waste or shortages. However, even with these promising results, we need to apply formal evaluation metrics to quantify this performance further and describe it more rigorously. Metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), or R-squared can be used to provide a numerical evaluation of the model's accuracy. Here are the formulas for the metrics:

Here are the formulas for the metrics:

Table 2.2 Actual no of meals vs. predicted no of meals

Actual No of meals	Predicted No of meals
163	165
152	152
131	132
251	252
217	216
220	221
133	133
222	223
145	146
192	192

1. Mean Absolute Error (MAE):

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \widehat{y}_i| \tag{2.6}$$

2. Mean Squared Error (MSE):

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2 \tag{2.7}$$

3. R-squared (R^2):

$$MSE = 1 - \frac{\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2} \tag{2.8}$$

where:

- y_i is the actual value.
- $\widehat{y}_i$ is the predicted value.
- $\overline{y}$ is the mean of the actual values.
- n is the total number of observations.

Here's a comprehensive comparison table outlining the differences and similarities between MAE, MSE, and R-squared, as shown in Table 2.3.

To evaluate our model's performance using the MSE, we first calculated the errors (the differences between the actual and predicted number of meals) and then squared each error to avoid negative values canceling each other out. Finally, we averaged these squared errors to find the MSE.

Table 2.3 Comparison between MAE, MSE, and *R*-squared

Aspect	Mean absolute error (MAE)	Mean squared error (MSE)	*R*-squared (R^2)
Definition	The average of the absolute differences between actual and predicted values	The average of the squared differences between actual and predicted values	The proportion of variance in the dependent variable that is predictable from the independent variables
Sensitivity to outliers	Less sensitive to outliers because it uses absolute values	More sensitive to outliers due to squaring the differences, which amplifies larger errors	Not directly affected by outliers but can be influenced if outliers affect the fit of the model
Range	Non-negative; $0 \leq \text{MAE} < \infty$	Non-negative; $0 \leq \text{MSE} < \infty$	$-\infty < R^2 \leq 1$; higher values indicate better fit
Interpretability	Easy to understand; represents average error in the same units as the data	Less interpretable due to squaring, which can exaggerate error. Units are squared compared to the original data	Can be interpreted as a percentage of variance explained by the model
Optimal value	MAE} $= 0$ indicates perfect predictions	MSE} $= 0$ indicates perfect predictions	$R^2 = 1$ indicates a perfect fit; $R^2 = 0$ means the model explains none of the variance
Use case	Good for general error measurement when all errors should contribute equally	Suitable when larger errors need to be penalized more heavily, e.g., to avoid large mistakes	Best for understanding how well the model explains the variability of the data
Advantages	Simple and intuitive; less influenced by large deviations	Highlights larger errors more prominently; useful for focusing on minimizing large errors	Provides a clear indication of how well the model performs compared to a baseline (e.g., mean)
Disadvantages	Treats all errors equally; might not emphasize significant deviations	Can be overly influenced by outliers, which may skew the evaluation	Does not provide error magnitude; a negative R^2 indicates a poor model

Here's how it looks for our model in Table 2.4.
MSE Calculation:

$$\text{MSE} = \frac{1}{10}\sum_{1}^{10}(\text{Error Squared}) = \frac{4+0+1+1+1+1+0+1+1+0}{10}$$

The MSE for our model is **1.0**. While this seems like a low value, we must be cautious when interpreting it, especially given that we only have 10 instances in our dataset. With such a small sample size, it's hard to determine whether this low MSE

Table 2.4 Calculating the error between actual and predicted values for model evaluation

Actual No of meals	Predicted No of meals	Actual No of meals − predicted No of meals (error)	Error squared
163	165	−2	4
152	152	0	0
131	132	−1	1
251	252	−1	1
217	216	1	1
220	221	−1	1
133	133	0	0
222	223	−1	1
145	146	−1	1
192	192	0	0

indicates a genuinely well-performing model or if it's simply due to limited data. To confidently claim that the model performs well, we would need to test it on a larger dataset. If the MSE remains consistently low with more data, we can be more assured that the model generalizes well and makes reliable predictions.

2.8 Polynomial Regression

Sometimes, the features $x_1, x_2, x_3, \ldots, x_n$ do not exhibit a linear relationship with the target variable y. This is where polynomial regression becomes valuable. Polynomial regression is an extension of linear regression where we construct new features based on the existing features by raising them to a certain degree, known as the polynomial degree (Myers et al. 2016; Rawlings et al. 1998).

In polynomial regression, the original features are transformed to include higher degree terms. Once these new features are constructed, we apply the same normal equation used in linear regression, but on the expanded set of features (Cheng and Schneeweiss 1998; Heiberger et al. 2009; Patil and Patil 2021).

Suppose we have a single feature x_1 and a target y. To apply polynomial regression:

1. Choose the Degree: Decide the degree n for the polynomial regression.
2. Construct New Features: Create new features by raising x_1 to powers from 1 up to n. For example, if $n = 3$, the new feature set would be $x_1, {x_1}^2, {x_1}^3$.
3. Apply the Normal Equation: Use the normal equation to find the coefficients:

$$\beta = \left(X^{\mathrm{T}} \cdot X\right)^{-1} \cdot \left(X^{\mathrm{T}} \cdot y\right) \tag{2.9}$$

Here, X now represents the matrix of transformed features (e.g., $[x_1, {x_1}^2, {x_1}^3]$). By applying polynomial regression, we enable the model to capture non-linear patterns in the data that linear regression cannot.

Table 2.5 Farmer's data

Tree age (years)	Fruit yield (pounds)
1	3
2	10
3	51
4	137
5	286
6	517
7	848
8	1294
9	1882
10	2619
11	3533
12	4632
13	5943
14	7475
15	9255
16	11,298
17	13,619
18	16,234
19	19,171
20	22,439

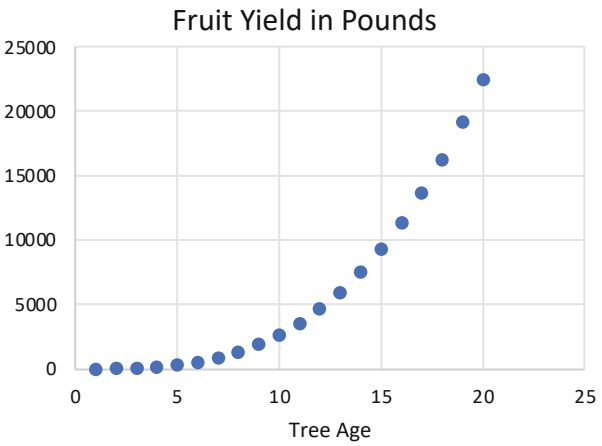


Fig. 2.2 Plotting the farmer's data

Now, let's go through an example of polynomial regression. Imagine a farmer asks you to predict the fruit yield of a tree based on its age. The data you have is as follows in Table 2.5.

When we plot this data (as shown in Fig. 2.2), it is evident that the relationship between tree age and fruit yield is not linear. However, to understand the performance of polynomial regression, we should first apply linear regression and calculate the MSE to see how well it performs with this data. After calculating the MSE

for linear regression, we will proceed to apply polynomial regression and observe how it compares.

After adding the intercept to the tree age feature to form X, we applied linear regression by solving the normal equation and obtained the coefficients: [−5374.04736842, 1089.17593985]. This result implies that the linear regression model suggests the relationship between the tree age and the fruit yield can be expressed as:

$$\text{FruitYield} = -5374.047 + 1089.175 \times \text{Tree } Age$$

When we plugged in the ages of the trees into this equation, we obtained the following predicted fruit yields, as shown in Table 2.6.

When we calculated the Mean Squared Error, it was 7,386,567.33—a significantly large value. This high error is logical because no single straight line can accurately represent the non-linear relationship evident in the plotted data in Fig. 2.2.

Now, let's apply polynomial regression using a degree of 2. To do this, we need to create a new feature matrix. Follow these steps:

1. Add the Intercept: Start by adding a column of ones to represent the intercept.
2. Degree 1 Features: For each feature in the original dataset, raise the values to the power of 1 and add them to the new feature matrix.
3. Degree 2 Features: For each feature in the original dataset, raise the values to the power of 2 and add them to the new feature matrix.

Table 2.6 Predicted fruit yield using linear regression

Tree age (years)	Actual fruit yield (pounds)	Predicted fruit yield (pounds)
1	3	−4285.0
2	10	−3196.0
3	51	−2107.0
4	137	−1017.0
5	286	72.0
6	517	1161.0
7	848	2250.0
8	1294	3339.0
9	1882	4429.0
10	2619	5518.0
11	3533	6607.0
12	4632	7696.0
13	5943	8785.0
14	7475	9874.0
15	9255	10,964.0
16	11,298	12,053.0
17	13,619	13,142.0
18	16,234	14,231.0
19	19,171	15,320.0
20	22,439	16,409.0

Table 2.7 The new features matrix

Intercept	Tree age in years x_1^1	Tree age in years x_1^2
1	1	1
1	2	4
1	3	9
1	4	16
1	5	25
1	6	36
1	7	49
1	8	64
1	9	81
1	10	100
1	11	121
1	12	144
1	13	169
1	14	196
1	15	225
1	16	256
1	17	289
1	18	324
1	19	361
1	20	400

4. Stopping Condition: Since we are using a degree of 2, we stop here. If the degree were higher, we would continue creating new features by raising the original features to higher degrees (e.g., 3, 4).

The new feature matrix would look like this, as shown in Table 2.7.

Next, we use the normal equation (as we did in linear regression) to solve for the coefficients and create the prediction equation:

$$\beta = \left(X^{\mathrm{T}} \cdot X\right)^{-1} \cdot \left(X^{\mathrm{T}} \cdot y\right) \tag{2.10}$$

With the coefficients, we can construct the polynomial regression model that predicts fruit yield based on the tree age.

After solving for the coefficients using the normal equation with a degree of 2, we obtained the following coefficients: [1593.9, −811.17334245, 90.49282297], this means the polynomial regression equation is:

$$\text{FruitYield} = 1593.9 - 811.173 \times x_1 + 90.492 \times {x_1}^2$$

Using this equation, we made predictions for the fruit yield based on the tree age. The predicted values improved over the linear regression model, as shown below in Table 2.8.

The MSE for the polynomial regression was 198,306.13. This is a significant improvement compared to the MSE from the linear regression model

Table 2.8 Predicted fruit yield using polynomial regression of degree 2

Tree age (years)	Actual fruit yield (pounds)	Predicted fruit yield (pounds)
1	3	873.0
2	10	334.0
3	51	−25.0
4	137	−203.0
5	286	−200.0
6	517	−15.0
7	848	350.0
8	1294	896.0
9	1882	1623.0
10	2619	2531.0
11	3533	3621.0
12	4632	4891.0
13	5943	6342.0
14	7475	7974.0
15	9255	9787.0
16	11,298	11,781.0
17	13,619	13,956.0
18	16,234	16,312.0
19	19,171	18,850.0
20	22,439	21,568.0

(7,386,567.33), but it is still quite high. This suggests that while the polynomial regression of degree 2 better fits the data than a simple linear model, it might not yet be capturing the full complexity of the underlying relationship.

Since the MSE for the degree-2 polynomial regression was significantly better than the linear regression, we can experiment with higher degrees to see if we achieve even better results. Let's try using a polynomial regression of degree 3. For this, we need to transform our original feature (tree age) by raising it to higher degrees up to the desired degree. For degree 3, we create new features by raising the values of the tree age feature to the first, second, and third powers, and we also include the intercept.

The resulting feature matrix looks like this, as shown in Table 2.9.

By solving for the coefficients using the normal equation, we can develop the prediction equation for a degree-3 polynomial regression.

After applying the normal equation to this feature matrix, we obtained the following coefficients: [1.09514964, 1.56784884, −3.94224721, 2.99793874], this means our polynomial regression equation is:

$$\text{Fruit Yield} = 1.095 + 1.567 \times x_1 - 3.94 \times {x_1}^2 + 2.997 \times {x_1}^3$$

With this degree-3 polynomial equation, the predictions were incredible, as shown in Table 2.10.

Table 2.9 The new features matrix of degree-3

Intercept	Tree age in years x_1^1	Tree age in years x_1^2	Tree age in years x_1^3
1	1	1	1
1	2	4	8
1	3	9	27
1	4	16	64
1	5	25	125
1	6	36	216
1	7	49	343
1	8	64	512
1	9	81	729
1	10	100	1000
1	11	121	1331
1	12	144	1728
1	13	169	2197
1	14	196	2744
1	15	225	3375
1	16	256	4096
1	17	289	4913
1	18	324	5832
1	19	361	6859
1	20	400	8000

Table 2.10 Predicted fruit yield using polynomial regression of degree 3

Tree age (years)	Actual fruit yield (pounds)	Predicted fruit yield (pounds)
1	3	2.0
2	10	12.0
3	51	51.0
4	137	136.0
5	286	285.0
6	517	516.0
7	848	847.0
8	1294	1296.0
9	1882	1881.0
10	2619	2620.0
11	3533	3532.0
12	4632	4633.0
13	5943	5942.0
14	7475	7477.0
15	9255	9256.0
16	11,298	11,297.0
17	13,619	13,617.0
18	16,234	16,236.0
19	19,171	19,171.0
20	22,439	22,439.0

When we calculated the Mean Squared Error, it was only 1.7517, which is astonishingly low! This result indicates that the polynomial regression of degree 3 perfectly captures the underlying relationship between tree age and fruit yield. The improvement from linear and degree-2 polynomial regressions to this degree-3 model is remarkable, showcasing how powerful polynomial regression can be for modeling non-linear relationships in data.

Let's write a Python code snippet to load the data, create a polynomial regression model, and test it by changing the degree:

```
import numpy as np
import pandas as pd

# Input data
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,
18, 19, 20])
y = np.array([3, 10, 51, 137, 286, 517, 848, 1294, 1882, 2619, 3533, 4632,
5943, 7475, 9255, 11298, 13619, 16234, 19171, 22439])

# Degree of the polynomial
degree = 3  # Change this value to fit higher-degree polynomials

# Constructing the design matrix for polynomial regression (including the
intercept)
X_poly = np.ones((X.shape[0], degree + 1))
for i in range(1, degree + 1):
    X_poly[:, i] = X ** i

# Calculating coefficients using the normal equation: beta = (X^T * X)^-1
* X^T * y
X_poly_T = X_poly.T
beta = np.linalg.inv(X_poly_T @ X_poly) @ (X_poly_T @ y)

# Making predictions
y_pred = X_poly @ beta

# Calculating Mean Squared Error (MSE)
mse = np.mean((y - y_pred) ** 2)

# Creating a table of actual vs predicted values (rounded)
results_df = pd.DataFrame({
    'Tree Age (Years)': X,
    'Actual Fruit Yield (Pounds)': y,
    'Predicted Fruit Yield (Pounds)': np.round(y_pred, 0)
})

# Printing results
print("Coefficients (including intercept):", beta)
print("Mean Squared Error:", mse)
print(results_df)
```

2.9 Regression in Real Life

Regression is not just a mathematical concept confined to classrooms and textbooks; it is a powerful tool with real-life applications that help shape the world around us. Think about your everyday experiences, from predicting the weather to understanding house prices or planning a budget for a project. Regression, especially linear and polynomial, plays a central role in making sense of the data and finding patterns that guide our decisions.

Imagine you are a farmer trying to optimize crop yields based on factors such as the age of your plants, the amount of water and sunlight they receive, or even soil conditions. Linear regression can help you draw a simple, direct relationship between these factors and the yield. However, nature is rarely simple, and that's where polynomial regression comes in, capturing the more complex relationships where yields might increase up to a certain point and then plateau or even decrease if conditions become excessive.

Regression isn't just for farming; it's essential in finance too. Think of a financial advisor trying to predict how an investment portfolio might perform based on current market trends, historical data, and economic indicators. By applying regression analysis, they can create a model that helps investors make informed choices. For linear cases, a straightforward relationship between variables like interest rates and stock returns may be sufficient. But when the stock market follows non-linear trends, polynomial regression becomes crucial to uncover hidden patterns, ensuring better risk management and investment decisions.

Healthcare is another field where regression proves invaluable. Doctors and researchers use it to understand the relationship between various factors—such as age, lifestyle habits, and genetic predispositions—and health outcomes. This helps in diagnosing diseases, planning treatments, and predicting how a patient might respond to different types of medication. For example, a linear model might relate a single factor like age to blood pressure, but in reality, health is often influenced by a mix of variables that interact in complex ways. A polynomial model can provide a better fit, allowing doctors to make more precise and personalized recommendations.

In more everyday terms, think of online shopping and product recommendations. Regression models power the algorithms that suggest products you might like based on your past purchases and browsing history. While a simple relationship might be drawn using linear regression, data scientists often need more intricate models to capture non-linear relationships between your interests and product characteristics, improving the quality of recommendations.

Even social science and psychology benefit from regression analysis. Researchers use it to study how variables like age, income, and education affect people's happiness or how different forms of therapy can influence mental health outcomes. Sometimes, the simplest linear models are not enough to reveal the true story, and more complex polynomial models help paint a clearer picture of human behavior.

Regression, whether linear or non-linear, is a bridge between raw data and actionable insight. It helps us understand patterns, make predictions, and make

smarter choices in our daily lives. The power of these models lies in their ability to break down complicated relationships into something we can use to make better decisions—whether in business, healthcare, agriculture, finance, or even how we navigate our personal lives.

2.10 Concluding Remarks

Regression, whether in its simplest linear form or in the more complex polynomial models, is an essential tool in the data scientist's toolkit. It bridges the gap between raw data and actionable insights, helping us model relationships and make accurate predictions. Throughout this chapter, we have explored the fundamental principles of regression, starting with linear regression and expanding into polynomial regression to capture more complex, non-linear relationships.

The examples we examined showed that while linear regression can sometimes provide a useful approximation, real-world data often requires more sophisticated models to truly represent the underlying patterns. Polynomial regression, with its ability to incorporate higher degree terms, proved how much more powerful it can be for fitting data that follows a curved trend. The significant drop in Mean Squared Error when moving from a linear to a degree-3 polynomial regression emphasized this point, demonstrating the importance of selecting the right model for the task at hand.

Regression analysis is more than just numbers and equations—it plays a pivotal role in making decisions in a variety of fields. From agriculture and finance to healthcare and marketing, understanding and applying regression can help unlock the hidden stories within data, guiding smarter choices and more effective strategies.

Chapter 3
Classification: Logistic Regression to Naïve Bayes



Abstract This chapter explores binary classification through two foundational algorithms: logistic regression and Naive Bayes. We begin by differentiating classification from regression tasks, emphasizing how logistic regression transforms linear predictions into probabilities for classifying data into categories. The chapter outlines the complete logistic regression pipeline, including the use of the sigmoid function, gradient descent optimization, and evaluation metrics like accuracy, precision, recall, and F1 score. We highlight the importance of testing model generalizability and address key challenges such as overfitting and underfitting.

The chapter then shifts focus to the Naive Bayes algorithm, particularly for text classification tasks. We discuss the unique challenges of textual data and introduce preprocessing techniques—such as lowercasing, tokenization, and stop word removal—to prepare data for analysis. Using probabilistic reasoning and Bayes' Theorem, Naive Bayes classifies text based on word frequencies and class priors, with Laplace smoothing and log probabilities ensuring robust computation. A practical example, classifying spam emails, demonstrates the algorithm's effectiveness despite its simplifying assumptions. Together, these models provide a strong foundation for tackling binary classification problems in diverse real-world scenarios.

3.1 Logistic Regression

We have seen how to predict real values using regression, where we estimate numbers like the average number of meals a restaurant serves, the expected fruit yield from a farm, or even the temperature forecast for the next day—all based on historical data. Now, let's dive into the second big part of supervised learning: Classification. In classification, our goal is not to predict a number but rather to categorize data into specific groups or *classes*. Think of it this way: instead of asking "How much will it rain tomorrow?" as you would with regression, we now ask "Will it rain tomorrow or not?" This subtle shift transforms the problem from one of measuring to one of deciding (Hosmer Jr et al. 2013; Kleinbaum et al. 2010).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_3

Let's start with an example close to everyday life. Imagine you're at home, planning a trip to the beach, and you want to decide whether to go or not based on certain factors: Is the sky clear? What's the temperature? Are there any warnings about rough tides? Here, each factor represents a feature that helps you decide between two outcomes: "Yes, I'll go" or "No, I'll stay home." This is binary classification—we're making a choice between two distinct classes: "Go" (class 1) and "Don't Go" (class 0).

Binary classification problems pop up everywhere, from detecting spam emails (Spam vs. Not Spam) to predicting whether a patient might have a certain disease (Has Disease vs. Healthy). It's a powerful tool that helps systems make decisions based on data. But classification isn't always limited to just two options (Agresti 2013; Hilbe 2009). Sometimes, we need to handle situations with more than two possible outcomes. For example, predicting the type of animal you're looking at—whether it's a dog, cat, bird, or lizard—would involve multiclass classification, as you have more than two classes to choose from. The same goes for detecting the type of tumor when there are multiple types to consider. Whether it's binary or multiclass, classification relies on the features we extract from the data, just like regression, to make accurate predictions about new, unseen information (Sperandei 2014; Tripepi et al. 2008).

There are many models for binary classification, ranging from simple to complex. Since we've already explored regression, let's start with a model that builds on that familiar concept: *Logistic Regression*.

Logistic Regression is a binary classification model that takes the principles of regression a step further. It aims to find the best possible values for the coefficients (or betas) in a linear regression equation, but with a key difference—it's focused on making categorical decisions. The goal is to find these coefficients such that the model can predict whether an instance falls into one of two classes (e.g., "Yes" or "No," "Spam" or "Not Spam," "Has Disease," or "Healthy") as accurately as possible (Cabrera 1994; Dayton 1992; Peng et al. 2002).

Now, let's recall what we discussed in the regression chapter: regression is often a straightforward process. It gives you an equation for prediction, and if the results don't meet your expectations, there isn't much you can do to make it better other than finding another model. It's a bit of a "take it or leave it" approach.

Logistic regression, on the other hand, is more flexible because it uses an *iterative approach* to improve its predictions. It doesn't stop at producing an initial set of coefficients—it keeps refining them in search of the lowest possible error. This iterative process is powered by an optimization algorithm called *gradient descent*. Think of gradient descent as a method where the model climbs down a hill, step by step, to reach the lowest point in terms of prediction error. Each step adjusts the betas slightly to reduce the error, striving for better results (Bishop and Nasrabadi 2006; Itoo et al. 2021).

Of course, reaching that perfect minimum error isn't guaranteed. The landscape of possible solutions can be bumpy, and sometimes the model might get stuck at a "good enough" point instead of finding the absolute lowest error (Menard 2001). But even so, the beauty of logistic regression lies in its persistent effort to minimize the

error as much as possible, and in many cases, it succeeds in finding an effective solution. We'll explore exactly how this process works and what it looks like in practice in the following sections.

To truly understand how logistic regression works, let's break down its process step by step. Logistic regression operates through a series of well-defined steps that guide it toward making the most accurate predictions possible. Here are the steps:

1. Form a Linear Equation: The model starts by using the input features and current coefficients (betas) to form a linear equation. This equation looks familiar from our time studying regression:

$$z = \beta_0 + \beta_1 \cdot x_1 + \beta_2 . x_2 + \cdots + \beta_n \cdot x_n \tag{3.1}$$

   The result, z, represents a linear combination of the input features.

   Before we move further, let's make a small change to our terminology for consistency and clarity. From now on, we will refer to the coefficients, previously called betas, as weights. This shift aligns with how these parameters are typically described in machine learning. So, the equation we've been using:

$$z = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \ldots + \beta_n \cdot x_n \tag{3.2}$$

   will now become:

$$z = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + w_n \cdot x_n \tag{3.3}$$

   The equation remains exactly the same, but we will refer to the coefficients as weights going forward. This new terminology is especially common when discussing optimization, gradient descent, and the training process in general. So, when we talk about updating weights or adjusting them based on the gradient of the error, we're simply talking about modifying those coefficients (previously betas) to make the model more accurate.
2. Transform z Using a Hypothesis Function: To convert z into a value that represents the probability of a certain class, logistic regression uses the sigmoid function:

$$\text{Hypothesis} = \frac{1}{1 + e^{-z}} \tag{3.4}$$

   This function squeezes z into a range between 0 and 1, which is perfect for probabilities. If the hypothesis output is 0.5 or higher, we classify the input as class A (e.g., "Yes," "Positive," or "1"). If it's below 0.5, we classify it as class B (e.g., "No," "Negative," or "0").
3. Calculate the Error of the Model: Once we have our predictions, it's time to evaluate how well our model performed. For logistic regression, we use a metric called log-loss (or cross-entropy loss). The log-loss function measures how far

our predictions are from the actual values, penalizing confident but incorrect predictions more heavily than less confident ones. This helps guide the model to become more accurate over time.

4. Calculate the Gradient of the Error: This is where we start using the power of gradient descent. To minimize the error, we need to know the slope or gradient of the error with respect to each weight. The gradient tells us how to adjust each weight to move in the direction that reduces the error.
5. Update the Weights: Using the gradients calculated in the previous step and a small, predefined value called the *learning rate*, we update each beta:

$$w_j = w_j - learning\ rate \times \text{gradient of error} \tag{3.5}$$

   This step helps the model "learn" by adjusting the weights to minimize the error.
6. Repeat Steps 1–5: The process doesn't stop after one round. The model repeats steps 1–5, each time using updated weights to calculate a new z, new predictions, a new error, and new gradients. This cycle continues until a stopping criterion is met, such as:

   - The error falls below a certain threshold.
   - The model completes a predetermined number of iterations (also called epochs).

These steps ensure that logistic regression keeps improving its predictions and minimizes its error over time. In the following sections, we'll dive into the specifics of gradient descent, the role of the learning rate, and how to choose stopping criteria effectively. For now, keep in mind that logistic regression is more than just a straightforward model—it's an adaptive learner that fine-tunes itself with every iteration to become more accurate.

3.1.1 Practical Example

To get a better understanding of how logistic regression works, let's walk through a practical example using a dataset about patients. Our goal is to train a model that can predict whether new patients are likely to have diabetes based on specific features.

Here's a snapshot of our dataset as shown in Table 3.1.

Features:

- Age (years)
- BMI (Body Mass Index)
- Blood Glucose Level (mg/dL)

Target: Class (Diabetes)—where 1 indicates the patient has diabetes, and 0 indicates they do not.

Table 3.1 Snapshot of the diabetes dataset

Patient ID	Age (years)	BMI	Blood glucose level (mg/dL)	Class (diabetes)
1	45	29.5	130	1
2	34	22	95	0
3	50	31.2	145	1
4	28	19.8	85	0
5	60	33.1	160	1
6	41	25	110	0
7	55	30.5	135	1
8	38	27.4	102	0
9	48	28.7	140	1
10	29	23.5	89	0

Table 3.2 Dataset after adding the intercept

Intercept	Age (years)	BMI	Blood glucose level (mg/dL)
1	45	29.5	130
1	34	22	95
1	50	31.2	145
1	28	19.8	85
1	60	33.1	160
1	41	25	110
1	55	30.5	135
1	38	27.4	102
1	48	28.7	140
1	29	23.5	89

Step 1: Prepare the Features Matrix

Before we start training the logistic regression model, we need to set up our features matrix. Just like in linear regression, we need to include an intercept term. The intercept ensures that our model can learn a bias term, which acts as a baseline prediction.

So, our features matrix X becomes as shown in Table 3.2.

Step 2: Initialize the Weights

Next, we need to initialize the weights that correspond to each feature. For simplicity, we'll start with all weights set to zero:

w_0 (intercept weight)	w_1 (weight for age)	w_2 (weight for BMI)	w_3 (weight for blood GL)
0	0	0	0

These initial weights will be updated during the training process as the model iterates to minimize the error and find the optimal values. Each iteration, the weights are adjusted based on the gradient of the error to improve the prediction accuracy.

Step 3: Calculate the Predictions Based on the Current Weights

Now that we have initialized our weights, it's time to calculate the predictions for each instance in the dataset using the current weights. This step will involve computing the value of z and then applying the sigmoid function to get the probability of the instance belonging to class 1 (e.g., having diabetes).

Let's walk through one prediction step by step, using Patient 1 as an example.

Calculate z:

$$z = w_0 \cdot \text{intercept} + w_1 \cdot Age + w_2 \cdot BMI + w_3 \cdot \text{Blood Glucose Level}$$

Plug in the initial weights (all zeros) and the features of Patient 1:

$$z = 0 \cdot 1 + 0 \cdot 45 + 0 \cdot 29.5 + 0 \cdot 130 = 0$$

Apply the Sigmoid Function:

Next, we plug z into the sigmoid function to transform it into a probability:

$$\text{Prediction} = \frac{1}{1 + e^{-z}}$$

Plugging ($z = 0$) into the sigmoid function gives:

$$\text{Prediction} = \frac{1}{1 + e^{0}} = \frac{1}{1 + 1} = 0.5$$

The model predicts a probability of 0.5, which indicates a 50–50 chance of Patient 1 having diabetes. This is essentially no better than random guessing, which isn't useful for decision-making. The reason this prediction is so uninformative is that our weights are still at their initial values of zero. The model hasn't been trained yet, so it isn't capable of making accurate predictions.

This step highlights why the training process is essential. We need to continue training so that the model can learn and adjust the weights to make better predictions. The next part of the training involves calculating the error of these initial predictions and using gradient descent to adjust the weights in a way that improves the model's accuracy over time.

Now that we understand how to compute the prediction for one instance, we need to repeat Step 3 for all instances in our dataset. This allows us to see the predictions for each patient. Using the initial weights, which are still set to zero, the computed z value for each patient remains 0, leading to the same prediction of 0.5 across the board when plugged into the sigmoid function.

Here is how the predictions look for each patient, as shown in Table 3.3.

At this stage, our predictions are not very helpful. The model's output of 0.5 for every patient means it's giving each patient an equal chance of having or not having diabetes—essentially flipping a coin. This outcome is expected at the start of training when the weights are all initialized to zero.

Table 3.3 The first predictions for each patient

Intercept	Age (years)	BMI	Blood glucose level (mg/dL)	Class (diabetes)	Prediction (p)
1	45	29.5	130	1	0.5
1	34	22	95	0	0.5
1	50	31.2	145	1	0.5
1	28	19.8	85	0	0.5
1	60	33.1	160	1	0.5
1	41	25	110	0	0.5
1	55	30.5	135	1	0.5
1	38	27.4	102	0	0.5
1	48	28.7	140	1	0.5
1	29	23.5	89	0	0.5

Step 4: Calculate the Error of the Model as a Whole

Once we've made predictions for all instances in the dataset, it's time to measure the error of the model based on these predictions compared to the actual values. This error gives us a sense of how well the model is performing. If the error is high at first and then reduces over multiple iterations, it means the model is learning and improving. If, however, the error stops decreasing or becomes very small, we might choose to stop training and use the model in real-life applications.

On the other hand, if the model continues to have high error even after many iterations, it might indicate that the model is too simple for the task. In this case, we might need to try a more complex model or apply techniques to improve the current model.

To measure the error, logistic regression uses a metric called log-loss (also known as cross-entropy loss). This metric evaluates how far each prediction is from the actual value, penalizing confident but incorrect predictions more than less confident ones.

The log-loss error for the entire model is calculated as follows:

$$\text{Error} = -\frac{1}{n}\sum y_i \; ln(p_i) + (1 - y_i)\ln(1 - p_i) \tag{3.6}$$

where:

- n is the total number of instances in the dataset.
- y_i is the actual class of the instance (1 for having diabetes, 0 for not having diabetes).
- p_i is the predicted probability for the instance.

For each prediction, we calculate the error by taking the natural log of the prediction probability and multiplying it by the actual classification, y_i. Then, we add the result of $(1 - y_i)$ multiplied by the natural log of $(1 - p_i)$. This

Table 3.4 The loss of each prediction

Patient ID	Actual class (y)	Prediction (p)	Loss
1	1	0.5	0.693
2	0	0.5	0.693
3	1	0.5	0.693
4	0	0.5	0.693
5	1	0.5	0.693
6	0	0.5	0.693
7	1	0.5	0.693
8	0	0.5	0.693
9	1	0.5	0.693
10	0	0.5	0.693

calculation is repeated for each prediction, and finally, we take the average of all these values to get the error of the model for the current iteration.

This log-loss error serves as an indicator of how well our model is performing: a lower error means better performance. In the training process, our goal is to reduce this error by adjusting the model's weights through gradient descent (covered in the next step). Each time we recalculate the error and see it drop, we know the model is moving in the right direction.

To better understand how the log-loss calculation works, let's compute the loss for Patient 1 based on the prediction and the actual value.

For Patient 1:

- The actual class $y = 1$
- The prediction $p = 0.5$

The log-loss formula for this instance is:

$$\text{Loss} = -(y \cdot \ln(p) + (1 - y) \cdot \ln(1 - p))$$

Substituting the values for Patient 1:

$$\text{Loss} = -(1 \cdot \ln(0.5) + (1 - 1) \cdot \ln(1 - 0.5))$$

$$\text{Loss} = -(\ln(0.5) + 0 \cdot \ln(0.5))$$

$$\text{Loss} = -\ln(0.5)$$

Calculating ln(0.5) gives approximately (−0.693), so the loss for Patient 1 is:

$$\text{Loss} \approx 0.693$$

We repeat this calculation for each patient in the dataset, applying the log-loss formula to each prediction and actual class. This will give us a loss value for each instance, as shown in Table 3.4.

To calculate the total error for the model, we take the average of all these loss values:

$$\text{Total Error} = \frac{1}{n} \sum \text{Loss}$$

Since each loss value is 0.693 in this case, and we have 10 instances, the total error becomes:

$$\text{Total Error} = \frac{1}{10} \times 0.693 \times 10 = 0.693$$

This total error represents how well (or poorly) our model is performing with the initial weights. Since the error is relatively high, we know there's plenty of room for improvement, and that's where gradient descent comes in. The goal is to reduce this error over time by adjusting the weights to make better predictions.

Step 5: Calculating the New Weights Using Gradient Descent

Now it is time to make the model learn from its mistakes, which is the essence of the supervised machine learning process. The learning process in logistic regression is an optimization problem—it means the system will try to minimize the error to enhance its performance. One of the most common optimization methods used in machine learning is gradient descent. The gradient descent algorithm will always try to find the best values of the weights individually to reduce the error of the system collectively.

Before we see how this works mathematically, let's look at a simple real-life example that's similar to how gradient descent works.

Imagine you have a table with four legs, but this table is wobbly. It's not stable, and it's not level—so there's an "error" in the level of the table. Your goal is to make the table stable. You'll do this by looking at which legs are moving and figuring out the gap between each leg and the floor. Once you identify the gap, you can try to put a piece of paper under the leg to compensate for the gap and push the table upward.

Here's the key: since the leg is going down toward the floor, the paper must go in the opposite direction of the error—it needs to go upward to push the leg up. And you have to be careful with the thickness of the paper. The thickness should match the gap, not be more than it, because if the paper is too thick, you'll create another error, with the table tilting in the opposite direction.

Now let's say you used a paper that was thicker than needed, and the table is still not level. You'll need to redo the process by finding a better thickness for the paper, this time reducing it. Since the table tilted up this time, you'll need to go in the opposite direction again, using a thinner piece of paper. You keep repeating this process, adjusting the paper back and forth in small steps, until the table reaches a level that you find acceptable.

This is similar to how gradient descent works. The model "sees" the error in its predictions and adjusts each weight slightly in the opposite direction of the error to bring it closer to the target. This back-and-forth adjustment process helps the model find the optimal values for the weights, reducing the error over time and improving the predictions.

Now, imagine a more complex setup with the table. This time, each of the four legs is a different length, and the floor itself isn't even. Yet, we still want the table to be level. In this scenario, the four legs and the uneven floor are all contributing to the wobble (or the error). To fix it, each leg will need to compensate individually for the specific gap it has with the floor to reduce the wobble collectively.

This is exactly what we deal with in supervised machine learning. Instead of legs, we have multiple weights associated with multiple features, each contributing to the final decision. In machine learning, some features may play a bigger role in decision-making than others. But each feature is still responsible for a part of the outcome.

To get the best performance, we need to find the optimal adjustment for each weight, considering the importance of the feature it's connected to. Each weight will be fine-tuned individually, based on how much that feature impacts the decision, with the ultimate goal of reducing the error collectively. This individual adjustment process is the core of gradient descent. It fine-tunes each weight, step by step, to bring the model closer to making accurate predictions.

Alright, so we have a multivariable system, and our goal is to find the rate of change for each variable individually in order to solve the problem in that system. This means we want to see how each individual weight (or feature) impacts the overall error, so we can adjust it to improve the model.

Time to open our math toolbox and see what tool will serve us best here. And the answer is: *partial differential equations*!

Partial derivatives allow us to focus on how each variable (or weight) individually affects the error while treating all other variables as constants. This is crucial in gradient descent because we don't want to adjust all weights randomly; instead, we want to understand the unique impact of each weight on the error and adjust each one based on its individual contribution.

By calculating the partial derivative of the error with respect to each weight, we can determine how to adjust that weight to reduce the error. In other words, we're finding the *slope of the error function* in the direction of each weight. This slope tells us the direction and magnitude of the change needed for each weight to move the model closer to its optimal solution.

The equation we use to calculate the *gradients* for the weights in logistic regression is as follows:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i) \cdot x_{ij} \tag{3.7}$$

where:

- J is the error (or cost) function.
- w_j is the weight associated with feature j.
- p_i is the predicted probability for instance i.
- y_i is the actual class label for instance i.
- x_{ij} is the value of feature j for instance i.
- n is the total number of instances.

Here's the intuition behind this equation:

We go to each feature x that corresponds to each weight in every instance of the data and tell it: "Okay pal, you were part of this error ($p - y$), and your contribution to this error is ($p - y$) x of your value in this instance."

The larger the value of this feature, the larger its contribution to adjusting for that error. In other words, if a feature has a high value in a particular instance, it has a bigger impact on that instance's error and thus on the adjustment we need to make for its corresponding weight.

We repeat this for all features and for all data points (or instances). Once we have calculated these contributions for every instance, we take the average of these gradients for each feature across all instances. This average becomes the gradient for the corresponding weight. By using this averaged gradient, we ensure that our weight adjustments are based on the entire dataset, not just one instance, making the learning process smoother and more stable.

Let's calculate the gradients for each weight for Patient 1 given the initial weights $w_0 = 0$, $w_1 = 0$, $w_2 = 0$, $w_3 = 0$.

Step 1: Calculate the Prediction Error

For Patient 1, we already computed the prediction p as 0.5. The actual class y is 1.

The error, which is ($p - y$), is:

$$\text{Error} = p - y = 0.5 - 1 = -0.5$$

Step 2: Calculate the Gradient for Each Weight

Now, we'll use the gradient equation:

$$\frac{\partial J}{\partial w_j} = (p - y) \cdot x_j$$

Gradient for w_0 (Intercept)

$$\frac{\partial J}{\partial w_0} = (p - y) \cdot \text{Intercept} = -0.5 \times 1 = -0.5$$

Gradient for w_1 (Age)

$$\frac{\partial J}{\partial w_1} = (p - y) \cdot Age = -0.5 \times 45 = -22.5$$

Gradient for w_2 (BMI)

$$\frac{\partial J}{\partial w_2} = (p - y) \cdot BMI = -0.5 \times 29.5 = -14.75$$

Gradient for w_3 (Blood Glucose Level)

$$\frac{\partial J}{\partial w_3} = (p - y) \cdot \text{Blood Glucose Level} = -0.5 \times 130 = -65$$

Summary of Gradients for Patient 1

- Gradient for w_0: -0.5
- Gradient for w_1: -22.5
- Gradient for w_2: -14.75
- Gradient for w_3: -65

These values represent the direction and magnitude we should adjust each weight to reduce the error for this instance. In practice, we would repeat this process for all patients, average the gradients for each weight, and use that average to update the weights, as shown in Table 3.5.

After calculating the average gradient for each weight across all instances, we have the following values:

- Average gradient for w_0: 0
- Average gradient for w_1: -4.4
- Average gradient for w_2: -1.765
- Average gradient for w_3: -11.45

The question now is, *how much of this gradient do we want to use to adjust the previous weights?*

We have flexibility here—we could apply anywhere from 1% of the gradient (or maybe less) up to 100%. Choosing between a high percentage and a low percentage each has its advantages and disadvantages.

- High Percentage of the Gradient: The main advantage of using a higher percentage is that the system will reduce the error faster, meaning it learns more quickly. However, the downside is that it might overshoot the minimum error. If we take too big a step each time, the system might get stuck in an infinite loop, bouncing back and forth without ever reaching the lowest error.
- Low Percentage of the Gradient: On the other hand, using a smaller percentage (like 1%, for example) might make the model more precise, helping it reach the lowest error possible. However, the trade-off is that the learning process will be slower since each weight adjustment will be very small.

Table 3.5 The gradients of each weight

Intercept	Age (years)	BMI	Blood glucose level (mg/dL)	Actual class (diabetes)	Prediction (p)		Gradient for w_0	Gradient for w_1	Gradient for w_2	Gradient for w_3
1	45	29.5	130	1	0.5		−0.5	−22.5	−14.75	−65
1	34	22	95	0	0.5		0.5	17	11	47.5
1	50	31.2	145	1	0.5		−0.5	−25	−15.6	−72.5
1	28	19.8	85	0	0.5		0.5	14	9.9	42.5
1	60	33.1	160	1	0.5		−0.5	−30	−16.55	−80
1	41	25	110	0	0.5		0.5	20.5	12.5	55
1	55	30.5	135	1	0.5		−0.5	−27.5	−15.25	−67.5
1	38	27.4	102	0	0.5		0.5	19	13.7	51
1	48	28.7	140	1	0.5		−0.5	−24	−14.35	−70
1	29	23.5	89	0	0.5		0.5	14.5	11.75	44.5
						Average	0	−4.4	−1.765	−11.45

In machine learning, we refer to this percentage as the learning rate. It controls how much of the gradient we apply to adjust each weight. The learning rate is typically set to a small value, like 1% or 2%, to balance speed and stability.

To apply this concept, we calculate the new weights using the following formula:

$$New \text{ Weight} = Old \text{ Weight} - \alpha \cdot \text{Gradient} \tag{3.8}$$

where α is the learning rate.

This formula means that we take a percentage of the gradient and adjust the old weight by that amount in the opposite direction of the gradient (hence the minus sign). By doing this, we make a controlled adjustment toward reducing the error, with each step getting us closer to the optimal weights.

Let's go ahead and calculate the *new weights* based on an example learning rate.

Given that the old weights are:

w_0 (intercept weight)	w_1 (weight for age)	w_2 (weight for BMI)	w_3 (weight for blood GL)
0	0	0	0

and the average gradients are:

- Gradient for w_0: 0
- Gradient for w_1: −4.4
- Gradient for w_2: −1.765
- Gradient for w_3: −11.45

and with a learning rate α of 0.01 (or 1%), we can calculate the new weights to be used in the next iteration as follows:

$$New \text{ Weight} = Old \text{ Weight} – \alpha \times \text{Gradient}$$

Plugging in the values for each weight:

- For w_0:

$$\mathrm{w}_0 = 0 - (0.01 \times 0) = 0$$

- For w_1:

$$w_1 = 0 - (0.01 \times -4.4) = 0 + 0.044 = 0.044$$

- For w_2:

$$w_2 = 0 - (0.01 \times -1.765) = 0 + 0.01765 = 0.01765$$

- For w_3:

$$w_3 = 0 - (0.01 \times -11.45) = 0 + 0.1145 = 0.1145$$

So, the new weights that will be used in the next iteration are:

w_0 (intercept weight)	w_1 (weight for age)	w_2 (weight for BMI)	w_3 (weight for blood GL)
0	0.044	0.01765	0.1145

These new weights are now slightly adjusted based on the average gradients and the learning rate. With these weights, the model will go through another iteration, recalculating the predictions, the error, the gradients, and then updating the weights again. This process will continue, with each iteration hopefully bringing the model closer to finding the optimal weights that minimize the error.

The code below shows how to implement logistic regression and apply it to our patients' data.

```
import numpy as np

# Patient Data (Age, BMI,Blood Glucose Level , Actual Class)
data = np.array([
    [45, 29.5, 130, 1],
    [34, 22.0, 95, 0],
    [50, 31.2, 145, 1],
    [28, 19.8, 85, 0],
    [60, 33.1, 160, 1],
    [41, 25.0, 110, 0],
    [55, 30.5, 135, 1],
    [38, 27.4, 102, 0],
    [48, 28.7, 140, 1],
    [29, 23.5, 89, 0]
])

# Features and Labels
X = data[:, :-1]  # Features (Age, BMI, Blood Glucose Level)
y = data[:, -1]   # Labels (0 or 1 for diabetes)

# Add intercept column of 1s
intercept = np.ones((X.shape[0], 1))
X = np.hstack((intercept, X))

# Parameters
learning_rate = 0.01  # Reduced learning rate
num_iterations = 10000  # Lower number of iterations

# Initialize weights (including intercept)
weights = np.zeros(X.shape[1])
loss = 0
# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Training the model
for i in range(num_iterations):
    # Linear model
```

```
        z = np.dot(X, weights)
        # Sigmoid activation
        predictions = sigmoid(z)

        # Compute gradients
        dw = np.dot(X.T, (predictions - y)) / y.size

        # Update weights
        weights -= learning_rate * dw

        # Optional: print loss every 1000 iterations
        if i % 1000 == 0:
            # Loss calculation with epsilon to avoid log(0)
            epsilon = 1e-10
            loss = -np.mean(y * np.log(predictions + epsilon) + (1 - y) *
np.log(1 - predictions + epsilon))
            print(f"Iteration {i}: Loss = {loss}")
    print(loss)
    # Final weights including intercept
    print("Final Weights (including intercept):", weights)

    # Final predictions (0 or 1)
    final_probabilities = sigmoid(np.dot(X, weights))
    final_predictions = (final_probabilities >= 0.5).astype(int)

    print("Final Predictions (0 or 1):", final_predictions)
```

After running 10,000 iterations, we observed the following:

- Initial Error: At the start, the error was quite high, but it gradually decreased over each iteration.
- Final Error: By the last iteration, the error had reduced to 2.55.
- Final Weights: The weights after training were: [−3.73052864, 2.06840141, −13.97416502, 2.33918303].

With these final weights, the model made its predictions for our dataset as shown in Table 3.6.

Table 3.6 Final predictions

Patient ID	Actual class (y)	Prediction (p)
1	1	0
2	0	0
3	1	1
4	0	0
5	1	1
6	0	0
7	1	0
8	0	0
9	1	1
10	0	0

3.1.2 Evaluation of Classification Models

When we compare these predictions to the actual values, we can see that some of them are correct while others are not. At this point, we only know that the error is 2.55, and based on our rule of "the lower, the better," we can say the model has improved. However, *this error value alone doesn't tell us much about how well the model is performing* on a practical level. This is why we need additional metrics specifically designed to evaluate *classification models*. These metrics will give us a clearer picture of how well the model is doing and help us interpret the accuracy and reliability of its predictions (Mitchell and Mitchell 1997; Murphy 2012).

To better understand the performance of our model, we'll introduce *classification metrics* that depend on the model's correct and incorrect predictions for each class (Dalianis and Dalianis 2018; Hossin and Sulaiman 2015; Naidu et al. 2023). These metrics allow us to see not just the overall error but also specific types of successes and mistakes, helping us interpret how well the model is doing.

When evaluating each class, we consider four main counts for each data point in relation to the class we're examining:

1. **True Positives (TP)**: This is when the model correctly predicts that an instance belongs to the class we're focusing on. For example, if we're evaluating Class 1 (patients who have diabetes), a True Positive would occur when the model predicts that a patient has diabetes (positive prediction), and in reality, that patient does indeed have diabetes (true prediction). This count represents cases where the model says "Yes, this data point belongs to this class," and the actual class agrees—hence, True Positive.
2. **True Negatives (TN)**: This is when the model correctly predicts that an instance does not belong to the class we're evaluating. Continuing with Class 1 (diabetes patients), a True Negative would be counted when the model predicts that a patient does not have diabetes (negative prediction), and in reality, the patient does not have diabetes (true prediction). Here, the model says "No, this data point doesn't belong to this class," and the actual class agrees—thus, True Negative.

Table 3.7 Count types, explanation, and implications

Metric	Condition	Implication for other classes
TP	Correctly classified as belonging to its true class	Becomes TN for all other classes
TN	Correctly classified as not belonging to the class in question	Could be TP, FP, or FN in other classes
FP	Incorrectly classified as belonging to a class it does not belong to	Becomes FN for the true class
FN	Incorrectly classified as a different class instead of its true class	Becomes FP for the incorrectly predicted class

3. **False Positives (FP)**: This occurs when the model incorrectly predicts that an instance belongs to the class we're looking at. For example, if the model predicts that a patient has diabetes (positive prediction for Class 1), but in reality, the patient does not have diabetes; this would be a False Positive. The model's prediction is "positive," but it's wrong—hence, False Positive.
4. **False Negatives (FN)**: This happens when the model incorrectly predicts that an instance does not belong to the class we're evaluating, when it actually does. For Class 1, a False Negative would be when the model predicts that a patient does not have diabetes (negative prediction), but in reality, the patient does have diabetes. The model's prediction is "negative," but it's incorrect—thus, False Negative.

Example Walkthrough
In our patients' data example:

- If we're examining Class 1 (diabetes patients):
  - A TP occurs if the model predicts 1 and the actual class is 1.
  - A TN occurs if the model predicts anything but 1 (like 0), and the actual class is also not 1.
  - An FP occurs if the model predicts 1 (indicating diabetes) but the actual class is 0.
  - An FN occurs if the model predicts 0 (indicating no diabetes), but the actual class is 1.

We'll perform this evaluation for each class separately, counting TP, TN, FP, and FN for each one. This means that the same instance may be counted differently across classes. For example, a True Positive in one class (say, Class A) might be counted as a True Negative in another class (say, Class B).

We can summarize the implications of each count (True Positive, True Negative, False Positive, False Negative) across classes using Table 3.7.

Based on the predictions, let's count the True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) for Class 1 (patients with diabetes) and Class 0 (patients with no diabetes). As shown in Table 3.8.

Table 3.8 Counting predictions with respect to each class

Patient ID	Actual class (y)	Prediction (p)	Count type for Class 1	Count type for Class 0
1	1	0	FN	FP
2	0	0	TN	TP
3	1	1	TP	TN
4	0	0	TN	TP
5	1	1	TP	TN
6	0	0	TN	TP
7	1	0	FN	FP
8	0	0	TN	TP
9	1	1	TP	TN
10	0	0	TN	TP

Summary of Counts for Class 1

- True Positives (TP): 3
  - Patients 3, 5, and 9 were correctly predicted as having diabetes.
- True Negatives (TN): 5
  - Patients 2, 4, 6, 8, and 10 were correctly predicted as not having diabetes.
- False Positives (FP): 0
  - There are no cases where the model predicted diabetes (1) but the actual class was non-diabetes (0).
- False Negatives (FN): 2
  - Patients 1 and 7 were predicted as not having diabetes (0) when they actually did (1).

So, the counts for Class 1 are:

- TP = 3
- TN = 5
- FP = 0
- FN = 2

Summary of Counts for Class 0

- True Positives (TP): 5
  - Patients 2, 4, 6, 8, and 10 were correctly predicted as not having diabetes (0).
- True Negatives (TN): 3
  - Patients 3, 5, and 9 were correctly predicted as having diabetes (1), so they are not mistakenly labeled as Class 0.
- False Positives (FP): 2

  - Patients 1 and 7 were predicted as not having diabetes (0) when they actually have diabetes (1).
- False Negatives (FN): 0
  - There are no cases where the model predicted a patient as having diabetes (1) when they actually did not.

So, the final counts for Class 0 are:

- TP = 5
- TN = 3
- FP = 2
- FN = 0

So, the totals are: TP = 8, TN = 8, FP = 2, FN = 2

Based on TP, TN, FP, and FN, we can calculate four key metrics that provide meaningful insights into how well (or poorly) the model is performing.

1. Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.9}$$

   Accuracy gives a general indication of how often the model's predictions are correct, expressed as a percentage between 0% and 100%. The higher the accuracy, the better. However, if the dataset is imbalanced (e.g., many instances in Class A but only a few in Class B), we might see high accuracy even if the model performs poorly on the underrepresented class. This limitation makes accuracy alone unreliable for imbalanced datasets.
2. Precision:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3.10}$$

   Precision tells us how well the model is performing in terms of false positives. A high precision means that the model produces few false positives, which is crucial in situations where false alarms are costly or harmful.

   For example, in diagnosing diabetes, if a doctor falsely diagnoses a patient with diabetes when they don't have it (a false positive), the consequences can be severe. The patient may have to follow a strict diet, undergo unnecessary treatments, or change their lifestyle—all of which can impact their well-being and daily life. In such cases, precision is essential to minimize false positives and reduce the impact of false alarms.
3. Recall:

Table 3.9 Final model performance

	Accuracy	Precision	Recall	F1 Score
Class 1	0.8	1.0	0.6	0.75
Class 0	0.8	0.71	1.0	0.83
Overall	0.8	0.8	0.8	0.8

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3.11}$$

Recall focuses on false negatives—cases where the model fails to identify instances that belong to the positive class. High recall means that the model produces few false negatives, which is critical in situations where missing a positive instance (FN) could have serious consequences.

For example, consider a situation with COVID-19 diagnoses. If a doctor incorrectly diagnoses a COVID-positive patient as negative (a false negative), the patient may continue to interact with others, potentially spreading the virus, which could be dangerous for vulnerable individuals. In cases like these, recall is vital to ensure that positive cases aren't missed.

4. F1 Score:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3.12}$$

The F1 score is the harmonic mean of precision and recall. A high F1 score indicates a balance where both precision and recall are high, making it a good measure when both false positives and false negatives have serious implications. The F1 score provides a balanced view of the model's performance, especially useful when there's an uneven class distribution or when both precision and recall are crucial.

Based on the counts of TP, TN, FP, and FN for Class 1 and Class 0, we have derived the following metrics, as shown in Table 3.9.

From these results, it's clear that the system is performing better in Class 0 than in Class 1. The model has perfect recall for Class 0, meaning it doesn't miss any instances of patients without diabetes, but it sacrifices precision with some false positives. On the other hand, for Class 1, while precision is high (no false positives), the recall is relatively lower, indicating that the model sometimes fails to identify patients who actually have diabetes.

This discrepancy in performance between the two classes highlights the need for an overall evaluation across all classes to get a comprehensive understanding of the model's performance. Simply focusing on accuracy or precision for one class can be misleading, especially when the classes are imbalanced or when false negatives and false positives carry different consequences.

The results we discussed earlier were obtained after training the model for 10,000 epochs (iterations). But what if we increased the number of epochs to 100,000 or even 1,000,000? At that point, the model would likely reach perfect scores in terms

of accuracy, precision, recall, and F1 score, each hitting 100%. So, does this mean our model is perfect and ready to be used in real life?

The answer is *no*.

To achieve meaningful results that truly reflect a model's performance in real-world scenarios, we must train the model on a much larger dataset than just the ten instances we used in our example. To develop a model that can generalize well, we need tens or even hundreds of thousands of data points. The reason for this is that a model's ability to generalize depends on the variety and quantity of data it has seen during training. If we only feed the model a small dataset, it may essentially just memorize those instances. As a result, when faced with new, unseen data, it may fail to classify them correctly.

3.1.2.1 The Importance of Testing on Unseen Data

Another crucial factor in evaluating a model's performance is to measure its metrics on data that the model has not seen during training. When the model is training, it adjusts its weights based on the data it's exposed to. Thus, achieving 100% accuracy on the training data is not particularly impressive because the model is essentially being tested on the exact same examples it was trained on. It's like a teacher giving students an example in class and then including that same example on the test—solving it correctly isn't particularly impressive since the students have seen it before.

The real challenge—and the true test of a model's capabilities—is to see how well it performs on new, unseen data. To do this, we split our dataset into two parts:

1. Training Dataset: The portion of the data used to train the model (usually around 80% of the total data).
2. Testing Dataset: The remaining portion (around 20% of the data) that the model has not seen during training.

By training the model on 80% of the data and then testing it on the unseen 20%, we can get a clearer picture of how well the model generalizes to new data. If the model performs well on this testing dataset, then we can be more confident that it can handle real-life data that it hasn't encountered before.

In summary:

- Training data is used to teach the model and adjust its weights.
- Testing data is used to evaluate how well the model generalizes to new, unseen data.

If the model achieves high metrics on the testing dataset, only then can we trust that it's capable of making accurate predictions in real-world applications.

After splitting the data into training and testing datasets, we can calculate the metrics for both sets to gain a deeper understanding of how well the model is learning.

3.1.2.2 Analyzing the Metrics on Training and Testing Data

Evaluating the model on both the training dataset and the testing dataset can reveal a lot about the learning process:

1. High Training and High Testing Metrics:
   - If the model achieves high metrics (like accuracy, precision, recall, and F1 score) on both the training and testing datasets, it means that the model is able to generalize well to unseen data. This is the ideal scenario since it indicates that the model has learned the patterns in the data without simply memorizing it.
2. High Training Metrics but Low Testing Metrics (Overfitting):
   - If the model performs very well on the training data but poorly on the testing data, it suggests that the model is memorizing the training data rather than learning general patterns. This issue is known as overfitting.
   - In the case of overfitting, the model becomes too specialized to the training data, which means it fails to generalize to new, unseen data. To address overfitting, we may need to:

     Use regularization techniques to reduce model complexity.
     Collect more data to improve generalization.
     Apply dropout or other techniques to prevent the model from becoming too dependent on specific features.

3. Low Training Metrics but High Testing Metrics:
   - If the model performs better on the testing data than on the training data, this is unusual and often indicates that the model is guessing rather than actually learning. In such cases, it's likely that the model is not making reliable predictions, and the results are due to random chance rather than true learning. This may indicate a problem with the data split, model initialization, or even data leakage.
4. Low Training and Low Testing Metrics (Underfitting):
   - If both the training and testing metrics are poor, it means that the model is underfitting. This indicates that the model is too simple to capture the underlying patterns in the data. In such cases, we may need to:

     Use a more complex model that can capture more intricate patterns.
     Improve the feature engineering process to provide the model with better inputs.
     Increase the training epochs or adjust the learning rate to allow the model more time to learn.

By comparing the metrics on the training and testing datasets, we can diagnose potential issues with the model and adjust accordingly.

- High training and high testing metrics: The model is generalizing well.
- High training but low testing metrics: Overfitting—needs adjustments to improve generalization.
- Low training but high testing metrics: Potential issue with data or model setup.
- Low training and low testing metrics: Underfitting—model needs enhancement or replacement.

3.2 Classifying Text with Naive Bayes

Throughout this book, we have explored various types of data, primarily tabular data, where features are represented as columns and data instances as rows. We also encountered image data, where features are extracted in the form of feature maps. However, there's another crucial type of data: text.

Text features differ significantly from tabular features. While tabular data is inherently two-dimensional (rows and columns), text data is fundamentally one-dimensional, consisting of a sequence of words (Bhuvaneswari and Kalaiselvi 2012; Chen et al. 2021; Huang and Li 2011; Yang 2018). Additionally, words are inherently ambiguous—a single word can have multiple meanings and serve different grammatical roles (e.g., noun, verb, adjective) (Flach and Lachiche 2004; Ren et al. 2009). This ambiguity makes processing text more challenging and far less straightforward than handling tabular data (Deshpande and Kumar 2018; Sabry 2023).

For example, in tabular data, a feature like temperature with a value of 25 °C consistently represents the same concept across all data points. In text, however, the meaning of a word like "run" varies significantly depending on the context ("I run a business" vs. "I went for a run").

In this chapter, we will explore a simple yet powerful method for dealing with text data: Naive Bayes. Despite its simplicity, Naive Bayes remains one of the most effective algorithms for text classification tasks, such as spam detection, sentiment analysis, and document categorization.

Let's start with a simple dataset consisting of 13 emails, as shown in Table 3.10. Out of these, ten emails have known labels—Spam or Not Spam—while the remaining three emails are unlabeled. Our goal is to classify these three unknown emails based on the patterns learned from the ten labeled examples using the Naive Bayes algorithm.

Table 3.10 Emails dataset

Email ID	Email text	Label
1	"Win a free iPhone now"	Spam
2	"Exclusive deal just for you"	Spam
3	"Meeting rescheduled to 3 PM"	Not spam
4	"Get free access to premium tools"	Spam
5	"Let's catch up over lunch"	Not spam
6	"Limited offer, act now"	Spam
7	"See you at the conference"	Not spam
8	"Congratulations! You won a prize"	Spam
9	"Your invoice is attached"	Not spam
10	"Earn money from home easily"	Spam
11	"Let's have a meeting"	Unknown
12	"Special offer on your order"	Unknown
13	"Don't miss this exclusive chance"	Unknown

3.2.1 Preprocessing Text for Naive Bayes Classification

It's essential to preprocess the text data to ensure consistency and improve the performance of our Naive Bayes classifier. Text data is often messy, containing unnecessary punctuation, inconsistent casing, and stop words. Proper preprocessing helps reduce noise, standardize the data, and create a cleaner representation of the text, making it easier for the classifier to identify patterns effectively.

Common Preprocessing Steps

- Lowercasing: Convert all text to lowercase to ensure uniformity (e.g., "FREE" → "free").
- Remove Punctuation: Remove special characters and punctuation (e.g., "win!" → "win").
- Tokenization: Split text into individual words (e.g., "Win a prize" → ["win", "a", "prize"]).
- Remove Stop Words: Eliminate common words that add little meaning (e.g., "a", "the", "is").
- Stemming or Lemmatization (Optional): Reduce words to their root forms (e.g., "running" → "run").
- Remove Non-Alphabetic Tokens: Eliminate numbers and symbols if they're not meaningful.

3.2.2 Naive Bayes Classification Steps

Step 1: Count Word Occurrences in Each Class
We start by counting the number of occurrences of each word in Spam and Not Spam emails. Here's the breakdown:

Spam Emails (Class 1: Spam)
Email Text
"Win a free iPhone now"
"Exclusive deal just for you"
"Get free access to premium tools"
"Limited offer, act now"
"Congratulations! You won a prize"
"Earn money from home easily"

Word Count in Spam:

win: 1
free: 2
iphone: 1
now: 1
exclusive: 1
deal: 1
just: 1
you: 2
get: 1
access: 1
premium: 1
tools: 1
limited: 1
offer: 1
act: 1
congratulations: 1
won: 1
prize: 1
earn: 1
money: 1
home: 1
easily: 1
Total Word Count in Spam Emails: 24 words

Not Spam Emails (Class 2: Not Spam)
Email Text
"Meeting rescheduled to 3 PM"
"Let's catch up over lunch"

"See you at the conference"
"Your invoice is attached"

Word Count in Not Spam:

meeting: 1
rescheduled: 1
to: 1
pm: 1
let's: 1
catch: 1
up: 1
over: 1
lunch: 1
see: 1
you: 1
at: 1
the: 1
conference: 1
your: 1
invoice: 1
attached: 1
Total Word Count in Not Spam Emails: 17 words

Step 2: Calculate Word Probabilities (Likelihoods)
Using the word counts from Step 1, we calculate the likelihood of each word given a class using the Maximum Likelihood Estimation (MLE) with Laplace Smoothing.

The formula is:

$$P(\text{word}|\text{Class}) = \frac{\text{Count of word in class} + 1}{\text{Total words in class} + V} \tag{3.13}$$

where:

Count of word in class: Number of times a word appears in the specific class.
Total words in class: Total word count in that class.
V: Vocabulary size (total number of unique words across both classes).
+1: Laplace smoothing to handle words with zero occurrences.
Vocabulary Size (V): Count of unique words across all emails = 39 unique words

Total Word Count in Spam Emails: 24
Total Word Count in Not Spam Emails: 17
Let's calculate the likelihood for a sample word, "Free":

$$P(\text{Free} \mid \text{Spam}) = \frac{2+1}{24+39} = \frac{3}{63} = 0.04$$

$$P(\text{Free} \mid \text{Not Spam}) = \frac{0+1}{17+39} = \frac{1}{56} = 0.017$$

Let's calculate the likelihood for a sample word, "Meeting":

$$P(\text{Meeting} \mid \text{Spam}) = \frac{0+1}{24+39} = \frac{1}{63} = 0.015$$

$$P(\text{Meeting} \mid \text{Not Spam}) = \frac{1+1}{17+39} = \frac{2}{56} = 0.03$$

We would repeat this process for every word in the vocabulary.

Step 3: Calculate Class Probabilities (*P*(class))
The prior probability of each class is calculated based on the number of emails in each class:

$$P(\text{Class}) = \frac{\text{Number of emails in class}}{\text{Total Number of Emial}} \tag{3.14}$$

Number of Spam Emails: 6
Number of Not Spam Emails: 4
Total Emails: 10

$$P(\text{Spam}) = \frac{6}{10} = 0.6$$

$$P(\text{Not Spam}) = \frac{4}{10} = 0.4$$

Testing Phase

When a new document (e.g., an email) arrives, we calculate the posterior probability for each class using Bayes' Theorem:

$$P(\text{class}|\text{document}) \propto P(\text{class}) \cdot \prod_{i=1}^{n} P(\text{word}_i \mid \text{class}) \tag{3.15}$$

Steps:
For each class:

Start with the prior probability *P*(class).
For each word in the document, retrieve the probability of the word given the class from the training phase.

Combine Probabilities (Sum of Log Probabilities): Instead of multiplying probabilities (which can cause numerical underflow with small probabilities), we use logarithms:

$$\log P(\text{class}|\text{document}) = \log P(\text{class}) + \sum_{i=1}^{n} \log P(\text{word}_i|\text{class}) \tag{3.16}$$

This converts multiplication into summation, making computation stable.

Choose the Class with the Highest Score:

Compare the final log probabilities of each class.
Assign the document to the class with the highest log probability.

Step 4: Classifying an Unknown Email with Naive Bayes
Now that we have preprocessed the text and calculated word probabilities and class probabilities, let's classify one of the unknown emails using the Naive Bayes formula. The email we want to classify is:
"Let's have a meeting"

Step 1: Compute the Probability of the Email Belonging to the Spam Class
Using the Naive Bayes formula, the probability of the email belonging to the Spam class can be expressed as:

$$P(\text{Spam}|\text{Email}) \propto P(\text{Spam}) \cdot P(\text{word}_1|\text{Spam}) \cdot P(\text{word}_2|\text{Spam}) \cdot P(\text{word}_3|\text{Spam})$$

To simplify calculations and prevent underflow (due to multiplying small probabilities), we take the logarithm of each probability:

$$\log P(\text{Spam}|\text{Email}) = \log P(\text{Spam}) + \sum \log P(\text{word}_i|\text{Spam})$$

From our earlier preprocessing and training:

- The class probability for Spam is:

$$P(\text{Spam}) = 0.6$$

So,

$$\log(0.6) = -0.52$$

- The words in the email are: "Let's," "have," "meeting."
Using the Laplace Smoothing formula for words that are not present in the Spam class:

$$P(\text{word}|\text{Spam}) = \frac{0+1}{24+39} = \frac{1}{63} = 0.015$$

Taking the logarithm:

$$\log(0.015) = -4.19$$

Since none of the words ("Let's," "have," "meeting") are in the Spam class, we calculate:

$$\log P(\text{Spam}|\text{Email}) = \log(0.6) + \log(0.015) + \log(0.015) + \log(0.015) = -13.1$$

Step 2: Compute the Probability of the Email Belonging to the Not Spam Class
From our earlier preprocessing and training:

- The class probability for Spam is:

$$P(\text{Not Spam}) = 0.4$$

So,

$$\log(0.4) = -0.91$$

- The words in the email are: "Let's," "have," "meeting."
Using the Laplace Smoothing formula for words that are not present in the Not Spam class:

$$P(\text{have}|\text{Not Spam}) = \frac{0+1}{17+39} = \frac{1}{56} = 0.017$$

Taking the logarithm:

$$\log(0.017) = -4.07$$

Now we retrieve the probability of the words that appeared in the Not Spam class: "let's" and "meeting."
The probability of each word given the Not Spam class is calculated using the formula with Laplace Smoothing:

$$P(\text{let's}|\text{Not Spam}) = \frac{1+1}{17+39} = \frac{2}{56} = 0.03$$

$$P(\text{meeting}|\text{Not Spam}) = \frac{1+1}{17+39} = \frac{2}{56} = 0.03$$

Taking the natural logarithm:

$$\log(0.03) = -3.5$$

The class probability for Not Spam is:

$$\log(P(\text{Not Spam})) = \log(0.4) = -0.92$$

The total probability of the email being Not Spam is:

$$-0.92 + (-3.5) + (-4.07) + (-3.5) = -12$$

Now we compare the log probability of being in the Spam class, which was approximately -13.1, with the probability of being in the Not Spam class, which is -12.

Since -12 is higher (less negative) than -13.1, we classify the email as: "Not Spam."

This means the email "Let's have a meeting" is more likely to belong to the Not Spam class based on our Naive Bayes calculations.

3.3 Concluding Remarks

In this chapter, we explored binary classification using logistic regression, one of the simplest yet powerful models in machine learning. We began by distinguishing between regression tasks, which predict continuous values, and classification tasks, where the goal is to categorize data into distinct classes. Logistic regression serves as a bridge between these two areas by leveraging the principles of linear regression while transforming its predictions into probabilities to classify instances.

We delved into the step-by-step process of logistic regression, from forming a linear equation to calculating predictions using the sigmoid function, optimizing the weights using gradient descent, and measuring performance using classification metrics. Throughout the chapter, we illustrated these concepts with practical examples, demonstrating how logistic regression attempts to minimize error iteratively to reach optimal predictions.

However, we also discussed that while logistic regression can achieve high accuracy on training data, it doesn't guarantee real-world effectiveness. The model's performance must be evaluated on unseen data to truly understand its generalizability. This is why we emphasized the importance of splitting data into training and

testing sets, ensuring that the model's predictions are not merely a result of memorization but rather a reflection of its ability to generalize to new situations.

We also explored the challenges of overfitting and underfitting:

- Overfitting occurs when the model performs exceptionally well on training data but fails to generalize to new data, indicating that it's too specialized.
- Underfitting, on the other hand, suggests that the model is too simple to capture the underlying patterns, leading to poor performance on both training and testing data.

By measuring metrics like precision, recall, accuracy, and F1 score on both training and testing datasets, we gain a clearer picture of the model's strengths and limitations. These metrics help us identify whether the model is making useful predictions or if adjustments are needed to improve its performance.

We explored the Naive Bayes algorithm as a powerful yet simple tool for text classification. Text data, unlike tabular data, presents unique challenges due to its sequential nature and the ambiguity of words. We addressed these challenges by introducing essential preprocessing techniques such as lowercasing, tokenization, and the removal of punctuation and stop words. These steps ensured cleaner and more consistent data for our classifier.

The core focus of the chapter was on understanding how Naive Bayes uses word frequencies and probabilistic reasoning to classify text into categories like spam or not spam. We covered the calculation of word likelihoods using Laplace smoothing, the computation of class priors, and the application of Bayes' Theorem to derive posterior probabilities. To manage numerical stability, logarithmic probabilities were used instead of direct multiplication, simplifying the calculations and avoiding numerical underflow.

Through a practical example, we demonstrated how an unknown email can be classified using the Naive Bayes model. Despite its strong independence assumptions, the algorithm proved effective in distinguishing between spam and non-spam emails.

Chapter 4
Decision Trees and Random Forests



Abstract This chapter explores Decision Trees and Random Forests, two essential machine learning techniques for classification and regression. Decision Trees provide a clear, interpretable framework by recursively splitting data based on feature values to create structured, rule-based models. Key concepts such as Gini Impurity and Information Gain are discussed to explain how optimal splits are determined for improved model accuracy.

We then introduce Random Forests as an ensemble method that builds multiple decision trees using bootstrapped samples and random feature selection. This approach reduces overfitting and increases predictive performance by aggregating the outputs of individual trees, leading to more robust and generalizable models.

The chapter compares the strengths and limitations of both methods—highlighting the simplicity and transparency of Decision Trees versus the enhanced accuracy and stability of Random Forests. Practical implementation examples demonstrate how these models are applied, evaluated using metrics such as accuracy, precision, and recall. Overall, this chapter offers a practical and conceptual understanding of how to effectively leverage tree-based models for real-world data problems.

4.1 Decision Trees in Classification

Decision trees represent a powerful and intuitive approach to machine learning. To classify an instance, decision trees attempt to identify the optimal way to split the data based on its features (Ali et al. 2012; Bernard et al. 2009). This process aims to achieve a state of consistency regarding the predicted class by grouping similar data points that are likely to belong to the same class (Breiman et al. 2017; Cerulli 2023; Quinlan 2014).

For example, consider a task to classify whether a customer "Owns a Car" (Yes or No) based on the following dataset, as shown in Table 4.1.

During training, a decision tree will analyze this dataset and try to split the data iteratively, selecting one feature at a time. Each split aims to maximize the consis-

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_4

Table 4.1 Car owners dataset

ID	Age (years)	Income (K$/year)	City Type (urban/rural)	Owns a Car (Yes/No)
1	25	40	Urban	No
2	30	70	Urban	Yes
3	32	80	Rural	Yes
4	35	50	Urban	No
5	26	100	Rural	Yes
6	28	60	Urban	No
7	33	50	Urban	Yes
8	25	35	Rural	No
9	28	90	Rural	Yes
10	23	45	Urban	No

tency within the resulting groups, where a group ideally contains instances belonging to the same target class ("Owns a Car"). In other words, the decision tree seeks the best division of data into groups that are as homogeneous as possible with respect to the target class.

To split the data and achieve a more homogeneous state, decision trees rely on two main approaches: split impurity and information gain. Both methods help the tree determine the quality of a potential split, but they approach the problem from slightly different perspectives.

Imagine you're organizing a set of playing cards into piles. Each pile should ideally contain cards of the same suit—hearts, diamonds, clubs, or spades. If you start with a shuffled deck, your initial state is very messy: all the suits are mixed together. As you start sorting the cards into piles, the groups become more "pure" because each pile begins to contain cards of just one suit. This process of reducing the "messiness" is similar to how decision trees work when they use split impurity.

Split impurity measures how "mixed" the groups are after a split. A perfect split would result in groups that are entirely pure—just like the piles of cards containing only one suit. In decision trees, split impurity is often calculated using metrics like the Gini impurity. For example, if you have a group of customers and after a split you find that one group contains all "Yes" responses for "Owns a Car" and another group contains all "No" responses, you've achieved perfect purity. If, however, both groups still contain a mix of "Yes" and "No," the split impurity will be higher, signaling a less effective split. The tree uses these calculations to decide which feature and threshold to split on (John Lu 2010; Shalev-Shwartz and Ben-David 2014).

Now, let's think about information gain. Imagine you're solving a mystery, like figuring out who ate the last cookie from the jar. At first, you might have no idea—it could be anyone in the house. But as you gather clues (e.g., someone was seen in the kitchen at the time of the crime), your uncertainty decreases. Every clue brings you closer to the answer by reducing your confusion about what happened. Information gain works in a similar way: it measures how much "uncertainty" is reduced after a split. A good split provides a lot of useful information, making it easier for the

decision tree to predict the target class. The more your uncertainty is reduced, the higher the information gain (Fratello and Tagliaferri 2018; Shaikhina et al. 2019).

The main difference between the two approaches lies in how they assess the quality of a split. While impurity focuses on reducing the "messiness" within groups, information gain emphasizes reducing the overall uncertainty in the system. Both methods aim for the same goal—finding the best split—but the choice of which to use often depends on the implementation. For instance, Gini impurity is commonly used in classification tasks because it's computationally efficient and works well for most cases. Information gain, on the other hand, relies on entropy, a concept borrowed from information theory, and may be preferred in scenarios where capturing uncertainty is particularly important (Maimon and Rokach 2014).

4.1.1 Preprocessing

Before we use Gini Impurity or Information Gain, we need to prepare the data for the split. In the previous customer data, the process involves the following steps:

1. Get the distinct values from the features (Age, Income, City Type).
   This step ensures that we capture all unique data points for each feature, which will be used to evaluate possible splits.
2. If the data is numeric (Age, Income), sort it.
   Sorting the data points in ascending order allows us to identify meaningful thresholds for potential splits.
3. Calculate the midpoints between the data points.
   Midpoints represent the average of every two consecutive distinct values. These midpoints serve as candidate thresholds to evaluate splits for the decision tree.

Based on the customer data, here are the results of this preprocessing:
Distinct Sorted Values:

- Age (Years): [25,30,32,35,26,28,33,25,23]
- Income (K$/Year): [35, 40, 45, 50, 60, 70, 80, 90, 100]
- City Type: [Urban, Rural]

Midpoints:
Midpoints are calculated as the average of two consecutive sorted values, using the formula:

$$\text{Midpoint} = \frac{\text{value}_i + \text{value}_{i+1}}{2} \tag{4.1}$$

This is done for each pair of consecutive values in the sorted list.

- Age (Years):
  - 24.0, 25.5, 27.0, 29.0, 31.0, 32.5, 34.0

- Income (K$/Year):
  - 37.5, 42.5, 47.5, 55.0, 65.0, 75.0, 85.0, 95.0

These midpoints will be used to evaluate splits in terms of reducing impurity or increasing information gain.

4.1.2 Using Gini Impurity

If we decide to use Gini Impurity, we need to evaluate every potential split point for each feature to find the one that results in the purest split. For numeric features, this involves testing each midpoint (calculated earlier), while for categorical features, we evaluate each distinct value.

The goal is to minimize the Gini impurity, which is calculated as:

$$G = 1 - \sum_{i=1}^{C} p_i^2 \tag{4.2}$$

where:

- G: Gini impurity for a group.
- C: The number of classes in the target variable.
- p_i: The proportion of instances in the group belongs to class i.

To evaluate a potential split:

1. Divide the dataset into two groups based on the split value (e.g., one group with values $\leq$ the split point and the other with values $>$ the split point for numeric features).
2. Calculate the Gini impurity for each group after the split.
3. Compute the weighted average Gini impurity for the split:

$$G_{\text{split}} = \frac{n_{\text{left}}}{n} G_{\text{left}} + \frac{n_{\text{right}}}{n} G_{\text{right}} \tag{4.3}$$

where:

- n_{left} and n_{right}: Number of instances in the left and right groups, respectively.
- G_{left} and G_{right}: Gini impurity of the left and right groups.
- n: Total number of instances.

The split with the lowest G_{split} is chosen as the best split for that feature. We repeat this process for all features and select the split that achieves the smallest overall Gini impurity.

To find the best split in the customer data using Gini Impurity, we start by examining one feature—in this case, Age. Here's the data we're working with:

Age (years)	Owns a Car (Yes/No)
25	No
30	Yes
32	Yes
35	No
26	Yes
28	No
33	Yes
25	No
28	Yes
23	No

To find the optimal split, we follow these steps:

1. Extract the feature and class values to analyze the relationship between the two.
2. Identify potential split points—in this case, the midpoints of the sorted unique values of "Age."
3. Evaluate each split point by calculating the Gini Impurity for the resulting groups (values less than or equal to the split vs. values greater than the split).
4. Aim to minimize the overall Gini Impurity G_{split}.

The ideal split would be a value where:

- All instances with "Age" values less than or equal to the split belong to one class (e.g., "No").
- All instances with "Age" values greater than the split belong to the other class (e.g., "Yes").

This results in a perfectly homogeneous split, with 100% purity or 0% impurity. While a perfect split might not always be achievable in real-world data, the goal is to find the split point that gets us as close as possible to this condition.

We have already calculated the midpoints for the "Age" feature:
24, 26.5, 29, 32.5, 36.5, 39, 42.5, 47.5, 52.5.

These are the potential splits for "Age." We will examine each one to find the split with the minimum Gini Impurity.

For the first midpoint, 24, we split the data as follows:

- Left group: Instances where Age ≤ 24.
- Right group: Instances where Age > 24.

After splitting:

- Left group has 1 instance:
  - Class "No": 1 instance (Customer ID 8).
  - Class "Yes": 0 instances.
- Right group has 9 instances:
  - Class "No": 4 instances (Customer IDs 1, 4, 6, 10).
  - Class "Yes": 5 instances (Customer IDs 2, 3, 5, 7, 9).

To measure how homogeneous each group is, we calculate the proportion of instances belonging to each class in the left and right groups.

For the left group:

- Total instances: 1.
- Proportion of "No": $p_{\text{No}} = \frac{1}{1} = 1$.
- Proportion of "Yes": $p_{\text{Yes}} = \frac{0}{0} = 0$.

Using the Gini Impurity formula:

$$G = 1 - p_{No}^2 + p_{Yes}^2 = 1 - 1^2 + 0^2 = 1$$

The left group is perfectly pure with a Gini Impurity of 0.

For the right group:

- Total instances: 9.
- Proportion of "No": $p_{\text{No}} = \frac{4}{9}$.
- Proportion of "Yes": $p_{\text{Yes}} = \frac{5}{9}$.

Using the Gini Impurity formula:

$$G = 1 - \left(\left(\frac{4}{9}\right)^2 + \left(\frac{5}{9}\right)^2 \right) \approx 0.4938$$

The right group has a Gini Impurity of approximately 0.4938.

The overall Gini Impurity is the weighted average of the impurities of the left and right groups:

$$G_{\text{split}} = \frac{n_{\text{left}}}{n} G_{\text{left}} + \frac{n_{\text{right}}}{n} G_{\text{right}}$$

where:

- $n_{\text{left}} = 1$, $G_{\text{left}} = 0$.
- $n_{\text{right}} = 9$, $G_{\text{right}} \approx 0.4938$
- $n = 10$ (total instances).

$$G_{\text{split}} = \frac{1}{10}(0) + \frac{9}{10}(0.4938) = 0.444$$

Thus, the Gini Impurity for the split at midpoint 24 is 0.4444. We will compare this value with the impurities from other splits to find the best split.

We repeat the same process for all the potential split points of the "Age" feature. After evaluating each split, the results look like this, as shown in Table 4.2.

Table 4.2 Results after evaluating each split

Midpoint	Left No	Left Yes	Right No	Right Yes	Gini Left	Gini Right	Weighted Gini Left	Weighted Gini Right	Gini Split
24	1	0	4	5	0	0.494	0	0.444	0.444
25.5	3	0	2	5	0	0.408	0	0.286	0.286
27	3	1	2	4	0.375	0.444	0.15	0.267	0.417
29	4	2	1	3	0.444	0.375	0.267	0.15	0.417
31	4	3	1	2	0.49	0.444	0.343	0.133	0.476
32.5	4	4	1	1	0.5	0.5	0.4	0.1	0.5
34	4	5	1	0	0.494	0	0.444	0	0.444

Table 4.3 Results for the income feature

Midpoint	Left No	Left Yes	Right No	Right Yes	Gini Left	Gini Right	Weighted Gini Left	Weighted Gini Right	Gini Split
37.5	1	0	4	5	0	0.494	0	0.444	0.444
42.5	2	0	3	5	0	0.469	0	0.375	0.375
47.5	3	0	2	5	0	0.408	0	0.286	0.286
55	4	1	1	4	0.32	0.32	0.16	0.16	0.32
65	5	1	0	4	0.278	0	0.167	0	0.167
75	5	2	0	3	0.408	0	0.286	0	0.286
85	5	3	0	2	0.469	0	0.375	0	0.375
95	5	4	0	1	0.494	0	0.444	0	0.444

The minimum Gini Impurity occurs at the midpoint 25.5, with the following values:

- Left group: 3 instances of "No," 0 instance of "Yes."
- Right group: 2 instances of "No," 5 instances of "Yes."
- Gini Split: 0.286.

This means that, based on the "Age" feature, the best split is at 25.5.

While this is the best split for "Age," the decision tree needs to evaluate all other features (e.g., "Income" or "City Type") to ensure that the selected split minimizes impurity across all potential split points for all features. We must repeat this process for each feature and compare their minimum Gini Impurity values to determine the optimal feature and split point.

Now, we need to repeat the same process on the Income feature. Just like we did for the "Age" feature, we calculate the midpoints for the sorted unique values of "Income" and evaluate each potential split based on the Gini Impurity. The goal remains the same: to find the split point that minimizes the overall Gini Impurity. The results are shown in Table 4.3.

From the table above, the minimum Gini Impurity occurs at the midpoint 65 with:

- Left group: 5 instances of "No," 1 instance of "Yes."
- Right group: 0 instances of "No," 4 instances of "Yes."
- Gini Split: 0.167.

When dealing with categorical data, like the "City Type" feature in the dataset, the approach to finding the best split is slightly different from numeric features. Unlike numeric features, where we evaluate midpoints, for categorical features, we test each distinct category (or subsets of categories for multiclass data) as a potential splitting criterion.

Here is the dataset for "City Type":

City Type (Urban/Rural)	Owns a Car (Yes/No)
Urban	No
Urban	Yes
Rural	Yes
Urban	No
Rural	Yes
Urban	No
Urban	Yes
Rural	No
Rural	Yes
Urban	No

For categorical features, the goal remains the same: to find the split that minimizes impurity. Here's how we handle it:

1. Identify unique categories in the feature.
   For "City Type," the unique categories are Urban and Rural.
2. Evaluate splits based on each category.
   For "City Type," we consider splitting the data into two groups:
   - One group contains instances where "City Type" is Urban.
   - The other group contains instances where "City Type" is Rural.
3. Calculate Gini Impurity for the resulting groups.
   For each split, calculate the Gini Impurity of the two resulting groups (Urban and Rural) and compute the weighted average Gini Impurity for the split.

Split the data into two groups based on "City Type":

- Group 1: "Urban" (Left Group)
  Instances: [No, Yes, No, No, Yes, No]
  - "No": 4 instances
  - "Yes": 2 instances
  - Total: 6 instances

$$G_{\text{Urban}} = 1 - \left(\left(\frac{4}{6}\right)^2 + \left(\frac{2}{6}\right)^2 \right) \approx 0.444$$

- Group 2: "Rural" (Right Group)

Instances: [Yes, Yes, No, Yes]

- "No": 1 instance
- "Yes": 3 instances
- Total: 4 instances

$$G_{\text{Rural}} = 1 - \left(\left(\frac{1}{4}\right)^2 + \left(\frac{3}{4}\right)^2 \right) = 0.375$$

Now, calculate the weighted Gini Impurity:

$$G_{\text{split}} = \frac{n_{\text{Urban}}}{n} G_{\text{Urban}} + \frac{n_{\text{Rural}}}{n} G_{\text{Rural}}$$

where:

- $n_{\text{Urban}} = 6$, $n_{\text{Rural}} = 4$, $n = 10$
- $G_{\text{Urban}} = 0.444$, $G_{\text{Rural}} = 0.375$

$$G_{\text{split}} = \frac{6}{10} 0.444 + \frac{4}{10}\ 0.375 = 0.4164$$

- The split based on "City Type" results in a Gini Impurity of approximately 0.416.
- This process can be repeated for any other categorical features.

By comparing the Gini Impurity for this categorical split with the best splits for other features (e.g., "Age" and "Income"), the decision tree can determine which feature provides the most homogeneous groups and should be used for splitting.

Handling Multiclass Categorical Data

When dealing with features that have multiple categories, the approach to finding the best split involves evaluating all possible ways to partition the categories into two groups. Unlike numeric features, where midpoints are tested, multiclass categorical features require testing subsets of categories to identify the split that minimizes impurity.

Key Steps:

Identify Unique Categories: List all distinct categories for the feature.

Generate Possible Splits: Partition the categories into two groups. For k categories, there are $2^{k-1} - 1$ possible splits.

Evaluate Splits: For each split, divide the data into two groups and calculate the impurity for each group and the overall weighted impurity.

Select the Best Split: Choose the split with the lowest weighted impurity, ensuring the most homogeneous groups.

Apply the Split

Now that we have calculated the Gini Impurity for all features, we can summarize the results:

- Age: Best Gini Impurity = 0.25
- Income: Best Gini Impurity = 0.167
- City Type: Best Gini Impurity = 0.416

The best split is determined by the feature with the lowest Gini Impurity, which is Income with a split at Income} = 65.

Figure 4.1 illustrates the decision tree after splitting the dataset on Income = 65. The data is divided into two groups:

1. Income ≤ 65: Contains instances where the income is ≤ 65.
2. Income > 65: Contains instances where the income is greater than 65.

This split reduces the impurity the most, creating more homogeneous groups and making it easier for the decision tree to classify the target variable, "Owns a Car."

After splitting the data at Income = 65, the resulting tree has two children nodes. If we look at the left child, it contains six instances: five labeled as "No" and one labeled as "Yes." Meanwhile, the right child has four instances, all labeled as "Yes." At this point, the question arises: should we stop splitting further, or continue analyzing the data in these child nodes?

This is where stopping criteria come into play. Stopping criteria are rules that determine when a node becomes a leaf node, signaling the end of further splitting. In essence, a leaf node represents a final decision point in the tree. The decision for a leaf is based on the majority class within that node. For example, in the left child



Fig. 4.1 Decision tree showing the split on Income = 65

Table 4.4 The left node after splitting by income

Age (years)	Income (K$/year)	City Type (Urban/Rural)	Owns a Car (Yes/No)
25	40	Urban	No
35	50	Urban	No
28	60	Urban	No
33	50	Urban	Yes
25	35	Rural	No
23	45	Urban	No

node, the majority class is "No," so the final decision for any new instance that lands in this node would be "No."

There are several common stopping criteria that guide this decision-making process. One intuitive stopping condition is when a node becomes pure, meaning all instances in the node belong to a single class. For example, the right child in this case is pure because all four instances belong to the "Yes" class. Since there is no ambiguity, the tree stops splitting this node and declares it a leaf with a final decision of "Yes."

Another stopping criterion is when further splitting does not significantly reduce impurity. For instance, if splitting a node results in child nodes with only marginally lower Gini Impurity (or other impurity measures like entropy), the tree may decide that further division isn't worth the effort and declare the node as a leaf.

A third common stopping rule is based on the minimum number of instances in a node. If a node contains fewer than a specified threshold of instances, splitting may not yield statistically meaningful results and could lead to overfitting. In such cases, the node becomes a leaf, with its decision based on the majority class of the instances it contains.

Lastly, the tree might stop growing when it reaches a predefined maximum depth. This prevents the tree from becoming too complex and overfitting the training data. Limiting the depth helps ensure that the tree generalizes well to unseen data.

When a node becomes a leaf, its role is to make a prediction for all future instances that follow the path to that node. The decision at a leaf is simple: it predicts the class with the highest frequency among the instances in the node. For example, in the left child node, the decision would be "No" because the majority of the instances are labeled as "No." The right child node would predict "Yes," as all its instances belong to the "Yes" class.

Since the right node of the tree is a leaf (all its instances are "Yes"), there is no need to split it further. However, the left child node is not 100% pure, as it contains both "No" and "Yes" instances. Therefore, we proceed to split the left child further using its data, as shown in Table 4.4.

Following the same process as before, we evaluate potential splits on the Age feature by testing its midpoints, as shown in Table 4.5.

From the table, the split at Age $= 30.5$ produces the lowest Gini Impurity $G_{\text{split}} = 0.167$. This is the best split for the left child node.

Table 4.5 Evaluating the left node based on Age

Midpoint	Left No	Left Yes	Right No	Right Yes	Gini Left	Gini Right	Weighted Gini Left	Weighted Gini Right	Gini Split
24	1	0	4	1	0	0.32	0	0.267	0.267
26.5	3	0	2	1	0	0.444	0	0.222	0.222
30.5	4	0	1	1	0	0.5	0	0.167	0.167
34	4	1	1	0	0.32	0	0.267	0	0.267

Table 4.6 Evaluating the left node based on Income

Midpoint	Left No	Left Yes	Right No	Right Yes	Gini Left	Gini Right	Weighted Gini Left	Weighted Gini Right	Gini Split
37.5	1	0	4	1	0	0.32	0	0.267	0.267
42.5	2	0	3	1	0	0.375	0	0.25	0.25
47.5	3	0	2	1	0	0.444	0	0.222	0.222
55	4	1	1	0	0.32	0	0.267	0	0.267

After determining that the left child node (from the previous split) requires further splitting, we repeat the process on the Income feature. The potential splits and their Gini Impurities are calculated as follows in Table 4.6:

Next, we evaluate City Type for the same node. After testing all possible splits, the overall weighted Gini Impurity for the split is calculated to be 0.267.

Now we compare the results of all features at this stage:

1. Age (split at 30.5): Gini Impurity = 0.167
2. Income: Best Gini Impurity = 0.222
3. City Type: Gini Impurity = 0.267

The lowest Gini Impurity is for the split on Age at 30.5, making it the best split at this point.

Figure 4.2 illustrates the dataset after the second split at Age = 30.5. This split further divides the left child node of the initial split (Income ≤ 65) into two new child nodes.

At this point, the left child of the split at Age = 30.5 becomes a leaf node because all its instances are labeled as "No." The right child, however, contains one "No" and one "Yes." Depending on the stopping criteria, we may choose to stop here or continue splitting further.

If we decide to continue, the only feature that can be processed is Age, since both Income and City Type have only one unique value in this subset. The midpoint of the remaining "Age" values is Age = 34.

Figure 4.3 illustrates the final decision tree. The green nodes represent the leaf nodes, all of which are 100% pure. This purity ensures that every leaf has a single class, making the tree ready to classify new instances accurately.

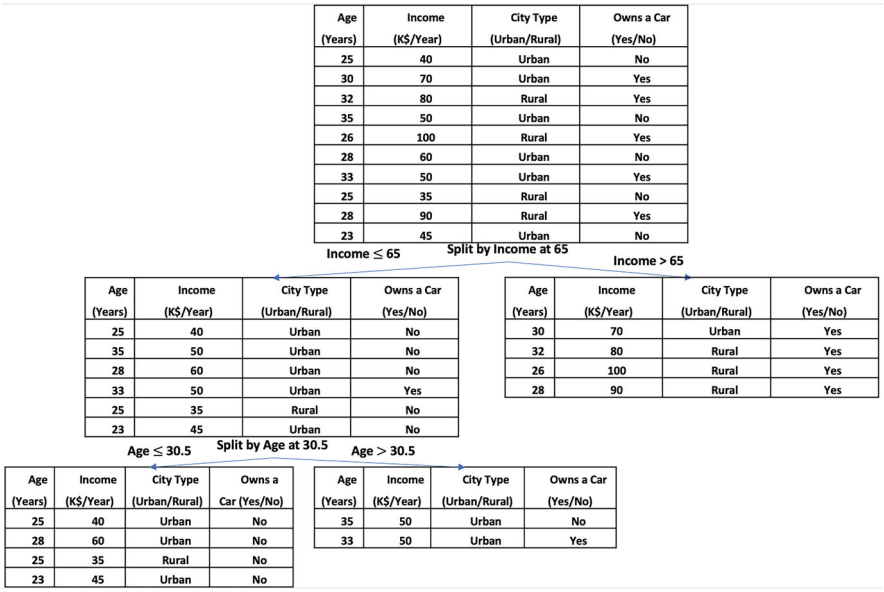


Fig. 4.2 Decision tree after the second split at Age = 30.5

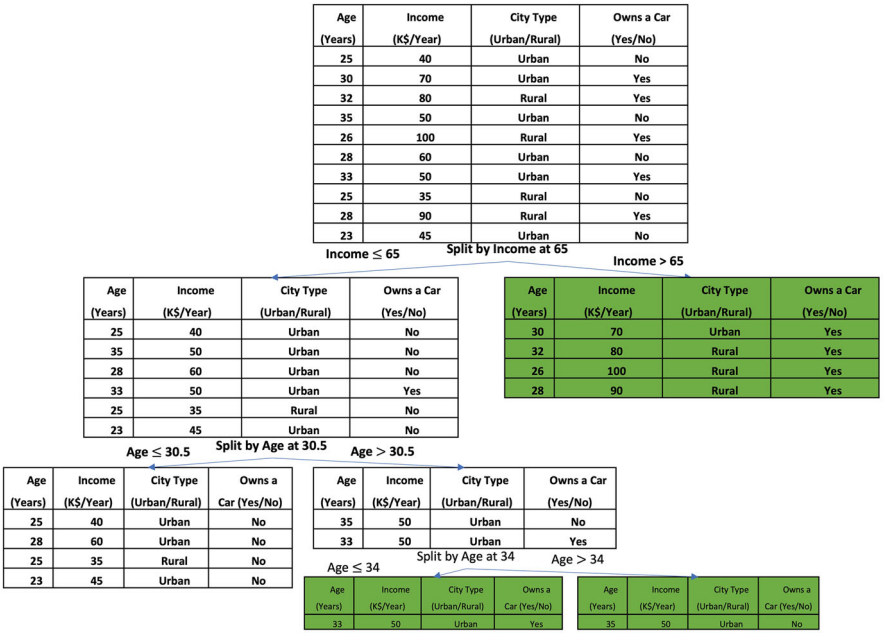


Fig. 4.3 The final decision tree

Predicting New Instances
When a new instance arrives at the tree:

1. It starts at the root node and moves along one of the branches based on the value of the split feature at that node.
2. At each node, the instance evaluates the splitting condition (e.g., Income >65) and moves to the corresponding child node.
3. This process continues until the instance reaches a leaf node. The class label assigned to that leaf becomes the predicted class for the instance.

This process allows the decision tree to classify instances efficiently based on the learned splits and provides a clear, interpretable path for decision-making.

4.1.3 Using Information Gain

Let's explore another method to create a decision tree: information gain. This method identifies the best split by measuring how much a split reduces entropy, which represents the uncertainty or disorder in the data. The goal is to find the split that provides the greatest reduction in entropy, thereby increasing the information gain.

Computing the Entropy of the Current Node
We calculate the entropy of the current node. Entropy helps us understand the impurity of the node based on the distribution of classes. The goal is to find the split that reduces entropy the most, thereby maximizing information gain. Since we are starting at the root, we first examine the distribution of instances across the classes.

Owns a Car	Count
No	5
Yes	5

The entropy H is calculated using the formula:

$$H_{\text{Node}} = -\sum_{i=1}^{n} p_i \log_2(p_i) \tag{4.4}$$

where:

- p_i is the proportion of instances belonging to class i.
- n is the total number of classes.

In this case:

- $p_{\text{No}} = \frac{5}{10} = 0.5$
- $p_{\text{Yes}} = \frac{5}{10} = 0.5$

Substitute these values into the formula to compute the entropy:

$$H_{\text{Node}} = -(0.5 \cdot \log_2(0.5) + 0.5 \cdot \log_2(0.5)) = 1$$

This entropy value represents the impurity of the root node.

Evaluate the Split Over the Midpoints
To determine the best split, we evaluate the potential splitting points based on the midpoints of the feature values in the dataset. For the feature Age, the possible splitting points are: 24.0, 25.5, 27.0, 29.0, 31.0, 32.5, 34.0

We calculate the entropy for each potential split by dividing the data into two groups:

- Left Group: Instances with values less than or equal to the splitting point.
- Right Group: Instances with values greater than the splitting point.

The goal is to identify the split that reduces the entropy the most, resulting in the highest information gain.

Let's evaluate the split for Age = 24.0. The data is divided as follows:

- Left Group (Age $\leq$ 24.0):
  - Instances: 23
  - Labels: No (1 instance)
- Right Group (Age $>$ 24.0):
  - Instances: 25, 30, 32, 35, 26, 28, 33, 25, 28
  - Labels: No, Yes, Yes, No, Yes, No, Yes, Yes, No**Compute the Entropy for Each Group**

1. Entropy of the Left Group:
   - Proportion of "No": $p_{\text{No}} = \frac{1}{1} = 1$
   - $H_{\text{Left}} = -(1 \cdot \log_2(1)) = 0$
2. Entropy of the Right Group:
   - Total instances: 9
   - Proportion of "Yes": $p_{\text{Yes}} = \frac{5}{9.}$
   - Proportion of "No": $p_{\text{No}} = \frac{4}{9}$
   - $H_{\text{Right}} = -\left(\frac{5}{9} \cdot \log_2\left(\frac{5}{9}\right) + \frac{4}{9} \cdot \log_2\left(\frac{4}{9}\right)\right) = 0.991$

Compute the Weighted Entropy of the Split
The weighted entropy of the split is given by:

$$H_{\text{split}} = \frac{|\text{Left Group}|}{|\text{Total Instances}|} H_{\text{left}} + \frac{|\text{Right Group}|}{|\text{Total Instances}|} H_{\text{Right}} \tag{4.5}$$

- $\frac{|\text{Left Group}|}{|\text{Total Instances}|} = \frac{1}{10}$
- $\frac{|\text{Right Group}|}{|\text{Total Instances}|} = \frac{9}{10}$

$$H_{\text{split}} = \frac{1}{10}(0) + \frac{9}{10}(0.991) = 0.891$$

Compute the Information Gain
The information gain for the split is:

$$\text{Information Gain} = H_{\text{Node}} - H_{\text{Split}} \tag{4.6}$$

$$\text{Information Gain} = 1 - 0.891 = 0.108$$

By comparing the information gain for all potential splits, we select the one with the highest information gain.

Selecting the Best Split Based on Information Gain
Information Gain for Age

From the calculations above, the best split for the feature Age occurs at the midpoint 25.5, which has the highest information gain of 0.396. As shown in Table 4.7.

Information Gain for Income

Now, we repeat the process for the feature Income and calculate the information gain at each midpoint. From the results, the best split for Income occurs at the midpoint 65, with the highest information gain of 0.61 as shown in Table 4.8.

Information Gain for City Type

The data is divided into two groups based on City Type:

1. Urban:
   - No: 4 instances
   - Yes: 2 instances
2. Rural:
   - No: 1 instance
   - Yes: 3 instances

Calculate Entropy for Each Group
1. Entropy for Urban:

$$p_{No} = \frac{4}{6} = 0.667, p_{Yes} = \frac{2}{6} = 0.333$$

Table 4.7 Evaluating information gain on Age

Midpoint	Left No	Left Yes	Right No	Right Yes	Entropy Left	Entropy Right	Weighted Entropy Left	Weighted Entropy Right	Information Gain
24	1	0	4	5	0	0.991	0	0.892	0.108
25.5	3	0	2	5	0	0.863	0	0.604	0.396
27	3	1	2	4	0.811	0.918	0.325	0.551	0.125
29	4	2	1	3	0.918	0.811	0.551	0.325	0.125
31	4	3	1	2	0.985	0.918	0.69	0.275	0.035
32.5	4	4	1	1	1	1	0.8	0.2	0
34	4	5	1	0	0.991	0	0.892	0	0.108

Table 4.8 Evaluating information gain on Income

Midpoint	Left No	Left Yes	Right No	Right Yes	Entropy Left	Entropy Right	Weighted Entropy Left	Weighted Entropy Right	Information Gain
37.5	1	0	4	5	0	0.991	0	0.892	0.108
42.5	2	0	3	5	0	0.954	0	0.764	0.236
47.5	3	0	2	5	0	0.863	0	0.604	0.396
55	4	1	1	4	0.722	0.722	0.361	0.361	0.278
65	5	1	0	4	0.65	0	0.39	0	0.61
75	5	2	0	3	0.863	0	0.604	0	0.396
85	5	3	0	2	0.954	0	0.764	0	0.236
95	5	4	0	1	0.991	0	0.892	0	0.108

$$H_{\text{Urban}} = -(0.667 \cdot \log_2(0.667) + 0.333 \cdot \log_2(0.333)) = 0.918$$

2. Entropy for Rural:

$$p_{No} = \frac{1}{4} = 0.25, p_{Yes} = \frac{3}{4} = 0.75$$

$$H_{\text{Rural}} = -(0.25 \cdot \log_2(0.25) + 0.75 \cdot \log_2(0.75)) = 0.811$$

Calculate Weighted Average Entropy
The weighted entropy for the split is:

$$H_{\text{Split}} = \frac{6}{10} \cdot 0.918 + \frac{4}{10} \cdot 0.811 = 0.551 + 0.324 = 0.875$$

The information gain is:

Information Gain $= H_{\text{Node}} - H_{\text{Split}}$
Information Gain $= 1.0 - 0.875 = 0.125$

Now we have the following information gains:

- Age: 0.396 at 25.5
- Income: 0.61 at 65
- City Type: 0.125

The highest information gain is for Income at 65, so we split the data based on this feature.

- Right Node: After the split, the right node (instances with income > 65) will become a leaf because all instances in this group belong to the class Yes.
- Left Node: For the left node (instances with income ≤ 65), the process is repeated. The highest information gain here is at 30.5, with Information Gain $= 0.316$.
- Left Node of this split: This group (instances with Age ≤ 30.5) becomes a leaf because all instances belong to the class No.
- Right Node of this split: This group (instances with Age > 30.5 and Income ≤ 65) splits further. The best split here is at 34, resulting in the final decision tree structure.

This process results in the same decision tree structure as depicted in Fig. 4.3.

While Gini impurity and information gain are both used to evaluate splits in decision trees, they can lead to different split decisions due to their distinct optimization objectives. Gini impurity focuses on minimizing the "mix" of classes within each split, while information gain aims to reduce overall uncertainty (entropy) in the system. These differences become apparent in datasets with imbalanced class distributions or when the splits lead to groups of very different sizes. For example, Gini impurity may prefer a split that creates a highly pure, but smaller group, as it directly rewards reductions in impurity. In contrast, information gain may favor a split that

distributes the reduction in entropy more evenly across groups, even if individual group purity is lower. This divergence in decision-making reflects their underlying mathematical approaches and highlights the importance of selecting the metric that aligns with the specific goals or constraints of a given classification problem.

4.1.4 Decision Trees in Regression

Decision trees can also be applied to regression tasks, using the same fundamental concept of splitting data based on features to achieve the best outcome. For regression, the goal is to minimize the variability (or error) of the target variable within each resulting group. If the feature is numeric, splits occur at the midpoints of sorted values. For categorical features, splits are based on distinct category groupings (Witten et al. 2005).

In a regression task, the best split is determined by the weighted variance of the target variable (House Price in this example). A split is considered optimal if it reduces the weighted variance the most.

Let's consider this dataset in Table 4.9:

Step 1: Find Midpoints for Numeric Features

For numeric features, calculate the midpoints between consecutive sorted values.

For House Size:

1. Sort the values: 1200, 1400, 1500, 1700, 1800, 1900, 2000, 2200
2. Midpoints:

Midpoints = 1300, 1450, 1600, 1750, 1850, 1950, 2100

For Number of Bedrooms:

Table 4.9 House prices dataset

House size (sq. ft)	Number of bedrooms	House price ($1000s)
1200	2	250
1400	3	300
1800	3	400
1700	4	450
1500	3	350
2000	4	500
1200	2	275
2200	5	600
1900	4	475
1500	3	375

1. Unique values: 2, 3, 4, 5
2. Midpoints:

$$\text{Midpoints} = 2.5, 3.5, 4.5$$

Step 2: Split Data into Left and Right Groups
For each midpoint:

1. Divide the dataset into:
   - Left Group: Data points with feature values $\leq$ the midpoint.
   - Right Group: Data points with feature values $>$ the midpoint.

Step 3: Calculate Variance for Each Group
Compute the variance of the target variable (House Price) for the left and right groups. Variance is calculated as:

$$\text{Variance} = \frac{\sum (\text{Price} - \text{Mean Price})^2}{m} \tag{4.7}$$

where m is the number of instances in that group.

Step 4: Calculate Weighted Variance of the Split
The weighted variance of a split is given by:

$$\text{Weighted Variance} = \frac{n_{\text{left}}}{n} \cdot \text{Variance}_{\text{left}} + \frac{n_{\text{right}}}{n} \cdot \text{Variance}_{\text{right}} \tag{4.8}$$

where:

- n_{left}: Number of instances in the left group.
- n_{right}: Number of instances in the right group.
- n: Total number of instances.

Step 5: Choose the Best Split
Among all possible splits, select the one with the lowest weighted variance.

Step 6: Recursively Split Until Stopping Criteria Are Met
Repeat the process for each resulting node until one of the following stopping criteria is met:

1. The node contains fewer than a minimum number of instances.
2. The variance reduction is below a predefined threshold.
3. A maximum tree depth is reached.

Figure 4.4 illustrates the final regression tree constructed for the dataset. The first split occurs on the feature with the lowest weighted variance, which is the number of bedrooms at the threshold of 3.5. This split divides the dataset into two groups:

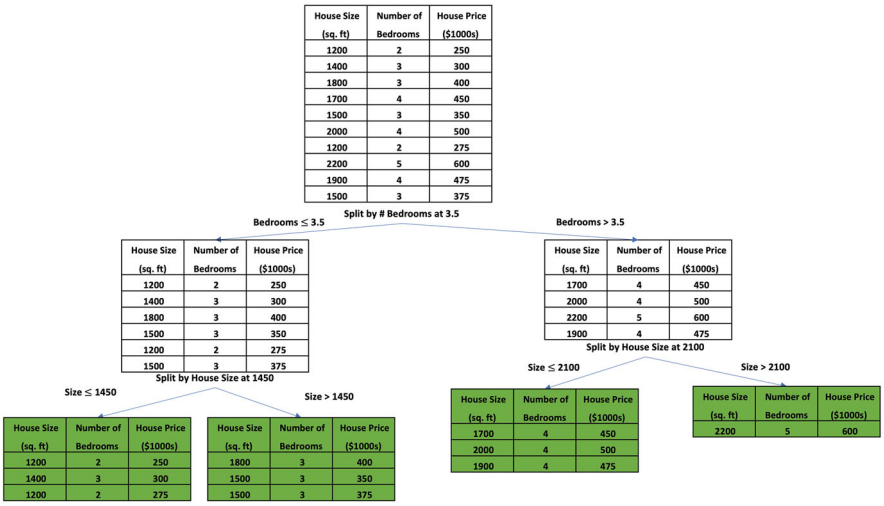


Fig. 4.4 Final regression tree

houses with three or fewer bedrooms in the left node and houses with more than three bedrooms in the right node.

For the left node, the next feature with the lowest variance is house size, with the optimal split at 1450 square feet. This creates two leaf nodes: one containing smaller houses with sizes less than or equal to 1450 square feet and the other with larger houses. Similarly, for the right node, the split is also based on house size, but this time at 2100 square feet, creating two leaf nodes. One group contains houses with sizes less than or equal to 2100 square feet, and the other group includes the larger houses exceeding this size.

In this example, the stopping criterion is to limit each node to a maximum of three instances. Once this criterion is met, the nodes become leaves, and no further splits are performed. Each leaf node holds the subset of data that satisfies its splitting path, and the prediction for that node is the average of the house prices within it.

Once the tree is trained, predicting the house price for a new instance involves traversing the tree from the root. The instance is evaluated at each node based on the splitting conditions, moving down one branch at a time until it reaches a leaf node. The prediction for the new instance is the average house price of all the data points in the corresponding leaf node.

4.2 Random Forests

When we have datasets with hundreds or thousands of data points and numerous features (e.g., 10–20 or more), constructing a single decision tree on such massive data may lead to inefficiencies and overfitting. A single, large decision tree tends to memorize the training data rather than generalize from it, reducing its predictive power on unseen data. The solution is not to rely on one large tree but instead to train multiple smaller trees (Boehmke and Greenwell 2019; Dinov 2018).

But how can we train multiple trees on the same dataset without simply replicating the same tree each time? The answer lies in sampling.

4.2.1 Using Subsamples to Train Trees

Instead of training each tree on the entire dataset, we randomly pick a subsample of the data to train each tree. For example:

- If we take 25% of the dataset for each tree, we can train four smaller trees. Each tree sees only a portion of the data, allowing it to develop a unique perspective on the problem.
- Once the trees are trained, they each make predictions for a new data point. To determine the final prediction, we use majority voting (for classification tasks) or average their outputs (for regression tasks).

This approach ensures that each tree is different, capturing diverse patterns in the data while reducing the risk of overfitting.

What if we used 10% of the dataset for each tree? This would allow us to train 10 trees if each data point is used only once. However, we can take this idea even further:

- Bootstrapping: Allow each data point to be used multiple times across different trees by sampling with replacement. This technique ensures that we can build as many trees as needed while introducing randomness into the training process.
- Random Feature Selection: Instead of allowing each tree to use all the features at every split, we can randomly select a subset of features for each split. This further diversifies the trees because each tree looks at the data differently.

With these two strategies—sampling data points and selecting random subsets of features—we create an ensemble of random trees. Each tree makes its prediction, and by aggregating their outputs, we get a more robust and accurate final prediction.

4.2.2 How Random Forests Work

This is the essence of a Random Forest:

1. Train multiple decision trees, each using a random subsample of the data.
2. For each tree, use a random subset of features at every split.
3. Aggregate the predictions of all the trees to make the final decision:
    - Use majority voting for classification tasks.
    - Use averaging for regression tasks.

This process reduces overfitting because the trees are less correlated due to the randomization introduced by sampling and feature selection (Biau and Scornet 2016; Breiman 2001).

Here's a full implementation in Python to show how to use Decision Trees for a classification task.

```
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, classification_report

# Load data from a CSV file
# Replace 'your_data.csv' with the path to your CSV file
data = pd.read_csv('your_data.csv')

# Display the first few rows of the dataset
print("Dataset preview:")
print(data.head())

# Assume the target column is named 'target' and the rest are features
# Replace 'target' with the actual column name of your target variable
features = data.drop('target', axis=1)
target = data['target']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(features, target,
test_size=0.2, random_state=42)

# Initialize and train the Decision Tree Classifier
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

# Predict on the test set
y_pred = clf.predict(X_test)

# Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy of the Decision Tree: {accuracy:.2f}")

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Visualize the Decision Tree structure
tree_rules = export_text(clf, feature_names=list(features.columns))
print("\nDecision Tree Structure:")
print(tree_rules)
```

Here's a full implementation to demonstrate how to use a Random Forest Classifier in Python. After loading the data, it is split into training and testing sets using an 80–20 split to ensure that the model is trained and evaluated on separate data. A RandomForestClassifier is then initialized with 100 trees (n_estimators = 100), and at each split, the model selects a random subset of features (max_features = 'sqrt') to encourage diversity among the trees.

```
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load data from a CSV file
# Replace 'your_data.csv' with the path to your CSV file
data = pd.read_csv('your_data.csv')

# Display the first few rows of the dataset
print("Dataset preview:")
print(data.head())

# Assume the target column is named 'target' and the rest are features
# Replace 'target' with the actual column name of your target variable
features = data.drop('target', axis=1)
target = data['target']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(features, target,
test_size=0.2, random_state=42)

# Initialize and train the Random Forest Classifier
rf_clf = RandomForestClassifier(n_estimators=100, max_features='sqrt',
random_state=42)
rf_clf.fit(X_train, y_train)

# Predict on the test set
y_pred = rf_clf.predict(X_test)

# Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy of the Random Forest: {accuracy:.2f}")

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Feature Importance
print("\nFeature Importance:")
feature_importance = pd.DataFrame({
    'Feature': features.columns,
    'Importance': rf_clf.feature_importances_
}).sort_values(by='Importance', ascending=False)
print(feature_importance)

# Visualize some predictions
print("\nSample Predictions:")
sample = X_test.iloc[:5]
print("Predictions:", rf_clf.predict(sample))
print("True Labels:", y_test.iloc[:5].values)
```

4.3 Evaluating Decision Trees and Random Forests in Classification Tasks

Metrics such as accuracy, precision, recall, and F1-score are commonly used to evaluate the quality of predictions, especially in classification tasks. Accuracy provides a straightforward measure of the proportion of correctly classified instances, but in datasets with imbalanced classes, precision and recall offer deeper insights into how well the model distinguishes between different classes.

Random Forests, being an ensemble of Decision Trees, bring an additional layer of complexity to evaluation. The aggregated nature of Random Forest predictions, through majority voting for classification or averaging for regression, helps mitigate overfitting and enhances performance. When evaluating Random Forests, feature importance scores are often analyzed to understand which features contribute most to the model's decisions. This interpretability is particularly useful in understanding the underlying patterns in the data and validating the model's relevance to the problem domain. Comparing metrics like accuracy and F1-score between Decision Trees and Random Forests often reveals that the ensemble model performs better, as it leverages the diversity among trees to produce more robust predictions (Dhivyaa et al. 2020; Wu et al. 2015).

Another aspect of evaluation involves examining how the models handle overfitting and underfitting. Decision Trees, if grown to their full depth, tend to overfit by capturing noise in the data. This is addressed by setting constraints such as maximum depth or minimum samples per leaf. Random Forests naturally counteract overfitting by averaging the outputs of many trees, each trained on different subsets of the data and features. This randomization reduces the likelihood of any single tree dominating the model's predictions. By comparing results from cross-validation and independent test datasets, practitioners can ensure that the models not only perform well on the data they have seen but also generalize effectively to new data.

In both models, confusion matrices are another valuable tool for evaluation, providing a detailed view of where the models succeed and where they fail. By examining false positives and false negatives, practitioners can identify specific patterns that the models struggle to classify.

4.4 Concluding Remarks

In this chapter, we have explored the intricacies of Decision Trees and Random Forests, two powerful techniques in machine learning. Decision Trees offer an intuitive and transparent approach to both classification and regression tasks, with their ability to split data based on feature values to form a tree structure that is easily interpretable. By understanding concepts such as Gini Impurity and Information Gain, we examined how optimal splits are identified to create homogeneous groups in the data.

Random Forests, on the other hand, demonstrated how ensemble methods can enhance model performance by combining multiple trees. The randomness introduced through bootstrapping and random feature selection reduces overfitting while leveraging the strengths of individual decision trees. This collective approach leads to more robust predictions and improved generalization.

The comparison between Decision Trees and Random Forests highlighted the trade-offs in complexity, interpretability, and performance. While Decision Trees are simple and interpretable, they are prone to overfitting. Random Forests mitigate this issue by aggregating predictions across multiple trees, making them a preferred choice for complex datasets.

Finally, we looked at practical implementation examples and evaluation metrics, emphasizing how these models can be applied to real-world problems. From analyzing feature importance to interpreting results through metrics such as precision, recall, and accuracy, this chapter provided a comprehensive guide to utilizing Decision Trees and Random Forests effectively.

Chapter 5
Ensemble Learning



Abstract This chapter focuses on ensemble learning, a powerful approach that combines multiple simple models to solve complex machine learning problems with greater accuracy and robustness. By leveraging the collective strength of weak learners—models that perform only slightly better than random chance—ensemble methods can produce high-performing predictive systems.

We examine two key techniques: Adaptive Boosting (AdaBoost) and Gradient Boosting. AdaBoost improves model accuracy by focusing on misclassified instances, assigning greater weight to difficult cases in each iteration. Gradient Boosting, in contrast, builds models sequentially by minimizing residual errors, refining predictions through stage-wise learning.

Both methods highlight the adaptability and precision of ensemble learning, showcasing its ability to enhance performance and reduce overfitting. Practical examples demonstrate the application of boosting techniques across various domains, from structured numerical data to more complex tasks. This chapter underscores the critical role of ensemble methods in building reliable, scalable machine learning solutions.

5.1 Weak Learners

As we discussed in the previous chapter on random forests, we can build a single model that consists of many smaller models working together. A random forest is one such example: it is a model composed of multiple decision trees collaborating to solve a single problem.

This concept is called ensemble learning. An ensemble is a collection of multiple machine learning models that work together to solve a problem. By using subsampling techniques, we can create a variety of models by altering the data and features used for training.

However, subsampling alone is not always sufficient to achieve accurate results. The complexity of the problem or the features may prevent the ensemble from performing well (Natekin and Knoll 2013; Wang and Sun 2021). To address this,

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_5

there are two main strategies for improving ensemble performance: Adaptive Boosting (AdaBoost) and Gradient Boosting (Friedman 2001; Schapire and Freund 2013).

These concepts rely on what are called weak learners to build an ensemble. A weak learner is a simple machine learning model, such as a decision tree with one parent and two children (a decision stump) or even a single neuron from a neural network (a perceptron).

The reason we prefer weak learners over strong learners, like a fully grown decision tree or a complete neural network, is to avoid overfitting the data. Strong learners may capture the noise in the data rather than the underlying patterns. By using weak learners that give only an approximate indication of the correct answer, we harness the wisdom of the crowd, where the combined predictions of many weak learners result in a more robust and accurate model (Schapire 2013; Ying et al. 2013).

Let's consider a dataset with three features and a target variable, as shown in Table 5.1.

Based on this data, we can create three stumps, each of which tries to split the labels based on one feature. For example, Stump 1 will attempt to split the data based on Temperature to create the most homogeneous groups (Rain or No Rain) in its child nodes, as we saw in the Decision Tree chapter.

Similarly, Stump 2 will evaluate Humidity and find the best threshold to split the data into the most homogeneous groups. Lastly, Stump 3 will perform the same process for Wind Speed.

By doing this, we create three decision stumps, each focused on one feature. Each stump provides a prediction based on its corresponding feature, and then we can apply a voting algorithm to combine the predictions and determine the final outcome.

Figure 5.1 demonstrates an example of a result of Stump 1 after processing all possible midpoints of the Temperature values. In this example, the stump determines the best threshold to be 25 °C, splitting the data as follows:

- If the temperature is less than or equal to 25 °C, the prediction is Rain.
- If the temperature is greater than 25 °C, the prediction is No Rain.

Table 5.1 Rainfall dataset

Instance	Temperature (°C)	Humidity (%)	Wind speed (km/h)	Target (Rain/No Rain)
1	23	80	10	Rain
2	30	60	20	No Rain
3	22	85	12	Rain
4	28	60	21	No Rain
5	27	75	11	Rain
6	29	65	19	No Rain
7	24	82	18	Rain
8	27	68	18	No Rain
9	21	90	17	Rain
10	31	85	18	No Rain

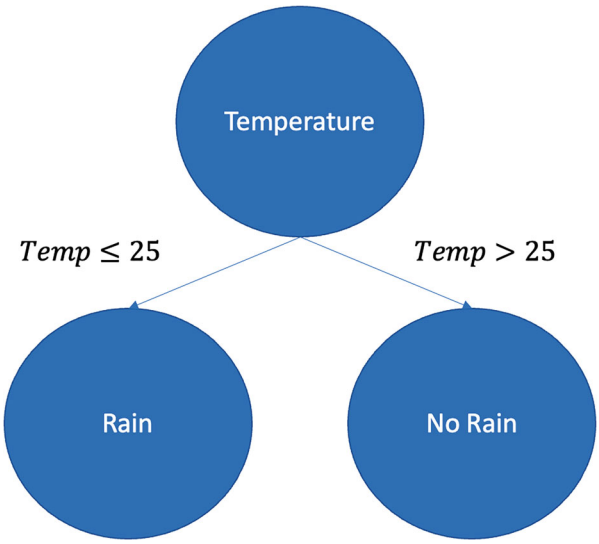


Fig. 5.1 Decision Stump based on Temperature

The same process is repeated for Humidity (Stump 2) and Wind Speed (Stump 3), creating simple models that work together to form a more powerful ensemble.

5.2 Adaptive Boosting (AdaBoost)

AdaBoost is a method designed to enhance the predictions of weak learners within an ensemble. The core idea behind AdaBoost is to iteratively adjust the importance, or weight, of each instance in the dataset.

In each iteration, instances that are misclassified by the current weak learner are assigned higher weights, while instances that are correctly classified are assigned lower weights. This dynamic weighting ensures that the weak learner in the next iteration focuses more on the harder-to-classify instances that were previously misclassified (Freund and Schapire 1997).

By focusing on the challenging instances, AdaBoost improves the ability of the weak learners to handle complex patterns in the data. Over multiple iterations, the ensemble grows stronger as it combines the predictions of these weak learners, with greater weight given to learners that perform better.

We start by assigning weight to each instance in the dataset. Initially, since we have 10 instances, all instances are given equal weights of $\frac{1}{10} = 0.1$. These weights represent the relative importance of each instance during training.

Additionally, to make the calculations easier and consistent, we encode the target values numerically:

- Rain is encoded as +1
- No Rain is encoded as -1

Table 5.2 Rainfall dataset with Initial Weights

Instance	Temperature (°C)	Humidity (%)	Wind Speed (km/h)	Target (Rain: +1, No Rain: −1)	Initial Weight
1	23	80	10	+1	0.1
2	30	60	20	−1	0.1
3	22	85	12	+1	0.1
4	28	60	21	−1	0.1
5	27	75	11	+1	0.1
6	29	65	19	−1	0.1
7	24	82	18	+1	0.1
8	27	68	18	−1	0.1
9	21	90	17	+1	0.1
10	31	85	18	−1	0.1

Here's the dataset with the initial weights assigned, as shown in Table 5.2.

5.2.1 Training the First Weak Learner

Since we have three features (Temperature, Humidity, and Wind Speed), we will train three decision stumps—one for each feature—and select the stump that achieves the lowest weighted error.

The process for training a stump follows the same approach as in a decision tree:

1. Obtain the distinct values of the feature.
2. Sort the values in ascending order.
3. Compute the midpoints between consecutive values as potential thresholds for splitting.
4. Evaluate each split to determine how well it separates the data, but instead of evaluating the split based on impurity (like Gini index or information gain), we calculate the weighted error.

5.2.2 Weighted Error for a Split

For each possible split, we calculate the weighted error using the formula:

$$\text{Error} = \sum_{i=1}^{n} w_i \cdot \mathbb{I}(y_i \neq h(x_i)) \tag{5.1}$$

where:

- w_i: Weight of the ith instance.
- y_i: True label (+1 for "Rain", −1 for "No Rain").
- $h(x_i)$: Prediction made by the stump for the ith instance (+1 or −1).
- $\mathbb{I}(y_i \neq h(x_i))$: Indicator function (1 if the instance is misclassified, 0 otherwise).

The first stump attempts to find the best split of data using the Temperature feature. The midpoints for potential splits are: 21.5, 22.5, 23.5, 25.5, 27.5, 28.5, 29.5, 30.5.

5.2.3 Evaluating the Split at Midpoints

At **21.5** midpoint:

- Left Child: Instances where Temperature ≤ 21.5
  - Instances: {9} (Instance 9 has label +1)
- Right Child: Instances where Temperature > 21.5
  - Instances: {1, 2, 3, 4, 5, 6, 7, 8, 10}
  - Labels: {+1, −1, +1, −1, +1, −1, +1, −1, −1}

Possible Predictions
The stump will try all possible prediction combinations for the left and right nodes to determine which combination of predictions results in the lowest weighted error by examining the misclassified instances in each node while taking into account the weights of these instances.

1. Left = −1, Right = −1
2. Left = −1, Right = +1
3. Left = +1, Right = −1
4. Left = +1, Right = +1

For each possibility, the weighted error is computed using formula (5.1).

Case 1: Left = −1, Right = −1
- Left Node:
  - Prediction: −1
  - Instances: {9}
  - Actual Label: +1 (misclassified)
  - Weighted Error: 0.1 (since 1 × 0.1)
- Right Node:
  - Prediction: −1
  - Misclassified Instances:{1, 3, 5, 7}
  - Weighted Error: 0.1 + 0.1 + 0.1 + 0.1 = 0.4
- Total Weighted Error: 0.1 + 0.4 = 0.5

Case 2: Left = −1, Right = +1

- Left Node:
  - Prediction: −1
  - Instances: {9}
  - Actual Label: +1 (misclassified)
  - Weighted Error: 0.1
- Right Node:
  - Prediction: +1
  - Misclassified Instances: {2, 4, 6, 8, 10}
  - Weighted Error: 0.1 + 0.1 + 0.1 + 0.1 + 0.1 = 0.5
- Total Weighted Error: 0.1 + 0.5 = 0.6

Case 3: Left = +1, Right = −1

- Left Node:
  - Prediction: +1
  - Instances: {9}
  - Actual Label: +1 (correctly classified)
  - Weighted Error: 0
- Right Node:
  - Prediction: −1
  - Misclassified Instances: {1, 3, 5, 7}
  - Weighted Error:0.1 + 0.1 + 0.1 + 0.1 = 0.4
- Total Weighted Error: 0 + 0.4 = 0.4

Case 4: Left = +1, Right = +1

- Left Node:
  - Prediction: +1
  - Instances: {9}
  - Actual Label:+1 (correctly classified)
  - Weighted Error: 0
- Right Node:
  - Prediction: +1
  - Misclassified Instances: {2, 4, 6, 8, 10}
  - Weighted Error: 0.1 + 0.1 + 0.1 + 0.1 + 0.1 = 0.5
- Total Weighted Error: 0 + 0.5 = 0.5

5.2.4 Selecting the Best Split

The split with the lowest total weighted error is:

- Case 3: Left = +1, Right = −1
- Total Weighted Error: 0.4

This process is repeated for all other midpoints to determine the best split across all thresholds. The split with the globally lowest error is selected as the decision for the stump.

All the midpoints should be processed in the same way, and the results are summarized in Table 5.3.

From the table, all midpoints have the left node predicting +1 (Rain) and the right node predicting −1 (No Rain). This pattern emerged because the lowest weighted error is achieved with this prediction rule for each split.

The lowest weighted error is 0.1, which occurs at two thresholds: 25.5 and 27.5. Since both splits result in the same weighted error, the stump can pick either threshold as the split point.

The stump selects one of these thresholds (e.g., 25.5) and sets the following rules for predictions:

- Left Node: Predict +1 if Temperature ≤ 25.5
- Right Node: Predict −1 if Temperature > 25.5

After training the second and third stumps on Humidity and Wind Speed, their results are summarized in Tables 5.4 and 5.5.

5.2.5 Comparing the Best Splits

The lowest weighted error for each stump is:

- Stump 1 (Temperature): 0.1 at 25.5 (Left: +1, Right: −1)
- Stump 2 (Humidity): 0.1 at 71.5 (Left: −1, Right: +1)
- Stump 3 (Wind Speed): 0.1 at 17.5 (Left: +1, Right: −1)

Table 5.3 Temperature midpoints left and right predictions

Threshold	Weighted Error	Left Prediction	Right Prediction
21.5	0.4	+1	−1
22.5	0.3	+1	−1
23.5	0.2	+1	−1
25.5	0.1	+1	−1
27.5	0.1	+1	−1
28.5	0.2	+1	−1
29.5	0.3	+1	−1
30.5	0.4	+1	−1

Table 5.4 Humidity midpoints left and right predictions

Threshold	Weighted Error	Left Prediction	Right Prediction
62.5	0.3	−1	+1
66.5	0.2	−1	+1
71.5	0.1	−1	+1
77.5	0.2	−1	+1
81.0	0.3	−1	+1
83.5	0.4	−1	+1
87.5	0.4	−1	+1

Table 5.5 Wind speed midpoints left and right predictions

Threshold	Weighted Error	Left Prediction	Right Prediction
10.5	0.4	+1	−1
11.5	0.3	+1	−1
14.5	0.2	+1	−1
17.5	0.1	+1	−1
18.5	0.2	+1	−1
19.5	0.3	+1	−1
20.5	0.4	+1	−1

The stump with the lowest weighted error should be selected. In this example, all stumps have the same weighted error, so we can choose any one of them. However, if the weighted errors differ, we must select the stump with the lowest error.

For this example, we will choose Stump 1 (Temperature), with a threshold of 25.5 °C and a weighted error of 0.1.

This means the Temperature stump is chosen as the first weak learner in the ensemble. Its prediction rule will be:

- Left Node: Predict +1 (Rain) if Temperature ≤ 25.5
- Right Node: Predict −1 (No Rain) if Temperature > 25.5

5.2.6 Updating the Weights

Since the Temperature stump has the lowest error, we will use it to update the weights of the misclassified instances so that the stumps in the next iteration focus more on these misclassified instances.

When we use Stump 1, which splits the data at 25.5 °C, the rule is:

- If the temperature is less than or equal to 25.5 °C, the prediction is +1 (Rain).
- If the temperature is greater than 25.5 °C, the prediction is −1 (No Rain).

The results will be as follows in Table 5.6.

Table 5.6 Results based on temperature split

Temperature (°C)	Target (Rain: +1, No Rain: −1)	Prediction
23	+1	+1
30	−1	−1
22	+1	+1
28	−1	−1
27	+1	−1
29	−1	−1
24	+1	+1
27	−1	−1
21	+1	+1
31	−1	−1

From this table, we can see that Instance 5 is the only misclassified instance.

Step 1: Calculate Weighted Error

The weighted error is the sum of the weights of the misclassified instances:

$$\text{Weighted Error} = 0.1 \ \{\text{weight of Instance } 5\}.$$

Step 2: Calculate Alpha

Alpha α determines the influence of this stump in updating the weights. It is calculated as:

$$\alpha = \frac{1}{2} \ln \frac{1 - \text{Error}}{\text{Error}} \tag{5.2}$$

Substituting the values:

$$\alpha = \frac{1}{2} \ln \frac{1 - 0.1}{0.1} \approx 1.099$$

Step 3: Update Weights

Using α, we update the weights as follows:

- If an instance is classified correctly:

$$w_i^{new} = w_i \cdot e^{-\alpha} \tag{5.3}$$

- If an instance is classified incorrectly:

$$w_i^{new} = w_i \cdot e^{\alpha} \tag{5.4}$$

- For Instance 5 (misclassified):

$$w_5^{new} = 0.1 \cdot e^{1.099} = 0.3$$

For correctly classified instances (e.g., Instance 1, 2, etc.):

$$w_i^{new} = 0.1 \cdot e^{-1.099} = 0.033$$

Step 4: Normalize Weights

Normalize the weights so their sum equals 1:
Total Weight = (9·0.033) + 0.3 = 0.597.
Normalized weights:

- For Instance 5:

$$w_5^{new} = \frac{0.3}{0.597} = 0.502$$

- For other instances:

$$w_i^{new} = \frac{0.3}{0.597} = 0.055$$

Final weights are shown in Table 5.7.

Table 5.7 Final weights after the update

Instance	Temperature (°C)	Humidity (%)	Wind Speed (km/h)	Target (Rain: +1, No Rain: −1)	Weight
1	23	80	10	+1	0.055
2	30	60	20	−1	0.055
3	22	85	12	+1	0.055
4	28	60	21	−1	0.055
5	27	75	11	+1	0.502
6	29	65	19	−1	0.055
7	24	82	18	+1	0.055
8	27	68	18	−1	0.055
9	21	90	17	+1	0.055
10	31	85	18	−1	0.055

In the next iteration, the stump will focus more on Instance 5, as its weight is significantly higher. This ensures the model learns from the error and improves classification for the challenging instances in subsequent iterations.

5.2.7 The Second Iteration

In the second iteration, the stumps will focus more on the misclassified instances from the previous iteration. These misclassified instances now have higher weights, making them more influential in determining the splits.

We have three stumps, each corresponding to a feature (Temperature, Humidity, and Wind Speed). The weighted errors for each stump's thresholds are as follows in Tables 5.8, 5.9, and 5.10.

5.2.8 Select the Best Stump

The minimum error for each stump is:

Table 5.8 Stump 1 (Temperature)

Threshold	Weighted Error	Left Prediction	Right Prediction
21.5	0.275	1	1
22.5	0.275	1	1
23.5	0.275	1	1
25.5	0.275	1	1
27.5	0.055	1	−1
28.5	0.110	1	−1
29.5	0.165	1	−1
30.5	0.220	1	−1

Table 5.9 Stump 2 (Humidity)

Threshold	Weighted Error	Left Prediction	Right Prediction
62.5	0.165	−1	1
66.5	0.110	−1	1
71.5	0.055	−1	1
77.5	0.275	1	1
81.0	0.275	1	1
83.5	0.275	1	1
87.5	0.275	1	1

Table 5.10 Stump 3 (Wind Speed)

Threshold	Weighted Error	Left Prediction	Right Prediction
10.5	0.275	1	1
11.5	0.165	1	−1
14.5	0.110	1	−1
17.5	0.055	1	−1
18.5	0.110	1	−1
19.5	0.165	1	−1
20.5	0.220	1	−1

Table 5.11 Results based on humidity split

Instance	Humidity (%)	Target (Rain: +1, No Rain: −1)	Prediction
1	80	+1	+1
2	60	−1	−1
3	85	+1	+1
4	60	−1	−1
5	75	+1	+1
6	65	−1	−1
7	82	+1	+1
8	68	−1	−1
9	90	+1	+1
10	85	−1	+1

- Temperature: 0.055 at a threshold of 27.5.
- Humidity: 0.055 at a threshold of 71.5.
- Wind Speed: 0.055 at a threshold of 17.5.

Since all errors are equal, we can choose any of these stumps. However, to ensure diversity in the ensemble and avoid reusing the same feature consecutively, we will select Stump 2 (Humidity), which splits at a threshold of 71.5 with a weighted error of 0.055.

Humidity Stump Rule
The selected rule for the humidity stump is:

- If Humidity ≤ 71.5, predict -1 (No Rain).
- If Humidity > 71.5, predict +1 (Rain).

Predictions with the Humidity Stump Rule are shown in Table 5.11.

Observation:

- Instance 5: This instance was misclassified in the previous iteration but is now correctly classified using the humidity stump.
- Instance 10: This instance was correctly classified in the previous iteration but is now misclassified with this stump.

Based on the weighted error of the humidity stump (Error = 0.055), we calculate α to determine how much to adjust the weights.

Step 1: Calculate α:

The formula for α is:

$$\alpha = \frac{1}{2} \ln \frac{1 - \text{error}}{\text{error}} = \frac{1}{2} \ln \frac{1 - 0.055}{0.055} = 1.421$$

Step 2: Update Weights:

We update the weights for each instance based on whether it was correctly or incorrectly classified:

Correctly classified: The weight decreases:

$$w_{new} = w \cdot e^{-\alpha}$$

Misclassified: The weight increases:

$$w_{new} = w \cdot e^{-\alpha}$$

Step 3: Normalize Weights:

All weights are normalized so their sum equals 1:

$$w_{\text{normalized}} = \frac{w_{new}}{\sum w_{new}}$$

Final Weights (Rounded):

After updating and normalizing, the final weights are shown in Table 5.12.

Table 5.12 Final weights after the second iteration

Instance	Temperature (°C)	Humidity (%)	Wind Speed (km/h)	Target	Final weight
1	23	80	10	+1	0.0291
2	30	60	20	−1	0.0291
3	22	85	12	+1	0.0291
4	28	60	21	−1	0.0291
5	27	75	11	+1	0.266
6	29	65	19	−1	0.0291
7	24	82	18	+1	0.0291
8	27	68	18	−1	0.0291
9	21	90	17	+1	0.0291
10	31	85	18	−1	0.5

5.2.9 The Third Iteration

The third iteration repeats the process using three stumps, each corresponding to one feature, and working with the updated weights from the previous iteration. After evaluating the midpoints of each feature, the minimum errors for the splits are as follows:

- Stump 1 (Temperature): Threshold at 27.5 with a weighted error of 0.0291
- Stump 2 (Humidity): Threshold at 83.5 with a weighted error of 0.1746
- Stump 3 (Wind Speed): Threshold at 17.5 with a weighted error of 0.0291

The two candidates with the lowest errors are Stump 1 and Stump 3. Since Stump 1 (Temperature) was used in a previous iteration, we select Stump 3 (Wind Speed) for this iteration. The decision rule for this stump is as follows:

- If Wind Speed ≤ 17.5, predict +1 (Rain).
- If Wind Speed > 17.5, predict -1 (No Rain).

The predictions for this iteration are as follows in Table 5.13.
Observations

- Instance 10, which was misclassified in the previous iteration, is now correctly classified.
- Instance 7, which was previously classified correctly, is now misclassified.

Now, we compute the α of the stump, which is 1.753, and update the new weights of the instances, following the same process as before.

5.2.10 The Prediction Phase

Now, we stop the training process, having completed three iterations. This gives us three winning stumps:

Table 5.13 Results based on Wind Speed split

Instance	Wind Speed (km/h)	Target (Rain: +1, No Rain: -1)	Prediction
1	10	+1	+1
2	20	-1	-1
3	12	+1	+1
4	21	-1	-1
5	11	+1	+1
6	19	-1	-1
7	18	+1	-1
8	18	-1	-1
9	17	+1	+1
10	18	-1	-1

1. Stump 1 (Temperature):
   - Splits at 25.5.
   - Prediction: +1 (Rain) if (Temperature $\leq$ 25.5); -1 (No Rain) if (Temperature} $>$ 25.5).
   - α: 1.099.
2. Stump 2 (Humidity):
   - Splits at 71.5.
   - Prediction: -1 (No Rain) if (Humidity $\leq$ 71.5); +1 (Rain) if (Humidity $>$ 71.5).
   - α: 1.421.
3. Stump 3 (Wind Speed):
   - Splits at 17.5.
   - Prediction: +1 (Rain) if (Wind Speed $\leq$ 17.5); -1 (No Rain) if (Wind Speed $>$ 17.5).
   - α: 1.753.

To predict the label of a new instance, we follow these steps:

1. Feed the instance to all the winning stumps:
   - Each stump provides a prediction (+1 or -1) based on its split rule.
2. Multiply each prediction by its respective α:
   - This weights the influence of each stump according to its performance during training.
3. Sum the weighted predictions:
   - Compute Weighted Sum

$$\text{Weighted } Sum = \sum_{t=1}^{T} \alpha_t \, h_t(x) \tag{5.5}$$

where:

- T: Total number of weak learners
- α_t: Weight of the tth weak learner
- $h_t(x)$: Prediction of the tth weak learner

4. Apply the sign function:
   - The final prediction is

$$H(\mathrm{x}) = \text{sign}(\text{Weighted } Sum) \tag{5.6}$$

where:

- +1: If the sum is positive (Rain).
- −1: If the sum is negative (No Rain).

This ensures that the strong learners (stumps with higher α) have more influence on the final prediction. This process effectively combines the wisdom of all three weak learners into a robust final prediction.

After applying the weighted sum and the sign function to the data, we obtain the following final predictions, as shown in Table 5.14.

It is evident from this table that the model achieves a training accuracy of 100%, as it correctly classifies all instances in the dataset.

This demonstrates the power of the ensemble learning approach, where the combination of weak learners compensates for their individual weaknesses. Although each stump misclassifies some instances, the weighted voting mechanism aggregates their predictions to produce a robust final decision. This effectively resolves the errors of individual weak learners and ensures accurate predictions.

5.2.11 AdaBoost in Regression and Multiclass Classification

So far, we've explored the power of AdaBoost in refining predictions and focusing on challenging cases in binary classification. But what happens when we step outside the binary realm? What about regression problems or scenarios where the target isn't just "Rain" or "No Rain," but perhaps one of several possible categories? Let's dive in.

5.2.11.1 AdaBoost for Regression Problems

When applying AdaBoost to regression, the approach shifts slightly. Instead of focusing on classification errors, AdaBoost for regression tackles prediction accuracy by minimizing the difference between predicted and actual values, often using a loss function such as the squared error. Here, weak learners aren't trying to decide between categories; they're making continuous predictions that are aggregated into a stronger model.

The process still emphasizes adaptability. Instances where predictions are off the mark—outliers, for example—get more weight in subsequent iterations. This forces the weak learners to focus on improving predictions for these tricky cases. Imagine building an ensemble that predicts house prices. If one weak learner underestimates a house's price significantly, the subsequent learners will prioritize narrowing that gap.

The outcome? A model that captures intricate relationships in the data, balancing the under- and over-predicted values in a way that enhances overall accuracy. This makes AdaBoost a solid choice for regression tasks, provided the weak learners (often shallow decision trees) are carefully tuned to avoid overfitting.

Table 5.14 Model's final predictions

					Alphas				
					1.099	1.421	1.753		
Instance	Temperature	Humidity	Wind Speed	Target	Stump 1	Stump 2	Stump 3	Weighted Sum	Final Prediction
1	23	80	10	+1	+1	+1	+1	4.273	+1
2	30	60	20	−1	−1	−1	−1	−4.273	−1
3	22	85	12	+1	+1	+1	+1	4.273	+1
4	28	60	21	−1	−1	−1	−1	−4.273	−1
5	27	75	11	+1	−1	+1	+1	2.075	+1
6	29	65	19	−1	−1	−1	−1	−4.273	−1
7	24	82	18	+1	+1	+1	−1	0.767	+1
8	27	68	18	−1	−1	−1	−1	−4.273	−1
9	21	90	17	+1	+1	+1	+1	4.273	+1
10	31	85	18	−1	−1	+1	−1	−1.431	−1

5.2.11.2 AdaBoost for Multiclass Classification

Now, let's switch gears to multiclass classification. Here, the task involves more than two categories, like predicting whether a fruit is an apple, orange, or banana. AdaBoost extends its binary foundation to handle multiple classes through clever adaptations.

One popular approach is one-vs.-all (or one-vs.-rest). This method breaks the multiclass problem into multiple binary problems, where each class is treated as "positive" against all others. For example, "Is this fruit an apple?" becomes a binary question, and the same logic applies to "Is this fruit an orange?" and so on. Each binary model gets its own AdaBoost ensemble, and their outputs are combined to make the final decision.

Another approach is direct multiclass boosting, where a single ensemble is built to predict all classes. Instead of binary classification rules, each weak learner outputs probabilities or scores for each class. The predictions are weighted and combined to determine the class with the highest confidence. This approach is particularly elegant because it avoids the need for separate models and handles the interplay between classes directly.

5.2.12 Why AdaBoost Stands Out

What's fascinating about AdaBoost is how seamlessly it adapts to these variations. Whether it's regression, binary classification, or multiclass problems, the algorithm's core principle remains unchanged: focus on the hardest-to-predict cases, iteratively refine predictions, and let the ensemble's collective wisdom shine.

5.2.13 AdaBoost Implantation

Here is the implement of AdaBoost:

```
from sklearn.ensemble import AdaBoostClassifier
  from sklearn.tree import DecisionTreeClassifier
  from sklearn.model_selection import train_test_split
  from sklearn.metrics import accuracy_score
  import pandas as pd

  # Create the dataset
  data = {
      "Temperature": [23, 30, 22, 28, 27, 29, 24, 27, 21, 31],
      "Humidity": [80, 60, 85, 60, 75, 65, 82, 68, 90, 85],
      "Wind Speed": [10, 20, 12, 21, 11, 19, 18, 18, 17, 18],
      "Target": [1, -1, 1, -1, 1, -1, 1, -1, 1, -1]
  }
  df = pd.DataFrame(data)

  # Split the data into features (X) and target (y)
  X = df[["Temperature", "Humidity", "Wind Speed"]]
  y = df["Target"]

  # Split into training and testing sets
  X_train,   X_test,   y_train,   y_test   =   train_test_split(X,   y,
test_size=0.3, random_state=42)

  # Create and fit the AdaBoost model
  base_estimator = DecisionTreeClassifier(max_depth=1)  # Weak learner
  model        =        AdaBoostClassifier(base_estimator=base_estimator,
n_estimators=50, random_state=42)
  model.fit(X_train, y_train)

  # Make predictions and evaluate the model
  y_pred = model.predict(X_test)
  accuracy = accuracy_score(y_test, y_pred)

  accuracy, y_pred
```

Here, we use a decision tree as the weak learner, with each tree limited to a maximum depth of 1, effectively creating decision stumps. These simple classifiers form the building blocks of our ensemble model. The AdaBoostClassifier is then applied, with the decision stumps as its base estimator. We specify the number of estimators as 50 in this example, meaning the model will iteratively train 50 weak learners, combining their predictions to improve overall accuracy.

5.3 Gradient Boosting

Gradient Boosting is another powerful ensemble learning method that is based on the concept of weak learners, where the next weak learner will try to enhance the prediction of the previous weak learners. It may use stumps just like we saw in

AdaBoost, but here each weak learner will try to predict the error of each split, not the actual class (Mayr et al. 2014).

For example, in AdaBoost, the left node and the right node give the prediction as an actual class, such as +1 or −1. In Gradient Boosting, however, the predictions of the left node and the right node will try to estimate the amount of error that would result if we split the data based on the current midpoint of the current feature being examined (Biau et al. 2019; Chen et al. 2015a; Friedman 2002).

Let's walk through the process of Gradient Boosting step by step using the same data we used in AdaBoost as shown in Table 5.1.

5.3.1 Initial Prediction

The first step in Gradient Boosting is to have an initial prediction. This can be a fixed value or the mean of the target values. This is one of the differences between AdaBoost and Gradient Boosting. In AdaBoost, each iteration has its own prediction, but in Gradient Boosting, all iterations modify the initial prediction to enhance it. In this example, the initial prediction will be the mean of the target values, which is 0 for all instances. As shown in Table 5.15.

5.3.2 The Residual Gradient (Error)

Now that we have an initial prediction, we need to calculate the residual error for each instance. The residual error represents the difference between the actual class value and the predicted value. It can be expressed as:

$$\text{Residual Error} = \text{Actual Class} - \text{Prediction} \tag{5.7}$$

Table 5.15 Rainfall dataset with initial predictions

Instance	Temperature (°C)	Humidity (%)	Wind Speed (km/h)	Target	Initial Prediction
1	23	80	10	+1	0
2	30	60	20	−1	0
3	22	85	12	+1	0
4	28	60	21	−1	0
5	27	75	11	+1	0
6	29	65	19	−1	0
7	24	82	18	+1	0
8	27	68	18	−1	0
9	21	90	17	+1	0
10	31	85	18	−1	0

Table 5.16 Residual error of each instance

Instance	Actual Class	Prediction	Residual Error
1	+1	0	+1
2	−1	0	−1
3	+1	0	+1
4	−1	0	−1
5	+1	0	+1
6	−1	0	−1
7	+1	0	+1
8	−1	0	−1
9	+1	0	+1
10	−1	0	−1

Using the dataset provided, where the initial prediction is 0 for all instances, the residual error for each instance would simply be equal to the actual class value (since Prediction = 0), as shown in Table 5.16.

These residual errors will now guide the training of the next weak learner, which aims to reduce these errors in subsequent iterations.

5.3.3 Iterative Steps in Gradient Boosting

In each iteration of Gradient Boosting, we perform the following steps to refine the model and minimize the residual error:

1. Compute the Midpoints of Each Feature:
   - Identify all distinct values of a feature, sort them in ascending order, and calculate the midpoints between consecutive values. These midpoints serve as potential split points.
2. Split the Data Based on the Midpoints:
   - For each midpoint of each feature, divide the data into two nodes: left (values $\leq$ midpoint) and right (values $>$ midpoint).
3. Compute the Average of the Residuals in Each Node:
   - For the data points in the left and right nodes, calculate the average of their residual errors.
4. Compute the Squared Error for Each Node:
   - For both the left and right nodes, compute the squared error using the formula:

$$\text{Squared Error} = \sum (\text{Residual} - \text{Average Residual})^2 \tag{5.8}$$

5. Compute the Total Error of the Split:
   - Sum up the squared errors of the left and right nodes to get the total error for the split.
6. Pick the Split Point with the Lowest Total Error:
   - Among all possible splits for all features, select the split point that minimizes the total error.
7. Split the Data Based on the Best Splitting Point:
   - Divide the dataset into left and right nodes using the best split point identified in the previous step.
8. Update Predictions for the Left Node:
   - Multiply the learning rate α by the average residual of the left node, and add this amount to the predictions of all data points in the left node.
9. Update Predictions for the Right Node:
   - Multiply the learning rate α by the average residual of the right node, and add this amount to the predictions of all data points in the right node.
10. Compute the Residual Error for the Next Iteration:
    - Recalculate the residual errors for all data points using the updated predictions:

$$\text{Residual Error} = \text{Actual Class} - \textit{New} \text{ Prediction}$$

These steps are repeated iteratively, with each weak learner focusing on the residual errors left by the previous learners. Over time, the combined model converges toward minimizing the total error and producing accurate predictions.

Iteration 1:

After computing the midpoints for each feature, the left and right predictions, and the total errors, the midpoint that minimizes the total error is selected.

Calculating for the Midpoint 21.5 on Temperature

1. Split the Data Based on the Midpoint 21.5:
   - Left Node: Includes instance 9 (Temperature ≤ 21.5).
   - Right Node: Includes all other instances (1, 2, 3, 4, 5, 6, 7, 8, 10).

2. Calculate Predictions for Each Node:
   - Left Node: The prediction is the average of the residuals for the left node:

$$\text{Prediction (Left Node)} = \frac{\textit{Sum} \text{ of Residuals in Left Node}}{\text{Number of Instances in Left Node}} = \frac{1}{1} = 1$$

- Right Node: The prediction is the average of the residuals for the right node:

$$\text{Prediction (Right Node)} = \frac{\textit{Sum of Residuals in Right Node}}{\text{Number of Instances in Right Node}}$$
$$= \frac{1 + (-1) + 1 + (-1) + 1 + (-1) + 1 + (-1) + (-1)}{9} = -0.111$$

3. Calculate Squared Errors for Each Node:

- Left Node:
  Residual = 1, Residual Prediction = 1.
  Squared Error (Left Node) = $(1 - 1)^2 = 0$.
- Right Node:
  Residuals = (1, −1, 1, −1, 1, −1, 1, −1, −1), Residual Prediction = −0.111.
  Squared Error (Right Node) = $\sum_{i=1}^{9} (\text{Residual}_i - \text{Prediction})^2$
  Substituting values:

$$(1 - (-0.111))^2 + (-1 - (-0.111))^2 + (1 - (-0.111))^2 + (-1 - (-0.111))^2$$
$$+ (1 - (-0.111))^2 + (-1 - (-0.111))^2 + (1 - (-0.111))^2$$
$$+ (-1 - (-0.111))^2 + (-1 - (-0.111))^2 = 8.888$$

4. Total Error for the Split:

- The total error is the sum of the squared errors for the left and right nodes:

$$\text{Total Error} = 0 + 8.888 = 8.888$$

Data after computing errors for all midpoints is shown in Table 5.17.

In this case, there are several options with the lowest error of 3.333, and we choose the midpoint 25.5 on the Temperature feature.

The Selected Split
- Feature: Temperature
- Midpoint: 25.5
- Left Prediction: 1.0
- Right Prediction: −0.666

Splitting the Data
The data is now split based on the Temperature ≤ 25.5:

Table 5.17 Total error for all midpoints

Feature	Midpoints	Left Prediction	Right Prediction	Total Error
Temperature	21.5	1	−0.111	8.888
Temperature	22.5	1	−0.25	7.5
Temperature	23.5	1	−0.428	5.714
Temperature	25.5	1	−0.666	3.333
Temperature	27.5	0.666	−1	3.333
Temperature	28.5	0.428	−1	5.714
Temperature	29.5	0.25	−1	7.5
Temperature	30.5	0.111	−1	8.888
Humidity	62.5	−1	0.25	7.5
Humidity	66.5	−1	0.428	5.714
Humidity	71.5	−1	0.666	3.333
Humidity	77.5	−0.6	0.6	6.4
Humidity	81	−0.333	0.5	8.333
Humidity	83.5	−0.142	0.333	9.523
Humidity	87.5	−0.111	1	8.888
Wind Speed	10.5	1	−0.111	8.888
Wind Speed	11.5	1	−0.25	7.5
Wind Speed	14.5	1	−0.428	5.714
Wind Speed	17.5	1	−0.666	3.333
Wind Speed	18.5	0.428	−1	5.714286
Wind Speed	19.5	0.25	−1	7.5
Wind Speed	20.5	0.111	−1	8.888

- Instances where Temperature ≤ 25.5 will belong to the left node.
- Instances where Temperature > 25.5 will belong to the right node.

Updating Predictions

Let the learning rate α be a constant, e.g., 0.1. The new predictions are updated as follows:

1. For instances in the left node:
   New Prediction = Old Prediction + ($\alpha \times 1.0$)
   Since the old prediction is 0, this becomes:
   New Prediction (Left Node) = $0 + (0.1 \times 1.0) = 0.1$
2. For instances in the right node:
   New Prediction = Old Prediction + ($\alpha \times -0.666$)
   Since the old prediction is 0, this becomes:
   New Prediction (Right Node) = $0 + (0.1 \times -0.666) = -0.0666$

Updated Predictions

The updated predictions for all instances based on the split are shown in Table 5.18.

Table 5.18 Updated predictions based on the temperature split

Instance	Temperature (°C)	Old Prediction	Node	New Prediction
1	23	0	Left Node	0.1
2	30	0	Right Node	−0.0666
3	22	0	Left Node	0.1
4	28	0	Right Node	−0.0666
5	27	0	Right Node	−0.0666
6	29	0	Right Node	−0.0666
7	24	0	Left Node	0.1
8	27	0	Right Node	−0.0666
9	21	0	Left Node	0.1
10	31	0	Right Node	−0.0666

Table 5.19 New predictions and the residual error

Instance	Actual Class	New Prediction	Residual Error
1	+1	0.1	+0.9
2	−1	−0.0666	−0.9334
3	+1	0.1	+0.9
4	−1	−0.0666	−0.9334
5	+1	−0.0666	+1.0666
6	−1	−0.0666	−0.9334
7	+1	0.1	+0.9
8	−1	−0.0666	−0.9334
9	+1	0.1	+0.9
10	−1	−0.0666	−0.9334

Recalculating the Residual Errors

The new residual errors are calculated as:

$$\text{Residual Error} = \text{Actual Class} - \textit{New} \text{ Prediction}$$

New predictions and the residual error are shown in Table 5.19.

This concludes the first iteration. The process will repeat, with the next weak learner focusing on minimizing the new residual errors.

After updating the predictions for all instances and applying the sign activation function, we classify each instance based on the following rule:

$$\text{Classified Prediction} = \begin{cases} +1 \text{ if Prediction} \geq 0 \\ -1 \text{ if Prediction} < 0 \end{cases} \tag{5.9}$$

Using the updated predictions from the split on Temperature ≤ 25.5, the results are shown in Table 5.20.

Table 5.20 Updated predictions from the split on Temperature

Instance	Temperature (°C)	New Prediction	Classified Prediction	Actual Class	Correct Classification?
1	23	0.1	+1	+1	Yes
2	30	−0.0666	−1	−1	Yes
3	22	0.1	+1	+1	Yes
4	28	−0.0666	−1	−1	Yes
5	27	−0.0666	−1	+1	No
6	29	−0.0666	−1	−1	Yes
7	24	0.1	+1	+1	Yes
8	27	−0.0666	−1	−1	Yes
9	21	0.1	+1	+1	Yes
10	31	−0.0666	−1	−1	Yes

Observation

- Instance 5 is misclassified.
  - Actual Class: +1
  - Classified Prediction: −1

This indicates that although the model has improved overall, there are still residual errors that need to be corrected in subsequent iterations. Instance 5 will receive a higher residual weight in the next iteration, ensuring that the model focuses on reducing its error.

Iteration 2

In the second iteration, based on the updated predictions and pseudo-residuals from the previous iteration, the process continues to minimize the residual errors.

Step 1: Determine the Best Split

From the computed errors for all midpoints across all features, results are shown in Table 5.21.

the split with the lowest total error is:

- Feature: Wind Speed
- Midpoint: 17.5
- Left Prediction: 0.941666
- Right Prediction: −0.627777
- Total Error: 2.821759

Step 2: Update Predictions for Each Node

Using the learning rate α of 0.1, the predictions are updated as follows:

- Left Node (Wind Speed $\leq$ 17.5):

$$\textit{New}\ \text{Prediction} = \textit{Old}\ \text{Prediction} + (\alpha \times \text{Left Prediction}) = \textit{Old}\ \text{Prediction} + (0.1 \times 0.941666)$$

Table 5.21 Total error of all the midpoints

Feature	Midpoint	Left Prediction	Right Prediction	Total Error
Temperature	21.5	0.9	−0.1	7.833
Temperature	22.5	0.9	−0.225	6.708
Temperature	23.5	0.9	−0.385	5.261
Temperature	25.5	0.9	−0.6	3.333
Temperature	27.5	0.622	−0.933	2.925
Temperature	28.5	0.4	−0.933	5
Temperature	29.5	0.233	−0.933	6.555
Temperature	30.5	0.103	−0.933	7.765
Humidity	62.5	−0.933	0.233	6.555
Humidity	66.5	−0.933	0.4	5
Humidity	71.5	−0.933	0.622	2.925
Humidity	77.5	−0.533	0.533	5.888
Humidity	81	−0.294	0.441	7.43
Humidity	83.5	−0.123	0.288	8.375
Humidity	87.5	−0.1	0.9	7.833
Wind Speed	10.5	0.9	−0.1	7.833
Wind Speed	11.5	0.983	−0.245	6.315
Wind Speed	14.5	0.955	−0.409	4.820
Wind Speed	17.5	0.941	−0.627	2.821
Wind Speed	18.5	0.4	−0.933	5
Wind Speed	19.5	0.233	−0.933	6.555
Wind Speed	20.5	0.103	−0.933	7.765

This results in adding 0.094166 to the predictions for instances in the left node.

Right Node (Wind Speed > 17.5):

$$\textit{New}\ \text{Prediction} = \textit{Old}\ \text{Prediction} + (\alpha \times \text{Right Prediction}) = \textit{Old}\ \text{Prediction} + (0.1 \times -0.627777)$$

This results in subtracting 0.0627777 from the predictions for instances in the right node.

Step 3: Compute Updated Predictions and Residuals

Based on the updated predictions, the residuals (errors) are recalculated as:

$$\text{Pseudo} - \text{Residual} = \text{Target} - \text{Prediction}$$

The updated predictions and residuals for all instances are shown in Table 5.22.

Table 5.22 Results of Iteration 2

Instance	Temperature	Humidity	Wind Speed	Target	Residual	Prediction
1	23	80	10	+1	+0.805	0.194
2	30	60	20	−1	−0.870	−0.129
3	22	85	12	+1	+0.805	0.194
4	28	60	21	−1	−0.870	−0.129
5	27	75	11	+1	+0.972	0.027
6	29	65	19	−1	−0.870	−0.129
7	24	82	18	+1	+0.962	0.037
8	27	68	18	−1	−0.870	−0.129
9	21	90	17	+1	+0.805	0.194
10	31	85	18	−1	−0.870	−0.129

Table 5.23 Model's final results

Instance	Target	Prediction	Classified Prediction	Correct?
1	+1	0.194167	+1	Yes
2	−1	−0.129444	−1	Yes
3	+1	0.194167	+1	Yes
4	−1	−0.129444	−1	Yes
5	+1	0.027500	+1	Yes
6	−1	−0.129444	−1	Yes
7	+1	0.037222	+1	Yes
8	−1	−0.129444	−1	Yes
9	+1	0.194167	+1	Yes
10	−1	−0.129444	−1	Yes

Observations

- The updated predictions bring us closer to minimizing the residuals.
- Instances like 5 and 7 are still showing relatively large residuals, meaning the model is focusing more on correcting their errors in the next iteration.
- The total error for this iteration is significantly reduced compared to the first iteration, indicating model improvement.

After applying the sign activation function to the predictions, the model achieves perfect classification on the training dataset. Despite some instances (such as 5 and 7) having relatively high residuals, the final classifications align perfectly with the actual target values as shown in Table 5.23.
Observations:

- 100% Training Accuracy: With only two iterations, the model classifies all instances correctly, demonstrating the efficiency of Gradient Boosting in minimizing errors iteratively.
- Residual Reduction: While the residuals for some instances, such as 5 and 7, remain relatively high, they no longer affect the classification results after applying the sign activation function.

Options Going Forward:

1. Stop Here: The model already achieves 100% training accuracy. Stopping here avoids overfitting and is often a practical choice when the training accuracy is sufficient.
2. Continue Iterations: We can further reduce the residuals to improve the model's fit, particularly for challenging instances. This might improve the generalization on unseen data but risks overfitting if the training error is reduced excessively.

5.3.4 Implantation

Below is the Python code to classify the data using Gradient Boosting:

```
import pandas as pd
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Create the dataset
data1 = {
    "Temperature": [23, 30, 22, 28, 27, 29, 24, 27, 21, 31],
    "Humidity": [80, 60, 85, 60, 75, 65, 82, 68, 90, 85],
    "Wind Speed": [10, 20, 12, 21, 11, 19, 18, 18, 17, 18],
    "Target": [1, -1, 1, -1, 1, -1, 1, -1, 1, -1],
}

# Convert data to a pandas DataFrame
df = pd.DataFrame(data1)

# Split the data into features (X) and target (y)
X = df[["Temperature", "Humidity", "Wind Speed"]]
y = df["Target"]

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Create and fit the Gradient Boosting model
model = GradientBoostingClassifier(n_estimators=50, learning_rate=0.1,
max_depth=3, random_state=42)
model.fit(X_train, y_train)

# Make predictions and evaluate the model
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

# Output the results
print("Predictions:", y_pred)
print("Accuracy:", accuracy)
```

5.4 AdaBoost and Gradient Boosting with Images

In image processing tasks, AdaBoost and Gradient Boosting play specialized roles, especially in classification, segmentation, and object detection. Their effectiveness hinges on the choice of weak learners and the iterative processes of splitting and error calculation, which are uniquely tailored to the characteristics of visual data (Natekin and Knoll 2013; Wade and Glynn 2020).

Both algorithms often employ decision stumps or shallow trees as weak learners. These simple models are particularly suited to image tasks, as they can capture fundamental patterns like edge contrasts, textures, or localized features. For instance, a decision stump might classify image regions based on thresholds for color intensity or gradient magnitude. While these individual decisions are rudimentary, their aggregation within an ensemble creates a robust model capable of identifying complex visual patterns.

In training these models, image preprocessing is critical. Each image is transformed into feature representations, such as raw pixel intensities, edge maps, or higher level descriptors like histograms of oriented gradients (HOG). The dataset of images is then split iteratively during training, with each weak learner determining thresholds for these features to divide the data into groups. For example, a decision stump might split images based on a threshold value for average brightness, grouping darker images separately from brighter ones.

These splits do not divide the images physically but instead split the dataset based on the features extracted from the images. For instance, in a dataset labeled as "Day" or "Night," the feature could be average intensity. A weak learner might split the dataset at a threshold intensity of 100, grouping images below this threshold as "Night" and those above as "Day." The goal of these splits is to create groups that are as homogeneous as possible with respect to their labels, minimizing classification errors.

Error calculations guide the refinement of these splits and the overall learning process. In AdaBoost, after a weak learner applies a split, the algorithm evaluates the misclassified images and assigns higher weights to these challenging areas. For example, if shadowy regions of an object are frequently misclassified, subsequent learners focus on these regions, refining the ensemble's accuracy in identifying such subtle patterns.

Gradient Boosting, in contrast, emphasizes modeling residual errors. In an image denoising task, for instance, the residual error might represent the remaining noise after an initial weak learner processes the image. The next learner then attempts to predict and correct these residuals. Each iteration minimizes the total error, such as mean squared error for pixel intensities or segmentation boundary mismatches, progressively improving the model's performance.

These iterative processes of splitting and error correction are particularly effective for image data, where overlapping or ambiguous features like shadows, textures, or similar colors are common. By dynamically adapting to the dataset and focusing on challenging aspects of the images, AdaBoost and Gradient Boosting excel at

capturing both broad visual structures and intricate details. Over iterations, these algorithms refine their ensembles to achieve high accuracy in diverse tasks, from object detection to segmentation, highlighting their adaptability and power in image processing.

5.5 Concluding Remarks

In this chapter, we explored ensemble learning, a methodology where the strength of many simple models is combined to tackle complex challenges in machine learning. At its core, ensemble learning demonstrates the power of collaboration, where individual models, though limited on their own, can achieve exceptional results when brought together.

Through our deep dive into techniques like Adaptive Boosting (AdaBoost) and Gradient Boosting, we saw how weak learners—models that only slightly outperform random guessing—can be iteratively refined and strategically weighted to create robust, high-performing ensembles. These methods exemplify the adaptability and precision that ensemble learning offers, making it a cornerstone in the toolkit of any machine learning practitioner.

AdaBoost's ability to dynamically focus on misclassified instances and Gradient Boosting's approach of minimizing residual errors through stepwise refinements illustrate the versatility and sophistication of these algorithms. Their application across a range of domains—from numerical datasets to image processing—further underscores their transformative potential.

Chapter 6
Unsupervised Learning: Clustering and Dimensionality Reduction



Abstract This chapter explores unsupervised learning, a machine learning approach that uncovers hidden patterns in unlabeled data. We introduce key techniques including K-Means Clustering, which groups data based on similarity, and DBSCAN, a density-based method adept at detecting irregularly shaped clusters and handling noise. Through real-world examples—such as segmenting coffee shop customers and analyzing taxi traffic—we illustrate how these algorithms reveal valuable insights for decision-making and optimization.

Additionally, we introduce Principal Component Analysis (PCA), a dimensionality reduction technique that transforms high-dimensional data into a more manageable form while preserving essential information. PCA enhances interpretability and computational efficiency, making it a vital tool for preprocessing complex datasets. This chapter provides a foundational understanding of how unsupervised learning techniques can be applied to explore structure, detect patterns, and simplify data in a wide range of applications.

6.1 K-Means Clustering

All the methods and approaches we have covered up until now were in the realm of supervised learning, where we have data along with the correct labels. The goal in supervised learning is to train a model to understand the features of the data and relate them to the correct labels so that, in the future, when it encounters new data, it can predict or classify the data based on the knowledge it gained during the training phase.

But life isn't always so neatly labeled, is it? Imagine walking into a party where you don't know anyone. You can't tell who's friends with whom just by looking, but as the night goes on, you notice patterns: a group laughing together in one corner, a few people intensely discussing books in another, and a couple of dancers by the stereo. Without anyone explicitly telling you, you've mentally clustered these people into groups based on their behavior, conversations, or other observable traits (Barlow 1989; Naeem et al. 2023). This is essentially the idea behind unsupervised

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_6

learning—learning patterns and structures in data without needing explicit labels (Mirkin 2005; J. Wu 2012).

In this chapter, we will explore this new paradigm of machine learning. Here, we have data but no correct labels. The goal is to enable a model to analyze the data, recognize patterns, and provide us with useful insights.

Let's start with one of the most popular techniques in unsupervised learning: K-Means clustering. K-Means is a simple yet powerful algorithm used to group similar data points into clusters based on a similarity metric, such as distance. But what does that really mean?

Imagine you're a teacher organizing students into study groups. You don't have any information about their academic strengths, but you do have their responses to a questionnaire about their interests. One student loves math puzzles, another enjoys building robots, and a third is passionate about poetry. Your goal is to form groups where students with similar interests are grouped together. K-Means does something similar, but instead of students, it works with data points in a multidimensional space.

Here's how it works:

1. Choose the number of clusters (K): First, decide how many groups you want to divide the data into. This is a bit like the teacher deciding that three study groups are enough.
2. Initialize centroids: Place K "centroids" (think of them as the central points of each cluster) randomly in the data space. Don't worry if they don't make sense at first—this is just the starting point.
3. Assign data points to clusters: Each data point is assigned to the cluster whose centroid it is closest to. It's like students gravitating toward the study group that aligns most with their interests.
4. Recalculate centroids: After assigning all the data points, adjust the centroids to be the mean of all the points in their cluster. This is like a group shifting its focus based on who's in it—if most students in a group love math puzzles, their discussions might naturally center on math.
5. Repeat steps 3 and 4: This process is repeated until the centroids no longer shift significantly, meaning the clusters have stabilized. At this point, you have your final groups.

6.1.1 Clustering Coffee Shop Customers

Imagine a coffee shop owner who wants to better understand their customers based on their internet usage and time spent in the café. The manager decides to divide customers into three clusters:

1. Quick Visitors—Customers who spend little time and use minimal internet.
2. Moderate Users—Customers who spend a moderate amount of time and use a decent amount of internet.
3. Heavy Users—Customers who spend long hours in the café and consume a lot of data.

This clustering could help the manager make strategic decisions, such as designing personalized offers or optimizing the seating arrangement. For instance:

- Quick Visitors might appreciate grab-and-go drinks or small snacks.
- Moderate Users might prefer sandwiches or light meals.
- Heavy Users may be interested in full meals, as they might be working or studying for extended periods.

Now, let's analyze the data using K-Means clustering. Here's the dataset the coffee shop collected shown in Table 6.1:

6.1.1.1 Step 1: Initializing Centroids

K-Means begins by initializing three centroids, which serve as the starting points for the clusters. In this example, let's randomly choose the following initial centroids:

- Centroid 1: (120, 500)
- Centroid 2: (145, 600)
- Centroid 3: (300, 1500)

These centroids are simply points in the data space, and they don't have to align with any actual data points initially. The algorithm will iteratively adjust these centroids to better reflect the data structure.

Table 6.1 Coffee shop dataset

Time online (minutes)	Data usage (MB)
10	50
15	100
300	1500
320	1600
120	500
130	550
12	55
145	600
250	1000
155	700
350	1500
355	1200

6.1.1.2 Step 2: Compute the Euclidean Distance Between Each Data Point and the Centroids

The next step is to calculate the distance between each data point and all the centroids. The algorithm uses Euclidean distance, which is calculated using the following formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{6.1}$$

Where:

- d is the distance between the data point (x_1, y_1) and the centroid (x_2, y_2).
- (x_1, y_1) are the coordinates of the data point.
- (x_2, y_2) are the coordinates of the centroid.

Let's calculate the distances for the first data point (10, 50) to all three centroids:

1. Distance to Centroid 1 (120, 500):

$$d = \sqrt{(120 - 10)^2 + (500 - 50)^2} \approx 463.13$$

2. Distance to Centroid 2 (145, 600):

$$d = \sqrt{(145 - 10)^2 + (600 - 50)^2} \approx 566.30$$

3. Distance to Centroid 3 (300, 1500):

$$d = \sqrt{(300 - 10)^2 + (1500 - 50)^2} \approx 1478.66$$

Assigning the Point to the Closest Centroid
Now that we have the distances:

- Distance to Centroid 1: 463.13
- Distance to Centroid 2: 566.30
- Distance to Centroid 3: 1478.66

Since the smallest distance is 463.13, the data point (10, 50) is assigned to Centroid 1.

Repeating for All Data Points
This process is repeated for all the data points in the dataset. Each data point is assigned to the centroid with the smallest Euclidean distance. Figure 6.1 shows the

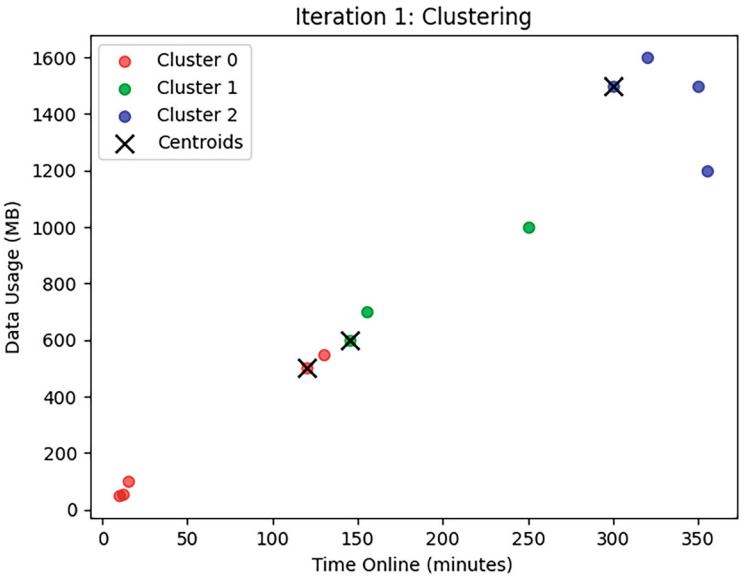


Fig. 6.1 Initial clustering of coffee shop customers

data points (colored based on their assigned clusters) and the centroids (black X marks) after the first iteration of clustering.

- Cluster 0 (Red): Represents the group of quick visitors with short time spent and low data usage.
- Cluster 1 (Green): Represents moderate users who spend a moderate amount of time and consume moderate data.
- Cluster 2 (Blue): Represents heavy users who spend the most time online and consume the largest amount of data.

6.1.1.3 Step 3: Update the Centroids

Now that we have assigned all the data points to their closest centroids, we update the centroids. The new centroid for each cluster is the average (mean) position of the data points in that cluster.

Here are the clusters after the first iteration:

- Cluster 0: (10, 50), (15, 100), (120, 500), (130, 550), (12, 55)
- Cluster 1: (145, 600), (250, 1000), (155, 700)
- Cluster 2: (300, 1500), (320, 1600), (350, 1500), (355, 1200)

We compute the average of each cluster to find the new centroids:

- Centroid 0:

$$\text{Mean} = \left(\frac{10+15+120+130+12}{5}, \frac{50+100+500+550+55}{5} \right) = (57.4, 251.0)$$

- Centroid 1:

$$\text{Mean} = \left(\frac{145+250+155}{3}, \frac{600+1000+700}{3} \right) = (183.33, 766.66)$$

- Centroid 2:

$$\text{Mean} = \left(\frac{300+320+350+355}{4}, \frac{1500+1600+1500+1200}{4} \right) = (331.25, 1450.0)$$

6.1.1.4 Step 4: Repeat Steps 2 and 3 Until Convergence

The K-Means algorithm now repeats the following steps:

1. Compute the distance between all data points and the new centroids.
2. Reassign each data point to the closest centroid.
3. Update the centroids based on the new cluster assignments.

This process continues until a stopping criterion is met. The stopping criteria could be:

- The centroids no longer move significantly between iterations (convergence).
- A maximum number of iterations is reached.

Here are the clusters after the last iteration:

- Cluster 0: (10, 50), (15, 100), (12, 55)
- Cluster 1: (145, 600), (250, 1000), (155, 700), (120, 500), (130, 550)
- Cluster 2: (300, 1500), (320, 1600), (350, 1500), (355, 1200)

In Fig. 6.2a, the second iteration shows the updated centroids and the reassignment of data points to their respective clusters. At this stage, the centroids have moved closer to the center of each cluster, reflecting a better alignment with the data points. Moving to Fig. 6.2b, the third iteration shows further adjustments as the centroids continue to shift. The data points progressively align more closely to their respective centroids, resulting in tighter groupings within each cluster.

By Fig. 6.2c, the final iteration is reached. At this point, no data points switch clusters, and the centroids have stabilized. This indicates that the algorithm has converged, meaning the positions of the centroids remain unchanged, and all points are optimally assigned to their clusters.

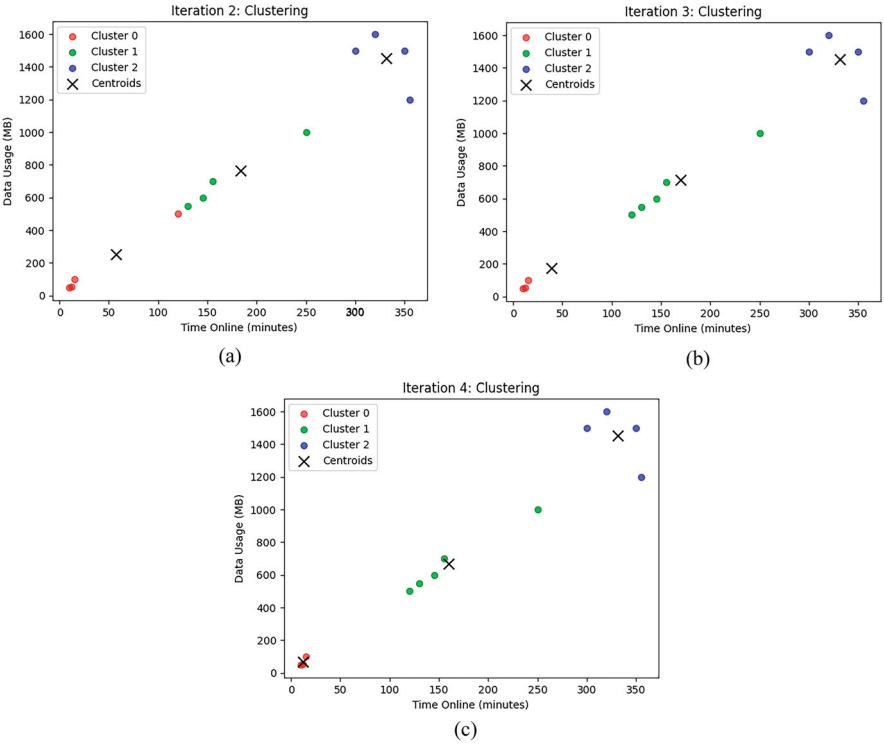


Fig. 6.2 Clustering progress after the second iteration, (**a**), third iteration, (**b**), and fourth iteration, (**c**), showing centroids moving closer to the center of their respective clusters

Throughout each iteration, it becomes evident that the data points switch clusters to move closer to their new centroids, while the centroids themselves gradually shift toward the center of their respective clusters. By the fourth iteration, the clustering process stabilizes, as all points remain in their assigned clusters, and the centroids no longer move.

At this stage, the K-Means algorithm has successfully grouped the coffee shop customers into three meaningful clusters: Quick Visitors, Moderate Users, and Heavy Users. This outcome provides the manager with actionable insights to tailor offerings and services to each customer group effectively.

6.1.2 Drawbacks of K-Means

While K-Means is a popular and powerful algorithm, it's not without its limitations. First, K-Means requires us to specify the number of clusters, K, upfront. But what if

we don't know how many natural groups exist? Picking the wrong K could lead to less meaningful results.

Additionally, the algorithm is sensitive to the placement of initial centroids. Starting with poor initial positions can lead to suboptimal clusters or require more iterations to converge. Think of it like finding your way to the center of a maze—if you start too far off, it'll take longer to reach your goal. K-Means is also affected by outliers, or unusual data points, which can drag centroids away from where they "should" be. For example, one extremely heavy internet user in our coffee shop might skew the results for an entire cluster.

6.1.3 K-Means Implantation

Here is the implementation of the K-Means algorithm.

```
import random
import matplotlib.pyplot as plt

data = [
    [10, 50],
    [15, 100],
    [300, 1500],
    [320, 1600],
    [120, 500],
    [130, 550],
     [12, 55],
    [145, 600],
     [250, 1000],
    [155, 700],
     [350, 1500],
    [355, 1200]

]

# Helper function to calculate Euclidean distance
def euclidean_distance(point1, point2):
    return ((point1[0] - point2[0])**2 + (point1[1] - point2[1])**2)**0.5

# Function to plot the data and centroids
def plot_clusters(data, centroids, clusters, title):
    colors = ['r', 'g', 'b', 'c', 'm', 'y']  # Use different colors for
different clusters

    # Plot data points
    for cluster_id, points in clusters.items():
        x_coords = [point[0] for point in points]
        y_coords = [point[1] for point in points]
        plt.scatter(x_coords, y_coords, c=colors[cluster_id],
label=f'Cluster {cluster_id}', alpha=0.6)

    # Plot centroids
    centroid_x = [centroid[0] for centroid in centroids]
    centroid_y = [centroid[1] for centroid in centroids]
    plt.scatter(centroid_x, centroid_y, c='black', marker='x', s=100,
label='Centroids')

    plt.title(title)
    plt.xlabel('Time Online (minutes)')
    plt.ylabel('Data Usage (MB)')
    plt.legend()
    plt.show()

# K-Means function
def kmeans(data, k, max_iterations=100):
    # Step 1: Initialize centroids randomly from the data points
    centroids = random.sample(data, k)

    for iteration in range(max_iterations):
```

```
        # Step 2: Assign points to the nearest centroid
        clusters = {i: [] for i in range(k)}  # Create empty clusters
        for point in data:
            distances = [euclidean_distance(point, centroid) for centroid
in centroids]
            closest_centroid = distances.index(min(distances))
            clusters[closest_centroid].append(point)

        # Plot the clusters and centroids after assignment
        plot_clusters(data, centroids, clusters, title=f'Iteration
{iteration + 1}: Clustering')

        # Step 3: Update centroids to the mean of assigned points
        new_centroids = []
        for i in range(k):
            if clusters[i]:  # Avoid empty cluster division by zero
                cluster_points = clusters[i]
                mean_x = sum(point[0] for point in cluster_points) /
len(cluster_points)
                mean_y = sum(point[1] for point in cluster_points) /
len(cluster_points)
                new_centroids.append([mean_x, mean_y])
            else:  # If a cluster is empty, reinitialize its centroid
randomly
                new_centroids.append(random.choice(data))

        # Check for convergence (if centroids do not change)
        if new_centroids == centroids:
            break
        centroids = new_centroids
        print("Clusters:")
        for cluster_id, points in clusters.items():
            print(f"Cluster {cluster_id}: {points}")
        print("\nCentroids:")
        for i, centroid in enumerate(centroids):
            print(f"Centroid {i}: {centroid}")

    # Final plot of clusters
    plot_clusters(data, centroids, clusters, title='Final Clustering')

    return clusters, centroids

# Apply K-Means
k = 3  # Number of clusters
clusters, centroids = kmeans(data, k)

# Output results
print("Clusters:")
for cluster_id, points in clusters.items():
    print(f"Cluster {cluster_id}: {points}")

print("\nCentroids:")
for i, centroid in enumerate(centroids):
    print(f"Centroid {i}: {centroid}")
```

6.2 Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

K-Means is not suitable for all applications. For instance, Fig. 6.3 displays taxi traffic data from New York City. If we attempt to cluster this data into two clusters—Highly Active Regions and Less Active Regions—based on taxi locations, the result is shown in Fig. 6.4.

The K-Means clustering algorithm did not provide meaningful results due to the nature of the data. Specifically, the data shows regions of high density (active regions) and low density (less active regions) in addition to some noisy data points.

This limitation arises because K-Means assumes clusters are spherical and equally dense, which does not align with the structure of the taxi location data (Celebi and Aydin 2016; Sindhu Meena and Suriya 2020).

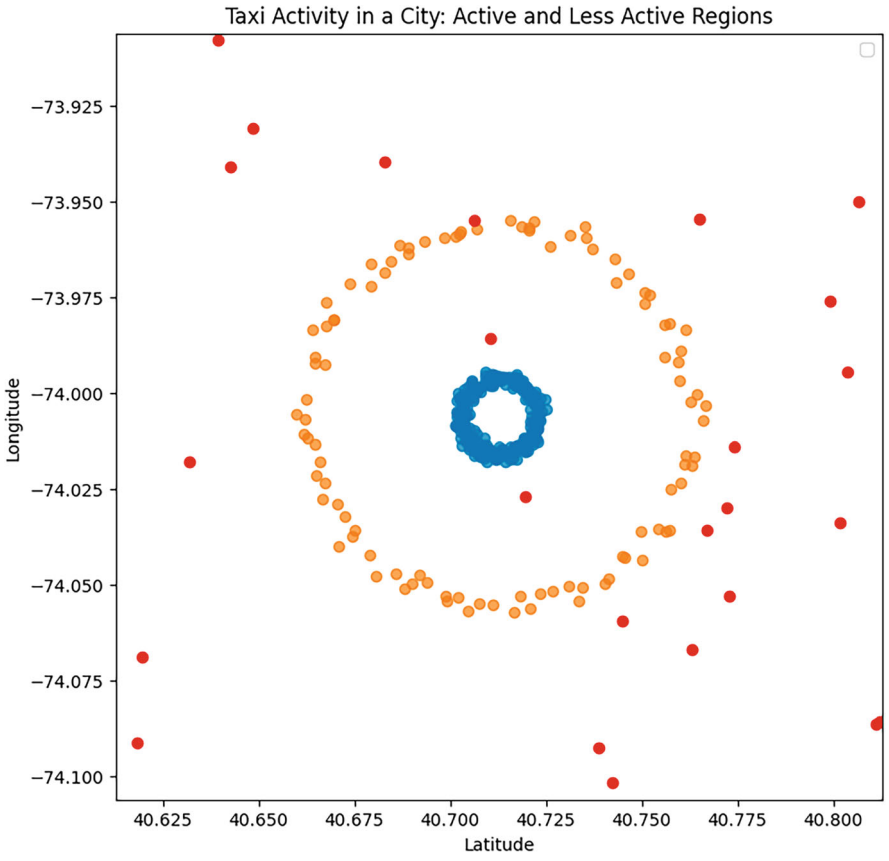


Fig. 6.3 Taxi location data in New York City

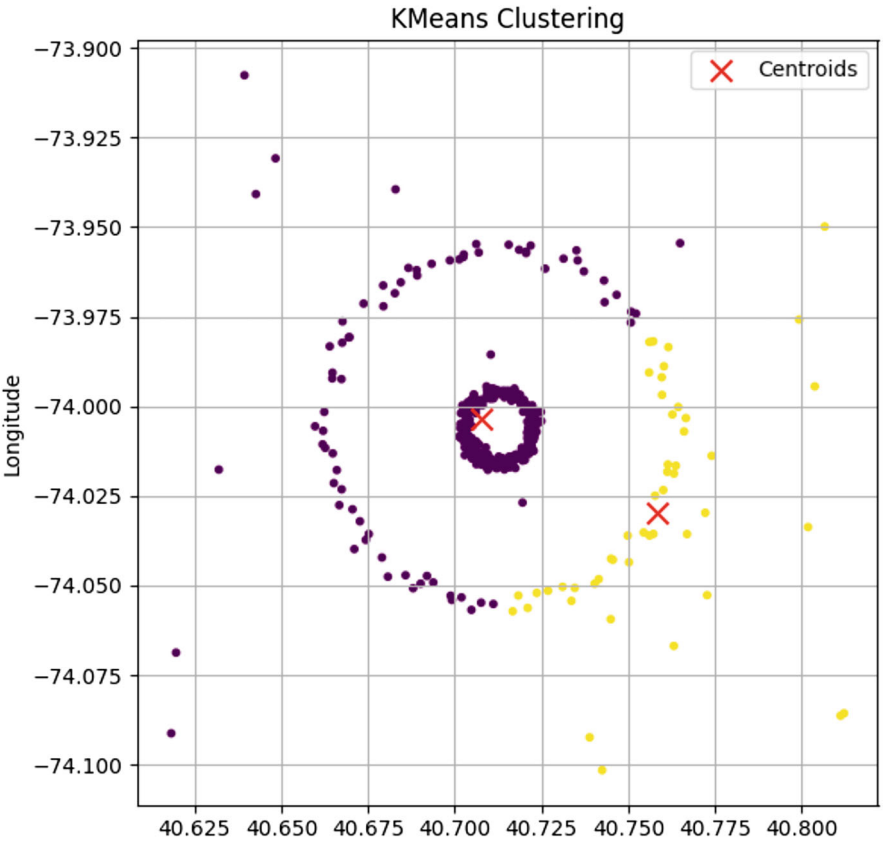


Fig. 6.4 K-Means clustering applied to taxi location data

To handle dense regions and noise, we need an alternative algorithm like DBSCAN. DBSCAN is better suited for this type of data as it identifies clusters based on density rather than shape or size, making it ideal for applications involving unevenly distributed and noisy data (Ester et al. 1996; Schubert et al. 2017; Xu and Wunsch 2005).

In DBSCAN, there are three types of points:

Core Points: Points that have at least a minimum number of neighboring points (including itself) within a specified radius.

Border Points: Points that do not meet the criteria to be core points but are within the neighborhood of a core point.

Noise Points: Points that are neither core points nor border points and are considered outliers.

To determine whether a point is a core, border, or noise point, DBSCAN relies on two parameters:

Epsilon (ε): The radius within which neighboring points are considered.
Minimum Points (MinPts): The minimum number of points (including the point itself) that must exist within the radius ε for a point to be classified as a core point.

Constrains:

If a point has at least MinPts within the radius ε, it is a core point.
If a point does not satisfy the core point condition but lies within the ε radius of a core point, it is a border point.
If a point does not satisfy either condition, it is labeled as noise (or outlier).

DBSCAN's ability to identify clusters based on density makes it highly effective for datasets with non-uniform distributions and noise, such as the taxi location data discussed earlier.

Let's take 27 points from the Taxi data and apply the DBSCAN algorithm to them, with $\varepsilon = 2.5$ and MinPts $= 6$. Below are the latitude and longitude coordinates of the points shown in Table 6.2:

Table 6.2 The latitude and longitude coordinates

	Longitude	Latitude
Point_0	−74.007155	40.7224453
Point_1	−74.009498	40.722516
Point_2	−74.00918	40.7227555
Point_3	−74.007357	40.7208115
Point_4	−74.006009	40.7210904
Point_5	−74.007126	40.7235542
Point_6	−74.007392	40.7215414
Point_7	−74.003292	40.7665837
Point_8	−74.002293	40.762694
Point_9	−74.000203	40.7643829
Point_10	−73.996826	40.7597029
Point_11	−73.991916	40.7596251
Point_12	−73.988856	40.7602451
Point_13	−73.990585	40.7559698
Point_14	−73.983495	40.7615378
Point_15	−73.994457	40.8037275
Point_16	−74.059423	40.7449325
Point_17	−74.101532	40.7424449
Point_18	−73.954742	40.7061704
Point_19	−73.930829	40.6483309
Point_20	−74.066866	40.7631101
Point_21	−74.068739	40.6196671
Point_22	−73.954495	40.7649111
Point_23	−74.013832	40.7740145
Point_24	−73.907648	40.6393788
Point_25	−73.949868	40.806597
Point_26	−73.939478	40.6829194

Table 6.3 The scaled latitude and longitude coordinates

	Longitude	Latitude
Point_0	−7400.7	4072.2
Point_1	−7400.9	4072.3
Point_2	−7400.9	4072.3
Point_3	−7400.7	4072.1
Point_4	−7400.6	4072.1
Point_5	−7400.7	4072.4
Point_6	−7400.7	4072.2
Point_7	−7400.3	4076.7
Point_8	−7400.2	4076.3
Point_9	−7400	4076.4
Point_10	−7399.7	4076
Point_11	−7399.2	4076
Point_12	−7398.9	4076
Point_13	−7399.1	4075.6
Point_14	−7398.3	4076.2
Point_15	−7399.4	4080.4
Point_16	−7405.9	4074.5
Point_17	−7410.2	4074.2
Point_18	−7395.5	4070.6
Point_19	−7393.1	4064.8
Point_20	−7406.7	4076.3
Point_21	−7406.9	4062
Point_22	−7395.4	4076.5
Point_23	−7401.4	4077.4
Point_24	−7390.8	4063.9
Point_25	−7395	4080.7
Point_26	−7393.9	4068.3

To make the calculations simpler and clearer for demonstration purposes, we scale the longitude and latitude values by a factor of 100. This scaling is not a part of the DBSCAN algorithm itself but is done here for better visualization and numerical clarity.

The scaled dataset is as follows in Table 6.3:

6.2.1 Step 1: Calculate the Distance Matrix

Using the Euclidean distance formula, we calculate the pairwise distance between each point in the dataset. The Euclidean distance between two points (x_1,y_1) and (x_2, y_2) is given by:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

This produces a distance matrix, where each entry at position (i,j) represents the distance between Point i and Point j. Below is the distance matrix for the scaled data as shown in Table 6.4:

6.2.2 Step 2: Determine Core Points and Initialize Clusters

1. Find Neighbors:
   - For each point in the dataset, calculate the number of neighboring points within a distance less than or equal to epsilon (ε). In this example, $\varepsilon = 2.5$.
   - Include the point itself as a neighbor.
2. Core Point Identification:
   - If the number of neighboring points (including itself) is greater than or equal to the minimum number of points (MinPts = 6 in this example), the point is classified as a core point.
3. Cluster Assignment:
   - If the core point is not assigned to any temporary cluster, create a new cluster and:
     - Add the core point to the cluster.
     - Add all its neighbors to a temporary working space and assign the same cluster to them temporarily.

6.2.3 Step 3: Process Remaining Points

1. Iterate Through Points:
   - Move to the next point in the dataset.
2. Neighbor Search for Each Point:
   - Find the neighbors of the current point within distance ε.
   - Check whether the point satisfies the MinPts requirement.
3. Cluster Expansion:
   Core Points:
   - If the point is already in a temporary cluster and satisfies the MinPts condition, add it permanently to the cluster.
   - Add its neighbors to the cluster and expand it.

Table 6.4 The distance matrix between points

	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14	P15	P16	P17	P18	P19	P20	P21	P22	P23	P24	P25	P26
P0	0	0.2	0.2	0.2	0.2	0.1	0.1	4.4	4.1	4.3	3.9	4	4.2	3.7	4.6	8.2	5.7	9.6	5.5	10.6	7.2	12	6.8	5.2	13	10.2	7.8
P1	0.2	0	0	0.3	0.4	0.3	0.2	4.5	4.1	4.3	3.9	4.1	4.3	3.8	4.7	8.3	5.5	9.4	5.7	10.8	7	11.9	6.9	5.2	13.1	10.3	8
P2	0.2	0	0	0.3	0.4	0.2	0.2	4.4	4.1	4.3	3.9	4.1	4.3	3.8	4.7	8.2	5.5	9.4	5.7	10.8	7	11.9	6.9	5.1	13.1	10.3	8
P3	0.2	0.3	0.3	0	0.1	0.3	0.1	4.6	4.2	4.4	4	4.2	4.4	3.9	4.7	8.4	5.7	9.7	5.5	10.5	7.3	11.8	6.9	5.4	12.9	10.3	7.8
P4	0.2	0.4	0.4	0.1	0	0.3	0.1	4.6	4.2	4.4	4	4.1	4.3	3.8	4.6	8.3	5.8	9.8	5.3	10.5	7.4	11.9	6.8	5.3	12.8	10.2	7.7
P5	0.1	0.3	0.2	0.3	0.3	0	0.2	4.3	3.9	4.1	3.8	3.9	4.1	3.6	4.5	8.1	5.6	9.6	5.5	10.7	7.2	12.1	6.7	5.1	13	10.1	7.9
P6	0.1	0.2	0.2	0.1	0.1	0.2	0	4.5	4.1	4.3	4	4.1	4.3	3.8	4.7	8.3	5.7	9.6	5.5	10.6	7.3	11.9	6.8	5.3	12.9	10.3	7.8
P7	4.4	4.5	4.4	4.6	4.6	4.3	4.5	0	0.4	0.4	0.9	1.3	1.6	1.7	2	3.8	6	10.1	7.8	13.9	6.4	16.1	4.9	1.3	15.9	6.7	10.5
P8	4.1	4.1	4.1	4.2	4.2	3.9	4.1	0.4	0	0.3	0.6	1.1	1.4	1.4	1.9	4.2	6	10.1	7.4	13.5	6.5	15.8	4.8	1.6	15.5	6.8	10.2
P9	4.3	4.3	4.3	4.4	4.4	4.1	4.3	0.4	0.3	0	0.6	1	1.2	1.3	1.7	4	6.2	10.4	7.4	13.5	6.7	16	4.6	1.7	15.6	6.6	10.2
P10	3.9	3.9	3.9	4	4	3.8	4	0.9	0.6	0.6	0	0.5	0.8	0.7	1.3	4.4	6.4	10.6	6.8	12.9	7	15.7	4.3	2.2	15	6.6	9.6
P11	4	4.1	4.1	4.2	4.1	3.9	4.1	1.3	1.1	1	0.5	0	0.3	0.4	0.9	4.4	6.9	11.1	6.5	12.7	7.5	16	3.8	2.6	14.7	6.3	9.3
P12	4.2	4.3	4.3	4.4	4.3	4.1	4.3	1.6	1.4	1.2	0.8	0.3	0	0.5	0.6	4.4	7.2	11.4	6.4	12.6	7.8	16.2	3.5	2.9	14.6	6.1	9.2
P13	3.7	3.8	3.8	3.9	3.8	3.6	3.8	1.7	1.4	1.3	0.7	0.4	0.5	0	0.9	4.8	7	11.2	6.1	12.3	7.7	15.7	3.7	2.9	14.3	6.5	8.9
P14	4.6	4.7	4.7	4.7	4.6	4.5	4.7	2	1.9	1.7	1.3	0.9	0.6	0.9	0	4.4	7.8	12	6.2	12.5	8.3	16.6	2.9	3.3	14.4	5.6	9
P15	8.2	8.3	8.2	8.4	8.3	8.1	8.3	3.8	4.2	4	4.4	4.4	4.4	4.8	4.4	0	8.8	12.3	10.5	16.8	8.3	19.8	5.6	3.5	18.6	4.5	13.3
P16	5.7	5.5	5.5	5.7	5.8	5.6	5.7	6	6	6.2	6.4	6.9	7.2	7	7.8	8.8	0	4.2	11.2	16.1	2	12.6	10.7	5.4	18.5	12.6	13.5
P17	9.6	9.4	9.4	9.7	9.8	9.6	9.6	10.1	10.1	10.4	10.6	11.1	11.4	11.2	12	12.3	4.2	0	15.1	19.5	4	12.7	14.9	9.3	22	16.5	17.3
P18	5.5	5.7	5.7	5.5	5.3	5.5	5.5	7.8	7.4	7.4	6.8	6.5	6.4	6.1	6.2	10.5	11.2	15.1	0	6.3	12.6	14.3	5.9	9	8.2	10.1	2.8
P19	10.6	10.8	10.8	10.5	10.5	10.7	10.6	13.9	13.5	13.5	12.9	12.7	12.6	12.3	12.5	16.8	16.1	19.5	6.3	0	17.8	14.1	11.9	15.1	2.5	15.9	3.6
P20	7.2	7	7	7.3	7.4	7.2	7.3	6.4	6.5	6.7	7	7.5	7.8	7.7	8.3	8.3	2	4	12.6	17.8	0	14.3	11.2	5.4	20.2	12.5	15.1
P21	12	11.9	11.9	11.8	11.9	12.1	11.9	16.1	15.8	16	15.7	16	16.2	15.7	16.6	19.8	12.6	12.7	14.3	14.1	14.3	0	18.5	16.4	16.2	22.2	14.4
P22	6.8	6.9	6.9	6.9	6.8	6.7	6.8	4.9	4.8	4.6	4.3	3.8	3.5	3.7	2.9	5.6	10.7	14.9	5.9	11.9	11.2	18.5	0	6	13.4	4.2	8.3
P23	5.2	5.2	5.1	5.4	5.3	5.1	5.3	1.3	1.6	1.7	2.2	2.6	2.9	2.9	3.3	3.5	5.4	9.3	9	15.1	5.4	16.4	6	0	17.1	7.2	11.8
P24	13	13.1	13.1	12.9	12.8	13	12.9	15.9	15.5	15.6	15	14.7	14.6	14.3	14.4	18.6	18.5	22	8.2	2.5	20.2	16.2	13.4	17.1	0	17.2	5.4
P25	10.2	10.3	10.3	10.3	10.2	10.1	10.3	6.7	6.8	6.6	6.6	6.3	6.1	6.5	5.6	4.5	12.6	16.5	10.1	15.9	12.5	22.2	4.2	7.2	17.2	0	12.4
P26	7.8	8	8	7.8	7.7	7.9	7.8	10.5	10.2	10.2	9.6	9.3	9.2	8.9	9	13.3	13.5	17.3	2.8	3.6	15.1	14.4	8.3	11.8	5.4	12.4	0

Border Points:

- If the point is in the temporary cluster but does not satisfy MinPts, classify it as a border point. Border points are part of the cluster but cannot expand it.

Noise Points:

- If a point has no neighbors within ε, classify it as a noise point. Noise points are not part of any cluster.

Applying DBSCAN to the Data

1. Start with Point P_0:
   - P_0 has 7 neighboring points within $\varepsilon = 2.5$: (P_0, P_1, P_2, P_3, P_4, P_5, P_6).
   - This is more than the minimum points (MinPts = 6), so P_0 is classified as a core point.
   - Since P_0 is not in any temporary cluster, a new cluster (Cluster 1) is created, and P_0 is added as a core point.
   - All its neighbors (P_0, P_1, P_2, P_3, P_4, P_5, P_6) are added temporarily to Cluster 1.
2. Move to Point P_1:
   - P_1 also has 7 points within ε: (P_0, P_1, P_2, P_3, P_4, P_5, P_6).
   - Since P_1 is already marked in Cluster 1 temporarily, it becomes a core point in Cluster 1.
   - All its neighbors (P_0, P_1, P_2, P_3, P_4, P_5, P_6) are added to Cluster 1 if they are not already in it.
3. Repeat for Points P_2 through P_6:
   - Each of these points has at least seven neighbors (including themselves) within ε, meeting the conditions for core points.
   - They all remain in Cluster 1, expanding it further.

   Result: Points P_0 through P_6 form Cluster 1.
4. Move to Point P_7:
   - P_7 has nine neighbors within ε: (P_7, P_8, P_9, P_{10}, P_{11}, P_{12}, P_{13}, P_{14}, P_{23}).
   - P_7 is not in any temporary cluster, so a new cluster (Cluster 2) is created.
   - P_7 becomes a core point in Cluster 2, and all its neighbors P_8 through P_{14} are added temporarily to Cluster 2.
5. Process Points P_8 through P_{14}:
   - Each point has at least six neighbors within ε, satisfying the condition for core points.
   - All these points are assigned to Cluster 2.

   Result: Points P_7 through P_{14} form Cluster 2.
6. Point P_{23} is a border point:
   - P23 has five neighbors within epsilon ε, not meeting the minimum threshold for neighbors.

- However, it is already part of Cluster 2 due to being a neighbor of points in that cluster.
- Since it does not satisfy the condition to become a core point (not enough direct neighbors), it is classified as a border point for Cluster 2.

Handle Remaining Points:

1. Noise Points:

- Points like P_{17} have no neighbors within ϵ, except themselves. These points are classified as noise.
- Other points, such as P_{16} and P_{20}, have neighbors but are not close to any core point, so they are also classified as noise.

2. Summary:

- Cluster 1: Points P_0 through P_6.
- Cluster 2: Points P_7 through P_{14}, with P_{23} as a border point.
- Noise Points: Points like P_{16}, P_{17}, P_{20}, and others that do not meet the conditions for clusters.

After processing the entire taxi dataset with epsilon (ϵ) = 2.5 and a minimum number of points = 15, we obtained the results shown in Fig. 6.5 and Table 6.5.

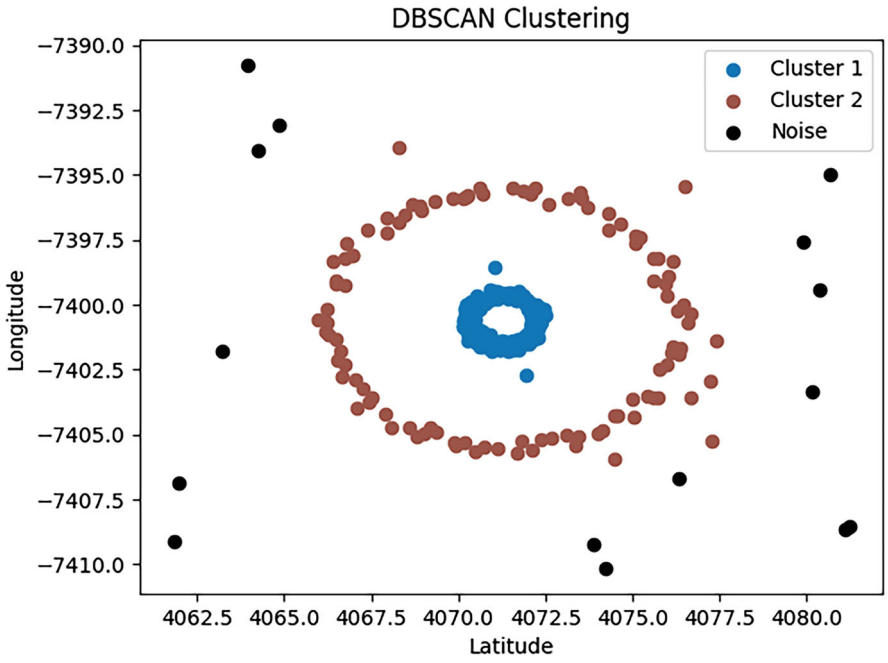


Fig. 6.5 Results of DBSCAN clustering on the taxi data

Table 6.5 The distance matrix between points highlighting the clusters

	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14	P15	P16	P17	P18	P19	P20	P21	P22	P23	P24	P25	P26
P0	0	0.2	0.2	0.2	0.2	0.1	0.1	4.4	4.1	4.3	3.9	4	4.2	3.7	4.6	8.2	5.7	9.6	5.5	10.6	7.2	12	6.8	5.2	13	10.2	7.8
P1	0.2	0	0	0.3	0.4	0.3	0.2	4.5	4.1	4.3	3.9	4.1	4.3	3.8	4.7	8.3	5.5	9.4	5.7	10.8	7	11.9	6.9	5.2	13.1	10.3	8
P2	0.2	0	0	0.3	0.4	0.2	0.2	4.4	4.1	4.3	3.9	4.1	4.3	3.8	4.7	8.2	5.5	9.4	5.7	10.8	7	11.9	6.9	5.1	13.1	10.3	8
P3	0.2	0.3	0.3	0	0.1	0.3	0.1	4.6	4.2	4.4	4	4.2	4.4	3.9	4.7	8.4	5.7	9.7	5.5	10.5	7.3	11.8	6.9	5.4	12.9	10.3	7.8
P4	0.2	0.4	0.4	0.1	0	0.3	0.1	4.6	4.2	4.4	4	4.1	4.3	3.8	4.6	8.3	5.8	9.8	5.3	10.5	7.4	11.9	6.8	5.3	12.8	10.2	7.7
P5	0.1	0.3	0.2	0.3	0.3	0	0.2	4.3	3.9	4.1	3.8	3.9	4.1	3.6	4.5	8.1	5.6	9.6	5.5	10.7	7.2	12.1	6.7	5.1	13	10.1	7.9
P6	0.1	0.2	0.2	0.1	0.1	0.2	0	4.5	4.1	4.3	4	4.1	4.3	3.8	4.7	8.3	5.7	9.6	5.5	10.6	7.3	11.9	6.8	5.3	12.9	10.3	7.8
P7	4.4	4.5	4.4	4.6	4.6	4.3	4.5	0	0.4	0.4	0.9	1.3	1.6	1.7	2	3.8	6	10.1	7.8	13.9	6.4	16.1	4.9	1.3	15.9	6.7	10.5
P8	4.1	4.1	4.1	4.2	4.2	3.9	4.1	0.4	0	0.3	0.6	1.1	1.4	1.4	1.9	4.2	6	10.1	7.4	13.5	6.5	15.8	4.8	1.6	15.5	6.8	10.2
P9	4.3	4.3	4.3	4.4	4.4	4.1	4.3	0.4	0.3	0	0.6	1	1.2	1.3	1.7	4	6.2	10.4	7.4	13.5	6.7	16	4.6	1.7	15.6	6.6	10.2
P10	3.9	3.9	3.9	4	4	3.8	4	0.9	0.6	0.6	0	0.5	0.8	0.7	1.3	4.4	6.4	10.6	6.8	12.9	7	15.7	4.3	2.2	15	6.6	9.6
P11	4	4.1	4.1	4.2	4.1	3.9	4.1	1.3	1.1	1	0.5	0	0.3	0.4	0.9	4.4	6.9	11.1	6.5	12.7	7.5	16	3.8	2.6	14.7	6.3	9.3
P12	4.2	4.3	4.3	4.4	4.3	4.1	4.3	1.6	1.4	1.2	0.8	0.3	0	0.5	0.6	4.4	7.2	11.4	6.4	12.6	7.8	16.2	3.5	2.9	14.6	6.1	9.2
P13	3.7	3.8	3.8	3.9	3.8	3.6	3.8	1.7	1.4	1.3	0.7	0.4	0.5	0	0.9	4.8	7	11.2	6.1	12.3	7.7	15.7	3.7	2.9	14.3	6.5	8.9
P14	4.6	4.7	4.7	4.7	4.6	4.5	4.7	2	1.9	1.7	1.3	0.9	0.6	0.9	0	4.4	7.8	12	6.2	12.5	8.3	16.6	2.9	3.3	14.4	5.6	9
P15	8.2	8.3	8.2	8.4	8.3	8.1	8.3	3.8	4.2	4	4.4	4.4	4.4	4.8	4.4	0	8.8	12.3	10.5	16.8	8.3	19.8	5.6	3.5	18.6	4.5	13.3
P16	5.7	5.5	5.5	5.7	5.8	5.6	5.7	6	6	6.2	6.4	6.9	7.2	7	7.8	8.8	0	4.2	11.2	16.1	2	12.6	10.7	5.4	18.5	12.6	13.5
P17	9.6	9.4	9.4	9.7	9.8	9.6	9.6	10.1	10.1	10.4	10.6	11.1	11.4	11.2	12	12.3	4.2	0	15.1	19.5	4	12.7	14.9	9.3	22	16.5	17.3
P18	5.5	5.7	5.7	5.5	5.3	5.5	5.5	7.8	7.4	7.4	6.8	6.5	6.4	6.1	6.2	10.5	11.2	15.1	0	6.3	12.6	14.3	5.9	9	8.2	10.1	2.8
P19	10.6	10.8	10.8	10.5	10.5	10.7	10.6	13.9	13.5	13.5	12.9	12.7	12.6	12.3	12.5	16.8	16.1	19.5	6.3	0	17.8	14.1	11.9	15.1	2.5	15.9	3.6
P20	7.2	7	7	7.3	7.4	7.2	7.3	6.4	6.5	6.7	7	7.5	7.8	7.7	8.3	8.3	2	4	12.6	17.8	0	14.3	11.2	5.4	20.2	12.5	15.1
P21	12	11.9	11.9	11.8	11.9	12.1	11.9	16.1	15.8	16	15.7	16	16.2	15.7	16.6	19.8	12.6	12.7	14.3	14.1	14.3	0	18.5	16.4	16.2	22.2	14.4
P22	6.8	6.9	6.9	6.9	6.8	6.7	6.8	4.9	4.8	4.6	4.3	3.8	3.5	3.7	2.9	5.6	10.7	14.9	5.9	11.9	11.2	18.5	0	6	13.4	4.2	8.3
P23	5.2	5.2	5.1	5.4	5.3	5.1	5.3	1.3	1.6	1.7	2.2	2.6	2.9	2.9	3.3	3.5	5.4	9.3	9	15.1	5.4	16.4	6	0	17.1	7.2	11.8
P24	13	13.1	13.1	12.9	12.8	13	12.9	15.9	15.5	15.6	15	14.7	14.6	14.3	14.4	18.6	18.5	22	8.2	2.5	20.2	16.2	13.4	17.1	0	17.2	5.4
P25	10.2	10.3	10.3	10.3	10.2	10.1	10.3	6.7	6.8	6.6	6.6	6.3	6.1	6.5	5.6	4.5	12.6	16.5	10.1	15.9	12.5	22.2	4.2	7.2	17.2	0	12.4
P26	7.8	8	8	7.8	7.7	7.9	7.8	10.5	10.2	10.2	9.6	9.3	9.2	8.9	9	13.3	13.5	17.3	2.8	3.6	15.1	14.4	8.3	11.8	5.4	12.4	0

The algorithm successfully clustered the data into two clusters, and the noise points did not affect the algorithm's performance. This behavior contrasts with the K-Means algorithm, which we observed earlier.

6.2.4 Implementation

Here is an implantation of the DBSCAN.

```
import pandas as pd
from sklearn.cluster import DBSCAN
import numpy as np

# Load the data from a CSV file
# Make sure the CSV file has columns named "Longitude" and "Latitude"
file_path = "Book2.csv"  # Update this with the actual path to your CSV
file
df = pd.read_csv(file_path)

# DBSCAN parameters
epsilon = 0.025  # Adjust based on the dataset scale
min_samples = 15

# Apply DBSCAN
dbscan = DBSCAN(eps=epsilon, min_samples=min_samples)
df['Cluster'] = dbscan.fit_predict(df[['Longitude', 'Latitude']])

# Classify points
core_samples_mask = np.zeros_like(dbscan.labels_, dtype=bool)
core_samples_mask[dbscan.core_sample_indices_] = True

df['Type'] = np.where(df['Cluster'] == -1, 'Noise',
                      np.where(core_samples_mask, 'Core', 'Border'))

# Save the results to a new CSV file
output_file = "dbscan_results.csv"
df.to_csv(output_file, index=False)

print(f"Results saved to {output_file}")
```

6.3 Principal Component Analysis (PCA)

Nowadays, we receive massive volumes of data. Sometimes this data is highly relevant to the application, while other times it is not (Bühlmann and Van De Geer 2011). Unimportant features in the dataset can compromise the performance of a model, leading to issues like overfitting or underfitting. Additionally, irrelevant or excessive data may require more computational resources, time to train, or a larger

number of parameters, which can increase the model's size. This makes it unsuitable for lightweight applications (Abdi and Williams 2010; Foulds et al. 2025; Jain 2010; Jolliffe and Cadima 2016).

6.3.1 Steps of PCA

To address these challenges, we need a method to reduce the dimensionality of the data while retaining its most important aspects. One widely used dimensionality reduction technique is Principal Component Analysis (PCA).

PCA is an unsupervised learning method that reduces the dimensionality of the dataset by transforming it into a smaller dataset. This smaller dataset captures the most significant features or principal components of the original data, minimizing information loss.

Imagine having these features for e-commerce customer analytics:

1. Customer Age: Age of the customer (in years).
2. Annual Income: Annual income in dollars.
3. Spending Score: Spending behavior score (1–100).
4. Shopping Frequency: Number of purchases made per month.
5. Website Engagement: Average engagement score with the website (1–10).
6. Cart Abandonment Rate: Percentage of shopping carts abandoned before checkout.
7. Average Purchase Value: Average value of each purchase (in dollars).
8. Discount Usage Rate: Percentage of purchases where discounts or coupons were used.
9. Customer Tenure: Duration of time the customer has been with the platform (in months).
10. Product Return Rate: Percentage of products returned by the customer.

These ten features provide a lot of information for e-commerce customer analytics. However, analyzing all of them at once can be overwhelming. It would be more efficient if we could reduce the number of features while still retaining the most meaningful information. This is where PCA comes into play.

To demonstrate, let's take a snapshot of 15 instances of this data and see how PCA can reduce its dimensionality. As shown in Table 6.6.

6.3.1.1 Step 1: Standardize the Data

Before applying PCA, the first step is to standardize the dataset. Standardization ensures that all features have the same scale and are not dominated by features with larger numerical values.

For example, in the dataset:

Table 6.6 E-commerce customer dataset

Customer age	Annual income	Spending score	Shopping frequency	Website engagement	Cart abandonment rate	Average purchase value	Discount usage rate	Customer tenure	Product return rate
29	117,627	26	12	8	50.28	294	34.8	88	29.47
41	149,892	22	13	2	31.36	436	71.41	14	27.03
69	97,308	39	12	5	42.86	274	45.99	89	15.36
24	61,107	44	3	8	75.17	36	8.29	49	13.68
57	40,668	91	1	3	51.9	377	1.99	67	15.61
18	116,944	48	13	2	76.15	386	39.06	100	33.61
67	131,388	60	4	9	6.63	291	96.46	84	13.48
59	37,191	28	13	7	19.66	46	12.69	91	31.69
19	44,358	7	17	3	66.23	308	60.98	75	40.91
68	121,539	17	2	1	95.95	437	10.1	63	21.84
66	71,560	49	9	3	91.31	263	86.81	55	15.41
22	55,615	27	5	6	74.65	257	41.39	23	14.27
47	120,105	75	15	2	4.62	144	24.63	13	14.03
51	106,479	39	14	6	22.7	367	3.5	11	15.91
59	117,919	23	7	2	27.6	35	22.9	22	33.67

- Annual Income is in the range of thousands.
- Spending Score is between 1 and 100.
- Website_Engagement is between 1 and 10.

If we apply PCA directly without standardizing, features with larger values (like Annual Income) will dominate the principal components, and the result may not represent the true structure of the data.

To standardize each feature X, we calculate the z-score for each data point using:

$$Z = \frac{X - \mu}{\sigma} \tag{6.2}$$

Where:

- X is the original value of the feature.
- μ is the mean of the feature.
- σ is the standard deviation of the feature.

After standardization:

- The mean of each feature becomes 0.
- The standard deviation becomes 1.

The data becomes as in Table 6.7:

6.3.1.2 Step 2: Compute the Covariance Matrix

Once the data is standardized, the next step is to compute the covariance matrix. The covariance matrix helps us understand how the features in the dataset vary with respect to each other. In PCA, the covariance matrix is essential for identifying the directions (principal components) that capture the most variance in the data.

The covariance between two features X and Y is calculated as:

$$Cov(X, Y) = \frac{1}{n-1} \sum_{i=1}^{n} \left(X_i - \overline{X}\right)\left(Y_i - \overline{Y}\right) \tag{6.3}$$

Where:

- X_i and Y_i are the values of features X and Y.
- $\overline{X}$ and $\overline{Y}$ are the mean values of features X and Y.
- n is the number of observations.

The resulting covariance matrix is symmetric and shows the covariance between every pair of features. If a covariance value is:

Table 6.7 E-commerce customer dataset after data standardization

Customer age	Annual income	Spending score	Shopping frequency	Website engagement	Cart abandonment rate	Average purchase value	Discount usage rate	Customer tenure	Product return rate
−0.932	0.695	−0.627	0.528	1.37	0.04	0.229	−0.089	1.02	0.78
−0.289	1.594	−0.81	0.725	−0.957	−0.616	1.292	1.168	−1.358	0.511
1.211	0.13	−0.031	0.528	0.207	−0.217	0.079	0.295	1.052	−0.777
−1.2	−0.878	0.199	−1.253	1.37	0.902	−1.702	−1	−0.234	−0.962
0.568	−1.447	2.355	−1.648	−0.569	0.096	0.85	−1.216	0.345	−0.749
−1.522	0.676	0.382	0.725	−0.957	0.936	0.917	0.057	1.406	1.237
1.104	1.079	0.933	−1.055	1.758	−1.472	0.207	2.028	0.891	−0.984
0.675	−1.544	−0.535	0.725	0.982	−1.021	−1.627	−0.849	1.116	1.025
−1.468	−1.344	−1.499	1.517	−0.569	0.592	0.334	0.81	0.602	2.043
1.157	0.804	−1.04	−1.451	−1.344	1.621	1.299	−0.937	0.216	−0.062
1.05	−0.587	0.428	−0.066	−0.569	1.461	−0.003	1.697	−0.041	−0.771
−1.308	−1.031	−0.581	−0.857	0.595	0.884	−0.048	0.137	−1.069	−0.897
0.032	0.764	1.621	1.121	−0.957	−1.542	−0.894	−0.439	−1.391	−0.923
0.247	0.385	−0.031	0.923	0.595	−0.916	0.775	−1.164	−1.455	−0.716
0.675	0.704	−0.765	−0.462	−0.957	−0.746	−1.709	−0.498	−1.101	1.244

- Positive, it indicates a direct (positive) relationship between the two features.
- Negative, it indicates an inverse (negative) relationship.
- Zero or close to zero, it suggests no linear relationship.

The covariance matrix helps PCA identify the directions (principal components) where the data has the maximum variance. These directions will later be used to transform the data into a new space with reduced dimensionality. The covariance matrix is shown in Table 6.8.

6.3.1.3 Step 3: Compute the Eigenvectors and Eigenvalues

The next step in PCA is to compute the eigenvectors and eigenvalues of the covariance matrix.

Why are Eigenvectors and Eigenvalues Important?

- Eigenvectors represent the directions (principal components) in which the data varies the most.
- Eigenvalues represent the magnitude of the variance along these directions.

The largest eigenvalues correspond to the principal components that explain the most variance in the dataset.

6.3.1.4 Steps to Compute Eigenvectors and Eigenvalues

1. Solve the characteristic equation of the covariance matrix:

$$Cv = \lambda \mathrm{v} \tag{6.4}$$

   Where:

   - C is the covariance matrix.
   - λ are the eigenvalues.
   - v are the eigenvectors.

2. The eigenvectors are then sorted in descending order of their corresponding eigenvalues.
3. The eigenvectors with the largest eigenvalues will form the new feature space.

Eigenvectors and eigenvalues are critical because they help us identify which directions (principal components) capture the most information (variance) in the data. By selecting the top eigenvectors, we can reduce the dimensionality of the dataset while retaining its most important features.

Table 6.8 The covariance matrix

	Customer age	Annual income	Spending score	Shopping frequency	Website engagement	Cart abandonment rate	Average purchase value	Discount usage rate	Customer tenure	Product return rate
Customer age	1.0714	0.1878	0.2644	−0.2983	−0.0917	−0.3058	−0.0108	0.063	0.0261	−0.3884
Annual income	0.1878	1.0714	−0.072	0.1367	−0.2117	−0.3291	0.3506	0.2784	−0.238	−0.0105
Spending score	0.2644	−0.072	1.0714	−0.3081	0.0297	−0.2818	−0.0075	−0.1057	0.0054	−0.6472
Shopping frequency	−0.2983	0.1367	−0.3081	1.0714	−0.1388	−0.3481	0.0188	0.1801	−0.0111	0.5164
Website engagement	−0.0917	−0.2117	0.0297	−0.1388	1.0714	−0.2775	−0.3462	0.0672	0.285	−0.3061
Cart abandonment rate	−0.3058	−0.3291	−0.2818	−0.3481	−0.2775	1.0714	0.2971	−0.0142	0.2382	0.0605
Average purchase value	−0.0108	0.3506	−0.0075	0.0188	−0.3462	0.2971	1.0714	0.2437	0.1029	−0.012
Discount usage rate	0.063	0.2784	−0.1057	0.1801	0.0672	−0.0142	0.2437	1.0714	0.1649	0.0196
Customer tenure	0.0261	−0.238	0.0054	−0.0111	0.285	0.2382	0.1029	0.1649	1.0714	0.3157
Product return rate	−0.3884	−0.0105	−0.6472	0.5164	−0.3061	0.0605	−0.012	0.0196	0.3157	1.0714

6.3.2 Example: Calculating Eigenvectors and Eigenvalues

Let's take a simple example with a 2x2 covariance matrix to illustrate how to calculate eigenvectors and eigenvalues step by step.

6.3.2.1 Step 1: Define the Covariance Matrix

Suppose we have the following covariance matrix C:

$$\begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$$

6.3.2.2 Step 2: Solve the Characteristic Equation

To compute the eigenvalues, we solve the characteristic equation:

$$\det(\mathrm{C} - \lambda I) = 0$$

Where:

- I is the identity matrix,
- λ represents the eigenvalues.

First, subtract λ along the diagonal of C:

$$C - \lambda I = \begin{bmatrix} 4-\lambda & 2 \\ 2 & 3-\lambda \end{bmatrix}$$

Now, compute the determinant of this matrix:

$$\det(C - \lambda I) = (4-\lambda)(3-\lambda) - (2)(2)$$

Simplify the determinant:

$$\det(C - \lambda I) = (4-\lambda)(3-\lambda) - 4 = \lambda^2 - 7\lambda + 8$$

Set the determinant to zero and solve for lambda:

$$\lambda^2 - 7\lambda + 8 = 0$$

Factoring the equation:

$$(\lambda - 1)(\lambda - 6) = 0$$

Thus, the eigenvalues are:

$$\lambda_1 = 6, \lambda_2 = 1$$

6.3.2.3 Step 3: Find the Eigenvectors

For each eigenvalue, solve for the corresponding eigenvector v using the equation:

$$(C - \lambda I)\, v = 0$$

For $\lambda_1 = 6$:
Substitute $\lambda = 6$ into $C - \lambda I$:

$$C - 6I = \begin{bmatrix} 4-6 & 2 \\ 2 & 3-6 \end{bmatrix} = \begin{bmatrix} -2 & 2 \\ 2 & -3 \end{bmatrix}$$

Now solve $(C - 6\mathrm{I})\, v = 0$:

$$\begin{bmatrix} -2 & 2 \\ 2 & -3 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

From the first row:

$$-2v_1 + 2v_2 = 0 \Rightarrow v_1 = v_2$$

The eigenvector corresponding to $\lambda_1 = 6$ is:

$$v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

For $\lambda_2 = 1$:
Substitute $\lambda = 1$ into $C - \lambda I$:

$$C - 1I = \begin{bmatrix} 4-1 & 2 \\ 2 & 3-1 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 2 & 2 \end{bmatrix}$$

Solve $(C - 1I)\, v = 0$:

$$\begin{bmatrix} 3 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

From the first row:

$$3v_1 + 2v_2 = 0 \Longrightarrow v_1 = -\frac{2}{3}v_2$$

The eigenvector corresponding to $\lambda_2 = 1$ is:

$$v_2 = \begin{bmatrix} -2/3 \\ 1 \end{bmatrix}$$

Final Results

- Eigenvalues: $\lambda_1 = 6$, $\lambda_2 = 1$.
- Eigenvectors:
  - For $\lambda_1 = 6$: $v_1 = \begin{bmatrix} 1 \\ 1. \end{bmatrix}$
  - For $\lambda_2 = 1$: $v_2 = \begin{bmatrix} -2/3 \\ 1 \end{bmatrix}$

Based on the costumer's data, we had the Eigenvalues and Eigenvectors are as follows in Table 6.9:

Now that we have the eigenvalues and their corresponding eigenvectors, we can determine which principal components (directions) contribute the most to the variance in the data.

The eigenvalues indicate the variance explained by each principal component. To decide which principal components to retain, we look at the eigenvalues and focus on the ones with the largest values, as they capture the most variance in the data. The eigenvectors corresponding to these eigenvalues form the directions (or axes) of the new feature space.

Variance Explained by Each Component

To evaluate how much variance each principal component contributes, we calculate the proportion of variance explained:

$$\text{Proportion of Variance} = \frac{\text{eigenvalue}_i}{\sum \text{eigenvalues}} \tag{6.5}$$

Table 6.9 The Eigenvalues and Eigenvectors

		Customer age	Annual income	Spending score	Shopping frequency	Website engagement	Cart abandonment rate	Average purchase value	Discount usage rate	Customer tenure	Product return rate
	Eigenvalues	Eigenvectors									
Customer age	2.432	−0.366	−0.283	0.266	0.055	−0.148	0.073	0.358	−0.594	−0.43	−0.138
Annual income	0.042	0.034	−0.094	0.625	0.001	−0.065	0.364	−0.433	0.124	−0.18	0.477
Spending score	1.899	−0.492	−0.393	0.048	0.054	−0.031	−0.007	−0.342	−0.136	0.668	−0.128
Shopping frequency	1.691	0.402	−0.135	0.222	−0.404	−0.015	0.496	0.403	−0.055	0.396	−0.196
Website engagement	1.411	−0.203	−0.425	−0.311	−0.361	−0.455	−0.029	0.222	0.393	−0.108	0.358
Cart abandonment rate	0.28	0.193	−0.425	−0.33	0.619	0.057	0.475	−0.027	0.152	−0.137	−0.137
Average purchase value	0.462	0.14	−0.035	0.327	0.53	−0.161	−0.348	0.471	0.094	0.327	0.325
Discount usage rate	0.961	0.119	0.009	0.289	0.061	−0.589	−0.167	−0.18	0.301	−0.113	−0.623
Customer tenure	0.859	0.132	0.357	−0.298	0.115	−0.625	0.233	−0.156	−0.455	0.163	0.227
Product return rate	0.676	0.576	−0.498	−0.052	−0.139	0.018	−0.431	−0.275	−0.356	−0.055	0.082

$$\text{Proportion of Variance Explained} = [0.22697878, 0.00388837, 0.17724268, 0.15787227, 0.13173468, 0.0261139, 0.04314393, 0.0897113, 0.08021839, 0.06309571]$$

6.3.2.4 Step 5: Choose the Number of Principal Components

Once we have the Proportion of Variance Explained (PVE) for each principal component, we need to determine the number of components that capture the majority of the variance in the original data.

Steps to Select the Best Number of PCs

1. Sort the Proportion of Variance Explained
   If the PVE is not already sorted in descending order, we first sort it. Sorting ensures we are analyzing the components that explain the most variance first.
   Example PVE (unsorted):
   [0.22697878, 0.00388837, 0.17724268, 0.15787227, 0.13173468, 0.0261139, 0.04314393, 0.0897113, 0.08021839, 0.06309571]
   After sorting (descending):
   [0.22697878, 0.17724268, 0.15787227, 0.13173468, 0.0897113, 0.08021839, 0.06309571, 0.04314393, 0.0261139, 0.00388837]
2. Compute the Cumulative Variance Explained
   Next, we calculate the cumulative sum of the sorted PVE. This shows how much total variance is captured as we add more components.
   Cumulative Variance Explained Example:
   Cumulative PVE = [0.2269, 0.4042, 0.5620, 0.6937, 0.7834, 0.8636, 0.9267, 0.9698, 0.9959, 1.0000]
3. Set a Threshold for the Variance to Be Retained.
   Decide on a threshold for the percentage of variance you want to retain. A typical threshold is 80–90%.
   - If the target is 80% (0.8), we look for the smallest number of components where the cumulative PVE exceeds this value.
4. Choose the Number of Principal Components.
   From the cumulative PVE, identify the number of components needed to reach or exceed the threshold.
   Example Analysis:
   - At 80% (0.8), the cumulative PVE reaches 0.8636 after adding six components.
   - Therefore, six principal components are sufficient to explain approximately 80% of the variance in the original data.

6.3.2.5 Step 6: Transform the Data Using the Selected Principal Components

After deciding how many principal components to retain (in this case, six components based on the 80% threshold), the next step is to project the original standardized data onto these principal components. This step reduces the dataset's dimensionality while preserving the maximum variance.

How to Transform the Data

1. Select the Top Principal Components:
   Use the eigenvectors corresponding to the top six eigenvalues (principal components).
2. Matrix Multiplication:
   Project the standardized data onto the selected principal components using matrix multiplication:

$$Z = X \cdot W \tag{6.6}$$

   Where:

   - X is the standardized original data.
   - W is the matrix of selected eigenvectors.
   - Z is the transformed data in the new feature space.

3. Result:
   After multiplying Matrix X by Matrix W, Table 6.10 shows that the transformed data Z will have six dimensions (principal components) instead of the original ten features.

Table 6.10 Transformed data Z with six principal components derived from original ten features

PC1	PC2	PC3	PC4	PC5	PC6
1.21961213	−0.4599818	−0.6420518	−1.1245099	0.49460138	0.12210759
1.36020119	2.67939487	0.14164804	0.3145646	0.87926982	−0.1882681
−0.5590711	0.36465152	−0.068318	−1.1579732	−0.7857481	−0.1404241
−1.2373428	−2.585485	−0.342838	0.66780397	1.30847229	−0.3897182
−2.3548357	−0.999892	1.60842157	0.57645211	−1.1313134	1.49643421
2.09123893	0.02034017	1.04945014	−0.3915287	−0.3130189	1.5057887
−2.0721546	1.15546338	−0.5339846	−2.930451	0.45152454	−0.413758
0.26866468	−1.7809389	−2.2018675	−0.4165907	−1.6091855	−0.3592558
3.46763	−0.9259832	−0.1927535	−0.2507481	−0.0324258	0.45828817
0.18102698	0.41461752	2.74752933	0.74166626	−0.8859173	−1.4185923
−0.4933131	0.15479939	1.42043528	−0.77311	0.04982482	−0.3810074
−0.2130083	−1.2980053	0.55867692	0.65076413	1.9847651	−0.320814
−1.3276628	1.66201937	−1.5038987	1.47893308	−0.0719821	1.26884565
−0.6224817	1.01406976	−0.8696273	1.05896655	0.58818593	0.41821417
0.29149617	0.58493017	−1.1708219	1.55576087	−0.9270528	−1.6578407

Matrix X

Customer age	Annual income	Spending score	Shopping frequency	Website engagement	Cart abandonment rate	Average purchase value	Discount usage rate	Customer tenure	Product return rate
−0.932	0.695	−0.627	0.528	1.37	0.04	0.229	−0.089	1.02	0.78
−0.289	1.594	−0.81	0.725	−0.957	−0.616	1.292	1.168	−1.358	0.511
1.211	0.13	−0.031	0.528	0.207	−0.217	0.079	0.295	1.052	−0.777
−1.2	−0.878	0.199	−1.253	1.37	0.902	−1.702	−1	−0.234	−0.962
0.568	−1.447	2.355	−1.648	−0.569	0.096	0.85	−1.216	0.345	−0.749
−1.522	0.676	0.382	0.725	−0.957	0.936	0.917	0.057	1.406	1.237
1.104	1.079	0.933	−1.055	1.758	−1.472	0.207	2.028	0.891	−0.984
0.675	−1.544	−0.535	0.725	0.982	−1.021	−1.627	−0.849	1.116	1.025
−1.468	−1.344	−1.499	1.517	−0.569	0.592	0.334	0.81	0.602	2.043
1.157	0.804	−1.04	−1.451	−1.344	1.621	1.299	−0.937	0.216	−0.062
1.05	−0.587	0.428	−0.066	−0.569	1.461	−0.003	1.697	−0.041	−0.771
−1.308	−1.031	−0.581	−0.857	0.595	0.884	−0.048	0.137	−1.069	−0.897
0.032	0.764	1.621	1.121	−0.957	−1.542	−0.894	−0.439	−1.391	−0.923
0.247	0.385	−0.031	0.923	0.595	−0.916	0.775	−1.164	−1.455	−0.716
0.675	0.704	−0.765	−0.462	−0.957	−0.746	−1.709	−0.498	−1.101	1.244

Matrix W

v_1	v_3	v_4	v_5	v_8	v_9
−0.3662526	0.26594084	0.05481981	−0.1476008	−0.5942059	−0.4299636
0.03365796	0.62524037	0.00119327	−0.0649963	0.12379926	−0.1801639
−0.491878	0.047718	0.05449437	−0.0310603	−0.1358439	0.66750921
0.40176621	0.22230771	−0.4039169	−0.0153329	−0.0549346	0.39641331
−0.2033103	−0.3109471	−0.3605633	−0.4545487	0.39309129	−0.1075712
0.1927136	−0.3295637	0.61906683	0.05736492	0.15164957	−0.1365885
0.13977657	0.3265506	0.53043644	−0.1611757	0.09439305	0.32746168
0.11858015	0.28885149	0.06099129	−0.5886044	0.30104317	−0.1128497
0.13166011	−0.2976366	0.11512569	−0.6246152	−0.4545277	0.16346021
0.57615915	−0.0519019	−0.1388831	0.01753624	−0.3556275	−0.0551975

6.3.3 Implementation

Here is the implementation of the PCA algorithm.

```
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Load the dataset
data_file = 'ecoomerce.csv'
df = pd.read_csv(data_file)

# Step 2: Data preprocessing
# Dropping non-numeric columns and handling missing values
df_numeric = df.select_dtypes(include=['float64', 'int64']).dropna()

# Standardize the features
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_numeric)

# Step 3: Apply PCA
pca = PCA(n_components=0.8)
pca_result = pca.fit_transform(scaled_data)

# Explained variance ratio
explained_variance_ratio = pca.explained_variance_ratio_

# Step 4: Visualize explained variance
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(explained_variance_ratio) + 1),
explained_variance_ratio.cumsum(), marker='o')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance by Principal Components')
plt.grid(True)
plt.show()

# Show PCA results as a DataFrame
pca_df = pd.DataFrame(pca_result, columns=[f"PC{i+1}" for i in
range(pca_result.shape[1])])
pca_df.head()
```

6.4 Concluding Remarks

In this chapter, we explored unsupervised learning, a paradigm that allows models to discover hidden patterns and structures in unlabeled data. We introduced key techniques such as K-Means Clustering, which partitions data into distinct groups based on similarity, and DBSCAN, a density-based approach that excels at identifying clusters in datasets with uneven density and noise.

Through practical examples, such as clustering coffee shop customers or analyzing taxi traffic data, we demonstrated how these algorithms provide actionable

insights, whether for business optimization or pattern recognition. However, we also highlighted the limitations of K-Means, including its sensitivity to initial centroids and its reliance on a predefined number of clusters. DBSCAN, on the other hand, addressed these challenges by being capable of handling non-spherical clusters and noise.

Finally, we introduced Principal Component Analysis, a powerful dimensionality reduction technique that preserves the most significant features of a dataset while minimizing redundancy. PCA helps simplify large datasets, making them easier to interpret and process without losing critical information.

Chapter 7
Introduction to Artificial Neural Networks



Abstract This chapter introduces Artificial Neural Networks (ANNs), highlighting their structure, training process, and practical applications. We begin by exploring the network architecture—input, hidden, and output layers—and the function of activation mechanisms like sigmoid, ReLU, and softmax in enabling non-linear decision boundaries and handling various classification tasks.

The training process is examined in detail through backpropagation, where gradients are computed and weights are updated using optimization techniques like gradient descent. Key loss functions, including Mean Squared Error and Cross-Entropy Loss, are discussed with mathematical clarity to illustrate how networks learn from errors.

Practical implementation is emphasized through step-by-step Python examples, demonstrating how to construct, train, and evaluate neural networks for both binary and multiclass classification. Finally, we extend the discussion to regression tasks, showing how ANNs can predict continuous outputs using linear activation in the final layer. This chapter provides a comprehensive foundation in building and applying neural networks across a range of real-world problems.

7.1 Revisiting Logistic Regression

If you're enthusiastic about learning neural networks and have jumped straight to this chapter, we highly recommend reading the previous chapters—particularly on regression and logistic regression. This is because we build upon those concepts here, following the same flow of ideas. Even if you're already familiar with regression and logistic regression, you might gain new insights from revisiting those chapters.

Before diving into neural networks, let's revisit logistic regression in a more abstract way to deepen our understanding and prepare us to extend it to more complex machine learning methods.

A model that uses logistic regression to classify data can be described as follows:

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_7

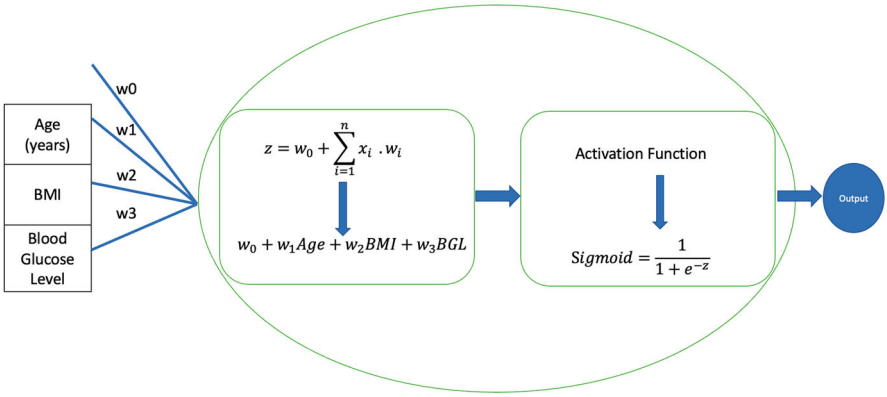


Fig. 7.1 Illustration of a logistic regression model

1. Input Layer: It begins with inputs that receive feature data.
2. Weights Assignment: Each input feature is associated with a weight that can be adjusted during training to enhance predictions.
3. Linear Equation: A linear equation is formed by multiplying the input features by their corresponding weights and adding the value of the intercept (bias) to the equation.
4. Activation Function: The result of the linear equation is then passed through a sigmoid function, producing a probability between 0 and 1.
5. Output Layer: This probability represents the predicted class label of the model.
6. Training Process: The model's predictions are compared with the actual labels, and the weights are updated using gradient descent to reduce prediction errors.

Figure 7.1 illustrates this process. The main components here include the input data, a computational unit that processes the inputs, and the final output.

Introducing Neural Network Terminology

Let's standardize the terminology using the language commonly employed in neural networks:

- The computational unit that receives features and weights will be referred to as a neuron.
- The intercept in the linear equation now becomes what we call a bias.
- The sigmoid function, which processes the output of the linear equation, is an example of what we call an activation function.

So now, we have a neuron, which is essentially a computational unit that receives features, weights, and a bias. Within this unit, a linear equation is solved, followed by applying an activation function. This neuron then produces an output.

7.2 From Logistic Regression to Artificial Neural Networks

The idea of abstracting logistic regression into a computational unit gives us the flexibility to build a network of these units. By adding as many units as needed, we can construct what is known as an Artificial Neural Network (ANN)—a network of interconnected computational units or *neurons* (Khan et al. 2020).

To illustrate, let's build upon the same example from the regression chapter, where we aimed to classify patients as diabetic or non-diabetic based on features such as age, BMI, and blood glucose level. By adding another computational unit (neuron) to the model, we can enhance its capacity to capture complex patterns, as shown in Fig. 7.2.

The process of training multiple neurons begins by feeding the features into each neuron individually. Each neuron has its own set of weights associated with the features, as well as its own bias term (previously known as the intercept).

In this setup, each neuron produces a distinct output based on the initial weights assigned to it. To ensure that neurons do not generate identical outputs, the weights are initialized with random values rather than zeros. This randomness helps break symmetry during training, allowing the neurons to learn unique patterns (Z. Zhang and Zhang 2018; Zou et al. 2009).

As illustrated in Fig. 7.2, the naming convention used distinguishes between the weights and biases for different neurons. For instance:

w_{11} refers to the weight associated with feature 1 for Neuron 1.
w_{21} refers to the weight associated with feature 2 for Neuron 1.
$bias_{01}$ indicates the bias term for Neuron 1.

By using this convention, we clearly indicate which weights and biases are tied to specific neurons, helping to maintain clarity as the network scales.

Here's the polished continuation of your text:

Let's take an example to see how the above network with two neurons produces outputs when given specific feature values and weights.

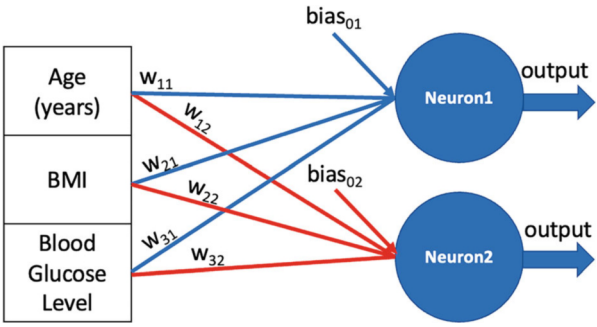


Fig. 7.2 A simple neural network with two neurons, each receiving inputs from features (age, BMI, blood glucose level)

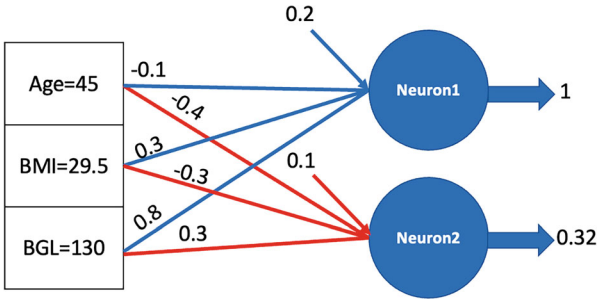


Fig. 7.3 Example outputs from a simple neural network with two neurons

From the logistic regression chapter, we had a patient with the following feature values:

- Age: 45
- BMI: 29.5
- Blood Glucose Level: 130

We will assign random weights and biases to each neuron:

- For Neuron 1: weights $= [-0.1, 0.3, 0.8]$ and bias $= 0.2$
- For Neuron 2: weights $= [-0.4, -0.3, 0.2]$ and bias $= 0.1$

Calculating the outputs:

- Neuron 1 produces an output of 1.
- Neuron 2 produces an output of 0.32.

These results are illustrated in Fig. 7.3.

7.3 Extending the Neural Network for a Single Output

In a standard logistic regression model, after making a prediction, the next step is to calculate the loss based on that prediction. The model then uses gradient descent to adjust the weights, reducing the error over time. At the end of the training, we would have a single output representing the prediction (Hassoun 1995; Yegnanarayana 2009).

However, in the current network, we have *two different outputs* from the two neurons. This raises the question: how do we proceed? Should we pick the highest value? The minimum value? Perhaps pick one at random or take the average of the results?

The answer is: none of the above.

The solution is to add a new neuron that takes these outputs as inputs, processes them, and then produces a single output value to represent the network's final prediction. This approach is illustrated in Fig. 7.4.

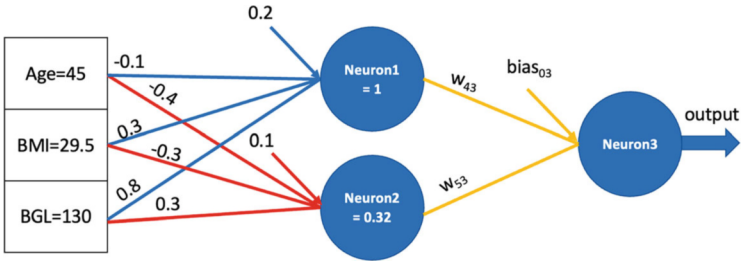


Fig. 7.4 Extending the neural network with an additional neuron (Neuron 3)

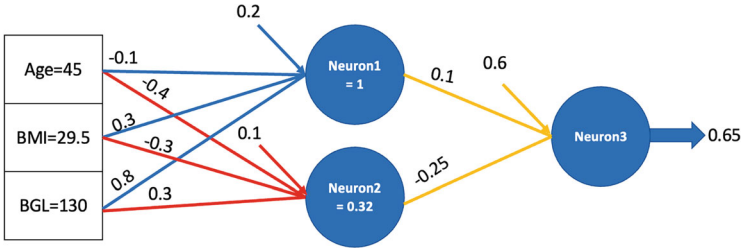


Fig. 7.5 The final step of the neural network

In this setup:

- The outputs of Neuron 1 and Neuron 2 become the inputs for Neuron 3.
- Each input to Neuron 3 will have its own weight, and the neuron will also have its own bias.
- Neuron 3 then produces a single output, which we can use to calculate the loss and adjust the weights throughout the network during training.

Now, the outputs from Neuron 1 and Neuron 2 become the inputs (or features) for Neuron 3. Let's assume that the weights for these connections are:

- $w_{43} = 0.1$ (weight from Neuron 1 to Neuron 3)
- $w_{53} = -0.25$ (weight from Neuron 2 to Neuron 3)

Additionally, let's set the bias for Neuron 3 to be 0.6.

With these values, the final output of the network, produced by Neuron 3, will be approximately 0.65. This process is illustrated in Fig. 7.5.

7.4 Calculating the Loss for the Network

The next step is to calculate the error of the prediction, just like we did with logistic regression. Since our problem is a binary classification, we will also use the log loss (binary cross-entropy) metric in the exact same way, using the following formula:

$$\text{Loss} = -\frac{1}{N}\sum_{i=1}^{N} y_i \ \log(\widehat{y}_i) + (1 - y_i) \ \log(1 - \widehat{y}_i) \tag{7.1}$$

Where:

- N is the number of training instances.
- y_i is the actual label for the ith instance.
- $\widehat{y}_i$ is the predicted probability from the network for the ith instance.

Now, let's calculate the loss for our example:

- For Patient 1, assume the original label (y) is 1.
- The prediction from the network ($\widehat{y}$) is 0.65.

Using the formula:

$\text{Loss} = -y \log(\widehat{y}) + (1 - y) \log(1 - \widehat{y})$
$\text{Loss} = -1 \times \log(0.65) + (1 - 1) \times \log(1 - 0.65)$
$\text{Loss} = -\log(0.65) \approx -(-0.4308) = 0.4308$

So, the loss for this instance is approximately 0.43.

Of course, to get the complete loss of the network, we need to provide the full training dataset and sum up the losses for all instances, then take the average. This gives us a measure of how well the network is performing on the entire dataset.

7.5 Backpropagation

Now it is time to find the best way to update the weights based on the error we have gotten. In logistic regression, we used gradient descent to compute the gradient of each weight based on the error. It was a straightforward approach, as each weight directly impacted the error. However, in an Artificial Neural Network, it is more complicated than that (Chauvin and Rumelhart 2013; Rojas and Rojas 1996; Wythoff 1993).

In an ANN, we have an input layer, some hidden layers where each can have a different number of neurons, and each of these interconnected layers affects the next layers. Finally, we have the output layer, which is influenced by all the previous layers.

So, the impact of each weight is not the same across the entire network, even for the same error. Some weights might have more impact than others. That's why we use a method called backpropagation to find the gradient of each of the weights based on its impact, starting from the output layer and going backward toward the input layer (Anderson and McNeill 1992).

In our simple network, we have already calculated the error at the output. Now, we need to adjust the weights to reduce this error in the next iterations. To do this, we must start with the output layer and work our way backward.

The first step is to compute the gradient for Neuron3 based on the error generated and the activation function used. To calculate the gradient for the output layer, we use:

$$\frac{\partial \text{Loss}}{\partial z} = \frac{\partial \text{Loss}}{\partial \widehat{y}} \times \frac{\partial \widehat{y}}{\partial z} \tag{7.2}$$

7.5.1 Step 1: Derivative of the Loss Function with Respect to the Predicted Output

The derivative of the log loss (cross-entropy) function with respect to the predicted output $\widehat{y}$ is:

$$\frac{\partial \text{Loss}}{\partial \widehat{y}} = -\frac{y}{\widehat{y}} + \frac{1-y}{1-\widehat{y}} \tag{7.3}$$

When $y = 1$:

$$\frac{\partial \text{Loss}}{\partial \widehat{y}} = -\frac{1}{\widehat{y}} \tag{7.4}$$

For our specific example where $\widehat{y} = 0.65$:

$$\frac{\partial \text{Loss}}{\partial \widehat{y}} = -\frac{1}{0.65} \approx -1.538$$

7.5.2 Step 2: Sigmoid Activation Function Derivative

The activation function used in Neuron3 is the sigmoid function, defined as:

$$\sigma(z) = \frac{1}{1+e^{-z}} \tag{7.5}$$

The derivative of the sigmoid function with respect to z is:

$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1-\sigma(z)) \tag{7.6}$$

Since $\widehat{y} = \sigma(z) = 0.65$, the derivative is:

$$\frac{\partial \widehat{y}}{\partial z} = 0.65 \times (1-0.65) = 0.2275$$

7.5.3 Step 3: Calculating the Gradient of the Loss with Respect to z

Now, we combine the two derivatives we found:

$$\frac{\partial \text{Loss}}{\partial z} = \frac{\partial \text{Loss}}{\partial \widehat{y}} \times \frac{\partial \widehat{y}}{\partial z}$$

Substituting the values:

$$\frac{\partial \text{Loss}}{\partial z} = -1.538 \times 0.2275 = -0.35$$

The gradient we calculated, -0.35, tells us how much we need to adjust the weights connected to Neuron3 to reduce the error. A negative value indicates that increasing z will help reduce the loss, so we should adjust the weights accordingly during the weight update step.

The next step is to calculate the contribution of each weight that is connected to Neuron 3 since each one contributed to this gradient. This step is similar to logistic regression, where we multiply the value of the output of the neuron associated with each of these weights. In this case, the outputs come from Neuron1 and Neuron2.

Now, we have the gradient for Neuron 3, which is $\delta_3 = -0.35$. To compute the gradient of $_{43}$ (the weight connected to Neuron 3 from Neuron1), we multiply this gradient by the output from Neuron1. Similarly, we do the same for Neuron2 to compute the gradient of w_{53}, and the same for the bias_{03}.

- For w_{43} (connected to Neuron1):

$$\frac{\partial \text{Loss}}{\partial w_{43}} = \delta_3 \times \text{output of Neuron1} = -0.35 \times 1 = -0.35$$

- For w_{53} (connected to Neuron2):

$$\frac{\partial \text{Loss}}{\partial w_{53}} = \delta_3 \times \text{output of Neuron2} = -0.35 \times 0.32 = -0.112$$

- For bias_{03}:

$$\frac{\partial \text{Loss}}{\partial \text{bias}_{03}} = \delta_3 \times 1 = -0.35 \times 1 = -0.35$$

The outputs from the neurons are 1 and 0.32, respectively. This means:

- The gradient for w_{43} is -0.35.
- The gradient for w_{53} is -0.112.
- The gradient for $bias_{03}$ is -0.35.

These gradients tell us how much to adjust each of these weights. A negative value indicates that we should increase the weight to reduce the error in the next iteration.

We have computed the gradient for each weight, and now we can update the weights based on the learning rate, just like we did in logistic regression. However, we must not do this yet because the current weight values are still needed to compute the gradients in the previous layers. Therefore, we must wait until all the layers finish computing their gradients before adjusting the weights.

Even when all the layers finish computing their gradients, we don't necessarily update the weights immediately. It depends on which mechanism we are using:

- If we use Stochastic Gradient Descent (SGD), we can adjust the weights right away after all the layers finish calculating their gradients for a single training example.
- If we use other mechanisms like Mini-batch Gradient Descent or Batch Gradient Descent, then we wait until the entire mini-batch or batch is processed. In these cases, we take the average of the gradients for each weight across the batch and adjust each weight based on this average gradient.

This approach helps to smooth out the updates and can lead to more stable convergence, especially in complex networks.

7.6 Propagating the Error Backward

Since we have calculated the gradient of the output node and the gradient for each weight, it is now time to step back to the previous layer to compute the gradient for each neuron in that layer. This will allow us to compute the gradient for each weight connected to each neuron in that layer.

The process of computing the gradient for all the hidden layers—regardless of how many layers there are or how many neurons each layer contains—will follow the same method (Agarap 2018; Azulay and Weiss 2019; W. Chen et al. 2015b).

Each neuron in a hidden layer will receive the gradients from all the neurons it is connected to in the next layer. However, to compute the gradient for each neuron in the current hidden layer, we not only need to consider the gradients from the next layer, but also take into account the derivative of the activation function used by the current neuron. This derivative is crucial because it determines how sensitive the neuron's output is to changes in its input, effectively controlling how much influence the neuron has on the overall network's learning process.

To calculate the gradient for a neuron in a hidden layer, we multiply the gradient of each neuron in the next layer by the weight connecting it to the current neuron and sum these contributions. We then multiply this sum by the derivative of the

activation function of the current neuron. This way, we determine how much each weight and activation function impact the gradient flow.

In other words, the gradient for a neuron in a hidden layer can be expressed as:

$$\delta_j = \left(\sum_k \delta_k^{\text{next}} \times w_{jk}^{\text{next}} \right) \times \sigma'(z_j) \tag{7.7}$$

Where:

- δ_j is the gradient for the current hidden neuron j.
- δ_k^{next} is the gradient of the next layer's neuron k.
- w_{jk}^{next} is the weight connecting neuron j in the current hidden layer to neuron k in the next layer.
- $\sigma'(z_j)$ is the derivative of the activation function of the current hidden neuron with respect to its input z_j.

This formula helps us compute the contribution of the current neuron to the gradients of the neurons in the next layer while also adjusting for the activation function. Once we have the gradients for all neurons in the current layer, we can proceed to adjust the weights connecting the previous layer to the current layer using the same technique we used earlier.

Now let's compute the gradient of Neuron1. To do this, we need to look at all the nodes that Neuron1 is connected to, get their gradients, multiply those gradients by the corresponding weights, sum all these values, and then multiply by the derivative of the sigmoid function.

The gradient for Neuron1 δ_1 is calculated as:

$$\delta_1 = (\delta_3 \times w_{43}) \times \sigma(z_1)(1 - \sigma(z_1)) \tag{7.8}$$

Substituting the values we have:

$$\delta_1 = (-0.35 \times 0.1) \times (1 \times (1 - 1)) = 0$$

So, the gradient of Neuron1 is 0, which means in this case, we don't need to compute the weight gradients connected to Neuron1 because the gradient of the neuron itself is zero.

This implies that Neuron1 has no effect on the error during this iteration, so its weights will not be updated in this step. This can happen when the neuron's activation reaches the saturation point (output of 0 or 1), causing the derivative of the sigmoid function to be zero.

Now we do the same for Neuron2. We need to get the gradient of the neurons connected to it in the next layer, multiply that by each weight, sum up these values, and then multiply the result by the derivative of the activation function for Neuron2.

The gradient for Neuron2 (δ_2) is calculated as:

$$\delta_2 = \delta_3 \times w_{53} \times \sigma'(z_2) \tag{7.9}$$

Substituting the values we have:

$$\delta_2 = (-0.35 \times -0.25) \times (0.32 \times (1 - 0.32)) \approx 0.01904$$

So, the gradient for Neuron2 is approximately 0.01904.

This means that Neuron2 will have some impact on the error, so we will need to adjust the weights connected to it in the previous layer based on this gradient.

Based on the gradient of Neuron2 ($\delta_2 = 0.01904$), we can now calculate the gradient of each weight connected to Neuron2 from the previous layer, which is the input layer. This is done by multiplying δ_2 by the values of the features that are feeding into Neuron2: Age, BMI, BGL, and the bias_{02}.

Here's how we compute the gradient for each weight:

1. Gradient for the weight connecting Age (45) to Neuron2:

$$\frac{\partial \text{Loss}}{\partial \text{w}_{12}} = \delta_2 \times Age = 0.01904 \times 45 = 0.8568$$

2. Gradient for the weight connecting BMI (29.5) to Neuron2:

$$\frac{\partial \text{Loss}}{\partial \text{w}_{22}} = \delta_2 \times BMI = 0.01904 \times 29.5 = 0.56168$$

3. Gradient for the weight connecting BGL (130) to Neuron2:

$$\frac{\partial \text{Loss}}{\partial \text{w}_{32}} = \delta_2 \times BGL = 0.01904 \times 130 = 2.4752$$

4. Gradient for the bias_{02}:

$$\frac{\partial \text{Loss}}{\partial \text{bias}_{02}} = \delta_2 \times 1 = 0.01904 \times 1 = 0.01904$$

- Gradient for Age weight: 0.8568
- Gradient for BMI weight: 0.56168
- Gradient for BGL weight: 2.4752
- Gradient for bias_{02} weight: 0.01904

These gradients tell us how much we should adjust each of the weights connected to Neuron2 during the weight update step.

7.7 Updating the Weights

Now that we have reached the input layer, the backpropagation phase for this iteration is complete. Depending on the mechanism we choose for updating the weights, we will decide the next step:

- If we choose Stochastic Gradient Descent, we can update the weights immediately based on the learning rate.
- If we choose Mini-batch Gradient Descent or Batch Gradient Descent, we need to process the entire mini-batch or batch first, then compute the average gradient for each weight, and finally update each weight based on its average gradient.

Let's assume we are using SGD, so we will directly update the weights using a learning rate of 0.1.

7.7.1 Step 1: Updating Weights Connected to Neuron2 from the Input Layer

Since the gradient of Neuron1 was $\delta_1 = 0$, the weights connected to Neuron1 (w_{11}, w_{21}, w_{31}) and its bias b_{01} will not change.

The gradients we calculated for Neuron2 are:

- Gradient for Age weight: 0.8568
- Gradient for BMI weight: 0.56168
- Gradient for BGL weight: 2.4752
- Gradient for bias b_{02}: 0.01904

We will now update these weights using the formula:

$$w_{new} = w_{old} - (\text{learning rate} \times \text{gradient}) \tag{7.10}$$

Let's update each weight:

1. Updating weight w_{12} (connected to Age):
   $w_{12,\text{new}} = w_{12} - (0.1 \times 0.8568) = -0.4 - 0.08568 = -0.485$
2. Updating weight w_{22} (connected to BMI):
   $w_{22,\text{new}} = w_{22} - (0.1 \times 0.56168) = -0.3 - 0.056168 = -0.356168$
3. Updating weight w_{32} (connected to BGL):
   $w_{32,\text{new}} = w_{32} - (0.1 \times 2.4752) = 0.3 - 0.24752 = 0.05248$
4. Updating bias b_{02}:
   $b_{02,\text{ new}} = b_{02} - (0.1 \times 0.01904) = 0.1 - 0.001904 = 0.098$

7.7.2 *Step 2: Updating Weights Connected to Neuron3*

Next, we move on to the weights connected to Neuron3:

- The gradient for w_{43}: −0.35
- The gradient for w_{53}: −0.112
- The gradient for bias b_{03}: −0.35

We will update these weights similarly using the learning rate of 0.1:

1. Updating weight w_{43} (connected to Neuron1):
   $$w_{43,new} = w_{43} - (0.1 \times -0.35) = 0.1 + 0.035 = 0.135$$
2. Updating weight w_{53} (connected to Neuron2):
   $$w_{53,\ new} = w_{53} - (0.1 \times -0.112) = -0.25 + 0.0112 = -0.239$$
3. Updating bias b_{03}:
   $$b_{03,\ new} = b_{03} - (0.1 \times -0.35) = 0.6 + 0.035 = 0.635$$

The updated network with the new weights is shown in Fig. 7.6.

The code below shows a full implantation of our simple neural network:

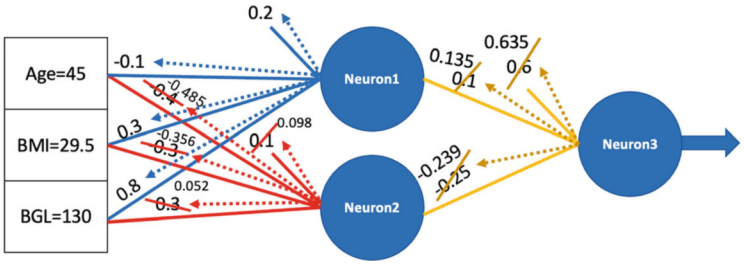


Fig. 7.6 Updated Neural Network After Weight Adjustments Using SGD with a Learning Rate of 0.1

```
import numpy as np

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of the sigmoid function
def sigmoid_derivative(x):
    return sigmoid(x) * (1 - sigmoid(x))

# Log-loss (binary cross-entropy) function
def log_loss(y_true, y_pred):
    return - (y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

# Derivative of the log-loss function
def log_loss_derivative(y_true, y_pred):
    return (y_pred - y_true) / (y_pred * (1 - y_pred))

# Define the neural network structure based on the diagram
class SimpleNeuralNetwork:
    def __init__(self):
        # Initialize weights as per the given diagram
        # Initialize weights and biases randomly
        self.weights_1 = np.random.randn(2, 3)  # Random weights to
Neuron1 and Neuron2
        self.bias_1 = np.random.randn(2)        # Random biases for
Neuron1 and Neuron2
        self.weights_2 = np.random.randn(2)     # Random weights to
Neuron3
        self.bias_2 = np.random.randn()         # Random bias for Neuron3

    # Forward pass
    def forward(self, inputs):
        # Layer 1 (hidden layer)
        self.z1 = np.dot(self.weights_1, inputs) + self.bias_1
        self.a1 = sigmoid(self.z1)  # Activation of Neuron1 and Neuron2

        # Layer 2 (output layer)
        self.z2 = np.dot(self.weights_2, self.a1) + self.bias_2
        self.a2 = sigmoid(self.z2)  # Output of Neuron3
        return self.a2

    # Backward pass (gradient computation)
    def backward(self, inputs, y_true):
        # Calculate output error
        y_pred = self.a2
        output_error = log_loss_derivative(y_true, y_pred) *
sigmoid_derivative(self.z2)

    # Calculate gradients for weights between Neuron3 and hidden layer
    grad_weights_2 = output_error * self.a1
    grad_bias_2 = output_error

    # Calculate hidden layer errors
    hidden_errors = output_error * self.weights_2 *
```

```
sigmoid_derivative(self.z1)

        # Calculate gradients for weights between inputs and hidden layer
        grad_weights_1 = np.outer(hidden_errors, inputs)
        grad_bias_1 = hidden_errors

        return grad_weights_1, grad_bias_1, grad_weights_2, grad_bias_2

    # Training function
    def train(self, X, y, epochs=1000, learning_rate=0.01):
        for epoch in range(epochs):
            for i in range(len(X)):
                inputs = X[i]
                y_true = y[i]

                # Forward pass
                self.forward(inputs)

                # Backward pass
                grad_w1, grad_b1, grad_w2, grad_b2 = self.backward(inputs,
y_true)

                # Update weights and biases
                self.weights_1 -= learning_rate * grad_w1
                self.bias_1 -= learning_rate * grad_b1
                self.weights_2 -= learning_rate * grad_w2
                self.bias_2 -= learning_rate * grad_b2

    # Prediction function
    def predict(self, X):
        predictions = []
        for i in range(len(X)):
            pred = self.forward(X[i])
            predictions.append(1 if pred >= 0.5 else 0)
        return predictions

# Load the dataset
import pandas as pd

# Load the dataset from the provided input
data = pd.read_csv('diabetes_data.csv')

# Prepare the input features and target
X = data[['Age', 'BMI', 'BGL']].values
y = data['Class'].values

# Scale the input features (optional but improves performance)
X = (X - X.mean(axis=0)) / X.std(axis=0)

# Initialize and train the neural network
nn = SimpleNeuralNetwork()
nn.train(X, y, epochs=100, learning_rate=0.01)

# Make predictions
predictions = nn.predict(X)
```

```
# Evaluate the model
accuracy = np.mean(predictions == y)
print(f'Accuracy: {accuracy * 100:.2f}%')

# Display the predictions
results = pd.DataFrame({
    'Age': data['Age'],
    'BMI': data['BMI'],
    'Blood Glucose Level': data['BGL'],
    'Actual': y,
    'Predicted': predictions
})
print(results)
```

7.8 Understanding the Code: Building a Simple Neural Network

The code provided here is a practical implementation of the concepts we've been exploring, including how multiple neurons can work together in a network to make predictions. Let's break down the code step by step to see how it builds on what you've learned so far.

7.8.1 The Basics: Activation and Loss Functions

To start, we define some core functions that are essential for our neural network:

- Sigmoid Function: This is the activation function we've been using throughout the chapter. It squashes the output of each neuron to a value between 0 and 1:

```
def sigmoid(x):
  return 1 / (1 + np.exp(-x))
```

- Sigmoid Derivative: This function helps us during backpropagation (more on that later) by measuring how sensitive the output is to changes in the input:

```
def sigmoid_derivative(x):
  return sigmoid(x) * (1 - sigmoid(x))
```

- Log Loss (Binary Cross-Entropy): Since we're solving a classification problem (like predicting whether a patient has diabetes), we use the log-loss function to measure how well the network's predictions match the actual labels:

```
def log_loss(y_true, y_pred):
  return - (y_true * np.log(y_pred) + (1 - y_true) * np.log(1 -
y_pred))
```

- Log Loss Derivative: This is used to calculate the gradient (i.e., how much we need to adjust the weights) during the training process:

```
def log_loss_derivative(y_true, y_pred):
  return (y_pred - y_true) / (y_pred * (1 - y_pred))
```

7.8.2 Defining Our Simple Neural Network

Here, we've implemented a network with two layers:

- Layer 1 (Hidden Layer): Consists of two neurons.
- Layer 2 (Output Layer): Consists of one neuron.

In the code, we define a class called SimpleNeuralNetwork:

```
class SimpleNeuralNetwork:
  def __init__(self):
    # Initialize random weights and biases
    self.weights_1 = np.random.randn(2, 3) # Weights for Neuron
1 and Neuron 2
    self.bias_1 = np.random.randn(2)     # Biases for Neuron 1 and
Neuron 2
    self.weights_2 = np.random.randn(2)   # Weights for Neuron 3
    self.bias_2 = np.random.randn()     # Bias for Neuron 3
```

This initialization follows the idea discussed in the chapter—assigning random weights to break symmetry and allow each neuron to learn unique patterns.

7.8.3 Forward Pass: Computing the Output

The forward pass is where we feed input data through the network to generate predictions. Here's how it works:

1. Hidden Layer: The inputs are multiplied by the weights, and a bias is added. The result is passed through the sigmoid activation function.
2. Output Layer: The outputs from the hidden layer are combined, and the final output is calculated using another sigmoid function.

```
def forward(self, inputs):
  # Layer 1: Hidden layer
  self.z1 = np.dot(self.weights_1, inputs) + self.bias_1
  self.a1 = sigmoid(self.z1)
  # Layer 2: Output layer
  self.z2 = np.dot(self.weights_2, self.a1) + self.bias_2
  self.a2 = sigmoid(self.z2)
  return self.a2
```

7.8.4 Backward Pass: Adjusting the Weights

The backward pass involves calculating how much each weight and bias contributed to the prediction error. This is known as backpropagation, which we introduced in the chapter.

- We start by calculating the error at the output layer.
- Then, we propagate this error backward through the hidden layer to adjust the weights.

```
def backward(self, inputs, y_true):
  # Calculate output error
  y_pred = self.a2
  output_error = log_loss_derivative(y_true, y_pred) *
sigmoid_derivative(self.z2)
  # Gradients for weights and biases between output and hidden
layers
  grad_weights_2 = output_error * self.a1
  grad_bias_2 = output_error
```

(continued)

```
  # Errors for the hidden layer
  hidden_errors = output_error * self.weights_2 *
sigmoid_derivative(self.z1)
  grad_weights_1 = np.outer(hidden_errors, inputs)
  grad_bias_1 = hidden_errors
  return grad_weights_1, grad_bias_1, grad_weights_2,
grad_bias_2
```

7.8.5 Training the Network

The train function repeatedly adjusts the weights using the gradients calculated in the backward pass. We do this for a set number of epochs to minimize the error:

```
def train(self, X, y, epochs=1000, learning_rate=0.01):
  for epoch in range(epochs):
    for i in range(len(X)):
      inputs = X[i]
      y_true = y[i]
      # Forward pass
      self.forward(inputs)
      # Backward pass
      grad_w1, grad_b1, grad_w2, grad_b2 = self.backward(inputs,
y_true)
      # Update weights and biases
      self.weights_1 -= learning_rate * grad_w1
      self.bias_1 -= learning_rate * grad_b1
      self.weights_2 -= learning_rate * grad_w2
      self.bias_2 -= learning_rate * grad_b2
```

7.8.6 Making Predictions

Finally, we can use the trained network to make predictions on new data:

```
def predict(self, X):
  predictions = []
```

(continued)

```
    for i in range(len(X)):
        pred = self.forward(X[i])
        predictions.append(1 if pred >= 0.5 else 0)
    return predictions
```

7.8.7 *Testing the Network with Real Data*

The code concludes by loading a dataset (diabetes data), normalizing the input features, and training the neural network. It then evaluates the accuracy of the model:

```
data = pd.read_csv('diabetes_data.csv')
X = data[['Age', 'BMI', 'BGL']].values
y = data['Class'].values
# Feature scaling
X = (X - X.mean(axis=0)) / X.std(axis=0)
# Initialize and train the neural network
nn = SimpleNeuralNetwork()
nn.train(X, y, epochs=100, learning_rate=0.01)
# Make predictions and evaluate
predictions = nn.predict(X)
accuracy = np.mean(predictions == y)
print(f'Accuracy: {accuracy * 100:.2f}%')
```

7.9 Activation Functions in Artificial Neural Networks

In the Logistic Regression chapter and earlier in this chapter, we have primarily used the sigmoid activation function. However, it's important to understand that activation functions play a crucial role in the performance of neural networks.

Why Do We Need Activation Functions?

Activation functions are essential because they introduce non-linearity into the neural network. Without activation functions, the network would be equivalent to a single linear transformation, no matter how many layers it has. This linearity means that the model would not be able to learn complex patterns or represent non-linear relationships in the data.

Here's a deeper explanation:

- Non-linearity: Activation functions allow the model to learn from errors and adjust to non-linear patterns in the data. This flexibility is necessary for the model to generalize well and avoid overfitting.
- Sensitivity to Input Changes: Without an activation function, small changes in the input features would result in small changes in the output, which might not capture the underlying complexities. Introducing a non-linear activation function can magnify these changes, making the network more sensitive to important patterns.

7.9.1 Common Activation Functions in Artificial Neural Networks

Table 7.1 is a comparison of some of the most commonly used activation functions, along with their formulas, best use cases, and derivatives.

7.9.2 Explanation of Activation Functions

1. Sigmoid: The sigmoid function squashes the input into a range between 0 and 1, making it useful for binary classification. However, it suffers from vanishing gradients for very large or very small inputs, making training slow.
2. Tanh (Hyperbolic Tangent): The tanh function scales inputs to a range between −1 and 1, centering the output around zero. This can lead to faster convergence

Table 7.1 Activation functions

Activation function	Formula	Best uses	Derivative
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	Binary classification, shallow networks	$\sigma'(z) = \sigma(z)(1 - \sigma(z))$
Tanh (Hyperbolic Tangent)	$\tan h(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Hidden layers in RNNs, problems where outputs need to be centered around zero	$\tan h'(z) = 1 - \tan h^2(z)$
ReLU (Rectified Linear Unit)	ReLU(z) = max (0, z)	Deep networks, image data, computational efficiency	$\text{ReLU}'(z) = \begin{cases} 1, z > 0 \\ 0, z \leq 0 \end{cases}$
Leaky ReLU	Leaky ReLU (z) = max(0.01z, z)	Avoids dying ReLU problem, networks with sparse activations	$\text{Leaky ReLU}'(z) = \begin{cases} 1, z > 0 \\ 0.01, z \leq 0 \end{cases}$
Softmax	$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$	Multiclass classification in the output layer	Softmax'(z_i) = Softmax(z_i) (1 − Softmax(z_i))

compared to the sigmoid function but still suffers from the vanishing gradient problem.
3. ReLU (Rectified Linear Unit): ReLU is widely used in deep networks due to its computational efficiency and ability to mitigate the vanishing gradient problem. It outputs the input directly if positive; otherwise, it outputs zero. However, neurons can become inactive if they always output zero during training (the "dying ReLU" problem).
4. Leaky ReLU: A variant of ReLU that allows a small, non-zero gradient when the input is negative, thus preventing neurons from dying.
5. Softmax: Softmax is used in the output layer of networks for multiclass classification. It converts raw output scores into probabilities, ensuring they sum up to 1.

Where to Use Each Activation Function
- Sigmoid: Use in output layers for binary classification.
- Tanh: Suitable for hidden layers in RNNs and when output needs to be centered around zero.
- ReLU: Best for hidden layers in CNNs and other deep networks, especially with image data.
- Leaky ReLU: Use when encountering the dying ReLU problem.
- Softmax: Ideal for the output layer in multiclass classification problems.

Extending Artificial Neural Networks for Multiclass Classification
So far, we've seen examples of binary classification, where we had only two distinct classes (e.g., predicting whether a customer will make a purchase or not). But what if we have multiple classes to distinguish between?

For instance, let's consider a scenario where we want to classify customers into one of the following categories:

1. High-Value Customer
2. Loyal Customer
3. Occasional Buyer
4. Inactive Customer

This classification is based on the following features:

- Customer Age
- Purchase Frequency
- Total Spending Last Year
- Product Category Preference

In this scenario, we have four classes, meaning the output layer in our Artificial Neural Network will need to have four nodes—one node for each class, as shown in Fig. 7.7. However, since there are multiple possible classes, we cannot use the sigmoid activation function in the output layer.

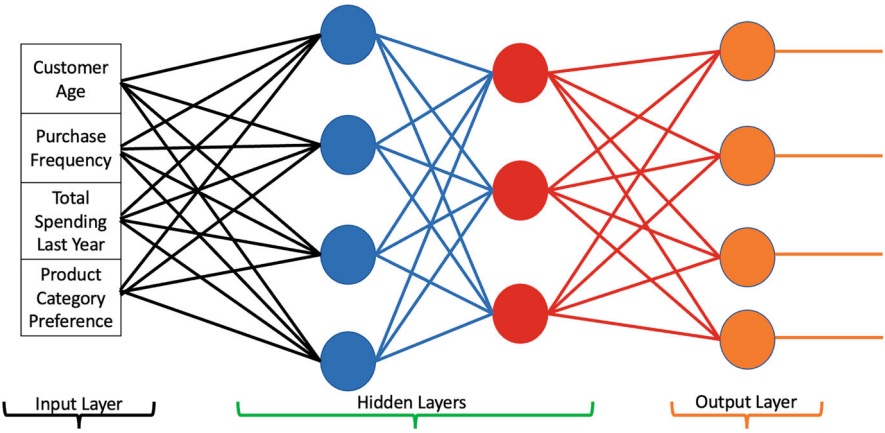


Fig. 7.7 Architecture of a neural network for multiclass classification

7.9.3 Why Use Softmax for Multiclass Classification?

The Softmax activation function is the ideal choice for multiclass classification problems where the classes are mutually exclusive. It is used in the output layer of the network when there are more than two classes.

Here's how the Softmax function works:

- Softmax takes the raw output from the last hidden layer and converts it into probabilities.
- It ensures that each output node has a value between 0 and 1, and importantly, that the sum of all output node probabilities equals 1.

This makes the Softmax function perfect for multiclass classification, as it allows the model to assign a probability to each class. The class with the highest probability is chosen as the model's prediction (Basheer and Hajmeer 2000).

7.9.4 Understanding the Softmax Output

Let's assume that after training, our network receives a set of inputs (features such as customer age, purchase frequency, etc.). The network then produces the following outputs from the four nodes in the output layer:

- Output Node 1 (High-Value Customer): 0.20
- Output Node 2 (Loyal Customer): 0.30
- Output Node 3 (Occasional Buyer): 0.45
- Output Node 4 (Inactive Customer): 0.05

These outputs are the probabilities for each class. Notice that:

- Each node's output is between 0 and 1.
- The sum of all probabilities equals 1.

Since Output Node 3 (corresponding to the class "Occasional Buyer") has the highest probability (0.45), the network classifies the input as belonging to this class.

How the Softmax Function Works Mathematically
The Softmax function is defined as:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}} \tag{7.11}$$

Where:

- z_i is the input to the Softmax function for the ith class.
- n is the total number of classes.
- e^{z_i} exponentiates the raw score for class i, making all values positive.
- The denominator normalizes these exponentiated values so that their sum is 1.

This normalization process allows the outputs to be interpreted as probabilities.

7.9.5 Example Calculation

Let's say the raw outputs (logits) from the final layer of our neural network are:

- $z_1 = 2.0$
- $z_2 = 2.5$
- $z_3 = 3.0$
- $z_4 = 1.5$

To convert these logits into probabilities using Softmax:

1. Exponentiate each value:
   - $e^{2.0} = 7.39$
   - $e^{2.5} = 12.18$,
   - $e^{3.0} = 20.09$,
   - $e^{1.5} = 4.48$,
2. Sum the exponentiated values:
   7.39 + 12.18 + 20.09 + 4.48 = 44.14
3. Divide each exponentiated value by the sum to get probabilities:
   - $\text{Softmax}(z_1) = \frac{7.39}{44.14} \approx 0.17$.
   - $\text{Softmax}(z_2) = \frac{12.18}{44.14} \approx 0.28$

- $\text{Softmax}(z_3) = \frac{20.09}{44.14} \approx 0.46$
- $\text{Softmax}(z_4) = \frac{4.48}{44.14} \approx 0.10$

Thus, the class with the highest probability (0.46) is predicted as the final output.

In multiclass classification problems, the Softmax activation function is indispensable. By converting raw outputs into probabilities, it allows the network to make confident predictions across multiple classes, ensuring that the sum of the probabilities is always 1. This makes it easier to interpret the results and utilize them in practical applications.

However, it's important to note that if we have a multiclass problem, the differences primarily affect the output layer only, as shown in Fig. 7.7. The structure of the hidden layers (i.e., the number of hidden layers and the number of neurons in each) remains the same whether we are dealing with binary or multiclass classification.

The choice of the number of hidden layers and neurons is crucial for the overall performance of the network:

- A low number of layers and neurons may result in underfitting, where the model is too simple to capture the underlying patterns in the data.
- Conversely, having too many layers and neurons can lead to overfitting, where the model becomes too complex and starts memorizing the training data rather than generalizing to new inputs.

Thus, while the output layer's configuration changes for multiclass classification, the architecture of the hidden layers needs to be carefully tuned to achieve optimal performance, regardless of whether it's a binary or multiclass classification problem.

7.10 Loss Function for Multiclass Classification with Softmax: Cross-Entropy Loss

Now that we understand how the Softmax activation function helps us classify data into multiple categories, the next question is: How does the network learn to make better predictions? This is where the loss function comes into play.

For multiclass classification problems using Softmax, the most commonly used loss function is called Cross-Entropy Loss (sometimes known as Categorical Cross-Entropy). Let's break down what this means and why it's useful.

7.10.1 What Is Cross-Entropy Loss?

Imagine you're trying to guess someone's favorite type of fruit. You might have several options to choose from—Apple, Banana, Cherry, Grape—but you can only pick one. Now, let's say you have some clues, and based on those, you think there's

a 70% chance it's Apple, 20% chance it's Banana, 5% chance it's Cherry, and 5% chance it's Grape. But when you finally find out, it turns out the correct answer was Banana.

The Cross-Entropy Loss function helps the network understand how wrong it was. It essentially compares the probability distribution predicted by the network (like your fruit guess) with the actual correct answer. If the network assigns a high probability to the correct class (e.g., Banana), the loss will be low. But if it assigns a low probability to the correct class, the loss will be high. The goal during training is to minimize this loss.

The Formula for Cross-Entropy Loss

The Cross-Entropy Loss for a single prediction is calculated using the following formula:

$$\text{Loss} = -\sum_{i=1}^{n} y_i . \log(\widehat{y}_i) \tag{7.12}$$

Here's what each term means:

- n: The total number of classes (e.g., 4 if we're classifying customers into 4 categories).
- y_i: The actual label (1 if the class is correct, 0 if not).
- $\widehat{y}_i$: The probability assigned by the network to class i (the output of the Softmax function).

Let's break it down with a real-world example:

Suppose our neural network is trying to classify a customer as either a High-Value Customer, Loyal Customer, Occasional Buyer, or Inactive Customer. The correct label is Loyal Customer. If the network outputs the following probabilities:

- High-Value Customer: 0.10
- Loyal Customer: 0.50
- Occasional Buyer: 0.30
- Inactive Customer: 0.10

The actual label is Loyal Customer, so:

- $y_1 = 0$
- $y_2 = 1$
- $y_3 = 0$
- $y_4 = 0$

The Cross-Entropy Loss would be calculated as:

$$\text{Loss} = -[0 \times \log(0.10) + 1 \times \log(0.50) + 0 \times \log(0.30) + 0 \times \log(0.10)]$$

This simplifies to:

$$\text{Loss} = -\log(0.50) = 0.693$$

The network was not terribly off, but it also wasn't super confident about the correct class. If the probability for the correct class had been higher, the loss would have been smaller.

7.10.2 The Derivative of Cross-Entropy Loss with Softmax

Now, let's talk about how the network uses this loss to improve its predictions during training. To update its weights, the network needs to calculate the gradient of the loss function with respect to its predictions. In simpler terms, it needs to understand how much each prediction contributed to the error.

Here's the beautiful part: when using Cross-Entropy Loss together with Softmax, the math simplifies nicely. The derivative of the loss with respect to the predicted probability $\widehat{y}_i$ is:

$$\frac{\partial \text{Loss}}{\partial \widehat{y}_i} = \widehat{y}_i - y_i \tag{7.13}$$

What does this mean?

- If the network's prediction $\widehat{y}_i$ is close to the actual label y_i (i.e., if the network was confident and correct), the gradient will be small.
- If the network's prediction was way off, the gradient will be large, which means the network will adjust its weights more significantly in the next round of training.

Let's revisit our customer classification example. If the network predicted a 50% probability for the correct class but should have been closer to 100%, the difference ($\widehat{y}_i - y_i = 0.50 - 1 = -0.50$) shows there's room for improvement. This feedback is what drives the learning process.

To summarize:

- The Softmax activation function converts raw outputs into probabilities that sum to 1.
- The Cross-Entropy Loss measures how well those probabilities match the actual classes.
- The derivative of the loss function helps the network understand how to adjust its weights to get better at making predictions.

This combination of Softmax and Cross-Entropy is powerful because it's efficient, intuitive, and works seamlessly for multiclass classification problems like customer segmentation.

7.11 Retouching on Gradient Descent

In Fig. 7.8, we have a neural network designed for multiclass classification with:

- Two hidden layers:
  - The first hidden layer has four neurons.
  - The second hidden layer has three neurons.

- The output layer has four neurons, corresponding to the customer categories.
- The hidden layers use the ReLU activation function.
- The output layer uses the Softmax activation function.

How Gradient Descent Works in this Network

1. Forward Pass:
   - We pass the input features (like Customer Age, Purchase Frequency, etc.) through the network.
   - Each hidden layer applies the ReLU activation function, and the output layer uses Softmax to generate class probabilities.
   - Once the forward pass is complete, we get a probability distribution across the four output classes.
2. Calculating the Error:
   - After obtaining the network's predictions, we compare them with the actual class labels using the Cross-Entropy Loss function.
   - This loss measures how far off the predictions are.

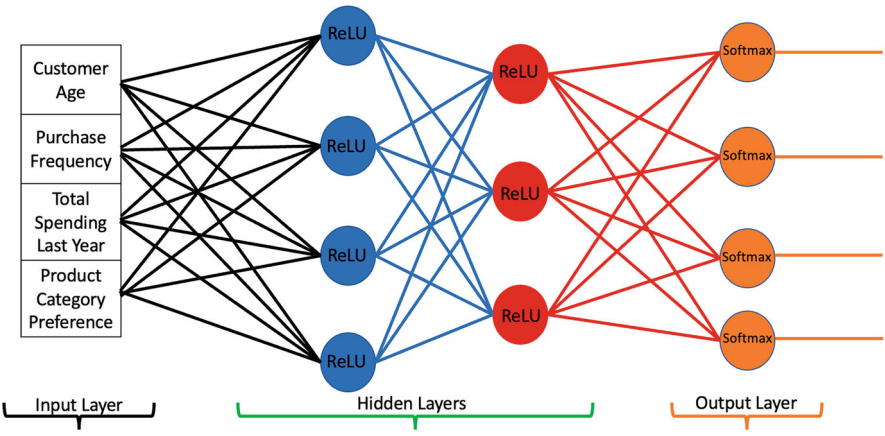


Fig. 7.8 Multiclass classification network with two hidden layers using ReLU and an output layer using Softmax

3. Backward Pass—Gradient Descent:
   - We begin backpropagation from the output layer, where we compute the gradient of the loss with respect to each output neuron.
   - Since we're using Softmax combined with Cross-Entropy Loss, the gradient calculation simplifies nicely.
4. Adjusting the Output Layer Weights:
   - For each output neuron:
     - Calculate the gradient using the derivative of the loss function.
     - To adjust the weights connecting the output layer to the previous hidden layer, multiply the gradient by the output values of the neurons from the last hidden layer (i.e., the inputs to the Softmax layer).
     - This tells us how much each weight contributed to the error and how much it needs to be adjusted.
5. Stepping Back to the Second Hidden Layer:
   - Now, we move to the second hidden layer (with three neurons).
   - For each neuron in this layer:
     - Calculate its gradient by summing up the contributions from the output layer neurons it's connected to. This involves multiplying the gradients from the output layer by the weights of the connections.
     - Multiply this sum by the derivative of the ReLU activation function for that neuron.
   - To adjust the weights connecting this hidden layer to the previous hidden layer:
     - Multiply the gradient of each neuron by the inputs it received from the first hidden layer. These inputs are the activations passed through ReLU in the first hidden layer.
     - This step determines how much each weight should be adjusted based on its contribution.
6. Stepping Back to the First Hidden Layer:
   - We repeat the process for the first hidden layer (with four neurons):
     - For each neuron, sum the gradients from the next hidden layer, multiply by the respective weights, and then apply the derivative of the ReLU function.
     - To adjust the weights connecting this layer to the input layer:
     - Multiply the gradient of each neuron by the original input features (like Customer Age, Purchase Frequency, etc.).
     - This ensures that each weight is adjusted based on how much the input features contributed to the error.

7. Updating the Weights:
   - Once all gradients are computed, we adjust the weights using gradient descent.
   - The adjustment is scaled by the learning rate to control the size of the weight updates.
   - If using Stochastic Gradient Descent, weights are updated after each training instance. For batch gradient descent, updates happen after processing an entire batch.
8. Ready for the Next Iteration:
   - After updating the weights, the network is ready for the next training iteration.
   - This process continues iteratively until the network achieves good performance on the training data.

7.12 Implementation

Below is the complete code to load data from a CSV file, preprocess it, build the neural network, and train it.

```
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
# Step 1: Load the dataset from a CSV file
data = pd.read_csv('customer_data.csv')
# Step 2: Inspect the dataset (uncomment if you want to see the data)
# print(data.head())
# Step 3: Preprocess the dataset
# Extract features and target
features = ['Customer Age', 'Purchase Frequency', 'Total Spending
Last Year', 'Product Category Preference']
X = data[features]
# Encode the target labels (High-Value Customer, Loyal Customer,
etc.)
label_encoder = LabelEncoder()
data['Class'] = label_encoder.fit_transform(data['Class']) #
```

(continued)

```
Encode class labels as integers
y = data['Class']
# Convert labels to one-hot encoding
y = to_categorical(y, num_classes=4)
# Step 4: Normalize the features
scaler = StandardScaler()
X = scaler.fit_transform(X)
# Step 5: Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
# Step 6: Build the model
model = Sequential([
  Dense(4, activation='relu', input_shape=(4,)), # First hidden
layer with 4 neurons
  Dense(3, activation='relu'),          # Second hidden layer with
3 neurons
  Dense(4, activation='softmax')          # Output layer with
4 neurons (Softmax)
])
# Step 7: Compile the model
model.compile(optimizer='adam',
       loss='categorical_crossentropy',
       metrics=['accuracy'])
# Step 8: Train the model
history = model.fit(X_train, y_train, epochs=20, batch_size=8,
validation_split=0.2)
# Step 9: Evaluate the model on the test set
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.2f}")
# Step 10: Make predictions (optional)
predictions = model.predict(X_test)
predicted_classes = np.argmax(predictions, axis=1)
# Convert predicted classes back to original labels
predicted_labels = label_encoder.inverse_transform
(predicted_classes)
# Print the first 10 predictions
print("Sample Predictions:", predicted_labels[:10])
```

The code implements a neural network with *two hidden layers* using *ReLU activation* and an output layer with *Softmax activation*. The network is trained on a customer classification dataset to classify customers into one of four categories: *High-Value Customer, Loyal Customer, Occasional Buyer*, or *Inactive Customer*.

Here's a step-by-step breakdown of how this code corresponds to the structure of the network in Fig. 7.8.

7.12.1 Step 1: Loading the Data

```
data = pd.read_csv('customer_data.csv')
```

- The data is loaded from a CSV file named `'customer_data.csv'`.
- The CSV is expected to contain columns for the input features and the target class.

7.12.2 Step 2: Inspecting the Data (Optional)

```
# print(data.head())
```

- This is commented out but can be used to preview the first few rows of the dataset.

7.12.3 Step 3: Preprocessing the Data

Extract Features and Target

```
features = ['Customer Age', 'Purchase Frequency', 'Total Spending
Last Year', 'Product Category Preference']
X = data[features]
```

- The features used in this network are exactly those shown in Fig. 7.8:
  - Customer Age
  - Purchase Frequency
  - Total Spending Last Year
  - Product Category Preference

Encode the Target Labels

```
label_encoder = LabelEncoder()
data['Class'] = label_encoder.fit_transform(data['Class'])
y = data['Class']
```

- The target column (i.e., customer class) is encoded as integers:
  - For example:

    `High-Value Customer` → `0`
    `Loyal Customer` → `1`
    `Occasional Buyer` → `2`
    `Inactive Customer` → `3`

One-Hot Encoding

```
y = to_categorical(y, num_classes=4)
```

- The target labels are converted to *one-hot encoding* because the output layer of the network uses Softmax activation.
  - Example: If the class is `High-Value Customer`, the one-hot encoding would be `[1, 0, 0, 0]`.

7.12.4 Step 4: Normalizing the Features

```
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

- **Standardization** ensures that all features have a mean of 0 and a standard deviation of 1, which helps the network converge faster during training.

7.12.5 Step 5: Splitting the Data

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

- The dataset is split into *80% training data* and *20% testing data*.

7.12.6 Step 6: Building the Neural Network Model

```
model = Sequential([
  Dense(4, activation='relu', input_shape=(4,)), # First hidden
layer with 4 neurons
  Dense(3, activation='relu'),          # Second hidden layer with
3 neurons
  Dense(4, activation='softmax')          # Output layer with
4 neurons (Softmax)
])
```

Correspondence to Fig. 7.8:

- **Sequential Model**:
  - The `Sequential` model in Keras allows us to build a neural network *layer by layer* in a straightforward manner.
  - It is used when our network has a simple, linear stack of layers where each layer has exactly one input tensor and one output tensor.
  - In this network, each layer's output serves as the input to the next layer, as shown in Fig. 7.8.
- **Dense Layers**:
  - The `Dense` layer is a fully connected (or densely connected) neural network layer, where each neuron in the layer is connected to every neuron in the previous and next layers.
  - The general syntax for a dense layer is:

```
Dense(units, activation='activation_function',
input_shape=(input_dim,))
```

`units`: The number of neurons in the layer.
`activation`: The activation function to be applied to the outputs of the neurons in that layer.
`input_shape`: Specifies the shape of the input data. This is only needed for the first layer.

- **Layer-by-Layer Breakdown**:
  First Hidden Layer:

```
Dense(4, activation='relu', input_shape=(4,))
```

  - **4 neurons**: This layer has four neurons.
  - **ReLU activation**: The Rectified Linear Unit (ReLU) activation function is used to introduce non-linearity into the model, helping it learn complex patterns.
  - **input_shape = (4,)**: The input shape is `(4,)` because we have four input features (Customer Age, Purchase Frequency, Total Spending Last Year, Product Category Preference).
  - This layer takes the raw inputs, applies weights, biases, and the ReLU function to transform them before passing to the next layer.

  Second Hidden Layer:

```
Dense(3, activation='relu')
```

  - **3 neurons**: This layer has three neurons.
  - **ReLU activation**: Again, the ReLU function is applied here to introduce non-linearity.
  - This layer receives the outputs from the first hidden layer and processes them similarly.

  Output Layer:

```
Dense(4, activation='softmax')
```

- **4 neurons**: The output layer has four neurons because we are performing multiclass classification with four possible categories.
- **Softmax activation**: The Softmax activation function is used here because it converts the raw scores into probabilities that sum to 1, allowing us to interpret the output as a probability distribution over the four classes.
- The neuron with the highest probability will determine the predicted class.

7.12.7 Step 7: Compiling the Model

```
model.compile(optimizer='adam',
      loss='categorical_crossentropy',
      metrics=['accuracy'])
```

- **Adam optimizer** is used for adjusting the weights.
- **Categorical Cross-Entropy Loss** is used since we have a multiclass classification problem with one-hot encoded labels.
- **Accuracy** is used as the evaluation metric.

7.12.8 Step 8: Training the Model

```
history = model.fit(X_train, y_train, epochs=20, batch_size=8,
validation_split=0.2)
```

- The model is trained for **20 epochs** with a **batch size of 8**.
- **20% of the training data** is used for validation during training.

7.12.9 Step 9: Evaluating the Model

```
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.2f}")
```

- The model's performance is evaluated on the test set.
- The **accuracy** on the test set is printed.

7.12.10 Step 10: Making Predictions (Optional)

```
predictions = model.predict(X_test)
predicted_classes = np.argmax(predictions, axis=1)
# Convert predicted classes back to original labels
predicted_labels = label_encoder.inverse_transform
(predicted_classes)
print("Sample Predictions:", predicted_labels[:10])
```

- The trained model is used to make predictions on the test set.
- The predicted class labels are converted back to their original form (e.g., `High-Value Customer`, `Loyal Customer`).
- The first ten predictions are printed.

This code builds and trains a neural network based on the architecture in Fig. 7.8. It uses *ReLU activation* in the hidden layers and *Softmax activation* in the output layer to classify customers into one of four categories based on input features like age, purchase frequency, total spending, and product preference.

7.13 Using Artificial Neural Networks for Regression Problems

We've explored binary classification and multiclass classification using ANNs. Now, let's shift our focus to using ANNs for regression problems, where the goal is to predict a continuous numerical value rather than discrete categories.

Example: Predicting Customer Lifetime Value (CLV)
Let's revisit our customer data example. Previously, we classified customers into categories like High-Value, Loyal, Occasional Buyer, or Inactive using features such as:

- Customer Age
- Purchase Frequency
- Total Spending Last Year
- Product Category Preference

However, in a regression scenario, instead of predicting a class, we want to estimate a numerical value—such as a customer's Lifetime Value (CLV). This value represents how much revenue a customer is expected to bring in over their relationship with the company.

In a regression network, the output layer will have a single neuron since we are predicting a continuous value rather than multiple categories.

7.13.1 Choosing the Right Activation Function for Regression

For regression problems, the choice of activation function in the output layer is critical. Unlike classification tasks, where we used the Softmax or Sigmoid functions, here we aim to predict a real number that can take any value within a reasonable range.

- Activation Function: The most appropriate activation function for the output layer in regression is linear (i.e., no activation function). This is because a linear activation function allows the network to produce outputs that can take on any continuous value, both positive and negative.
  - Formula: $f(x) = x$
  - Derivative: The derivative of a linear function is 1, which simplifies gradient calculations during backpropagation.

7.13.2 Loss Function for Regression

For regression tasks, the loss function commonly used is the Mean Squared Error (MSE). The MSE measures how far the network's predictions are from the actual continuous values.

- Mean Squared Error (MSE):

$$\text{Loss} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \widehat{y}_i) \tag{7.14}$$

  - N: Number of data points.
  - y_i: Actual value for the ith instance,
  - $\widehat{y}_i$: Predicted value for the ith instance

- Derivative of MSE:
  The gradient of the MSE with respect to the network's prediction ($\widehat{y}_i$) is:

$$\frac{\partial \text{Loss}}{\partial \widehat{y}_i} = -2(y_i - \widehat{y}_i) \tag{7.15}$$

This derivative indicates how much we need to adjust the weights to reduce the error for each prediction.

Summary
In regression problems, we adjust the ANN architecture as follows:

- The output layer uses a linear activation function to predict continuous values.
- The loss function is Mean Squared Error to minimize the difference between predicted and actual values.
- The derivative of the linear activation function is straightforward, allowing the network to learn efficiently without restricting the range of predicted values.

7.14 Concluding Remarks

In this chapter, we explored the foundational concepts and practical implementations of Artificial Neural Networks, extending our understanding of regression and logistic regression into more complex machine learning models. By abstracting logistic regression into computational units (neurons) and linking them into networks, we have demonstrated how ANNs can handle more intricate patterns and relationships in data.

We began with the basics, revisiting logistic regression and introducing neural network terminology, emphasizing the roles of weights, biases, activation functions, and layers. By progressively building from simple single-layer networks to deeper architectures, we illustrated how these networks can be scaled to solve more complex tasks.

Key Takeaways Include
1. Understanding the Network Structure: We examined how input layers, hidden layers, and output layers interact, and how neurons process inputs to produce predictions. The role of activation functions, particularly the sigmoid, ReLU, and softmax functions, was emphasized in achieving non-linearity and handling different types of classification problems.
2. Training Neural Networks: We delved into backpropagation, highlighting how errors are propagated backward through the network to adjust weights using gradient descent. The mathematical derivations for computing gradients, particularly for common loss functions like Mean Squared Error and Cross-Entropy Loss, were carefully explained.
3. Practical Implementation: The chapter provided step-by-step code examples, showing how to build, train, and evaluate simple neural networks using Python.

The implementation of both binary and multiclass classification networks illustrated the flexibility of ANNs in various real-world scenarios.

4. Extending to Regression Tasks: Finally, we discussed using ANNs for regression problems, where the output layer uses a linear activation function to predict continuous values. This showcased the versatility of neural networks beyond classification tasks.

Chapter 8
Convolutional Operations

Abstract This chapter explores the foundational and advanced techniques of feature extraction in computer vision using convolutional methods. We begin with the use of simple filters to detect edges, textures, and shapes—transforming raw pixel data into structured feature maps that enable neural networks to interpret visual information more effectively.

Building on this, we introduce *Cubixel kernels*, an advanced concept that extends beyond traditional 2D convolutions. By incorporating the volumetric properties and aspect ratios of pixels, Cubixel kernels extract more nuanced features across both spatial dimensions and color channels. This approach enhances the network's ability to capture complex visual patterns and inter-channel relationships, paving the way for more accurate and robust image analysis.

8.1 Extracting Features from Images

In the previous chapter, we learned how to train an ANN on tabular data by simply feeding one instance at a time to the neural network. This approach works well for ANNs because the input layer expects a one-dimensional vector, such as 10, 30, 24, 2.5.

But what if we want to train an ANN to classify images, for example, to distinguish between cats and dogs? As we discussed in the feature extraction chapter, images are inherently two-dimensional, defined by their width and height, and consist of pixels that represent color intensities. If the image is in color using the RGB format, each pixel has three values: one for red, one for green, and one for blue, as illustrated in Fig. 8.1 and Table 8.1.

So, how do we feed these image features into an ANN?

The solution is to flatten the image into a one-dimensional vector. We start with the red channel: we take the first row, then concatenate the second row, followed by the third row, continuing this process until all rows of the red channel are flattened into a single vector. Once the red channel is completed, we then concatenate all the rows from the green channel, followed by all the rows from the blue channel (Z. Li et al. 2022).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_8



Fig. 8.1 Lena image with a 32×32 region extracted around the eye area, showing individual pixels as colored squares

Figure 8.2 shows an example of a 3×3 segment of an image with its three channels (Red, Green, and Blue) before and after flattening it into a single vector.

Now that we have a one-dimensional vector, we can indeed feed it into a neural network to classify the image based on its features. However, this approach is not very efficient and often fails to fully capture the rich information present in the image. By simply using the raw pixel values, we are missing opportunities to better understand the structure and patterns within the image (Yao et al. 2022).

If we leverage some image processing techniques, we can extract higher level features from the image. These features provide the neural network with additional context and information beyond just the pixel values, which can significantly improve the accuracy of classification.

These specialized image processing techniques are known as convolutional operations. Through these operations, we can transform the original image in ways that highlight its important features, such as edges, textures, and shapes. By extracting these features, we give the neural network a much deeper understanding of the image content, which leads to better performance in tasks like object detection and image classification.

To extract features from images using convolutional operations, we use a filter (also known as a kernel), which is essentially a small matrix with specific values. This filter convolves over the pixels of the image to calculate new values based on the filter's values and the values of the surrounding pixels. The result is a transformed image known as a feature map (Dhruv and Naskar 2020).

The new feature map highlights specific patterns or features in the original image, depending on the kernel used and the values of adjacent pixels. For instance, different filters can be used to detect vertical edges, horizontal edges, corners, textures, and other patterns. Each filter is designed with a specific set of values to emphasize a particular type of feature (Chitradevi and Srimathi 2014; Marion 2013).

Table 8.1 RGB values for each pixel in the 16×16 area of the extracted 32×32 region, displaying the numerical color intensities that define each pixel's color

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	[214, 177, 170]	[215, 180, 169]	[215, 183, 181]	[223, 187, 180]	[219, 188, 168]	[214, 181, 167]	[211, 167, 136]	[200, 111, 100]	[198, 83, 85]	[204, 83, 96]	[207, 97, 107]	[214, 107, 103]	[214, 115, 122]	[220, 112, 119]	[224, 122, 121]	[225, 123, 118]
2	[214, 181, 175]	[217, 188, 180]	[220, 189, 176]	[219, 194, 171]	[218, 185, 173]	[206, 160, 135]	[196, 95, 86]	[196, 80, 93]	[202, 83, 94]	[205, 85, 91]	[211, 99, 109]	[217, 103, 105]	[218, 111, 105]	[223, 121, 121]	[224, 121, 117]	[221, 126, 122]
3	[217, 184, 176]	[218, 189, 180]	[219, 193, 174]	[217, 185, 163]	[206, 143, 109]	[183, 81, 80]	[192, 74, 84]	[193, 92, 96]	[202, 98, 109]	[213, 94, 97]	[210, 96, 100]	[210, 99, 105]	[218, 103, 99]	[218, 115, 114]	[217, 116, 112]	[217, 109, 109]
4	[223, 192, 169]	[222, 192, 173]	[218, 179, 161]	[201, 133, 93]	[184, 75, 79]	[187, 77, 88]	[198, 76, 81]	[201, 85, 87]	[209, 90, 95]	[210, 88, 94]	[201, 89, 92]	[208, 88, 89]	[210, 107, 110]	[213, 102, 104]	[215, 99, 100]	[211, 96, 102]
5	[223, 195, 176]	[213, 175, 147]	[193, 108, 81]	[176, 67, 74]	[182, 61, 67]	[187, 74, 80]	[196, 77, 82]	[200, 76, 77]	[199, 77, 83]	[204, 82, 91]	[201, 82, 80]	[198, 83, 90]	[203, 93, 86]	[196, 88, 88]	[201, 80, 78]	[203, 91, 86]
6	[209, 169, 147]	[179, 93, 74]	[154, 53, 69]	[168, 69, 82]	[184, 57, 62]	[181, 70, 78]	[192, 72, 77]	[188, 72, 78]	[185, 65, 70]	[173, 57, 77]	[170, 62, 66]	[177, 63, 73]	[182, 68, 69]	[180, 67, 76]	[184, 71, 75]	[184, 76, 71]
7	[165, 74, 70]	[153, 52, 64]	[154, 53, 70]	[175, 59, 73]	[175, 65, 68]	[173, 61, 74]	[180, 65, 72]	[183, 68, 74]	[176, 66, 76]	[169, 64, 73]	[142, 42, 61]	[151, 39, 61]	[148, 37, 58]	[141, 31, 57]	[152, 48, 60]	[139, 41, 58]
8	[150, 46, 69]	[158, 47, 61]	[155, 43, 66]	[174, 59, 68]	[164, 52, 67]	[156, 41, 57]	[138, 35, 60]	[141, 36, 60]	[157, 55, 70]	[152, 52, 71]	[130, 32, 60]	[124, 33, 63]	[125, 37, 65]	[123, 30, 63]	[117, 24, 56]	[120, 27, 60]
9	[145, 47, 75]	[162, 52, 66]	[149, 42, 65]	[152, 46, 65]	[148, 36, 60]	[141, 37, 65]	[118, 30, 59]	[116, 19, 57]	[123, 28, 55]	[128, 33, 59]	[115, 25, 59]	[110, 25, 60]	[114, 27, 62]	[114, 22, 55]	[107, 19, 62]	[111, 25, 61]
10	[144, 35, 64]	[143, 36, 61]	[142, 33, 57]	[143, 31, 55]	[129, 30, 63]	[126, 24, 52]	[115, 20, 59]	[108, 20, 57]	[111, 24, 59]	[110, 22, 57]	[110, 20, 55]	[108, 22, 59]	[109, 22, 58]	[108, 18, 56]	[109, 19, 57]	[107, 15, 52]
11	[144, 41, 65]	[135, 23, 57]	[126, 17, 53]	[121, 22, 51]	[115, 26, 63]	[118, 19, 58]	[110, 18, 54]	[110, 22, 60]	[110, 20, 62]	[106, 24, 53]	[112, 28, 69]	[105, 22, 56]	[105, 15, 56]	[108, 20, 55]	[109, 21, 55]	[103, 15, 53]
12	[136, 32, 62]	[120, 27, 58]	[118, 19, 59]	[111, 18, 55]	[104, 16, 53]	[103, 19, 57]	[109, 23, 60]	[109, 21, 67]	[108, 20, 57]	[104, 26, 60]	[111, 26, 60]	[103, 23, 60]	[104, 25, 65]	[103, 14, 54]	[99, 13, 49]	[101, 10, 54]
13	[132, 35, 61]	[111, 20, 57]	[104, 18, 53]	[102, 18, 59]	[104, 24, 61]	[100, 18, 60]	[106, 17, 53]	[106, 17, 56]	[106, 16, 56]	[106, 27, 67]	[102, 17, 56]	[109, 21, 57]	[104, 19, 54]	[104, 14, 57]	[95, 15, 57]	[92, 17, 67]
14	[124, 24, 56]	[101, 11, 46]	[106, 15, 56]	[101, 20, 61]	[102, 19, 57]	[101, 21, 59]	[102, 20, 54]	[107, 24, 57]	[114, 21, 56]	[102, 17, 55]	[99, 12, 55]	[106, 25, 63]	[106, 18, 61]	[105, 19, 63]	[91, 11, 59]	[94, 19, 60]
15	[115, 17, 50]	[101, 11, 55]	[96, 11, 52]	[98, 11, 54]	[99, 23, 67]	[105, 24, 63]	[115, 34, 60]	[133, 49, 71]	[131, 34, 68]	[96, 14, 52]	[104, 16, 56]	[112, 36, 62]	[118, 29, 65]	[97, 16, 58]	[86, 11, 62]	[82, 8, 53]
16	[105, 27, 60]	[92, 16, 60]	[94, 13, 54]	[101, 12, 59]	[106, 23, 55]	[119, 37, 67]	[139, 54, 76]	[148, 61, 84]	[144, 49, 78]	[101, 22, 62]	[113, 32, 67]	[131, 54, 79]	[132, 50, 87]	[102, 18, 61]	[86, 9, 55]	[80, 8, 56]

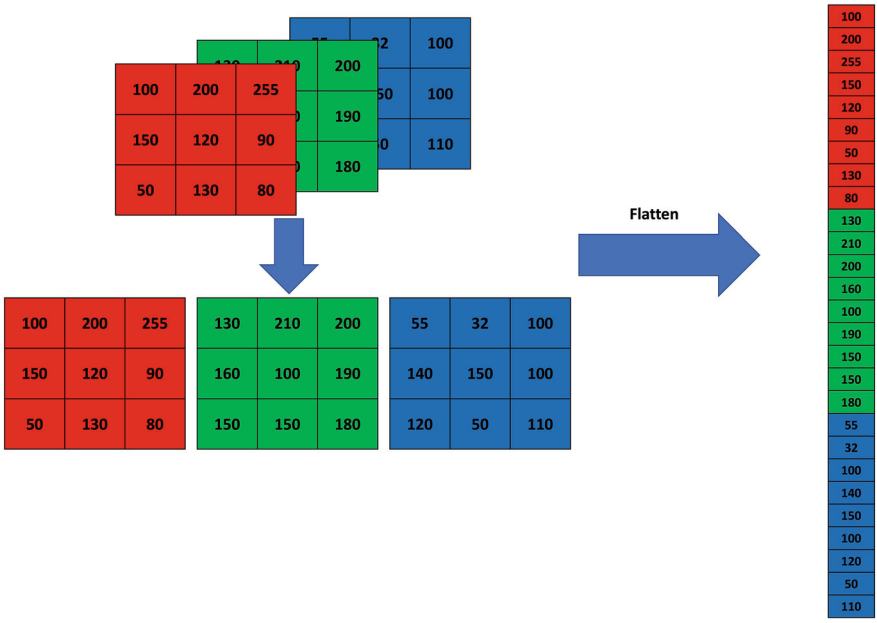


Fig. 8.2 Flattening an RGB image segment

When we apply a single filter, we obtain one feature map that emphasizes a specific characteristic of the image. By applying multiple filters, each with its own set of values, we can generate multiple feature maps. These feature maps collectively capture various aspects of the image, such as shapes, edges, and textures.

Instead of feeding the raw pixel values of the original image into a neural network, we can now feed these feature maps, which contain richer and more informative representations of the image. This allows the neural network to better understand and classify the image, as it now has access to more meaningful features extracted through convolutional operations (Awcock and Thomas 1995; Petrou and Petrou 2010).

A filter is essentially a matrix of values, and the arrangement of these values within the matrix determines the type of features that the filter will extract from the image. The values in the filter dictate how it interacts with the pixels of the image to produce a feature map that highlights specific characteristics.

For example, as shown in Fig. 8.3, different filters can be used to detect different types of features:

- Filter (a) is designed to detect vertical edges.
- Filter (b) focuses on detecting horizontal edges.
- Filter (c) emphasizes areas with high intensity.
- Filter (d) captures diagonal patterns or edges.

To apply a filter to an image, follow these steps:

1. Start at the top-left corner of the image.
2. Align the center of the filter with the pixel you want to transform.

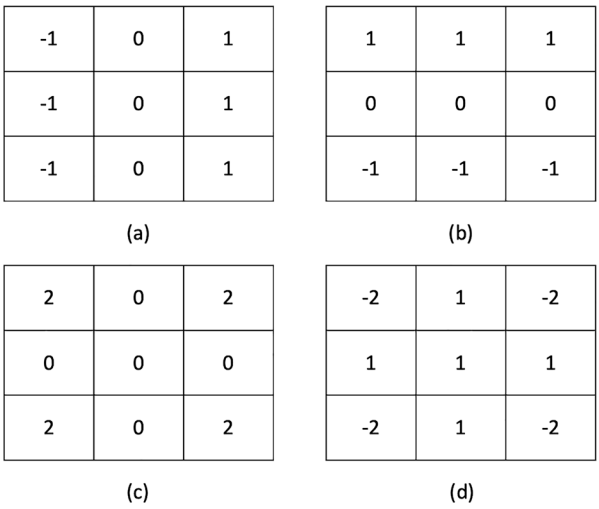


Fig. 8.3 Examples of common convolutional filters

3. Apply the convolution operation to that region:
   - Mathematically, this involves taking the element-wise product of the filter values and the corresponding pixel values in the image. Then, sum up all these products to get a single value.
   - For example, if the filter is a (3×3) matrix F and the corresponding (3×3) patch of the image is P, the convolution at pixel location (i,j) is calculated as:

$$(F * P)_{i,j} = \sum_{m=-k}^{k} \sum_{n=-k}^{k} F[m,n] \cdot P[i+m, j+n] \tag{8.1}$$

   - where:
     - $(F * P)_{i,j}$: The output value in the feature map at position $((i,j)$.
     - $F[m,n]$: The filter (or kernel) value at position (m,n).
     - $P[i+m, j+n]$: The pixel value in the input image shifted by m and n relative to the center pixel (i,j).
     - k: Half the size of the filter (i.e., for a 3×3 filter, $k = 1$; for a 5×5 filter, $k = 2$).
     - The summations run over the filter dimensions, effectively sliding the filter across the image to compute the convolution result.
   - The resulting value replaces the original pixel at (i, j) in the new feature map.
4. Move to the next pixel by sliding the filter one step to the right.
5. Repeat steps 2, 3, and 4 until the filter has been applied across the entire image.

This process results in a new image, known as a feature map, which highlights a specific type of feature based on the filter used. For instance, if the filter is designed to detect vertical edges, the resulting feature map will emphasize vertical lines in the image.

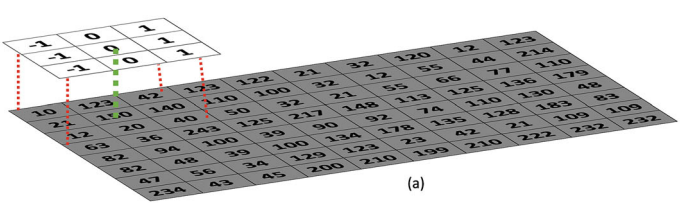
10	123	42	123	122	21	32	120	12	123
21	150	140	110	100	32	12	55	44	214
12	20	40	50	32	21	55	66	77	110
63	36	243	125	217	148	113	125	136	179
82	94	100	39	90	92	74	110	130	48
82	48	39	100	134	178	135	128	183	83
47	56	34	129	123	23	42	21	109	109
234	43	45	200	210	199	210	222	232	232

(a)

-1	0	1
-1	0	1
-1	0	1

(b)

Fig. 8.4 (**a**) A grayscale image where each pixel value ranges from 0 to 255. (**b**) A filter designed to detect vertical edges



-1 x 10	0 x 123	1 x 42
-1 x 21	0 x 150	1 x 140
-1 x 12	0 x 20	1 x 40

(b)

Fig. 8.5 (**a**) The filter being applied to the image, starting at pixel $i = 1, j = 1$. (**b**) The element-wise multiplication of the filter values with the corresponding pixels in the image

Let's see an example of how to apply a filter on an image. Figure 8.4a shows a grayscale image (not RGB), where each pixel has a single intensity value ranging from 0 to 255. Figure 8.4b displays the filter that we want to apply.

In Fig. 8.5a, we demonstrate how the filter is applied to the image. We begin by placing the filter at the top-left corner. However, since each value in the filter must have a corresponding pixel value in the image, the first row, first column, last row, and last column are skipped to avoid calculation errors. This results in the loss of some edge pixels. To prevent this, we can use padding — adding rows and columns of zeros around the edges of the image. This allows us to apply the convolution on every pixel, including the edges, without losing any information, as shown in Fig. 8.6.

In this example, we don't mind losing the outer rows and columns, so we start by placing the filter on the pixel at $i = 1, j = 1$ (since the counting starts from 0), as illustrated in Fig. 8.5a.

Each value in the filter is multiplied by its corresponding value in the image, relative to the center of the filter, as shown in Fig. 8.5b. The resulting value for this pixel in the new feature map is calculated as:

$$(-1 \times 10) + (0 \times 123) + (1 \times 42) + (-1 \times 21) + (0 \times 150) + (1 \times 140) + (-1 \times 12) + (0 \times 20) + (1 \times 40) = 179$$

0	0	0	0	0	0	0	0	0	0	0	0
0	10	123	42	123	122	21	32	120	12	123	0
0	21	150	140	110	100	32	12	55	44	214	0
0	12	20	40	50	32	21	55	66	77	110	0
0	63	36	243	125	217	148	113	125	136	179	0
0	82	94	100	39	90	92	74	110	130	48	0
0	82	48	39	100	134	178	135	128	183	83	0
0	47	56	34	129	123	23	42	21	109	109	0
0	234	43	45	200	210	199	210	222	232	232	0
0	0	0	0	0	0	0	0	0	0	0	0

Fig. 8.6 Adding padding to the original image with rows and columns of zeros

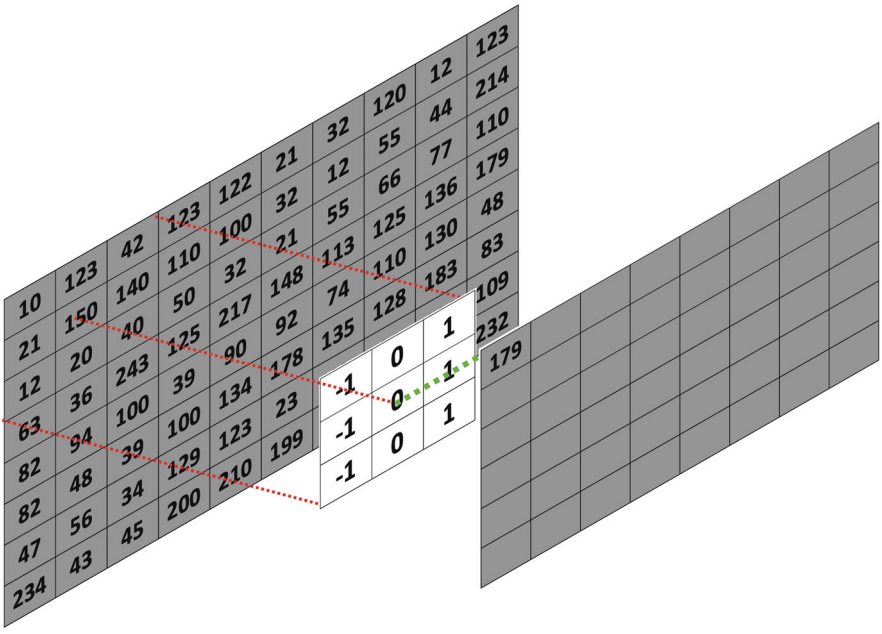


Fig. 8.7 The result of applying the convolution operation at pixel $i = 1, j = 1$

The computed value is then placed in the corresponding position in the feature map, as illustrated in Fig. 8.7.

Figure 8.7 illustrates the resulting feature map after the first convolution operation is applied to the pixel at $i = 1, j = 1$, producing a value of 179. After this, we move to

the next pixel in the image, performing the convolution using the same filter to obtain a new value in the feature map. Once all the columns of the original image have been processed, we move to the next row, repeating the convolution operation across all the columns, and continue row by row until the entire image is covered.

The resulting feature map shown in Fig. 8.7 is smaller than the original image because we did not use padding. Each time we apply a convolution filter without padding, the output feature map decreases in size. If we continue to apply more convolutional filters to the resulting feature map, it will continue to shrink. To preserve the original size of the image after applying a convolution, padding is necessary.

8.2 Implementing Convolutional Operations Using a Convolutional Layer

In this section, we'll demonstrate how to apply convolutional operations on an image using Python and TensorFlow.

```
import matplotlib.pyplot as plt
import tensorflow as tf
import numpy as np
from PIL import Image
# Load the image and ensure it's in RGB format
img = Image.open('Lenna_(test_image).png').convert('RGB')
img = img.resize((256, 256))  # Resize for simplicity if needed

# Convert the image to a NumPy array and normalize it
img_array = np.array(img, dtype='float32') / 255.0  # Normalize to [0, 1]
img_processed = np.expand_dims(img_array, axis=0)  # Add batch dimension

# Define a Conv2D layer to extract feature maps
conv_layer = tf.keras.layers.Conv2D(filters=4, kernel_size=(3, 3),
strides=(1, 1), padding='same', activation=None)
feature_maps = conv_layer(img_processed)

# Extract the feature maps
feature_maps = feature_maps[0].numpy()

# Plot the original image
plt.figure(figsize=(12, 6))
plt.subplot(1, 5, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

# Plot the resulting feature maps
for i in range(4):
    plt.subplot(1, 5, i + 2)
    plt.imshow(feature_maps[:, :, i], cmap='gray')
    plt.title(f"Feature Map {i + 1}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

At the core of convolutional operations in neural networks is the convolutional layer, which applies multiple filters to the input image. This layer enables the network to detect various features like edges, textures, and patterns, which are crucial for understanding the content of the image. By using a convolutional layer, we can efficiently extract multiple feature maps that serve as the foundation for deeper analysis in subsequent layers. Let's set up a convolutional layer to extract these features from our loaded image:

```
# Define a Conv2D layer to extract feature maps
conv_layer = tf.keras.layers.Conv2D(filters=4, kernel_size=
(3, 3), strides=(1, 1), padding='same', activation=None)
feature_maps = conv_layer(img_processed)
```

In the code above, we define a convolutional layer using TensorFlow's "Conv2D" function:

- filters = 4: Extracts four different feature maps.
- kernel_size = (3, 3): Uses a 3×3 filter matrix.
- strides = (1, 1): Moves the filter one pixel at a time both horizontally and vertically.
- padding = 'same': Maintains the same spatial dimensions for the output as the input by adding zero-padding around the edges.
- activation = None: No activation function is applied, so the output consists of raw convolution values.

This convolutional layer is applied to our preprocessed image, resulting in a feature map tensor with the shape "(1, 256, 256, 4)":

- The "4" refers to the number of feature maps generated by the four filters, each capturing different aspects of the image's features.

Figure 8.8 shows the original image alongside the resulting feature maps obtained after applying the convolutional layer.



Fig. 8.8 Original Image and Corresponding Feature Maps Extracted by the Convolutional Layer

8.3 Cubixel Kernel for Feature Extraction

The traditional neural network architecture relies on 2D convolutional filters to extract features such as edges, textures, and patterns from images. However, this approach primarily focuses on 2D spatial information and often lacks the ability to capture deeper volumetric relationships inherent in complex image data. To overcome these limitations, the Cubixel kernel was introduced as an innovative approach to feature extraction (Aburass 2024).

Unlike conventional convolutional filters that operate on flat, two-dimensional pixels, a Cubixel kernel operates on Cubixels—a three-dimensional representation of pixels where the dimensions are derived from the RGB color components, as discussed in Chap. 2. This allows the kernel to extract features not only based on color intensity but also on the volumetric properties of image data, providing a richer and more detailed understanding of spatial relationships.

In a Cubixel representation:

- Volume (V): Represents the combined intensity of the RGB channels.
- Aspect Ratio (A): Captures the spatial relationship between the different color components.

By convolving a Cubixel kernel over an image, we can extract features that include depth and structural information, which are often missed by traditional 2D convolutions.

Mathematical Representation
The Cubixel convolution can be defined mathematically as:

$$C(i,j) = \sum_{m=-k}^{k} \sum_{n=-k}^{k} V(i+m, j+n).K_c(m,n) \tag{8.2}$$

$$C(i,j) = \sum_{m=-k}^{k} \sum_{n=-k}^{k} A(i+m, j+n).K_c(m,n) \tag{8.3}$$

Where:

- $C(i, j)$: The convolved feature at position (i, j).
- $V(i, j)$ and $A(i, j)$: The volume and aspect ratio at position (i, j).
- $\underline{K_c}(m, n)$: The Cubixel kernel values.
- The summations run over the kernel dimensions, centered around the pixel at (i, j).

8.3.1 Understanding Volume (V)

In the traditional pixel representation, each pixel carries color information through its Red, Green, and Blue (RGB) components. However, in the Cubixel paradigm,

each pixel is expanded into a three-dimensional "cubixel" where its dimensions are directly tied to its RGB values:

- The width of the cubixel corresponds to the Red component.
- The height corresponds to the Green component.
- The depth corresponds to the Blue component.

Thus, each cubixel can be thought of as a tiny 3D cube embedded within a 2D plane, representing the color intensity in three dimensions. The Volume (V) of a cubixel is computed as the product of these three dimensions:

$$\mathrm{V} = \mathrm{R} \times \mathrm{G} \times \mathrm{B} \tag{8.4}$$

This volume metric captures the intensity of the color in a way that goes beyond just a flat RGB value. By converting pixels into cubixels, the convolutional layer can now analyze not only the color but also the intensity and spatial relationship in a 3D space. The volume allows for the detection of regions in an image where the intensity of colors changes sharply, which is often indicative of edges or significant features.

8.3.2 Understanding Aspect Ratio (A)

The Aspect Ratio (A) of a cubixel is another important property. While volume captures the overall intensity, aspect ratio focuses on the relative proportions of the cubixel's dimensions. This property is crucial for detecting shapes, edges, and textures that are not purely dependent on color intensity but on how the colors are distributed spatially.

For instance, the aspect ratio can be computed as:

$$\mathrm{A} = \frac{R}{G}, \frac{G}{B} \; or \; \frac{B}{R} \tag{8.5}$$

The idea is to analyze how the color components relate to one another. For example, if the red component is much larger than the green and blue components, it indicates a dominance of the red hue in that region. By examining the aspect ratios, the Cubixel kernel can detect patterns that would otherwise be missed by traditional 2D filters.

How Volume (V) and Aspect Ratio (A) Work Together
In the Cubixel kernel, the convolution operation is modified to not only consider the raw pixel values but also to adjust the volumes and aspect ratios based on the filters. Here's how it works:

1. Volume-Based Analysis: When the Cubixel kernel is applied to an image, it adjusts the volumes of the cubixels. This can highlight regions where the color intensity changes, thereby enhancing features like edges or textures. For example,

if a certain region of an image has a high volume contrast with its neighbors, it may indicate the presence of an object boundary.
2. Aspect Ratio-Based Analysis: Simultaneously, the aspect ratio analysis focuses on the shape and distribution of colors. By comparing the aspect ratios of neighboring cubixels, the kernel can detect patterns such as elongated textures or repetitive structures, which are common in textures like fabrics or natural scenes.
3. Adaptive Feature Detection: The Cubixel kernel uses both volume and aspect ratio information to detect features adaptively. For instance, edges are identified where there are significant changes in either volume or aspect ratio. This multidimensional approach allows for more nuanced feature extraction compared to traditional convolutional filters.

The use of V and A in the Cubixel kernel provides a richer representation of the image, especially useful in areas like medical imaging, where subtle variations in color intensity and texture can indicate important features. By leveraging both volume and aspect ratios, this method improves the network's ability to identify complex patterns that are not easily detectable with conventional 2D kernels.

This specialized convolution extracts features based on both the volume and aspect ratio of Cubixels, making it particularly effective for tasks requiring detailed spatial analysis, such as medical imaging and 3D object recognition.

The integration of the Cubixel kernel into a neural network enhances its feature extraction capabilities by capturing volumetric and spatial nuances that are otherwise missed. This new layer can be used alongside traditional Conv2D layers, allowing the model to benefit from both 2D and 3D feature extraction.

Let's implement the Cubixel Kernel using Python and TensorFlow.

```
#Cubixel Kernel
class CubixelLayer(tf.keras.layers.Layer):
    def __init__(self, num_kernels, kernel_size=(3, 3),
volume_threshold=1.0, **kwargs):
        super(CubixelLayer, self).__init__(**kwargs)
        self.num_kernels = num_kernels
        self.kernel_size = kernel_size
        self.volume_threshold = volume_threshold
        self.edge_color = np.array([255, 0, 0], dtype=np.float32)  # Red
color for edges
        self.kernels = self.add_weight(
            name='kernels',
            shape=(num_kernels, kernel_size[0], kernel_size[1], 1, 1),
            initializer='random_normal',
            trainable=True
        )

    def call(self, inputs):
        # Calculate cubixel properties
        volumes, aspect_ratios = self.calculate_cubixel_properties(inputs)

        kernel_outputs = []
        for i in range(self.num_kernels):
            kernel = self.kernels[i]
            altered_volumes, altered_aspect_ratios =
self.apply_magnetic_kernel((volumes, aspect_ratios), kernel)
            local_threshold =
self.calculate_local_threshold(altered_volumes)

            # Detect edges at the current scale
            edges = tf.cast(altered_volumes, tf.float32) >
tf.cast(local_threshold, tf.float32)
            edges |= tf.cast(altered_aspect_ratios, tf.float32) >
tf.cast(local_threshold, tf.float32)

            # Apply edge color and store each kernel's output separately
            edge_colored_image = tf.cast(edges, tf.float32)[...,
tf.newaxis] * self.edge_color
            kernel_outputs.append(edge_colored_image)

        # Stack all kernel outputs along a new axis (axis=0)
        # This will give you a tensor of shape [num_kernels, height,
width, channels]
        multi_scale_edges = tf.stack(kernel_outputs, axis=0)

        return multi_scale_edges


    @staticmethod
    def calculate_cubixel_properties(image):
        # Normalize the image to a range of 0-1
        image_normalized = image / 255.0

        # Calculate volumes as the product of the RGB values (considered
```

```
as dimensions)
        # Assuming image shape: [batch, height, width, channels]
        # Volumes are calculated for each pixel
        volumes = image_normalized[..., 0] * image_normalized[..., 1] *
image_normalized[..., 2]

        # Calculate aspect ratios
        # Example: You could define aspect ratios based on the relative
proportions of RGB values
        # Adjust the calculation based on your specific definition of
aspect ratio
        aspect_ratios = image_normalized[..., 0] / image_normalized[...,
 ]  # Placeholder, replace with your aspect ratio calculation

        return volumes, aspect_ratios

    def apply_magnetic_kernel(self, cubixel_properties, kernel):
        kernel_tf = tf.reshape(kernel, [self.kernel_size[0],
self.kernel_size[1], 1, 1])
        # Ensure that the input to the convolution is 4D by adding a
channel dimension.
        altered_volume = tf.nn.conv2d(cubixel_properties[0][...,
tf.newaxis], kernel_tf, strides=[1, 1, 1, 1], padding="SAME")
        altered_aspect_ratio = tf.nn.conv2d(cubixel_properties[1][...,
tf.newaxis], kernel_tf, strides=[1, 1, 1, 1], padding="SAME")
        # Remove the last dimension to maintain the original shape.
        return altered_volume[..., 0], altered_aspect_ratio[..., 0]
    def calculate_local_threshold(self, image, neighborhood_size=9):
        pad_size = neighborhood_size // 2
        image = tf.expand_dims(image, -1) if len(image.shape) == 3 else
image

        padded_images = tf.pad(
            image,
            paddings=[[0, 0], [pad_size, pad_size], [pad_size, pad_size],
[0, 0]],
            mode='REFLECT'
        )

        patches = tf.image.extract_patches(
            images=padded_images,
            sizes=[1, neighborhood_size, neighborhood_size, 1],
            strides=[1, 1, 1, 1],
            rates=[1, 1, 1, 1],
            padding='VALID'
        )

        local_thresholds = tf.reduce_mean(patches, axis=-1, keepdims=True)

        # Ensure the output has the same shape as the input, minus the
channel dimension
        local_thresholds = tf.squeeze(local_thresholds, -1)

        return local_thresholds
```

```
from PIL import Image

# Load the image and ensure it's in RGB format
img = Image.open('Lenna_(test_image).png').convert('RGB')
img = img.resize((256, 256))  # Resize for simplicity if needed

# Convert the image to a NumPy array and normalize it
img_array = np.array(img, dtype='float32') / 255.0  # Normalize to [0, 1]
img_processed = np.expand_dims(img_array, axis=0)  # Add batch dimension

# Define a Conv2D layer to extract feature maps

# Normalize the data
# Define the volume threshold
volume_threshold = 1.0
number_of_kernals = 4
# Initialize the custom layer with the defined volume threshold
cubixel_layer = CubixelLayer(number_of_kernals,
                             kernel_size=(3, 3),
                             volume_threshold=volume_threshold)

# Normalize the data

# Apply the custom layer to the first 10 images
processed_images = cubixel_layer(img_processed)

# Rest of your plotting code...
# Assuming processed_images is a tensor of shape [num_kernels, num_images,
height, width, channels]

num_kernels = processed_images.shape[0]  # Number of kernels in the
CubixelLayer
num_images = processed_images.shape[1]   # Number of images processed

# Set up the plotting
# Calculate the total number of subplots needed (original + number of
kernels for each image)
plt.figure(figsize=(20, num_images * 2))

for i in range(num_images):
    # Original image subplot
    plt.subplot(num_images, num_kernels + 1, i * (num_kernels + 1) + 1)
    plt.imshow(img)
    plt.title("Original")
    plt.axis("off")

    # Processed images subplots
    for j in range(num_kernels):
        # Select the processed image for the current kernel
        processed_image = processed_images[j, i, :, :, 0]  # Assuming the
last dimension is for grayscale

        plt.subplot(num_images, num_kernels + 1, i * (num_kernels + 1) + 2
+ j)
```

```
        plt.imshow(processed_image, cmap='gray')  # Use a grayscale
colormap
        plt.title(f"Kernel {j+1}")
        plt.axis("off")

# Adjust layout to prevent overlapping titles and axes
plt.tight_layout()
# Display the plot
plt.show()
```

This implementation uses a custom TensorFlow layer called CubixelLayer, which performs the following key steps:

Cubixel Property Calculation: The layer calculates the volume and aspect ratio for each pixel based on the RGB values.

Magnetic Convolution: The computed Cubixel properties are convolved using custom filters to detect complex patterns.

Edge Detection: The layer applies adaptive thresholding to detect edges and highlight them in the output feature maps.

1. Defining the Cubixel Layer

The `CubixelLayer` class is a custom TensorFlow layer that applies a series of filters on the input image to extract feature maps based on Cubixel properties.

```
class CubixelLayer(tf.keras.layers.Layer):
  def __init__(self, num_kernels, kernel_size=(3, 3),
volume_threshold=1.0, **kwargs):
    super(CubixelLayer, self).__init__(**kwargs)
    self.num_kernels = num_kernels
    self.kernel_size = kernel_size
    self.volume_threshold = volume_threshold
   self.edge_color = np.array([255, 0, 0], dtype=np.float32) # Red
color for edges
    # Initialize random kernels
    self.kernels = self.add_weight(
      name='kernels',
      shape=(num_kernels, kernel_size[0], kernel_size[1], 1, 1),
      initializer='random_normal',
      trainable=True
    )
```

- **num_kernels**: Number of filters used for feature extraction.
- **kernel_size**: The size of each filter (e.g., 3×3).
- **volume_threshold**: A threshold used to detect edges.
- **kernels**: A set of learnable filters initialized with random values.

2. Calculating Cubixel Properties
The `calculate_cubixel_properties` method computes two key properties:

- Volume: Product of the normalized RGB values.
- Aspect Ratio: Ratio of the red channel to the green channel, providing information about the color distribution.

```
def calculate_cubixel_properties(image):
  image_normalized = image / 255.0
  volumes = image_normalized[..., 0] * image_normalized[..., 1] *
image_normalized[..., 2]
  aspect_ratios = image_normalized[..., 0] / (image_normalized
[..., 1] + 1e-5)
  return volumes, aspect_ratios
```

3. Applying the Magnetic Kernel
The `apply_magnetic_kernel` method performs convolutions on the calculated volumes and aspect ratios using the custom kernels.

```
def apply_magnetic_kernel(self, cubixel_properties, kernel):
  kernel_tf = tf.reshape(kernel, [self.kernel_size[0], self.
kernel_size[1], 1, 1])
  altered_volume = tf.nn.conv2d(
    cubixel_properties[0][..., tf.newaxis], kernel_tf, strides=
[1, 1, 1, 1], padding="SAME"
  )
  altered_aspect_ratio = tf.nn.conv2d(
    cubixel_properties[1][..., tf.newaxis], kernel_tf, strides=
[1, 1, 1, 1], padding="SAME"
  )
  return altered_volume[..., 0], altered_aspect_ratio[..., 0]
```

The convolution operation extracts features based on the volumetric and aspect ratio properties, providing richer feature maps compared to standard 2D filters.

4. Edge Detection and Visualization
To highlight edges, the `calculate_local_threshold` method computes an adaptive threshold based on the local neighborhood of each pixel.

```
def calculate_local_threshold(self, image,
neighborhood_size=9):
  pad_size = neighborhood_size // 2
  padded_images = tf.pad(
    image, paddings=[[0, 0], [pad_size, pad_size], [pad_size,
pad_size], [0, 0]], mode='REFLECT'
  )
  patches = tf.image.extract_patches(
    images=padded_images,
    sizes=[1, neighborhood_size, neighborhood_size, 1],
    strides=[1, 1, 1, 1],
    rates=[1, 1, 1, 1],
    padding='VALID'
  )
  local_thresholds = tf.reduce_mean(patches, axis=-1,
keepdims=True)
  return tf.squeeze(local_thresholds, -1)
```

5. Applying the Cubixel Layer to an Image

The following code demonstrates how to apply the Cubixel layer to the "Lena" image and visualize the extracted features.

```
from PIL import Image
img = Image.open('Lenna_(test_image).png').convert('RGB')
img = img.resize((256, 256))
img_array = np.array(img, dtype='float32') / 255.0
img_processed = np.expand_dims(img_array, axis=0)
cubixel_layer = CubixelLayer(num_kernels=4, kernel_size=
(3, 3), volume_threshold=1.0)
processed_images = cubixel_layer(img_processed)
```

6. Visualizing the Results

The processed feature maps are visualized to see how the Cubixel kernels extract edges and patterns from the image.

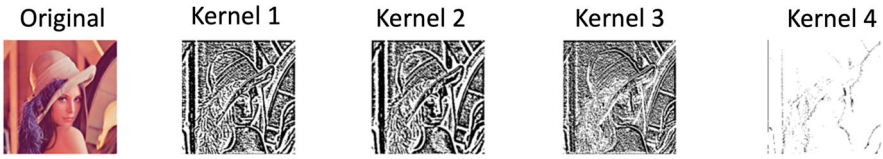


Fig. 8.9 The original "Lena" image and the corresponding feature maps generated using four different Cubixel kernels

```
plt.figure(figsize=(20, 8))
num_kernels = processed_images.shape[0]
for j in range(num_kernels):
  plt.subplot(1, num_kernels, j + 1)
  processed_image = processed_images[j, 0, :, :, 0]
  plt.imshow(processed_image, cmap='gray')
  plt.title(f"Kernel {j+1}")
  plt.axis("off")
plt.tight_layout()
plt.show()
```

The Cubixel layer effectively extracts features by leveraging both volumetric properties and aspect ratios of the RGB channels. This results in enhanced feature maps that capture intricate details such as edges and textures. Figure 8.9 shows the Lena image after applying four different Cubixel kernels, each revealing unique patterns and structures.

8.4 Concluding Remarks

We started by exploring how simple filters can be used to detect edges, textures, and shapes in images, thereby transforming raw pixel data into more meaningful feature maps. This step is crucial for enabling neural networks to understand visual content in a structured and efficient way.

We then progressed to more advanced techniques, introducing the concept of Cubixel kernels. By leveraging the volumetric properties and aspect ratios of pixels, these kernels go beyond traditional 2D convolutions to extract richer and more detailed features. This innovative approach allows us to capture not only spatial patterns but also deeper relationships inherent in color channels, leading to enhanced feature extraction.

Chapter 9
Convolutional Neural Networks



Abstract This chapter introduces Convolutional Neural Networks (CNN), a powerful deep learning architecture designed for image analysis. CNNs combine convolutional operations with neural networks to automatically extract hierarchical features from raw image data, making them especially effective for tasks like image classification.

We explore the core components of CNNs, including convolutional layers for pattern detection, pooling layers for dimensionality reduction, and fully connected layers for final classification. The chapter also covers the role of activation functions such as ReLU in introducing non-linearity, and the backpropagation process used to optimize filter weights during training.

To enhance performance and generalization, we discuss techniques such as dropout and pooling, which help reduce overfitting and computational complexity. By progressively learning more abstract features through stacked layers, CNNs offer a robust framework for capturing complex visual patterns and delivering high-accuracy predictions in real-world image processing tasks.

9.1 Convolutional Neural Networks

Now that we have learned how to extract features from images using convolutional operations, we can leverage these features to classify images using an Artificial Neural Network. By integrating convolutional operations with an ANN, we form what is known as a Convolutional Neural Network (Aburass et al. 2022; K. Wang et al. 2023; Zhao 2022).

A CNN is a type of neural network that first extracts features from images using convolutional layers. These features are then processed and passed into a neural network consisting of an input layer, hidden layers, and an output layer, similar to what we covered in the ANN chapter. The advantage of CNNs is their ability to automatically detect and learn hierarchical features directly from raw image data, making them especially effective for image classification tasks.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_9

As we saw in the chapter on convolutional operations, a "Conv2D" layer can be used to extract features from images, producing various feature maps. We can stack multiple "Conv2D" layers, allowing each layer to process the feature maps generated by the previous layer, ultimately refining the extracted features further. The feature maps from the final convolutional layer are then fed into the neural network for classification.

Let's look at how a series of convolutional layers can be used to build a CNN:

```
import tensorflow as tf
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
Flatten, Dense
from tensorflow.keras.models import Sequential
# Example model to demonstrate feature extraction using multiple
Conv2D layers
model = Sequential([
  # First Convolutional Layer
  Conv2D(filters=32, kernel_size=(3, 3), padding='same',
activation='relu', input_shape=(64, 64, 3)),
  # Second Convolutional Layer
  Conv2D(filters=64, kernel_size=(3, 3), padding='same',
activation='relu'),
  # Third Convolutional Layer
  Conv2D(filters=128, kernel_size=(3, 3), padding='same',
activation='relu'),
  # Flatten the feature maps
  Flatten(),
  # Fully connected ANN layers
  Dense(128, activation='relu'),
  Dense(10, activation='softmax') # Assuming we have 10 output
classes
])
# Display the model architecture
model.summary()
```

In the sequential CNN model, the process begins by applying the first convolutional layer to the input image. For instance, the following layer:

```
Conv2D(filters=32, kernel_size=(3, 3), padding='same',
activation='relu', input_shape=(64, 64, 3))
```

takes an input image with a shape of $64 \times 64 \times 3$, representing the height, width, and color channels (RGB). The layer uses 32 filters, each of size 3×3, to perform

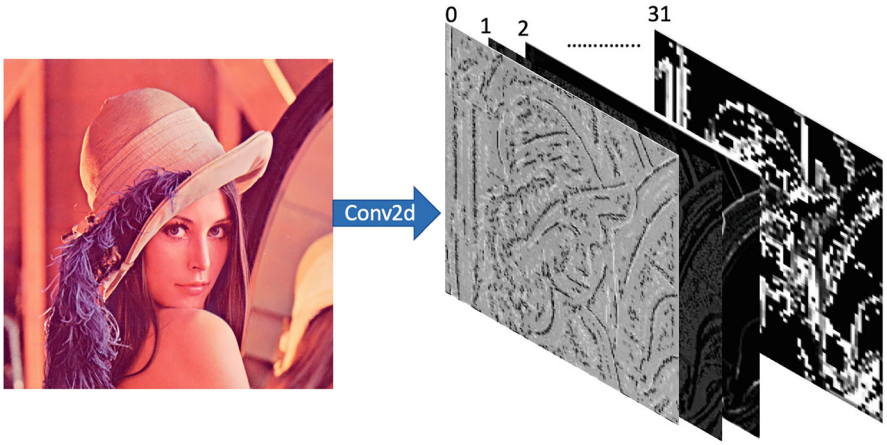


Fig. 9.1 Output of the first Conv2D layer applied to Lena image

convolutional operations across the input image. As a result, it generates 32 feature maps, each capturing specific features (e.g., edges, textures) identified by the filters. These feature maps are stacked on top of each other, forming an output with the shape 64 × 64 × 32. This output is then passed to the next convolutional layer for further processing (Z. Wang et al. 2021).

Figure 9.1 demonstrates the output of the first "Conv2D" layer when applied to the Lena image. The resulting feature maps, each of size 64 × 64, are shown stacked together, highlighting the features detected by the filters.

After the first convolutional layer, the output 64 × 64 × 32 feature maps become the input for the second convolutional layer. This second layer, defined as:

```
Conv2D(filters=64, kernel_size=(3, 3), padding='same',
activation='relu')
```

uses 64 filters, each of size 3 × 3. Here each filter processes the full stack of 32 channels in the input feature maps simultaneously. This results in the generation of 64 new feature maps, where each map captures a distinct pattern from the combined input data.

- The input to this layer is 64 × 64 × 32 (height, width, and channels).
- The output is 64 × 64 × 64, consisting of 64 feature maps of size 64 × 64.

The "padding = 'same'" parameter ensures that the spatial dimensions of the feature maps remain the same after applying the convolutional filters, preserving the 64 × 64 resolution. Figure 9.2 below illustrates the input and output of this second convolutional layer using the Lena image:

By stacking convolutional layers in this manner, the CNN continues to refine its understanding of the image by detecting more complex patterns. This hierarchy

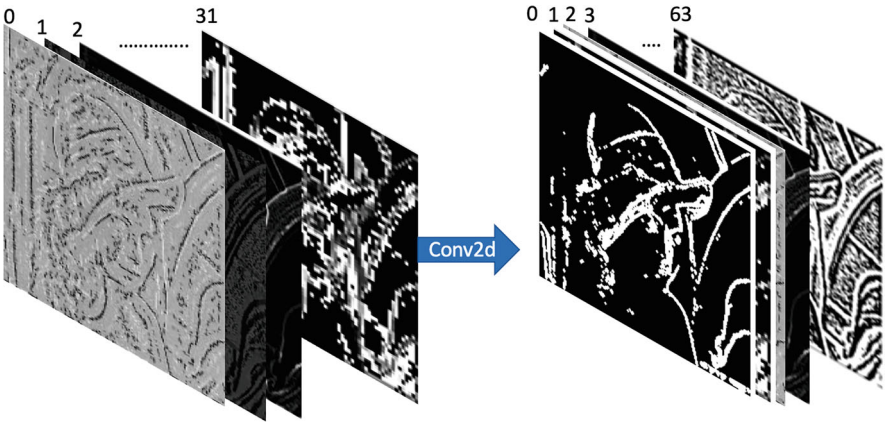


Fig. 9.2 Input and output of the second Conv2D layer applied to Lena image

allows the model to move from detecting simple edges to identifying textures and ultimately recognizing high-level structures in the image.

The same logic extends to the third convolutional layer. The input for this layer consists of 64 feature maps, each with dimensions 64 × 64. The layer, defined as:

```
Conv2D(filters=128, kernel_size=(3, 3), padding='same',
activation='relu')
```

uses 128 filters, each of size 3 × 3, to produce 128 new feature maps. Each filter processes all 64 channels (feature maps) of the input tensor simultaneously, extracting even more complex features. Essentially, each 3 × 3 filter in the layer is actually 3 × 3 × 64, as it spans all 64 channels of the input tensor.

- Input: 64 × 64 × 64 (64 feature maps, each 64 × 64).
- Output: 64 × 64 × 128 (128 feature maps, each 64 × 64).

All three convolutional layers use the ReLU activation function, ensuring that all feature map values are non-negative, which helps the network learn non-linear relationships effectively. Figure 9.3 illustrates the input and output of the third convolutional layer when applied to the Lena image:

Once the third convolutional layer has produced its feature maps, the next step is to feed these maps into the Flatten layer. The purpose of the flattening operation is to convert the 3D feature maps into a 1-dimensional vector, which can then serve as the input to the fully connected ANN.

The Flatten layer is defined as:

```
Flatten()
```

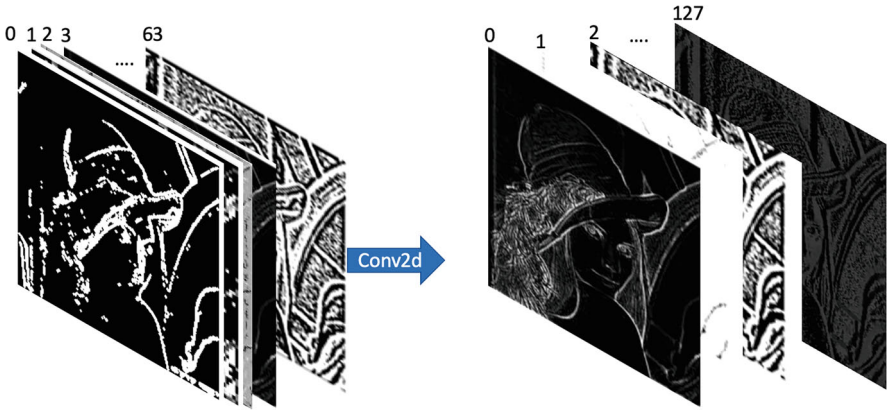


Fig. 9.3 Input and output of the third Conv2D layer applied to Lena image

Since the output from the third convolutional layer is a set of 128 feature maps, each of size 64 × 64, the flattening process converts this 64 × 64 × 128 tensor into a single 1-dimensional vector of length 524,288 (64 × 64 × 128).

- Input to Flatten: 64 × 64 × 128 feature maps.
- Output from Flatten: A 1D vector of length 524,288xs.

Figure 9.4 illustrates the transformation of the 64 × 64 × 128 feature maps into a single 1D vector:

If we were to pass the original image (64 × 64 × 3) directly to the neural network without using convolutional layers, the input to the network would be a 1-dimensional vector of length 12,288 (64 × 64 × 3). While this might seem simpler, it would not allow the network to effectively capture spatial hierarchies or localized features from the image.

By using convolutional layers, we increased the feature space to a much larger input size (524,288), representing more abstract and refined patterns in the image. This expanded representation gives the neural network greater power to classify the image by leveraging the extracted features.

After the flattened vector is prepared, it is fed into the fully connected layers of the neural network. These layers are responsible for making the final classification based on the features extracted by the convolutional layers.

The fully connected layers in this example are defined as:

```
# Fully connected ANN layers
Dense(128, activation='relu'),
Dense(10, activation='softmax') # Assuming we have 10 output
classes
```

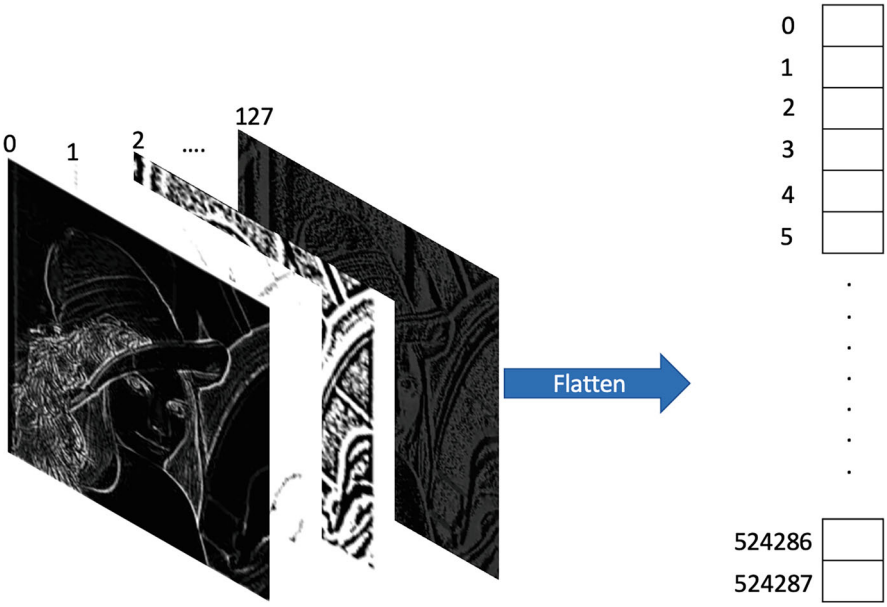


Fig. 9.4 Flattening the feature maps from the third Conv2D layer

1. First Fully Connected Layer (Hidden Layer):
   - This layer consists of 128 neurons.
   - It uses the ReLU activation function, which introduces non-linearity to the network and ensures that only positive values are passed forward.
   - The input to this layer is the 1-dimensional vector produced by the Flatten layer.

2. Output Layer:
   - This layer consists of ten neurons, one for each class in the classification task (assuming ten classes).
   - It uses the Softmax activation function, which converts the outputs into probabilities, with the sum of probabilities across all classes equal to 1. This allows the network to predict the most likely class for the input image. Figure 9.5 shows the fully connected network. The 1-dimensional vector serves as the input to the network, connecting to the 128 neurons in the hidden layer. These neurons are then connected to the ten output neurons:

Figure 9.6 below illustrates the complete Convolutional Neural Network architecture.

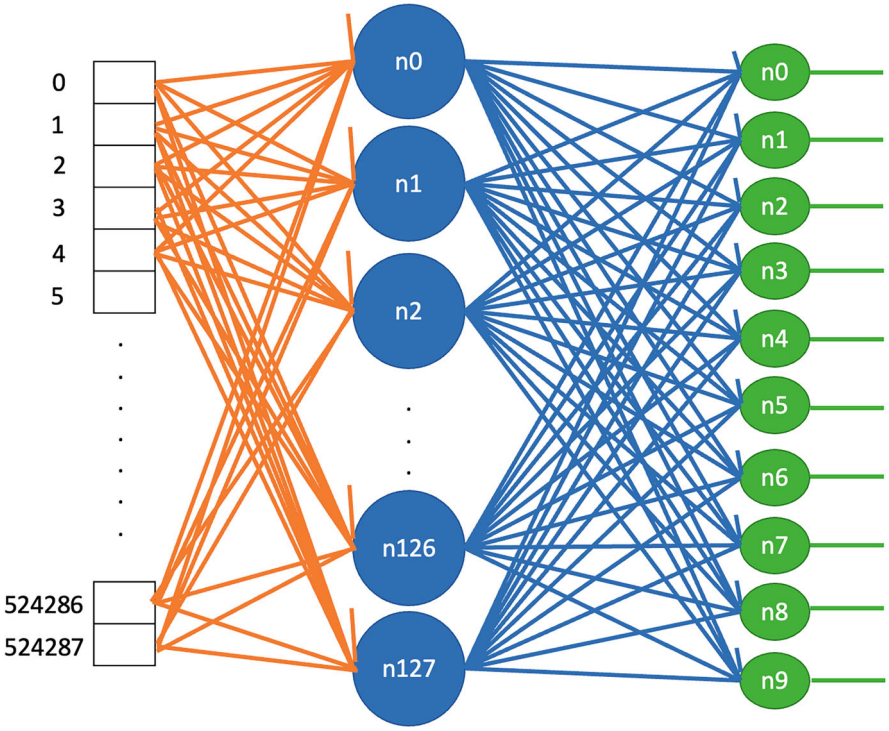


Fig. 9.5 Fully connected layers of the neural network

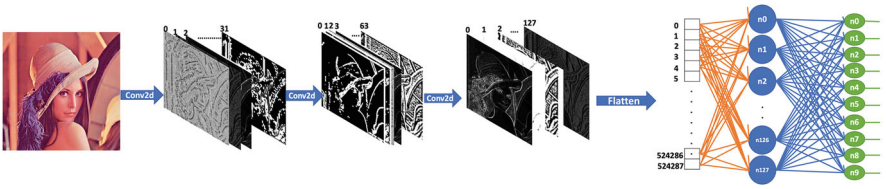


Fig. 9.6 Full convolutional neural network architecture

9.2 Conv2D Filters and Backpropagation

At first, the filters in a CNN can have random values, predefined values, or be initialized based on a specific distribution algorithm. However, to make the CNN more effective, these filters must evolve during training to better extract information from the input images. This process of improvement is achieved through backpropagation and gradient descent, similar to how weights are updated in an Artificial Neural Network. Let's explore how the filters are updated during training,

starting from the Flatten layer and moving backward through the CNN (Lydia and Francis 2019; Z. Zhang 2016).

9.2.1 *From the Flatten Layer to the Convolutional Layers*

When the ANN reaches the Flatten layer (or its input layer) during backpropagation, the Flatten layer receives a 1-dimensional vector of gradients from the neural network. Then, it reshapes this vector back to the shape of its previous Conv2D layer, in a reverse operation of the flattening process (Alshawabkeh et al. 2022; Howard et al. 2017). For example, if the previous Conv2D layer had a shape of $64 \times 64 \times 128$, the Flatten layer will reshape the gradient vector to that shape. This allows the previous Conv2D layer to work on these gradients and enhance its filters. Figure 9.7 illustrates the role of the Flatten layer during backpropagation by reshaping the gradient vector back to the shape of its previous Conv2D layer. This process is crucial as it ensures the gradients maintain the required structure, allowing the Conv2D layer to effectively process the gradients and update its filters. The figure demonstrates how a 1-dimensional gradient vector from the ANN is reshaped into a 3D tensor matching the spatial dimensions and channels of the Conv2D layer output (e.g., $64 \times 64 \times 128$) (Deisenroth et al. 2020; Géron 2022).

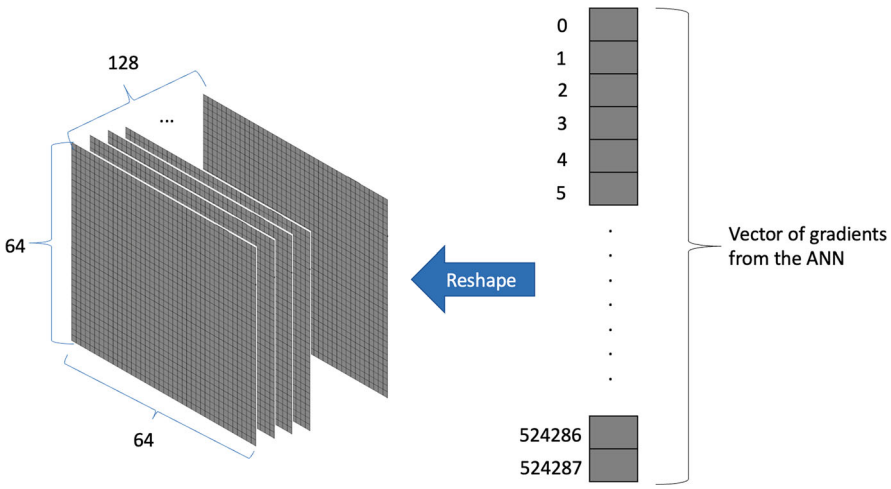


Fig. 9.7 Role of the flatten layer in backpropagation

9.2.2 *Processing of the Reshaped Gradient Tensor by the Derivative of ReLU*

Once the gradient tensor is reshaped back to the shape of layer3 output (e.g., 64 × 64 × 128), the next step in backpropagation involves processing this gradient tensor using the derivative of the ReLU activation function (Hecht-Nielsen 1992; Lee et al. 2016; Zhou 2018). This step determines how the gradients flow backward through the network, as illustrated in Fig. 9.8.

The reshaped gradient tensor, denoted as ∇L_{Layer3}, retains the same shape as layer3 output before the Flatten operation: 64 × 64 × 128. Each channel in this tensor corresponds to a feature map produced by a single filter in the Conv2D layer3. During this step, each channel is processed independently through the derivative of the ReLU activation function. The derivative ensures that gradients are only propagated through neurons that were active during the forward pass—those where the pre-activation output, Z, was >0.

ReLU introduces non-linearity into the network by setting all negative values to zero during the forward pass. Its derivative ensures selective gradient propagation during backpropagation. The derivative of ReLU is mathematically defined as:

$$\mathrm{f}'(\mathrm{z}) = \begin{cases} 1 \; if \; z > 0 \\ 0 \; if \; z \leq 0 \end{cases} \tag{9.1}$$

Here, z is the pre-activation output (before ReLU). Using this derivative, the gradients are updated by an element-wise (Hadamard) product between ∇L_{Layer3} and $\mathrm{f}'(z)$. The resulting gradient tensor, ∇L_{Z_3}, for each channel c is computed as:

$$\nabla L_{Z_3} = \nabla L_{\text{Layer3}} \odot \mathrm{f}'(Z) \tag{9.2}$$

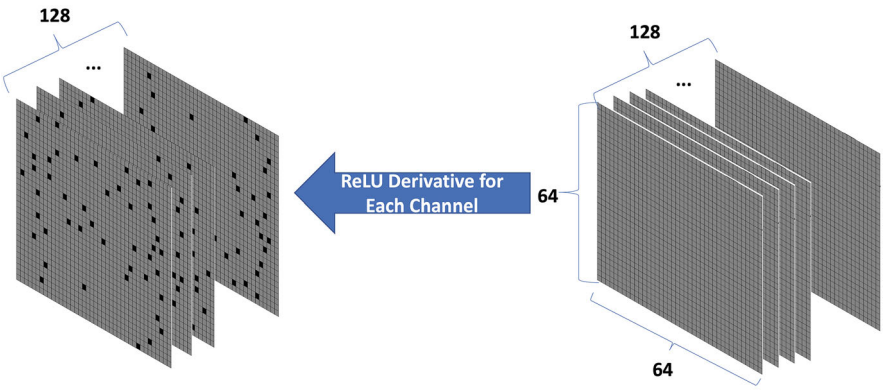


Fig. 9.8 Processing gradients through the ReLU derivative

For each channel, neurons where $Z > 0$ will have their gradients preserved, while those where $Z \leq 0$ will have their gradients zeroed out. As depicted in Fig. 9.8, this process ensures that only activated neurons contribute to the gradients passed back to the filters.

After applying the derivative of ReLU to all 128 channels, the processed gradients are combined (stacked) into a tensor of shape $64 \times 64 \times 128$. This combined tensor represents the adjusted gradients for all channels and is used in the subsequent steps of backpropagation, such as updating the Conv2D filters or propagating gradients to the previous layer (Ekman 2021).

9.2.3 Computing the Gradient of the Convolutional Layer

Now, after processing the gradient tensor based on the derivative of the ReLU, we calculate the gradient of the current convolutional layer. This step involves determining how the current layer contributes to the overall gradient so that it can be propagated backward to the previous layer. The process involves rotating the filters, convolving them with the corresponding gradient channels, and stacking the results to compute the full gradient tensor for this layer.

1. Rotating the Filters

   In the forward pass, each filter W_k (where k denotes the filter index) is used to compute feature maps via convolution. During backpropagation, to correctly reverse this operation, we rotate each filter by 180°. Mathematically, the rotated filter W_k^{rot180} for a 3×3 filter W_k is:

$$W_k^{\text{rot180}} = \begin{bmatrix} w_{33} & w_{32} & w_{31} \\ w_{23} & w_{22} & w_{21} \\ w_{13} & w_{12} & w_{11} \end{bmatrix} \tag{9.3}$$

   Here, W_k is flipped along both dimensions (height and width) to perform the reverse operation.

2. Convolving Rotated Filters with Gradient Channels

   Let:

   - $\nabla L_{Z_3}^{(k)}$ be the gradient of the loss L with respect to the kth output feature map (i.e., the processed gradient channel for filter k).
   - W_k^{rot180} be the rotated kth filter.

   The gradient of the loss with respect to the output feature map X, contributed by the kth filter, is computed by performing a 2D convolution of $\nabla L_{Z_3}^{(k)}$ with W_k^{rot180}:

$$\nabla L_X^{(k)} = W_k^{rot180} * \nabla \mathrm{L}_{Z_3}^{(k)} \tag{9.4}$$

Here:

- * represents the 2D convolution operation.
- $\nabla L_X^{(k)}$ is the gradient tensor contributed by the kth filter.

This operation computes how the gradient of the kth feature map influences the output of the current convolutional layer.

3. Stacking the Resulting Gradients

After computing $\nabla L_X^{(k)}$ for all filters ($k = 0,1, 2, \ldots, K$), the total gradient with respect to the output feature map is obtained by summing these gradients across all filters. Mathematically, the total gradient is:

$$\nabla L_{X_3} = \sum_{k=0}^{K} \nabla \mathrm{L}_X^{(k)} \tag{9.5}$$

where:

- ∇L_{X_3} is the full gradient tensor for the output to the current convolutional layer.
- K is the total number of filters in the layer.

The summation aggregates the gradient contributions from all filters to produce the total gradient tensor ∇L_{X_3}, which has the same shape as the output feature map.

The resulting gradient tensor ∇L_{X_3}:

- Has the same spatial dimensions and depth as the output feature map of the current layer.
- Represents the gradient of the loss L with respect to the output of this convolutional layer.
- Is passed to the previous layer for further backpropagation.

As shown in Fig. 9.9, each rotated filter (e.g., W_k^{rot180}) is convolved with its corresponding gradient channel $\nabla \mathrm{L}_{Z_3}^{(k)}$. This operation generates individual gradient contributions $\nabla L_X^{(k)}$, which are stacked (summed element-wise) to produce the final gradient tensor ∇L_{X_3}. This tensor is then propagated to the previous layer, ensuring that the backward pass properly accounts for all gradient contributions from the current convolutional layer (Weidman 2019).

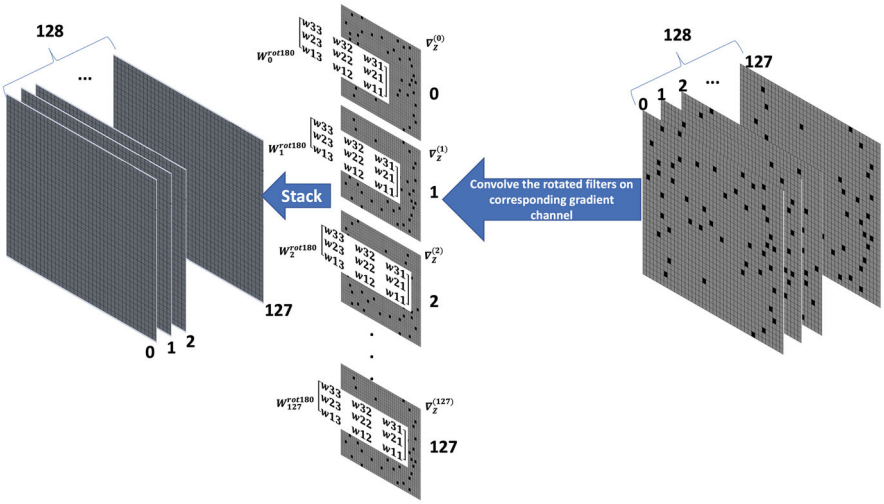


Fig. 9.9 Computing the gradient of the convolutional layer

9.2.4 The Gradient of the Filter Weights

This process reverses the forward pass operation, where the filter W_k and input X interact to produce the feature map X_k. Specifically, during backpropagation, the gradient of the loss with respect to the filter weights is calculated by aggregating contributions from the input and the gradient tensor ∇L_{X_3}.

9.2.4.1 Gradient of Filter Weights via Convolution

For a specific filter W_k, the gradient of its weights, denoted as ΔW_k, is determined by examining how the input tensor X contributes to the backpropagated loss gradient $\nabla L_{X_{3,k}}$. Mathematically, this can be expressed as:

$$\Delta W_k = \sum_{i=1}^{h_{\text{out}}} \sum_{j=1}^{w_{\text{out}}} \nabla L_{X_{3,k}}[i,j].X_{\text{region}}[i,j] \tag{9.6}$$

Here:

- ΔW_k is a tensor representing the gradient of the weights for filter W_k. If the filter size is $f_h \times f_w$, then ΔW_k has the same dimensions ($f_h \times f_w$).
- $\nabla L_{X_{3,k}}[i,j]$ is the backpropagated gradient value for the kth feature map at position (i, j).

- $X_{\text{region}}[i,j]$ is the region of the input tensor X that corresponds to the spatial position (i, j) in the feature map. This region has dimensions matching the filter size ($f_h \times f_w$).

For each weight $W_k[m, n]$ in the filter, the contribution to the gradient is computed as:

$$\Delta W_k[m,n] = \sum_{i=1}^{h_{\text{out}}} \sum_{j=1}^{w_{\text{out}}} \nabla L_{X_{3,k}}[i,j] \cdot X[i+m-1, j+n-1] \tag{9.7}$$

where:

- m, n index the spatial dimensions of the filter.
- $(i + m - 1, j + n - 1)$ align the input tensor X with the filter at position (i, j).

This operation ensures that each region of the input tensor X contributes to the gradient computation for the corresponding filter weights.

When the input tensor X has multiple channels (C_{in}), the filter W_k also has a separate set of weights for each input channel. To compute the gradient for the kth filter weight, we sum the contributions from all channels:

$$\Delta W_k[m,n] = \sum_{c=1}^{C_{\text{in}}} \sum_{i=1}^{h_{\text{out}}} \sum_{j=1}^{w_{\text{out}}} \nabla L_{X_{3,k}}[i,j] \cdot X_c[i+m-1, j+n-1] \tag{9.8}$$

Here:

- X_c is the cth channel of the input tensor X.
- $W_{k,\ c}\ [m, n]$ is the weight in the kth filter corresponding to the cth input channel.

Alternatively, this can be expressed as:

$$\Delta W_k = \sum_{c=1}^{C_{in}} \nabla L_{X_{3,k}} * X_c \tag{9.9}$$

where * denotes the convolution operation.

This summation ensures that all input channels contribute to the gradient computation for the filter weights.

9.2.5 *Incorporating Sliding Window in Gradient Computation*

The sliding window mechanism plays a critical role in calculating gradients for the filter weights during backpropagation. This mirrors the forward pass where the filter slides across the input tensor X to compute the feature map.

- Sliding Window in the Forward Pass
  In the forward pass, the filter W_k slides over the input tensor X with a specified stride, and at each spatial position (i, j), it computes a dot product between the filter weights and the corresponding region of X.
- Sliding Window in Backpropagation
  During backpropagation, the process is reversed. For each value in the gradient tensor $\nabla L_{X_{3,k}}[i, j]$, the corresponding input region from X is used to update the filter weights W_k. This involves sliding a window of size $f_h \times f_w$ over the input tensor X, aligned with each position of (i, j) in $\nabla L_{X_{3,k}}$.

At each step of the sliding window:

1. The gradient value $\nabla L_{X_{3,k}}[i, j]$ is multiplied by the corresponding region of X.
2. The resulting values are accumulated into ΔW_k, the gradient of the filter weights.

Mathematically:

$$\Delta W_k[m, n] = \sum_{i=1}^{h_{\text{out}}} \sum_{j=1}^{w_{\text{out}}} \nabla L_{X_{3,k}}[i, j] \cdot X[i + m, j + n] \tag{9.10}$$

Here:

- (m, n) index the filter dimensions $(f_h \times f_w)$.
- $(i + m$ and $j + n)$ align the filter's spatial position with the input region.

In Summary:

1. Extract the gradient tensor $\nabla L_{X_{3,k}}$
2. Slide a window of size $f_h \times f_w$ across the input tensor X, aligned with each position (i, j) in $\nabla L_{X_{3,k}}$.
3. For each window:
   - Multiply the gradient value $\nabla L_{X_{3,k}}$ [i, j] with the corresponding input region.
   - Accumulate these values into the filter gradient ΔW_k.
4. Repeat the process for all input channels and sum across channels to compute the full filter gradient.

This approach ensures a direct correspondence between the forward pass and backpropagation, effectively updating the filter weights based on the sliding window interactions with the input tensor.

Figure 9.10 represents the process of backpropagation in the third convolutional layer, specifically highlighting the relationship between the input tensor, the filters, and the gradient tensor. On the left, the input tensor corresponds to the feature maps generated by the second convolutional layer. This tensor serves as the input X. During backpropagation, a sliding window mechanism is applied to this input tensor, where regions of the input, denoted as $X_{\text{region}}[i, j]$, are extracted to align with the spatial positions of the gradient tensor $\nabla L_{X_{3,k}}$. Each of these regions has dimensions

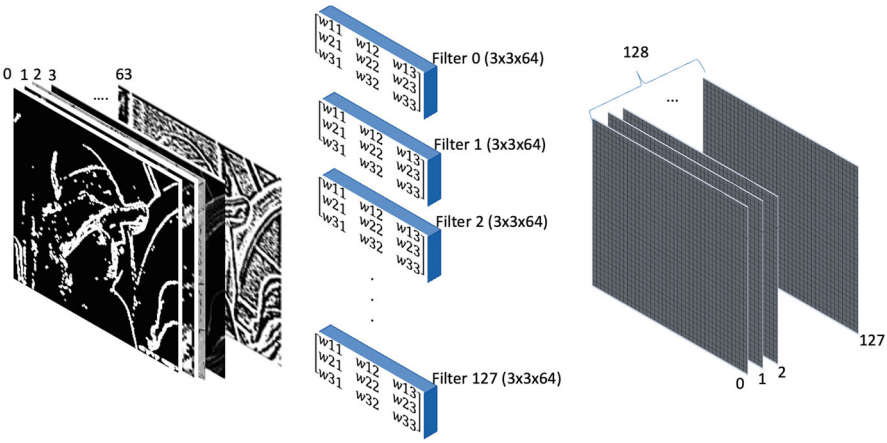


Fig. 9.10 Computing gradients of filter weights

matching the filter size $f_h \times f_w$ and provides the necessary input data to compute the gradients for the filters (Goodfellow et al. 2016; A. Zhang et al. 2023).

In the center of Fig. 9.10 are the filters of the third convolutional layer. These filters represent the learnable parameters W_k that map the input tensor to the output feature maps during the forward pass. In the backward pass, the filters are updated by aggregating contributions from the input regions $X_{\text{region}}[i, j]$ and the gradient tensor values $\nabla L_{X_{3,k}}\ [i, j]$. Each filter's gradient, ΔW_k, is computed by summing the element-wise products of $\nabla L_{X_{3,k}}\ [i, j]$ and the corresponding regions of the input tensor. This process ensures that the filters are adjusted to minimize the loss by considering the spatial and channel-wise interactions between the input and the gradient tensor.

On the right side of Fig. 9.10 is the gradient tensor $\nabla L_{X_{3,k}}$, which represents the backpropagated gradient from the subsequent layers of the network. This tensor carries information about how the output of the third convolutional layer influenced the loss function. Each value in $\nabla L_{X_{3,k}}$ corresponds to a specific spatial position and determines the weight of the contribution from the corresponding input region $X_{\text{region}}[i, j]$ to the filter gradient. The interaction between $\nabla L_{X_{3,k}}$ and the input tensor, mediated by the filters, is the key to computing ΔW_k.

9.2.6 Gradient Flow Mechanism for Layer Shape Adjustment

During backpropagation in a convolutional neural network, gradients from deeper layers must be reshaped to match the input dimensions of the preceding layers. This adjustment ensures proper weight updates and the smooth flow of gradients throughout the network. In the case of the third convolutional layer (Layer 3), with dimensions $64 \times 64 \times 128$, its gradient tensor must be reshaped to align with the

dimensions of the second convolutional layer (Layer 2), which has dimensions $64 \times 64 \times 64$.

The gradient tensor of Layer 3, denoted as ∇L_{X_3}, is used to compute the gradient tensor for Layer 2, ∇L_{X_2}. This computation involves convolving the gradient tensor from Layer 3 with rotated versions of Layer 3's filters. The mathematical operation for this process is described by:

$$\nabla L_{X_{2,k}(i,j,c)} = \sum_{k=1}^{128} \nabla L_{X_{3,k}} * W_{k,c}^{rot180} \tag{9.11}$$

Here, $\nabla L_{X_{3,k}}$ represents the gradient of the kth output channel of Layer 3, while $W_{k,c}^{rot180}$ is the 180°-rotated filter connecting the kth output channel of Layer 3 to the cth input channel of Layer 2. The convolution operation ensures that the gradient for each channel in Layer 2 integrates contributions from all 128 channels in Layer 3. Importantly, "same" padding is used during the convolution, which preserves the spatial dimensions (64×64) throughout this process. As a result, the final gradient tensor for Layer 2 has the shape $64 \times 64 \times 64$, matching its original structure.

The aggregation of 128 gradient channels from Layer 3 into the 64 channels of Layer 2 is achieved during backpropagation through a process that involves summing partial contributions across all 128 channels. This is how it works in detail:

Layer 3 generates 128 output channels (or feature maps), each contributing a gradient to Layer 2. These gradients are represented as a 2D matrix of size 64×64, denoted as $\nabla L_{X_{3,k}}$, where k runs from 1 to 128. Each of these 128 gradient matrices contributes to the computation of gradients for Layer 2, which has 64 filters. A single filter in Layer 2, indexed as c, is connected to all 128 channels of Layer 3. The weights of this filter are represented as a tensor $W_{k,c}$, where k represents the channel in Layer 3 and c corresponds to the filter index in Layer 2.

During backpropagation, each gradient slice $\nabla L_{X_{3,k}}$ from Layer 3 is convolved with the 180°-rotated version of the corresponding filter $W_{k,c}^{rot180}$. This operation calculates the contributions of the gradient from channel k of Layer 3 on channel c of Layer 2. Once these contributions are computed for all 128 channels, they are summed element-wise across all spatial positions (i, j) to produce the final gradient for channel c in Layer 2.

Each of the 64 filters in Layer 2 integrates information from all 128 channels of Layer 3. This is achieved by summing the rotated filter contributions and the corresponding gradients from each channel of Layer 3. The number of channels in Layer 2 is determined by the number of filters it contains, which is why the final gradient tensor for Layer 2 has dimensions $64 \times 64 \times 64$.

Figure 9.11 demonstrates the gradient flow and reshaping process during backpropagation. On the right, we see the $64 \times 64 \times 128$ gradient tensor from Layer 3, where each of the 128 channels contributes to the gradients for Layer 2. In the middle, the rotated $3 \times 3 \times 64$ filters are shown, which are convolved with each channel of the gradient tensor from Layer 3.

The results of these convolutions are aggregated to form the gradient tensor of Layer 2, shown on the left with dimensions $64 \times 64 \times 64$. This aggregation ensures

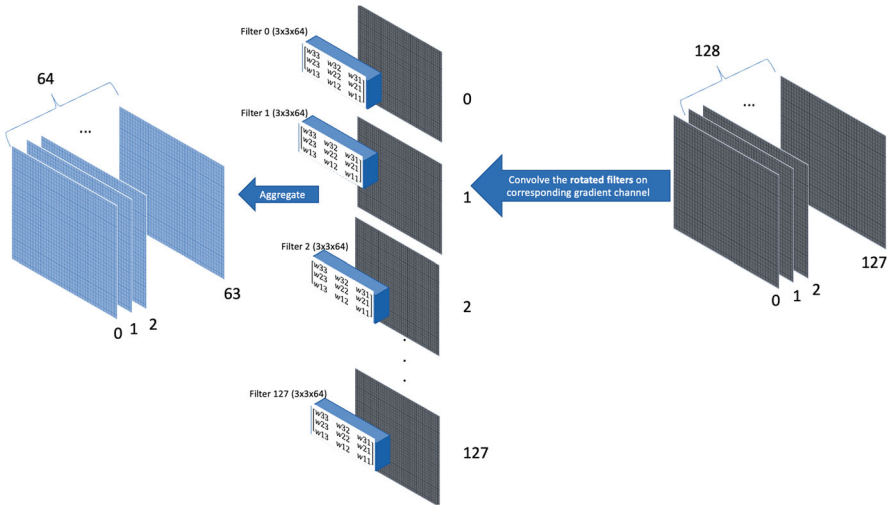


Fig. 9.11 Gradient flow during backpropagation

that the contributions from all 128 channels in Layer 3 are combined into the 64 channels of Layer 2 while preserving the spatial dimensions.

9.2.7 Updating the Filters Weights

Once the gradients of the filter weights, ΔW_k, have been computed using the process described, the next step in the training process involves updating these filter weights to reduce the loss. This is achieved by applying the gradient descent optimization rule, which adjusts the weights based on the computed gradients and the learning rate.

During the weight update step, each filter weight $W_k[m, n]$ is updated using the following equation:

$$W_k^{new}[m,n] = W_k[m,n] - \alpha \cdot \Delta W_k[m,n] \tag{9.11}$$

where:

- $W_k[m, n]$ is the current weight of the filter at position (m, n).
- α is the learning rate.
- $\Delta W_k[m, n]$ is the gradient of the loss with respect to the weight at position (m, n).

This weight update process is repeated for all the weights in each filter and across all filters in the convolutional layer. By iteratively applying this process over multiple training steps, the filters in the convolutional layer are fine-tuned to minimize the loss function, enabling the network to learn feature representations that are increasingly relevant for the task.

Summary of Gradient Computation Process
The backpropagation process for computing gradients in a CNN can be summarized as follows:

1. Compute Activation Gradients:
   - The gradient tensor received from the deeper layer is element-wise multiplied with the derivative of the activation function applied to the pre-activation outputs of the current layer. This step filters gradients to propagate only through activated neurons.
2. Calculate Input Gradients:
   - Each filter in the current layer is rotated 180°, and the gradient tensor is convolved with these rotated filters. This computes the gradient with respect to the input of the current layer, preserving the spatial alignment necessary for backpropagation.
3. Calculate Filter Gradients:
   - Using the input feature maps from the forward pass and the gradient tensor, a sliding window mechanism is applied to compute the gradient for each filter's weights. These gradients are accumulated across all input regions contributing to each filter.
4. Reshape Gradients for the Previous Layer:
   - After the input gradients are computed, the resulting gradient tensor is automatically aligned to match the shape of the previous layer's output. This alignment ensures compatibility for propagating gradients backward to the preceding layer.
5. Repeat for All Layers:
   - The process is repeated iteratively for all preceding convolutional layers, ensuring gradients are propagated backward through the network.
6. Update Filter Weights:
   - Once gradients are computed for all layers, the filter weights are updated using the gradient descent rule, enabling the CNN to optimize its filters during training.

9.3 The Pooling Layer

As we have seen in the convolutional layer, each layer will try to extract more features by applying more filters to the input data. However, as the network grows, the feature maps will get bigger and bigger, and not every bit of these feature maps is important. Some regions may contain noise, irrelevant data, or even zeros, which can

lead the model to overfit, increase training time, and consume more computational resources. The solution is the pooling layer (Caruana et al. 2000).

The pooling layer reduces the size of a feature map from a Conv2D layer by down-sampling its spatial dimensions (width and height) while retaining the most important features. By selectively keeping critical information and discarding less relevant details, pooling improves the network's performance in several ways:

- Enhancing generalization: Reducing overfitting by discarding unnecessary information.
- Lowering computational cost: Decreasing the size of intermediate representations for faster training and inference.
- Preserving key features: Retaining only the most dominant patterns that matter for classification or prediction.

There are two common types of pooling operations:

- Max Pooling: Keeps the maximum value in each pooling window, ensuring the most prominent activation (or feature) is preserved. For example, Fig. 9.12 illustrates how a 2×2 max-pooling operation is applied to a 4×4 feature map. In this case, the largest value from each 2×2 window is selected, resulting in a reduced 2×2 feature map that highlights the most important features.
- Average Pooling: Computes the average of all values in each pooling window, resulting in a smoother representation of the feature map. Figure 9.13 demonstrates this process. Using the same 4×4 input as Fig. 9.12, average pooling selects the mean of each 2×2 window to produce a 2×2 feature map that maintains the average trend of the features.

Pooling layers are strategically placed after convolutional layers to ensure that the extracted features are both relevant and computationally manageable. By reducing the resolution while retaining key information, pooling helps the network focus on the bigger picture instead of pixel-level details, which may not always contribute to

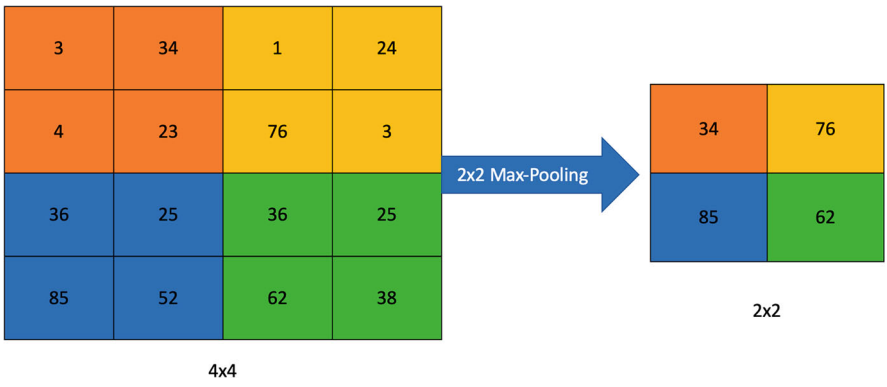


Fig. 9.12 Max pooling

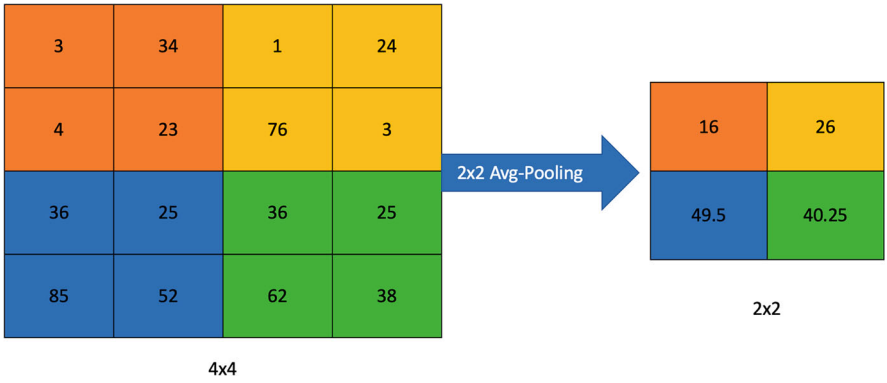


Fig. 9.13 Average pooling

the learning process. This down-sampling step is essential for the scalability and effectiveness of modern convolutional neural networks.

If we incorporate a MaxPooling2D layer after each Conv2D layer in our previous network, each pooling layer will follow its respective Conv2D layer, as shown in the modified model below:

```
import tensorflow as tf
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential

# Example model to demonstrate feature extraction using multiple Conv2D
layers
model = Sequential([
    # First Convolutional Layer
    Conv2D(filters=32, kernel_size=(3, 3),padding='same',
activation='relu', input_shape=(64, 64, 3)),
    MaxPooling2D(pool_size=(2, 2)),
    # Second Convolutional Layer
      Conv2D(filters=64, kernel_size=(3, 3) ,padding='same',
activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    # Third Convolutional Layer
    Conv2D(filters=128, kernel_size=(3, 3) ,padding='same',
activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    # Flatten the feature maps
    Flatten(),

    # Fully connected ANN layers
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')  # Assuming we have 10 output classes
])

# Display the model architecture
model.summary()
```

Layer (type)	Output Shape
conv2d_6 (Conv2D)	(None, 64, 64, 32)
conv2d_7 (Conv2D)	(None, 64, 64, 64)
conv2d_8 (Conv2D)	(None, 64, 64, 128)
flatten_2 (Flatten)	(None, 524288)
dense_4 (Dense)	(None, 128)
dense_5 (Dense)	(None, 10)

Fig. 9.14 Model architecture without pooling layers

Layer (type)	Output Shape
conv2d (Conv2D)	(None, 64, 64, 32)
max_pooling2d (MaxPooling2D)	(None, 32, 32, 32)
conv2d_1 (Conv2D)	(None, 32, 32, 64)
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 64)
conv2d_2 (Conv2D)	(None, 16, 16, 128)
max_pooling2d_2 (MaxPooling2D)	(None, 8, 8, 128)
flatten (Flatten)	(None, 8192)
dense (Dense)	(None, 128)
dense_1 (Dense)	(None, 10)

Fig. 9.15 Model architecture with pooling layers

If we print the summary of the first network (without pooling layers), the output shape for each layer would look like Fig. 9.14, where the Flatten layer has a shape of 524,288. However, after applying a pooling layer after each Conv2D layer, as shown in the modified architecture, the output shapes change significantly, as shown in Fig. 9.15.

The reduction in spatial dimensions is evident at every step:

- Each pooling layer halves the spatial size (width and height), resulting in progressively smaller feature maps.
- By the Flatten layer, the feature map dimensions are reduced to 8192, which is a significant decrease compared to the earlier 524,288.

This reduction not only decreases the memory and computational requirements of the model but also helps prevent overfitting by reducing the number of parameters passed to the fully connected layers.

9.3.1 Backpropagation Through the Pooling Layers

Backpropagation through pooling layers involves the selective redistribution of gradients from the subsequent layer to the positions in the preceding layer that influenced the forward pass. Unlike convolutional layers, pooling layers do not have learnable parameters, so the focus is on propagating gradients appropriately.

When gradients arrive at the Flatten layer from the fully connected layers, they exist as a one-dimensional vector. The Flatten layer's responsibility during backpropagation is to reshape this vector into a three-dimensional tensor matching the dimensions of the last pooling layer's output. If the output of the last pooling layer was of shape $8 \times 8 \times 128$, the gradient vector is reshaped into this same size. This reshaped tensor aligns spatially and channel-wise with the feature maps produced during the forward pass, ensuring that the gradients are ready to flow backward through the pooling layer.

In the MaxPooling layer, backpropagation uses the indices of the maximum values from each pooling window during the forward pass. Let X be the input tensor to the MaxPooling layer with dimensions $h \times w \times c$, where h and w represent the spatial dimensions, and c is the number of channels. If the pooling window is $p \times p$, the MaxPooling operation selects the maximum value X_{ij} in each $p \times p$ window for every channel. During backpropagation, the gradient tensor from the subsequent layer is distributed to the position corresponding to $\max(X_{ij})$, while all other positions within the window receive a gradient of zero. Mathematically, if G represents the gradient tensor received by the MaxPooling layer, the gradient G_{pool} propagated backward can be written as:

$$G_{\text{pool}}(i,j,k)\begin{cases} G(i',j',k), \text{if } X(i,j,k) = \max(X_{i,j,k}) \\ 0, \qquad\qquad \text{otherwise} \end{cases} \tag{9.12}$$

Here, i' and j' correspond to the spatial location of the maximum value in the $p \times p$ window.

Once the gradients pass through the MaxPooling layer, they flow into the preceding convolutional layer. As gradients continue to propagate backward through the network, the same process is applied recursively at each pooling and convolutional layer.

During backpropagation through an average pooling layer, the gradient from the subsequent layer, $G_{\text{avg}}(i',j',k)$, is distributed equally to all positions within the pooling window that contributed to the output. If the pooling window has

dimensions $p \times p$, the gradient for each element in the input tensor, $G_{\text{pool}}(i,j,k)$, is calculated as:

$$G_{\text{pool}}(i,j,k) = \frac{G(\mathrm{i}',\mathrm{j}',\mathrm{k})}{p \times p} \tag{9.13}$$

where:

- $i^{'}$ and $j^{'}$ represent the spatial indices in the output gradient tensor.
- i and j represent the corresponding indices in the input tensor for the pooling window.
- k is the channel index.
- $p \times p$ is the size of the pooling window.

This ensures that the gradient contribution from the output is uniformly spread across all elements of the input region within the pooling window.

9.4 The Dropout Layer

During the training of a CNN, some connections in the dense layer may become stronger than others due to the nature of the extracted features. This imbalance can result in certain connections becoming redundant or obsolete, which may lead to overfitting. Overfitting occurs when the model learns to overly rely on specific connections and fails to generalize well to unseen data (Srivastava et al. 2014).

To address this issue, a dropout layer is introduced to add randomness to the training process. The dropout layer randomly "turns off" a fraction of the neurons in the layer during each training iteration. By doing so, the network is forced to explore alternative connections and dependencies, preventing it from relying too heavily on specific neurons. This randomness helps the network generalize better by encouraging it to learn more robust feature representations.

In practice, the dropout layer is typically placed after a dense layer. During training, it randomly deactivates a subset of neurons in the dense layer and connects the remaining active neurons to the next layer. With each iteration, a different set of neurons is activated and deactivated, introducing variability in the connections. This process ensures that the model is not overly dependent on any single set of neurons.

For example, a dropout layer can be defined in Keras as follows:

```
Dense(128, activation='relu'),
# Adding a dropout layer with a dropout rate of 0.5
Dropout(rate=0.5)
Dense(256, activation=relu)
Dense(10, activation='softmax')
```

The "rate" parameter specifies the fraction of neurons to deactivate during training. In this case, 50% of the neurons will be turned off at random in each iteration.

It is important to note that dropout is only applied during the training phase. During inference (prediction), all neurons remain active, and their outputs are scaled by the dropout rate to maintain consistency with the training process.

By incorporating dropout layers into the network architecture, the model becomes less prone to overfitting, making it more effective and reliable when applied to new data. Dropout layers are a simple yet powerful tool for improving the generalization of deep learning models.

9.5 Concluding Remarks

In this chapter, we explored the fundamental principles and architecture of Convolutional Neural Networks (CNNs), a powerful tool for processing and analyzing image data. By integrating convolutional operations with artificial neural networks, CNNs enable the automatic extraction of hierarchical features directly from raw image inputs, making them highly effective for tasks like image classification.

We examined the core components of CNNs, beginning with convolutional layers that extract meaningful patterns such as edges and textures. These layers were followed by pooling operations, which reduced the spatial dimensions of feature maps while retaining key information. Finally, we discussed how fully connected layers process the extracted features to perform the final classification. This structure, combined with the ability to stack multiple convolutional layers, allows CNNs to progressively learn more abstract and complex representations of the input data.

The chapter also detailed the backpropagation process in CNNs, emphasizing how gradients are propagated through the network to optimize filter weights. This mechanism, paired with activation functions such as ReLU, enables the network to capture non-linear relationships effectively.

Additionally, we highlighted the role of pooling layers, dropout layers, and other techniques to improve generalization and mitigate overfitting. These approaches reduce the computational burden while enhancing the model's robustness and performance.

Now, after having learnt all this, you will be able to understand what stays behind the image generated on the cover, and how Convolutional Neural Networks are essential to the process of generating realistic and meaningful images in modern AI systems.

Chapter 10
Recurrent Neural Networks



Abstract This chapter introduces Recurrent Neural Networks (RNNs), emphasizing their unique ability to model and process sequential data such as text, time series, and language. Unlike feedforward networks, RNNs maintain a hidden state across time steps, allowing them to capture temporal dependencies and contextual relationships within sequences.

We examine the core architecture of RNNs, detailing weight initialization, the forward pass for prediction, and Backpropagation Through Time (BPTT) for training. Concepts like one-hot encoding, hidden state transitions, and cross-entropy loss are explored through practical examples to illustrate how RNNs learn from sequential patterns. Overall, this chapter provides a foundational understanding of how RNNs operate and their importance in sequence modeling tasks.

10.1 Recurrent Neural Networks

As discussed in the Naive Bayes chapter, the nature of text embeds its features within it. Words can have multiple meanings—a word might function as a noun, pronoun, verb, or adjective—and the context of the text can significantly alter its meaning. Additionally, the dependency of a word on previous words in a sentence gives text a sequential nature, where the present meaning often relies on something from the past.

A foundational neural network suitable for handling sequential data like text or time series is the *Recurrent Neural Network (RNN)* (Graves and Graves 2012; Mandic and Chambers 2001).

An RNN consists of neurons, just like a standard neural network. These neurons are interconnected, and the connections have weights and biases, similar to a traditional neural network. However, the key difference is that the output of a neuron, after completing its calculation, is fed forward to the next neuron. Additionally, this same output is reused in the calculation of the next data point. This forms a self-loop in the neuron, where the output is both passed to the next neuron and reintroduced as an input, along with the new data point (Chollet and Chollet 2021; Sutskever et al. 2014).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_10

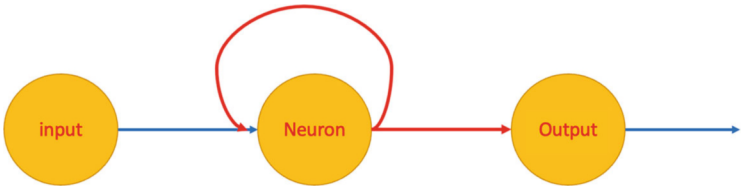


Fig. 10.1 A simple recurrent neural network

One common application of RNNs in text processing is *text auto-completion*, where the goal is to predict the next character or word. For example, consider the sentence: "Hello, how are you?" The first data point here is "Hello." The neuron computes some values based on the weights and biases and then passes these computations to the next layer. When processing the next word, "how," the neuron uses the state from the previous word (Hello) to influence its computation. This process continues with the word "are," where the neuron incorporates both the new input and the previous state into its calculation.

This sequential computation creates a form of *dependency between words*, which is essential for generating coherent and contextually relevant text. In essence, RNNs excel at capturing temporal and contextual relationships within sequential data, making them highly effective for tasks such as language modeling, machine translation, and speech recognition.

Figure 10.1 illustrates a simple Recurrent Neural Network. The input can be one word or one character at a time. Based on the weights and biases, the neuron computes the output and passes it to the next layer. When the next word or character is fed into the network, the output from the previous neuron's state—generated from processing the previous word or character—will be fed back into the neuron alongside the current input.

In this simple example, the output of the RNN represents the prediction of the next word or next character, depending on the network's configuration.

Let's explore a simple example of training an RNN to predict the next character in a sentence: "hello, how are you."

10.1.1 Step 1: Set the Hidden State Size and the Sequence Size

In an RNN, the hidden state represents the internal memory of the neuron. It stores the computed output from the previous step and feeds it back into the neuron along with the next input. The hidden state size determines the dimensionality of this memory (e.g., 2D, 3D, 4D). A larger hidden state size enables the network to learn more complex features and patterns from the text, while a smaller size might limit its learning capacity. In this example, we'll use a hidden state size of 4.

Another critical parameter in RNNs is the sequence size. The sequence size defines how many input instances (e.g., characters in this case) are processed before

performing gradient descent to update the weights. It acts like a short-term memory, allowing the network to maintain context over a defined number of previous inputs. In this example, we'll use a sequence size of 5.

This means the network will process five characters at a time, where each character depends on the previous hidden state. After processing these five characters, the network will perform gradient descent to update the weights and biases based on the error calculated.

10.1.2 Step 2: Tokenization

First, we split the training sentence into tokens. Since this application focuses on predicting the next character, we split the text into unique characters.

The output is a set of unique characters in the sentence:

```
['w', 'o', ' ', 'l', 'h', 'e', 'r', 'y', 'u', 'a']
```

Next, we assign an index to each of these characters so we can process them as numbers in the model:

```
{'w': 0, 'o': 1, ' ': 2, 'l': 3, 'h': 4, 'e': 5, 'r': 6, 'y': 7, 'u': 8, 'a':
9}
```

This numerical representation allows the RNN to process the characters as numerical inputs, making it easier to compute and predict the next character in the sequence.

10.1.3 Step 3: Initialize the Weights

In our simple RNN, as illustrated in Fig. 10.2, there are three main connections, each requiring weights and biases for proper computation.

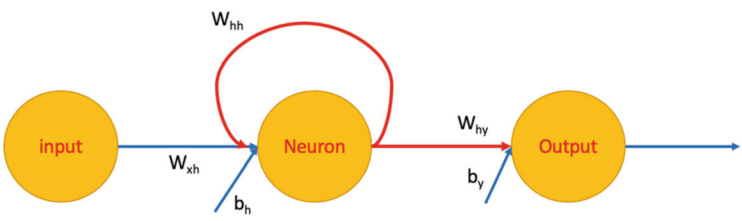


Fig. 10.2 Simple RNN with weights

1. Input to Hidden Neuron (W_{xh})
   - This connection maps the input layer (character vector) to the hidden state.
   - The weight matrix (W_{xh}) must have a size of (hidden state size, vocabulary size).
   - In our example:

$$W_{xh} \in \mathbb{R}^{4 \times 10} \tag{10.1}$$

   - 4 represents the hidden state size.
   - 10 represents the vocabulary size (unique characters).

2. Hidden Neuron to Itself (W_{hh})
   - This connection represents the recurrent connection, where the hidden state is passed back into itself in the next time step.
   - The weight matrix (W_{hh}) must have a size of (hidden state size, hidden state size).
   - In our example:

$$W_{xh} \in \mathbb{R}^{4 \times 4} \tag{10.2}$$

3. Hidden Neuron Bias (b_h)
   - Each hidden neuron requires a bias vector to adjust its output.
   - The bias vector b_h has a size equal to the hidden state size.
   - In our example:

$$b_h \in \mathbb{R}^4 \tag{10.3}$$

4. Hidden Neuron to Output (W_{hy})
   - This connection maps the hidden state to the output layer (vocabulary prediction).
   - The weight matrix (W_{hy}) must have a size of (vocabulary size, hidden state size).
   - In our example:

$$W_{hy} \in \mathbb{R}^{10 \times 4} \tag{10.4}$$

5. Output Bias (b_y)
   - Each output neuron requires a bias vector to adjust its predictions.
   - The bias vector b_y has a size equal to the vocabulary size.
   - In our example:

$$b_y \in \mathbb{R}^{10} \tag{10.5}$$

All the weights (W_{xh}, W_{hh}, W_{hy}) and biases (b_h, b_y) are initialized with random values to break symmetry and allow the RNN to start learning effectively.

10.1.4 Step 4: One-Hot Encode the Input

In an RNN, the unique characters in the text must be represented numerically to enable mathematical operations during training. However, directly using numerical indices might introduce unintended relationships between the characters. To address this, we use a technique called one-hot encoding.

In one-hot encoding, each character is represented as a vector of zeros with a single '1' at the position corresponding to that character's index in the vocabulary. This ensures that each character has a unique representation without implying any inherent order or hierarchy between them.

For example, based on our vocabulary mapping:

```
{'w': 0, 'o': 1, ' ': 2, 'l': 3, 'h': 4, 'e': 5, 'r': 6, 'y': 7, 'u': 8, 'a':
9}
```

The one-hot encoding of the character 'a' would look like this:

```
[0,0,0,0,0,0,0,0,0,1]
```

Similarly, the one-hot encoding of the character 'e' would look like this:

```
[0,0,0,0,0,1,0,0,0,0]
```

In this representation, each vector is the same size as the vocabulary size, which in our case is 10. The position of the '1' indicates the presence of a specific character, while all other positions remain '0'.

When processing a sentence, each character in the sequence will be converted into its corresponding one-hot encoded vector. These vectors will serve as the input to the RNN, allowing it to perform calculations with a consistent numerical format.

10.1.5 Step 5: Prepare the Target

Since the goal of our simple RNN is to predict the next character in a sequence, the target vector must represent the next character for every character in the input

sequence. Essentially, each character in the sequence is paired with the next character as its target.

To ensure the dimensions of the input and target vectors match and to prevent indexing errors during training, we can shift the sequence by one position. This means we remove the first character from the sequence and append it to the end.

For example, if the original sequence is:

```
[h, e, l, l, o]
```

The target sequence becomes:

```
[e, l, l, o, h]
```

In this format:

- The first character 'h' corresponds to the target 'e'.
- The second character 'e' corresponds to the target 'l'.
- And so on.

This alignment ensures that each character's index in the input vector matches the index in the target vector. For example:

- The character 'h' (index 0) will have its target as 'e', which can also be found at index 0 in the target vector.

It's important to note that we do not use the raw characters directly for the target sequence. Instead, we represent each target character using its one-hot encoded vector.

By using one-hot encoding for both inputs and targets, we ensure that the network can perform matrix calculations without encountering dimensional mismatches.

At this stage:

- Our input is a sequence of one-hot encoded characters.
- Our target is a shifted version of the input, also represented in one-hot encoding.

10.1.6 Step 6: Initialize the Hidden State Vector

In an RNN, each hidden state depends on the previous hidden state to maintain the temporal relationships between sequential inputs. To achieve this dependency, we need a hidden state vector that acts as a memory, carrying information from one step to the next. At the beginning of the sequence, there is no previous hidden state available. Therefore, we start with a vector of zeros, where the size of this vector matches the hidden state size. In our example, this size is 4, meaning our initial hidden state vector will have four zero values.

When the first character is processed, it uses this initial zero vector as its previous hidden state. The RNN computes a new hidden state based on the first character's input and the initial hidden state. Once this computation is complete, the resulting hidden state is stored and passed along with the next input character. This process repeats for every character in the sequence, with each hidden state carrying information from the previous states.

However, the hidden state vector is not infinite. Its persistence depends on the sequence size parameter. The sequence size determines how many characters are processed before the hidden state vector is reset to zeros. For example, if the sequence size is 5, the initial hidden state vector will be zeros at the start. The RNN will process characters 0–4, updating the hidden state at each step. But when it reaches the fifth character, a new hidden state vector of zeros is reinitialized, and the process starts again.

This reset mechanism ensures that the RNN does not carry dependencies over excessively long sequences, preventing issues such as vanishing gradients or over-reliance on earlier states. It also aligns with how gradient descent is performed, as the network updates its weights after processing each sequence.

10.1.7 Step 7: The Forward Pass

The forward pass begins by retrieving the one-hot encoding of the first character in the sequence. This vector serves as the input to the RNN for the first time step. The hidden state for this character is then computed using the following formula:

$$h_t = \tan h(W_{xh} \cdot x + W_{hh} \cdot h_{t-1} + b_h) \tag{10.6}$$

In this equation, the input vector x (the one-hot encoded character) is multiplied by the input-to-hidden weight matrix W_{xh}. Then, the previous hidden state h_{t-1} is multiplied by the hidden-to-hidden weight matrix W_{hh}. These two results are summed along with the hidden bias vector b_h, and the combined value is passed through a tan$h$ activation function. The tanh function ensures non-linearity and keeps the hidden state values bounded between -1 and 1.

At this stage, since we are processing the first character, the previous hidden state (h_{t-1}) is initialized to zeros. As a result, the hidden state calculation for the first character simplifies slightly because there's no contribution from a prior state.

Once the hidden state is computed for the current character, it is stored in the hidden state vector. This ensures that the next character can reference this state as its previous state when it is processed.

The computed hidden state is then passed to the output neuron to predict the next character. At this stage, the hidden state is multiplied by the hidden-to-output weight matrix (W_{hy}), and the output bias vector (b_y) is added. The resulting values are then passed through a softmax activation function to produce a probability distribution

over the vocabulary. This probability distribution represents the RNN's prediction for the next character in the sequence.

The loss is then calculated by comparing the predicted output with the actual next character (from the target sequence). This comparison is typically done using a cross-entropy loss function, which quantifies the difference between the predicted probabilities and the true target.

This process is repeated for each character in the sequence, with the hidden state being updated at each step and passed forward to the next character. After processing a sequence of characters equal to the sequence size, the hidden state vector is reset to zeros, and the forward pass begins again for the next sequence.

To understand how the RNN predicts the second character starting from the first character, we'll walk through the computations step by step.

We begin by initializing the weights and biases randomly. Let's assume the initial matrices and vectors are as follows:

$$W_{xh} = \begin{bmatrix} -0.13 & -0.078 & -0.175 & 0.089 & -0.025 & 0.053 & 0.102 & 0.086 & 0.066 & 0.024 \\ 0.182 & 0.065 & -0.084 & 0.053 & 0.032 & -0.144 & 0.093 & 0.121 & -0.22 & 0.062 \\ -0.154 & -0.211 & -0.057 & -0.081 & 0.103 & 0.025 & -0.024 & 0.123 & 0.053 & 0.262 \\ 0.21 & -0.066 & -0.006 & -0.025 & 0.085 & 0.176 & -0.007 & -0.002 & -0.098 & 0.086 \end{bmatrix}$$

$$W_{hh} = \begin{bmatrix} 0.111 & 0.053 & 0.252 & -0.133 \\ 0.124 & -0.047 & -0.04 & -0.07 \\ 0.201 & -0.169 & 0.243 & 0.108 \\ 0.057 & 0.177 & 0.056 & -0.078 \end{bmatrix}$$

$$W_{hy} = \begin{bmatrix} -0.171 & -0.002 & -0.14 & 0.011 \\ -0.033 & -0.031 & -0.046 & 0.04 \\ -0.166 & 0.05 & -0.122 & 0.151 \\ -0.078 & -0.082 & 0.062 & 0.005 \\ -0.053 & -0.066 & -0.065 & -0.004 \\ -0.071 & 0.03 & -0.073 & 0.025 \\ -0.003 & 0.163 & -0.297 & -0.253 \\ -0.065 & -0.143 & 0.032 & 0.04 \\ 0.077 & -0.038 & -0.08 & 0.182 \\ -0.003 & 0.069 & 0.083 & -0.014 \end{bmatrix}$$

$$b_h = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$b_y = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

The hidden state vector (h_s) is initialized to:

$$h_s = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

In our sentence, "hello how are you," the first character is 'h'. From our unique vocabulary:

```
{'w': 0, 'o': 1, ' ': 2, 'l': 3, 'h': 4, 'e': 5, 'r': 6, 'y': 7, 'u': 8, 'a': 9}
```

The one-hot encoding of the character 'h' is:

$$x = [0, 0, 0, 0, 1, 0, 0, 0, 0, 0]$$

This vector represents the input for the first character and will be fed into the RNN.

Next, we reshape this one-hot vector into a format suitable for matrix multiplication.

$$x = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

We then compute the hidden state (h_t) using the formula:

$$h_t = \tan h(W_{xh} \cdot x + W_{hh} \cdot h_{t\text{-}1} + b_b)$$

1. First, we multiply the input vector (x) by the input-to-hidden weight matrix (W_{xh}).

$$\begin{bmatrix} -0.13 & -0.078 & -0.175 & 0.089 & -0.025 & 0.053 & 0.102 & 0.086 & 0.066 & 0.024 \\ 0.182 & 0.065 & -0.084 & 0.053 & 0.032 & -0.144 & 0.093 & 0.121 & -0.22 & 0.062 \\ -0.154 & -0.211 & -0.057 & -0.081 & 0.103 & 0.025 & -0.024 & 0.123 & 0.053 & 0.262 \\ 0.21 & -0.066 & -0.006 & -0.025 & 0.085 & 0.176 & -0.007 & -0.002 & -0.098 & 0.086 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

2. Next, we add the result of multiplying the previous hidden state ($h_{t\text{-}1}$) (which is currently [0, 0, 0, 0]) with the hidden-to-hidden weight matrix (W_{hh}).

$$\begin{bmatrix} -0.13 & -0.078 & -0.175 & 0.089 & -0.025 & 0.053 & 0.102 & 0.086 & 0.066 & 0.024 \\ 0.182 & 0.065 & -0.084 & 0.053 & 0.032 & -0.144 & 0.093 & 0.121 & -0.22 & 0.062 \\ -0.154 & -0.211 & -0.057 & -0.081 & 0.103 & 0.025 & -0.024 & 0.123 & 0.053 & 0.262 \\ 0.21 & -0.066 & -0.006 & -0.025 & 0.085 & 0.176 & -0.007 & -0.002 & -0.098 & 0.086 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$+ \begin{bmatrix} 0.111 & 0.053 & 0.252 & -0.133 \\ 0.124 & -0.047 & -0.04 & -0.07 \\ 0.201 & -0.169 & 0.243 & 0.108 \\ 0.057 & 0.177 & 0.056 & -0.078 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

3. We then add the hidden bias vector (b_h).

$$\begin{bmatrix} -0.13 & -0.078 & -0.175 & 0.089 & -0.025 & 0.053 & 0.102 & 0.086 & 0.066 & 0.024 \\ 0.182 & 0.065 & -0.084 & 0.053 & 0.032 & -0.144 & 0.093 & 0.121 & -0.22 & 0.062 \\ -0.154 & -0.211 & -0.057 & -0.081 & 0.103 & 0.025 & -0.024 & 0.123 & 0.053 & 0.262 \\ 0.21 & -0.066 & -0.006 & -0.025 & 0.085 & 0.176 & -0.007 & -0.002 & -0.098 & 0.086 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$+ \begin{bmatrix} 0.111 & 0.053 & 0.252 & -0.133 \\ 0.124 & -0.047 & -0.04 & -0.07 \\ 0.201 & -0.169 & 0.243 & 0.108 \\ 0.057 & 0.177 & 0.056 & -0.078 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

4. The resulting sum is passed through the tanh activation function, producing the new hidden state (h_t).

$$h_t = \tanh h \left(\begin{bmatrix} -0.13 & -0.078 & -0.175 & 0.089 & -0.025 & 0.053 & 0.102 & 0.086 & 0.066 & 0.024 \\ 0.182 & 0.065 & -0.084 & 0.053 & 0.032 & -0.144 & 0.093 & 0.121 & -0.22 & 0.062 \\ -0.154 & -0.211 & -0.057 & -0.081 & 0.103 & 0.025 & -0.024 & 0.123 & 0.053 & 0.262 \\ 0.21 & -0.066 & -0.006 & -0.025 & 0.085 & 0.176 & -0.007 & -0.002 & -0.098 & 0.086 \end{bmatrix} \cdot \right.$$

$$\left. \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.111 & 0.053 & 0.252 & -0.133 \\ 0.124 & -0.047 & -0.04 & -0.07 \\ 0.201 & -0.169 & 0.243 & 0.108 \\ 0.057 & 0.177 & 0.056 & -0.078 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} -0.024 \\ 0.031 \\ 0.102 \\ 0.084 \end{bmatrix}$$

This new hidden state is then stored in the hidden state vector so it can be used for the next character.

After computing the hidden state vector (h_t) for the first character, it is passed to the output neuron to predict the next character in the sequence. This happens through the following formula:

$$y = \text{softmax}\left(W_{hy} \cdot h_t + b_y\right)$$

$$y = \text{softmax}\left(\begin{bmatrix} -0.171 & -0.002 & -0.14 & 0.011 \\ -0.033 & & & \\ -0.166 & -0.031 & -0.046 & 0.04 \\ -0.078 & 0.05 & -0.122 & 0.151 \\ -0.053 & -0.082 & 0.062 & 0.005 \\ -0.071 & -0.066 & -0.065 & -0.004 \\ -0.003 & 0.03 & -0.073 & 0.025 \\ -0.065 & 0.163 & -0.297 & -0.253 \\ 0.077 & -0.143 & 0.032 & 0.04 \\ -0.003 & -0.038 & -0.08 & 0.182 \\ & 0.069 & 0.083 & -0.014 \end{bmatrix} \cdot \begin{bmatrix} -0.024 \\ 0.031 \\ 0.102 \\ 0.084 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}\right)$$

$$= \begin{bmatrix} 0.0994 \\ 0.1002 \\ 0.1009 \\ 0.1009 \\ 0.0995 \\ 0.1001 \\ 0.0957 \\ 0.1007 \\ 0.1007 \\ 0.1013 \end{bmatrix}$$

The hidden state (h_t) is multiplied by the hidden-to-output weight matrix (W_{hy}), and the result is added to the output bias vector (b_y). The sum is then passed through a softmax activation function to produce a probability distribution over the entire vocabulary. The softmax ensures that all probabilities sum to 1, and each value represents the likelihood of the corresponding character being the next one in the sequence.

Assuming the output probability distribution indicates the highest probability corresponds to index 9 in the vocabulary, the predicted character is 'a'.

However, the actual next character in our sequence is 'e', which corresponds to the one-hot encoded vector: [0,0,0,0,0,1,0,0,0,0].

From the probability distribution generated by the softmax function, we look at the probability corresponding to 'e'. This probability is: 0.1001.

Using this probability, we calculate the cross-entropy loss. The cross-entropy loss function is given by:

$$L = -\log_e(p) \tag{10.7}$$

Where:

- L is the loss value.
- p is the predicted probability for the correct character ('e').

Substituting the values:

$$L = -\log_e(0.1001) = 2.301$$

This loss value quantifies how far off the prediction was from the actual target character. A higher loss value indicates a larger error, while a lower loss value suggests a more accurate prediction.

This process will now repeat for the next character in the sequence, using the updated hidden state vector (h_t) and feeding the next input character into the RNN.

10.1.8 Step 8: Backpropagation Through Time

In order to update the weights and improve the network's ability to predict the next character accurately, backpropagation through time (BPTT) is used. This process adjusts the weights and biases by propagating the error backward through the network over multiple time steps. The number of time steps depends on the sequence size defined earlier. In our example, the sequence size is 5, so the network will process five characters in the forward pass, producing five hidden states and corresponding outputs before performing backpropagation.

The backpropagation process starts at the output neuron after the final character in the sequence is processed. The error signal is calculated based on the difference between the predicted probabilities (from the softmax layer) and the actual one-hot encoded target values. This error is then propagated backward through each time step to update the weights and biases.

First, the error from the output neuron is calculated as the difference between the predicted probability distribution and the actual one-hot encoded vector. This difference is used to calculate the gradient of the output-to-hidden weight matrix (W_{hy}) and the output bias vector (b_y). The error signal is then propagated backward to calculate the gradient of the hidden state (h_t).

At each time step, the gradient of the hidden state is adjusted using the tanh derivative, which accounts for how the hidden state was computed during the forward pass. Specifically, the derivative of the *tan*h activation function is applied:

$$\frac{\partial L}{\partial h_t} = \left(1 - h_t^2\right) \cdot \delta \tag{10.8}$$

where L represents the loss function, h_t is the hidden state at time t, and δ represents the gradient of the loss with respect to the output.

Next, the gradient for the input-to-hidden weight matrix (W_{xh}) is computed based on the gradient of the hidden state and the input vector at the corresponding time step. Similarly, the gradient for the hidden-to-hidden weight matrix (W_{hh}) is computed based on the hidden state from the previous time step.

The bias gradients for both the hidden state (b_h) and output (b_y) are also updated at each time step. Finally, the gradient from the current time step is propagated backward to the previous hidden state to ensure that dependencies across time steps are preserved.

Mathematically, this process can be summarized as:

1. Compute the error at the output layer:

$$\delta_y = p_y - y \tag{10.9}$$

where p_y is the predicted probability and y is the actual one-hot encoded target.

2. Update the output weight matrix (W_{hy}) and output bias (b_y):

$$\Delta W_{hy} += \delta_y \cdot h_t^T \tag{10.10}$$

$$\Delta b_y += \delta_y \tag{10.11}$$

3. Propagate the error to the hidden state:

$$\delta_h = W_{hy}^T \cdot \delta_y + \delta_{h_{\text{next}}} \tag{10.12}$$

4. Apply the *tan*h derivative to calculate the raw hidden state gradient:

$$\delta_{h_{raw}} = \left(1 - h_t^2\right) \cdot \delta_h \tag{10.13}$$

5. Update the input-to-hidden weights (W_{xh}):

$$\Delta W_{\text{x}h} += \delta_{h_{raw}} \cdot x_t^T \tag{10.14}$$

6. Update the hidden-to-hidden weights (W_{hh}):

$$\Delta W_{hh} += \delta_{h_{raw}} \cdot h_{t-1}^T \tag{10.15}$$

7. Update the hidden bias (b_h):

$$\Delta b_h += \delta_{raw} \tag{10.16}$$

8. Pass the gradient backward to the previous hidden state:

$$\delta_{\text{next}} = W_{hh}^T \cdot \delta_{raw} \tag{10.17}$$

This entire process is repeated for each time step in reverse order, starting from the last character in the sequence and moving backward to the first character. After completing the backward pass, the gradients of all weights and biases are accumulated, and an optimization algorithm (e.g., Stochastic Gradient Descent or Adam) is applied to update the weights and biases.

The key aspect of Backpropagation Through Time is that it allows the network to learn dependencies across multiple time steps, enabling it to capture long-term relationships in sequential data. However, if the sequence size is too large, the gradients might vanish or explode, which can impact training stability. Techniques like gradient clipping are often used to address these issues.

10.1.9 Step 9: Update the Weights Based on the Learning Rate

After performing Backpropagation Through Time and calculating the gradients for each weight and bias in the network, the final step is to update the weights and biases to reduce the loss and improve the RNN's predictions in future iterations.

The learning rate α plays a crucial role in this step, controlling how large a step the network takes in the direction of the negative gradient. A smaller learning rate results in smaller updates, leading to more stable but slower convergence. Conversely, a larger learning rate speeds up convergence but can overshoot the optimal weights, causing instability.

The weight and bias updates follow the gradient descent optimization rule. Mathematically, the updates can be expressed as:

$$W_{xh} \leftarrow W_{xh} - \alpha \cdot \Delta W_{xh} \tag{10.18}$$

$$W_{hh} \leftarrow W_{hh} - \alpha \cdot \Delta W_{hh} \tag{10.19}$$

$$W_{hy} \leftarrow W_{hy} - \alpha \cdot \Delta W_{hy} \tag{10.20}$$

$$b_h \leftarrow b_h - \alpha \cdot \Delta b_h \tag{10.21}$$

$$b_y \leftarrow b_y - \alpha \cdot \Delta b_y \tag{10.22}$$

The updated weights and biases will now be used in the next forward and backward passes. Over multiple training iterations, this adjustment process allows the network to learn better representations of the sequential data, improving its ability to predict the next character accurately.

10.2 Implementation

Here is the implantation of this simple RNN:

```
import numpy as np

# Input sentence
sentence = "hello how are you"
chars = list(set(sentence))
print(chars)
vocab_size = len(chars)
char_to_idx = {char: i for i, char in enumerate(chars)}
print(char_to_idx)
idx_to_char = {i: char for i, char in enumerate(chars)}
print(idx_to_char)

# Hyperparameters
hidden_size = 4  # Size of hidden state
sequence_size = 5  # Sequence length
learning_rate = 0.01

# One-hot encoding
def one_hot(index, size):
    vec = np.zeros(size)
    vec[index] = 1
    return vec

# Initialize parameters
global Wxh, Whh, Why, bh, by
Wxh = np.round( np.random.randn(hidden_size, vocab_size) * 0.1,3)
Whh = np.round( np.random.randn(hidden_size, hidden_size) * 0.1,3)
Why = np.round( np.random.randn(vocab_size, hidden_size) * 0.1,3)



bh = np.zeros((hidden_size, 1))
by = np.zeros((vocab_size, 1))

# Activation function (softmax)
def softmax(x):
    exp_x = np.exp(x - np.max(x))
    return exp_x / exp_x.sum(axis=0)

# Forward and Backward pass
def rnn_forward_backward(sentence):
    global Wxh, Whh, Why, bh, by
    inputs = [one_hot(char_to_idx[char], vocab_size) for char in sentence]
    targets = inputs[1:] + [inputs[0]]

    loss = 0
    for i in range(0, len(inputs) - sequence_size + 1, sequence_size):
        hs = [np.zeros((hidden_size, 1))]
        ys = []
        ps = []

        # Forward pass
        for t in range(sequence_size):
```

```
            x = inputs[i + t].reshape(vocab_size, 1)
            ht = np.tanh(np.dot(Wxh, x) + np.dot(Whh, hs[-1]) + bh)
            yt = np.dot(Why, ht) + by
            pt = softmax(yt)
            loss += -np.log(pt[np.argmax(targets[i + t])])
            hs.append(ht)
            ys.append(yt)
            ps.append(pt)

        # Backward pass
        dWxh, dWhh, dWhy = np.zeros_like(Wxh), np.zeros_like(Whh),
np.zeros_like(Why)
        dbh, dby = np.zeros_like(bh), np.zeros_like(by)
        dh_next = np.zeros_like(hs[0])
        for t in reversed(range(sequence_size)):
            dy = ps[t]
            dy[np.argmax(targets[i + t])] -= 1
            dWhy += np.dot(dy, hs[t+1].T)
            dby += dy
            dh = np.dot(Why.T, dy) + dh_next
            dh_raw = (1 - hs[t+1] ** 2) * dh
            dbh += dh_raw
            dWxh += np.dot(dh_raw, inputs[i + t].reshape(1, -1))
            dWhh += np.dot(dh_raw, hs[t].T)
            dh_next = np.dot(Whh.T, dh_raw)

        # Gradient Clipping
        for dparam in [dWxh, dWhh, dWhy, dbh, dby]:
            np.clip(dparam, -5, 5, out=dparam)

        # Update parameters
        Wxh -= learning_rate * dWxh
        Whh -= learning_rate * dWhh
        Why -= learning_rate * dWhy
        bh -= learning_rate * dbh
        by -= learning_rate * dby

    print(f"Loss: {loss}")
    print("Gradients:")
    print("dWxh:", dWxh)
    print("dWhh:", dWhh)
    print("dWhy:", dWhy)
    print("dbh:", dbh)
    print("dby:", dby)
# Run RNN
rnn_forward_backward(sentence)
```

10.3 Concluding Remarks

In this chapter, we explored the fundamental principles and mechanics of Recurrent Neural Networks (RNNs), highlighting their critical role in processing sequential data such as text and time-series information. Unlike traditional feedforward neural networks, RNNs maintain a hidden state that captures temporal dependencies, enabling them to understand context and relationships across sequential inputs.

We delved into the architecture of RNNs, including the initialization of weights and biases, the forward pass for computing predictions, and the backward pass using Backpropagation Through Time (BPTT) for gradient optimization. Key concepts such as one-hot encoding, hidden state management, and loss calculation using cross-entropy were illustrated through practical examples.

Despite their capabilities, RNNs face challenges such as vanishing and exploding gradients, which can hinder their performance on long sequences. Techniques such as gradient clipping and optimized architectures like Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs) were briefly alluded to as solutions to these issues.

Chapter 11
Transfer Learning



Abstract This chapter focuses on Transfer Learning, a technique that enables the reuse of pre-trained deep learning models to tackle new tasks, especially when data or computational resources are limited. By leveraging the powerful feature extraction capabilities of existing models, transfer learning significantly reduces training time and improves performance on specialized problems.

We explore two key strategies: *freezing layers*, where only the final layers are retrained for a new task—ideal for small datasets—and *fine-tuning layers*, where all layers are made trainable to refine pre-trained weights, suited for larger datasets. Practical implementations using models like VGGNet, DenseNet, and InceptionV3 illustrate how these methods can be applied effectively.

The chapter also introduces the integration of custom components, such as the Cubixel Layer, to enhance spatial feature extraction. Through experiments on datasets like CIFAR-10, we demonstrate how transfer learning adapts powerful models to new domains with minimal effort.

Transfer learning proves to be a scalable, efficient approach for extending deep learning to a wide range of applications, from image classification to medical imaging. As deep learning advances, transfer learning remains a key method for accelerating innovation while maintaining high accuracy and adaptability.

11.1 Transfer Learning

Training a CNN can be costly in terms of time and resources. Sometimes, the application we are trying to solve using a CNN may not have sufficient data to train an efficient network. Generally, the more data we have, the better the CNN becomes at extracting meaningful features. Therefore, we need a method to save training time, reduce resource usage, and address the challenge of limited data. This method is known as *transfer learning* (Naskinova 2023; Perlich et al. 2014; Zoph et al. 2017).

In transfer learning, we can download a *pre-trained CNN architecture* that has already been trained on a massive dataset. These architectures are typically robust in detecting features due to their complex and deep structures (Hosna et al. 2022; Weiss

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3_11

et al. 2016; Zhuang et al. 2020). For instance, they might have been trained on datasets like ImageNet, which contains millions of images spanning hundreds of classes (Deng et al. 2009).

After downloading a pre-trained architecture, we can leverage its feature extraction capabilities. All we need to do is *modify the final layer* of the CNN to match the classes in our specific problem (Islam et al. 2022; Jumphoo et al. 2024; Srikantamurthy et al. 2023).

For example, let's say we have a deep CNN architecture pre-trained on ImageNet, which has *1000 classes.* If we want to use it for a *cancer classification problem with only three types of cancer*, we can *remove the final layer* (which corresponds to the 1000 classes) and *replace it with a new layer* tailored for our three cancer classes. This approach allows the powerful feature extraction capabilities of the CNN to be reused effectively on our smaller dataset, providing accurate results based on our specific classes.

When using transfer learning, we have two main options:

1. **Freeze the convolutional layers:** This means the layers will retain their original pre-trained weights and will not be updated during training on the new dataset.
2. **Fine-tune the convolutional layers:** This allows the pre-trained weights to be updated slightly to better align with the new dataset. However, this approach differs from training a CNN from scratch, as the initial weights are not random but already optimized from the pre-training phase.

Transfer learning, therefore, provides a significant advantage by reducing both the training time and the amount of data required, while still delivering high-performance results.

11.2 Transfer Learning with Freeze Using VGGNet

VGG (Visual Geometry Group) is a deep CNN architecture developed by the Visual Geometry Group at the University of Oxford. The most popular versions are VGG16 and VGG19, referring to the number of weight layers in the network (Hu et al. 2023; Simonyan and Zisserman 2014).

- VGG16 Architecture Overview:
  - Input: 224×224 RGB Image.
  - Convolutional Layers: 13 convolutional layers grouped into five blocks (each followed by max-pooling layers).
  - Fully Connected Layers: Three fully connected layers.
  - Output: Softmax layer for classification.

Training Dataset

VGG was initially trained on the ImageNet dataset, which contains 1.2 million images across 1000 classes.

Now, let's perform transfer learning with freezing convolutional layers on the CIFAR-10 dataset, which contains 60,000 images across 10 classes.

Code Example: Transfer Learning with Frozen VGGNet Layers for CIFAR-10

```
# Import necessary libraries
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize image data to [0, 1] range
x_train = x_train / 255.0
x_test = x_test / 255.0

# One-hot encode the labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Resize CIFAR-10 images to match VGG16 input (224x224)
x_train = tf.image.resize(x_train, (224, 224))
x_test = tf.image.resize(x_test, (224, 224))

# Load pre-trained VGG16 model without the top layer
(include_top=False)
base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# Freeze all convolutional layers in VGG16
for layer in base_model.layers:
    layer.trainable = False

# Build a new model on top of the pre-trained base
model = Sequential([
    base_model,
    Flatten(),  # Flatten feature maps
    Dense(256, activation='relu'),  # Add a dense layer
    Dense(10, activation='softmax')  # Final layer for 10 CIFAR-10
classes
])
# Compile the model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# Train the model
history = model.fit(
    x_train, y_train,
    validation_data=(x_test, y_test),
    epochs=10,
    batch_size=32
)
# Evaluate the model
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
# Plot training history
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

Explanation of the Code

1. Load and Preprocess Data:
   - CIFAR-10 images are resized to 224×224 to match the VGG16 input size.
   - Pixel values are normalized to the range [0,1].
2. Load VGG16:
   - The VGG16 model is loaded with "weights = 'imagenet'" and "include_top = False" to exclude the fully connected layers.
   - All convolutional layers are frozen (layer.trainable = False).
3. Add Custom Layers:
   - "Flatten()" flattens the feature maps.
   - "Dense(256)" adds a fully connected layer with ReLU activation.
   - "Dense(10)" creates an output layer for 10 classes with softmax activation.
4. Compile and Train:
   - The model is compiled with the Adam optimizer and categorical cross-entropy loss.
   - The model is trained on CIFAR-10 for 10 epochs.
5. Evaluate and Visualize:
   - The model is evaluated on the test set.
   - Training and validation accuracy are plotted.

11.3 Transfer Learning Without Freeze Using DenseNet

DenseNet (Densely Connected Convolutional Networks) is a deep CNN architecture designed to improve information and gradient flow between layers. In DenseNet, each layer is directly connected to every other layer in a feedforward fashion. This reduces the vanishing gradient problem and encourages feature reuse (Ester et al. 1996; G. Huang et al. 2016; L. Peng et al. 2022).

- DenseNet Key Features:
  - Dense connections between layers (each layer gets input from all preceding layers).
  - Fewer parameters compared to traditional architectures like ResNet.
  - Common versions: DenseNet-121, DenseNet-169, and DenseNet-201 (number indicates layer count).

Training Dataset

DenseNet is typically pre-trained on the ImageNet dataset, which contains 1.2 million images across 1000 classes.

Now, let's perform transfer learning without freezing convolutional layers on the CIFAR-10 dataset, which contains 60,000 images across 10 classes.

Code Example: Transfer Learning with Fine-Tuning DenseNet for CIFAR-10

```
# Import necessary libraries
import tensorflow as tf
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

# Normalize image data to [0, 1] range
x_train = x_train / 255.0
x_test = x_test / 255.0

# One-hot encode the labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Resize CIFAR-10 images to match DenseNet input (224x224)
x_train = tf.image.resize(x_train, (224, 224))
x_test = tf.image.resize(x_test, (224, 224))

# Load pre-trained DenseNet121 model without the top layer
(include_top=False)
base_model = DenseNet121(weights='imagenet', include_top=False,
input_shape=(224, 224, 3))

# Allow all layers to be trainable (fine-tuning)
for layer in base_model.layers:
    layer.trainable = True

# Build a new model on top of the pre-trained base
model = Sequential([
    base_model,
    Flatten(),  # Flatten feature maps
    Dropout(0.5),  # Add dropout for regularization
    Dense(256, activation='relu'),  # Add a dense layer
    Dense(10, activation='softmax')  # Final layer for 10 CIFAR-10
classes
])

# Compile the model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# Train the model
history = model.fit(
    x_train, y_train,
    validation_data=(x_test, y_test),
    epochs=10,
    batch_size=32
)

# Evaluate the model
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

# Plot training history
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

Explanation of the Code

1. Load and Preprocess Data:
   - CIFAR-10 images are resized to 224×224 to match the DenseNet input size.
   - Pixel values are normalized to the range [0,1].
2. Load DenseNet121:
   - The DenseNet121 model is loaded with "weights = 'imagenet'" and "include_top = False" to exclude the fully connected layers.
   - All convolutional layers are set as trainable (layer.trainable = True) for fine-tuning.
3. Add Custom Layers:
   - "Flatten()" flattens the feature maps.
   - "Dropout(0.5)" reduces overfitting.
   - "Dense(256)" adds a fully connected layer with ReLU activation.
   - "Dense(10)" creates an output layer for 10 classes with softmax activation.
4. Compile and Train:
   - The model is compiled with a lower learning rate (0.0001) to ensure fine-tuning does not destabilize pre-trained weights.
   - The model is trained on CIFAR-10 for 10 epochs.
5. Evaluate and Visualize:
   - The model is evaluated on the test set.
   - Training and validation accuracy are plotted.

Key differences between freezing and fine-tuning are shown in Table 11.1:

Table 11.1 Between freezing and fine-tuning trade-off

Aspect	Freezing layers	Fine-tuning layers
Trainable layers	Only the top layers are trainable	All layers are trainable
Learning rate	Higher learning rate can be used	Lower learning rate is preferred
Training time	Faster	Slower
Dataset size required	Small dataset sufficient	Larger dataset recommended

Fine-tuning DenseNet can yield better results when sufficient data is available, as the pre-trained layers can adapt more closely to the new dataset.

11.4 Transfer Learning with a Custom Cubixel Layer Using Inception Model on CIFAR-10 Dataset

The Inception model, specifically InceptionV3, is a deep convolutional neural network (CNN) architecture designed to efficiently handle image classification tasks by reducing computational overhead without compromising accuracy. InceptionV3 is built upon the concept of Inception modules, which consist of parallel convolutional layers with different kernel sizes (1×1, 3×3, and 5×5). These parallel convolutions allow the model to capture features at multiple scales, ensuring both fine-grained and large-scale features are extracted. The architecture also incorporates factorized convolutions, auxiliary classifiers, and global average pooling, reducing the number of parameters and improving training efficiency. Pre-trained on the ImageNet dataset (He et al. 2019; Szegedy et al. n.d.).

11.4.1 Cubixel Layer

The Cubixel layer introduces an innovative approach to feature extraction in image processing by treating each pixel as a three-dimensional cube rather than a flat two-dimensional entity. In this representation, the RGB values of each pixel correspond to the cube's width, length, and height, forming a volumetric structure. This paradigm allows the Cubixel layer to extract spatial relationships, depth information, and color variations that traditional layers may overlook. Additionally, the layer calculates adaptive thresholds, detects edge properties, and measures cube sizes, offering a richer set of features. This approach is particularly beneficial for tasks requiring high sensitivity to subtle image textures and edges, such as medical imaging or detailed texture analysis (Aburass 2024).

11.4.2 Code Implementation

Below is an example of integrating the Cubixel Layer into an InceptionV3 model for transfer learning on the CIFAR-10 dataset.

```
import tensorflow as tf
from tensorflow.keras.applications import InceptionV3
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout,
GlobalAveragePooling2D
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# Custom Cubixel Layer adjusted for feature maps
class Custom3DCubeLayer(tf.keras.layers.Layer):
    def __init__(self, **kwargs):
        super(Custom3DCubeLayer, self).__init__(**kwargs)

    def call(self, inputs):
        adaptive_thresholds = self.calculate_adaptive_threshold(inputs)
        gap_images = tf.map_fn(lambda x: self.extract_features(x[0],
x[1]),
                               (inputs, adaptive_thresholds),
dtype=tf.float32)
        return gap_images

    @tf.function
    def extract_features(self, feature_map, threshold):
        # Apply operations on feature maps (not assuming RGB channels)
        cube_sizes = self.calculate_cube_sizes(feature_map)
        cube_properties = self.calculate_cube_properties(cube_sizes)
        similarities = self.compute_similarity(cube_properties,
threshold)
        normalized_similarity = tf.math.sigmoid(similarities)
        edge_image = self.detect_edges(similarities, threshold)
        return edge_image

    @staticmethod
    def calculate_cube_sizes(feature_map):
        # Normalize feature map instead of assuming RGB values
        normalized_feature_map = tf.nn.sigmoid(feature_map)  #
Normalize values between 0 and 1
        cube_sizes = normalized_feature_map * 2  # Example scaling
        return cube_sizes

    @staticmethod
    def calculate_cube_properties(cube_sizes):
        # Example feature map property extraction
        volume = tf.reduce_prod(cube_sizes, axis=-1)  # Compute volume
across feature channels
   aspect_ratio = cube_sizes[..., 0] / (cube_sizes[..., 1] + 1e-6)  #
Avoid division by zero
        surface_area = tf.reduce_sum(cube_sizes, axis=-1)  # Simplified
```

```
surface area
        combined_properties = tf.stack([volume, aspect_ratio,
surface_area], axis=-1)
        return combined_properties

    @staticmethod
    def compute_similarity(cube_properties, threshold):
        similarity = tf.zeros_like(cube_properties)
        for dx in [-1, 0, 1]:
            for dy in [-1, 0, 1]:
                if dx == 0 and dy == 0:
                    continue
                shifted_properties = tf.roll(cube_properties,
shift=[dx, dy], axis=[0, 1])
                similarity += tf.abs(cube_properties -
shifted_properties)
        return similarity

    def detect_edges(self, similarity, threshold):
        levels = tf.where(
            similarity > threshold * 2, 3,
            tf.where(similarity > threshold, 2,
                     tf.where(similarity > threshold / 2, 1, 0)))
        return tf.cast(levels, tf.float32)

    @staticmethod
    def calculate_adaptive_threshold(feature_map):
        avg_size = tf.reduce_mean(feature_map, axis=[1, 2, 3])
        size_variability = tf.math.reduce_std(feature_map, axis=[1, 2,
3])
        return avg_size + size_variability

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Create a Dataset Pipeline with Resizing
batch_size = 32

train_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train))
train_dataset = train_dataset.map(lambda x, y: (tf.image.resize(x,
(299, 299)), y),
                                  num_parallel_calls=tf.data.AUTOTUNE)
train_dataset =
train_dataset.batch(batch_size).prefetch(tf.data.AUTOTUNE)
```

```
# Load pre-trained InceptionV3 model
base_model = InceptionV3(weights='imagenet', include_top=False,
input_shape=(299, 299, 3))

# Unfreeze all layers for fine-tuning
for layer in base_model.layers:
    layer.trainable = True

# Build model with Cubixel Layer
model = Sequential([
    base_model,
    Custom3DCubeLayer(),  # Integration of Custom Cubixel Layer
    GlobalAveragePooling2D(),
    Dropout(0.5),
    Dense(256, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
history = model.fit(
    train_dataset,
    validation_data=test_dataset,
    epochs=10,
)

# Evaluate the model
test_loss, test_accuracy = model.evaluate(test_dataset)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

# Plot training history
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

11.4.3 Integration of Cubixel Layer with Transfer Learning

The integration of the Cubixel Layer with InceptionV3 enhances the model's ability to interpret complex image features. Positioned after the pre-trained convolutional base, the Cubixel layer processes the extracted feature maps to emphasize spatial relationships and fine-grained details. This synergy allows the Inception model to

benefit from both the global feature extraction of its convolutional layers and the localized feature enhancements of the Cubixel layer.

By combining the strengths of both paradigms, the resulting architecture becomes highly capable of identifying subtle patterns, making it particularly powerful for datasets like CIFAR-10. Fine-tuning the entire model further optimizes the pre-trained weights to adapt specifically to the new dataset while preserving the robust feature extraction capabilities of the pre-trained layers.

11.5 Concluding Remarks

In this chapter, we explored the concept and practical implementation of Transfer Learning, a powerful technique that addresses the challenges of training deep convolutional neural networks from scratch, particularly when faced with limited data and computational resources. Transfer learning leverages pre-trained models, enabling them to be fine-tuned or adapted for new tasks while preserving their robust feature extraction capabilities.

We examined two primary strategies for transfer learning:

1. Freezing Layers: Pre-trained weights are retained in convolutional layers, and only the final layers are retrained for the specific task. This approach is efficient and ideal for smaller datasets.
2. Fine-Tuning Layers: All layers are made trainable, allowing slight updates to pre-trained weights. Fine-tuning is more effective when sufficient labeled data is available and requires careful learning rate adjustment to prevent overfitting.

Practical examples with VGGNet and DenseNet demonstrated these strategies, showcasing their application to the CIFAR-10 dataset. Additionally, we explored the integration of custom layers, such as the Cubixel Layer with the InceptionV3 model, to enhance feature extraction and capture complex spatial relationships in images.

Overall, transfer learning provides a scalable and efficient solution for training deep learning models, significantly reducing computational costs and training time while maintaining high performance. As deep learning continues to evolve, transfer learning will remain a cornerstone technique in applying pre-trained architectures to diverse domains, including image classification, medical imaging, and beyond.

References

Abdi, H., & Williams, L. J. (2010). Principal component analysis. *Wiley Interdisciplinary Reviews: Computational Statistics*, *2*(4), 433–459.

Aburass, S. (2024). Cubixel: a novel paradigm in image processing using three-dimensional pixel representation. *Multimedia Tools and Applications*. https://doi.org/10.1007/s11042-024-20081-6

AbuRass, S., Huneiti, A., & Al-Zoubi, M. B. (2020). Enhancing Convolutional Neural Network using Hu's Moments. *International Journal of Advanced Computer Science and Applications*, *11*(12), 130–137. https://doi.org/10.14569/IJACSA.2020.0111216

Aburass, S., Huneiti, A., & Al-Zoubi, M. B. (2022). Classification of Transformed and Geometrically Distorted Images using Convolutional Neural Network. *Journal of Computer Science*, *18*(8). https://doi.org/10.3844/jcssp.2022.757.769

Agarap, A. F. M. (2018). A neural network architecture combining gated recurrent unit (GRU) and support vector machine (SVM) for intrusion detection in network traffic data. *Proceedings of the 2018 10th International Conference on Machine Learning and Computing*, 26–30.

Agresti, A. (2013). *Categorical data analysis*. John Wiley & Sons.

Ali, J., Khan, R., Ahmad, N., & Maqsood, I. (2012). Random forests and decision trees. *International Journal of Computer Science Issues (IJCSI)*, *9*(5), 272.

Al-Selwi, S. M., Hassan, M. F., Abdulkadir, S. J., Muneer, A., Sumiea, E. H., Alqushaibi, A., & Ragab, M. G. (2024). RNN-LSTM: From applications to modeling techniques and beyond—Systematic review. *Journal of King Saud University-Computer and Information Sciences*, 102068.

Alshawabkeh, M., Ryalat, M. H., Dorgham, O. M., Alkharabsheh, K., Btoush, M. H., & Alazab, M. (2022). A hybrid convolutional neural network model for detection of diabetic retinopathy. *International Journal of Computer Applications in Technology*, *70*(3/4), 179. https://doi.org/10.1504/IJCAT.2022.130886

Anderson, D., & McNeill, G. (1992). Artificial neural networks technology. *Kaman Sciences Corporation*, *258*(6), 1–83.

Awcock, G. J., & Thomas, R. (1995). *Applied image processing*. Springer.

Azulay, A., & Weiss, Y. (2019). Why do deep convolutional networks generalize so poorly to small image transformations? *Journal of Machine Learning Research*, *20*(184), 1–25.

Barlow, H. B. (1989). Unsupervised learning. *Neural Computation*, *1*(3), 295–311.

Basheer, I. A., & Hajmeer, M. (2000). Artificial neural networks: fundamentals, computing, design, and application. *Journal of Microbiological Methods*, *43*(1), 3–31.

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

S. Aburass, I. Aljarah, *Classical Machine Learning*,
https://doi.org/10.1007/978-3-032-04399-3

Bernard, S., Heutte, L., & Adam, S. (2009). On the selection of decision trees in random forests. *2009 International Joint Conference on Neural Networks*, 302–307.

Bhuvaneswari, R., & Kalaiselvi, K. (2012). Naive Bayesian classification approach in healthcare applications. *International Journal of Computer Science and Telecommunications*, *3*(1), 106–112.

Biau, G., Cadre, B., & Rouvière, L. (2019). Accelerated gradient boosting. *Machine Learning*, *108*, 971–992.

Biau, G., & Scornet, E. (2016). A random forest guided tour. *Test*, *25*(2), 197–227.

Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning* (Vol. 4, Issue 4). Springer.

Boehmke, B., & Greenwell, B. M. (2019). *Hands-on machine learning with R*. Chapman and Hall/CRC.

Breiman, L. (2001). Random forests. *Machine Learning*, *45*, 5–32.

Breiman, L., Friedman, J., Olshen, R. A., & Stone, C. J. (2017). *Classification and regression trees*. Routledge.

Brownlee, J. (2020). *Data Preparation for Machine Learning: Data Cleaning, Feature Selection, and Data Transforms in Python*. Machine Learning Mastery. https://books.google.com/books?id=uAPuDwAAQBAJ

Bühlmann, P., & Van De Geer, S. (2011). *Statistics for high-dimensional data: methods, theory and applications*. Springer Science & Business Media.

Cabrera, A. F. (1994). Logistic regression analysis in higher education: An applied perspective. *Higher Education: Handbook of Theory and Research*, *10*, 225–256.

Caruana, R., Lawrence, S., & Giles, C. (2000). Overfitting in Neural Nets: Backpropagation, Conjugate Gradient, and Early Stopping. In T. Leen, T. Dietterich, & V. Tresp (Eds.), *Advances in Neural Information Processing Systems* (Vol. 13). MIT Press. https://proceedings.neurips.cc/paper_files/paper/2000/file/059fdcd96baeb75112f09fa1dcc740cc-Paper.pdf

Celebi, M. E., & Aydin, K. (2016). *Unsupervised learning algorithms* (Vol. 1). Springer.

Cerulli, G. (2023). Tree Modeling. In *Fundamentals of Supervised Machine Learning: With Applications in Python, R, and Stata* (pp. 201–267). Springer.

Chatzimparmpas, A., Martins, R. M., Kucher, K., & Kerren, A. (2022). FeatureEnVi: Visual Analytics for Feature Engineering Using Stepwise Selection and Semi-Automatic Extraction Approaches. *IEEE Transactions on Visualization and Computer Graphics*, *28*(4), 1773–1791. https://doi.org/10.1109/TVCG.2022.3141040

Chauvin, Y., & Rumelhart, D. E. (2013). *Backpropagation: theory, architectures, and applications*. Psychology press.

Chen, H., Hu, S., Hua, R., & Zhao, X. (2021). Improved naive Bayes classification algorithm for traffic risk management. *EURASIP Journal on Advances in Signal Processing*, *2021*(1), 30.

Chen, T., He, T., Benesty, M., Khotilovich, V., Tang, Y., Cho, H., Chen, K., Mitchell, R., Cano, I., & Zhou, T. (2015a). Xgboost: extreme gradient boosting. *R Package Version 0.4-2*, *1*(4), 1–4.

Chen, W., Wilson, J., Tyree, S., Weinberger, K., & Chen, Y. (2015b). Compressing neural networks with the hashing trick. *International Conference on Machine Learning*, 2285–2294.

Cheng, C., & Schneeweiss, H. (1998). Polynomial regression with errors in the variables. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, *60*(1), 189–199.

Chitradevi, B., & Srimathi, P. (2014). An overview on image processing techniques. *International Journal of Innovative Research in Computer and Communication Engineering*, *2*(11), 6466–6472.

Chollet, F., & Chollet, F. (2021). *Deep learning with Python*. Simon and Schuster.

Dalianis, H., & Dalianis, H. (2018). Evaluation metrics and evaluation. *Clinical Text Mining: Secondary Use of Electronic Patient Records*, 45–53.

Dasu, T., & Johnson, T. (2003). *Exploratory Data Mining and Data Cleaning*. Wiley. https://books.google.com/books?id=HStUfJnCvT4C

Dayton, C. M. (1992). Logistic regression analysis. *Stat*, *474*, 574.

Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for machine learning*. Cambridge University Press.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image databases. *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–255.

Deshpande, A., & Kumar, M. (2018). *Artificial intelligence for big data: complete guide to automating big data solutions using artificial intelligence techniques*. Packt Publishing Ltd.

Dhivyaa, C. R., Sangeetha, K., Balamurugan, M., Amaran, S., Vetriselvi, T., & Johnpaul, P. (2020). Skin lesion classification using decision trees and random forest algorithms. *Journal of Ambient Intelligence and Humanized Computing*, 1–13.

Dhruv, P., & Naskar, S. (2020). *Image Classification Using Convolutional Neural Network (CNN) and Recurrent Neural Network (RNN): A Review* (pp. 367–381). https://doi.org/10.1007/978-981-15-1884-3_34

Dinov, I. D. (2018). *Data science and predictive analytics*. Springer.

Domingos, P. (2012). A few useful things to know about machine learning. *Communications of the ACM*, *55*(10), 78–87. https://doi.org/10.1145/2347736.2347755

Dong, G., & Liu, H. (2018). *Feature engineering for machine learning and data analytics*. CRC Press.

Dorgham, O. (2008). Performance of a 2D-3D image registration system using (lossy) compressed x-ray CT. *Annals of the BMVA*, *2008*(3), 1–11.

Duboue, P. (2020). *The art of feature engineering: essentials for machine learning*. Cambridge University Press.

Ekman, M. (2021). *Learning deep learning: Theory and practice of neural networks, computer vision, natural language processing, and transformers using TensorFlow*. Addison-Wesley Professional.

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Kdd*, *96*(34), 226–231.

Fahrmeir, L., Kneib, T., Lang, S., & Marx, B. D. (2022). Regression models. In *Regression: Models, methods and applications* (pp. 23–84). Springer.

Fernandes, A. A. A., Koehler, M., Konstantinou, N., Pankin, P., Paton, N. W., & Sakellariou, R. (2023). Data Preparation: A Technological Perspective and Review. *SN Computer Science*, *4*(4), 425. https://doi.org/10.1007/s42979-023-01828-8

Flach, P. A., & Lachiche, N. (2004). Naive Bayesian classification of structured data. *Machine Learning*, *57*, 233–269.

Foulds, J., Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2025). *Data Mining: Practical Machine Learning Tools and Techniques*. Elsevier.

Fox, J. (2015). *Applied regression analysis and generalized linear models*. Sage Publications.

Fratello, M., & Tagliaferri, R. (2018). Decision trees and random forests. *Encyclopedia of Bioinformatics and Computational Biology: ABC of Bioinformatics*, *1*(S3), 374.

Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, *55*(1), 119–139.

Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 1189–1232.

Friedman, J. H. (2002). Stochastic gradient boosting. *Computational Statistics & Data Analysis*, *38*(4), 367–378.

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems*. O'Reilly Media, Inc.

Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). *Deep learning* (Vol. 1, Issue 2). MIT press Cambridge.

Grau, I., Nápoles, G., Bonet, I., & García, M. M. (2013). Backpropagation through time algorithm for training recurrent neural networks using variable length instances. *Computación y Sistemas*, *17*(1), 15–24.

Graves, A., & Graves, A. (2012). *Supervised sequence labelling*. Springer.

Hahs-Vaughn, D. L. (2024). *Applied multivariate statistical concepts*. Routledge.
Han, J., Pei, J., & Tong, H. (2022). *Data Mining: Concepts and Techniques*. Morgan Kaufmann. https://books.google.com/books?id=NR1oEAAAQBAJ
Hassoun, M. H. (1995). *Fundamentals of artificial neural networks*. MIT press.
Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning*. Citeseer.
He, K., Girshick, R., & Dollar, P. (2019, October). Rethinking ImageNet Pre-Training. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
Headrick, T. C. (2009). *Statistical simulation: Power method polynomials and other transformations*. Chapman and Hall/CRC.
Hecht-Nielsen, R. (1992). Theory of the backpropagation neural network. In *Neural networks for perception* (pp. 65–93). Elsevier.
Heiberger, R. M., Neuwirth, E., Heiberger, R. M., & Neuwirth, E. (2009). Polynomial regression. *R through Excel: A Spreadsheet Interface for Statistics, Data Analysis, and Graphics*, 269–284.
Hilbe, J. M. (2009). *Logistic regression models*. Chapman and hall/CRC.
Hosmer Jr, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression*. John Wiley & Sons.
Hosna, A., Merry, E., Gyalmo, J., Alom, Z., Aung, Z., & Azim, M. A. (2022). Transfer learning: a friendly introduction. *Journal of Big Data*, *9*(1), 102.
Hossin, M., & Sulaiman, M. N. (2015). A review on evaluation metrics for data classification evaluations. *International Journal of Data Mining & Knowledge Management Process*, *5*(2), 1.
Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. http://arxiv.org/abs/1704.04861
Hu, Z., Wang, Z., Jin, Y., & Hou, W. (2023). VGG-TSwinformer: Transformer-based deep learning model for early Alzheimer's disease prediction. *Computer Methods and Programs in Biomedicine*, *229*, 107291. https://doi.org/10.1016/j.cmpb.2022.107291
Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2016). *Densely Connected Convolutional Networks*. http://arxiv.org/abs/1608.06993
Huang, Y., & Li, L. (2011). Naive Bayes classification algorithm based on small sample set. *2011 IEEE International Conference on Cloud Computing and Intelligence Systems*, 34–39.
Islam, M. N., Hasan, M., Hossain, M. K., Alam, M. G. R., Uddin, M. Z., & Soylu, A. (2022). Vision transformer and explainable transfer learning models for auto detection of kidney cyst, stone and tumor from CT-radiography. *Scientific Reports*, *12*(1). https://doi.org/10.1038/s41598-022-15634-4
Itoo, F., Meenakshi, & Singh, S. (2021). Comparison and analysis of logistic regression, Naïve Bayes and KNN machine learning algorithms for credit card fraud detection. *International Journal of Information Technology*, *13*(4), 1503–1511.
Jain, A. K. (2010). Data clustering: 50 years beyond K-means. *Pattern Recognition Letters*, *31*(8), 651–666.
James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning* (Vol. 112, Issue 1). Springer.
John Lu, Z. Q. (2010). *The elements of statistical learning: data mining, inference, and prediction*. Oxford University Press.
Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, *374*(2065), 20150202.
Jumphoo, T., Phapatanaburi, K., Pathonsuwan, W., Anchuen, P., Uthansakul, M., & Uthansakul, P. (2024). Exploiting Data-Efficient Image Transformer-Based Transfer Learning for Valvular Heart Diseases Detection. *IEEE Access*, *12*, 15845–15855. https://doi.org/10.1109/ACCESS.2024.3357946
Khan, A., Sohail, A., Zahoora, U., & Qureshi, A. S. (2020). A survey of the recent architectures of deep convolutional neural networks. *Artificial Intelligence Review*, *53*(8), 5455–5516. https://doi.org/10.1007/s10462-020-09825-6

Kleinbaum, D. G., Klein, M., Kleinbaum, D. G., & Klein, M. (2010). Introduction to logistic regression. *Logistic Regression: A Self-Learning Text*, 1–39.

Lee, J. H., Delbruck, T., & Pfeiffer, M. (2016). Training deep spiking neural networks using backpropagation. *Frontiers in Neuroscience*, *10*, 508.

Lewis-Beck, C., & Lewis-Beck, M. (2015). *Applied regression: An introduction* (Vol. 22). Sage Publications.

Li, J., Cheng, K., Wang, S., Morstatter, F., Trevino, R. P., Tang, J., & Liu, H. (2018). Feature Selection. *ACM Computing Surveys*, *50*(6), 1–45. https://doi.org/10.1145/3136625

Li, Z., Liu, F., Yang, W., Peng, S., & Zhou, J. (2022). A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(12), 6999–7019. https://doi.org/10.1109/TNNLS.2021.3084827

Liu, H., & Motoda, H. (2012). *Feature Selection for Knowledge Discovery and Data Mining*. Springer US. https://books.google.com/books?id=aaDbBwAAQBAJ

Lydia, A. A., & Francis, F. S. (2019). Convolutional neural network with an optimized backpropagation technique. *2019 IEEE International Conference on System, Computation, Automation and Networking (ICSCAN)*, 1–5.

Maimon, O. Z., & Rokach, L. (2014). *Data mining with decision trees: theory and applications* (Vol. 81). World scientific.

Mandic, D. P., & Chambers, J. (2001). *Recurrent neural networks for prediction: learning algorithms, architectures and stability*. John Wiley & Sons, Inc.

Marion, A. (2013). *Introduction to image processing*. Springer.

Mayr, A., Binder, H., Gefeller, O., & Schmid, M. (2014). The evolution of boosting algorithms. *Methods of Information in Medicine*, *53*(06), 419–427.

Menard, S. (2001). *Applied logistic regression analysis*. SAGE Publications.

Mirkin, B. (2005). *Clustering for data mining: a data recovery approach*. Chapman and Hall/CRC.

Mitchell, T. M., & Mitchell, T. M. (1997). *Machine learning* (Vol. 1, Issue 9). McGraw-Hill New York.

Montgomery, D. C., Peck, E. A., & Vining, G. G. (2021). *Introduction to linear regression analysis*. John Wiley & Sons.

Murphy, K. P. (2012). *Machine learning: a probabilistic perspective*. MIT press.

Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press. https://books.google.com/books?id=wrZNEAAAQBAJ

Myers, R. H., Montgomery, D. C., & Anderson-Cook, C. M. (2016). *Response surface methodology: process and product optimization using designed experiments*. John Wiley & Sons.

Naeem, S., Ali, A., Anam, S., & Ahmed, M. M. (2023). An unsupervised machine learning algorithms: Comprehensive review. *International Journal of Computing and Digital Systems*.

Naidu, G., Zuva, T., & Sibanda, E. M. (2023). A review of evaluation metrics in machine learning algorithms. *Computer Science On-Line Conference*, 15–25.

Nargesian, F., Samulowitz, H., Khurana, U., Khalil, E. B., & Turaga, D. S. (2017). Learning feature engineering for classification. *IJCAI*, *17*, 2529–2535.

Naskinova, I. (2023). Transfer learning with NASNet-Mobile for Pneumonia X-ray classification. *Asian-European Journal of Mathematics*, *16*(01). https://doi.org/10.1142/S1793557122502400

Natekin, A., & Knoll, A. (2013). Gradient boosting machines, a tutorial. *Frontiers in Neurorobotics*, *7*, 21.

Patil, S., & Patil, S. (2021). Linear with polynomial regression: Overview. *International Journal of Applied Research*, *7*(8), 273–275.

Peckov, A. (2012). A machine learning approach to polynomial regression. *Ljubljana, Slovenia*, URL: http://kt.ijs.si/theses/phd_aleksandar_peckov.pdf

Peng, C.-Y. J., Lee, K. L., & Ingersoll, G. M. (2002). An introduction to logistic regression analysis and reporting. *The Journal of Educational Research*, *96*(1), 3–14.

Peng, L., Wang, C., Tian, G., Liu, G., Li, G., Lu, Y., Yang, J., Chen, M., & Li, Z. (2022). Analysis of CT scan images for COVID-19 pneumonia based on a deep ensemble framework with

DenseNet, Swin transformer, and RegNet. *Frontiers in Microbiology*, *13*. https://doi.org/10.3389/fmicb.2022.995323

Perlich, C., Dalessandro, B., Raeder, T., Stitelman, O., & Provost, F. (2014). Machine learning for targeted display advertising: Transfer learning in action. *Machine Learning*, *95*(1), 103–127. https://doi.org/10.1007/s10994-013-5375-2

Petrou, M. M. P., & Petrou, C. (2010). *Image processing: the fundamentals*. John Wiley & Sons.

Pyle, D. (1999). *Data Preparation for Data Mining*. Elsevier Science. https://books.google.com/books?id=hhdVr9F-JfAC

Quinlan, J. R. (2014). *C4. 5: programs for machine learning*. Elsevier.

Rawlings, J. O., Pantula, S. G., & Dickey, D. A. (1998). *Applied regression analysis: a research tool*. Springer.

Ren, J., Lee, S. D., Chen, X., Kao, B., Cheng, R., & Cheung, D. (2009). Naive bayes classification of uncertain data. *2009 Ninth IEEE International Conference on Data Mining*, 944–949.

Rojas, R., & Rojas, R. (1996). The backpropagation algorithm. *Neural Networks: A Systematic Introduction*, 149–182.

Ryan, T. P. (2008). *Modern regression methods*. John Wiley & Sons.

Sabry, F. (2023). *Naive Bayes Classifier: Fundamentals and Applications* (Vol. 31). One Billion Knowledgeable.

Schapire, R. E. (2013). Explaining adaboost. In *Empirical inference: festschrift in honor of vladimir N. Vapnik* (pp. 37–52). Springer.

Schapire, R. E., & Freund, Y. (2013). Boosting: Foundations and algorithms. *Kybernetes*, *42*(1), 164–166.

Schmidt, R. M. (2019). Recurrent neural networks (rnns): A gentle introduction and overview. *ArXiv Preprint ArXiv:1912.05911*.

Schubert, E., Sander, J., Ester, M., Kriegel, H. P., & Xu, X. (2017). DBSCAN revisited, revisited: why and how you should (still) use DBSCAN. *ACM Transactions on Database Systems (TODS)*, *42*(3), 1–21.

Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, *45*(11), 2673–2681.

Scott, S., & Matwin, S. (1999). Feature engineering for text classification. *ICML*, *99*, 379–388.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems* (Vol. 28). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf

Shaikhina, T., Lowe, D., Daga, S., Briggs, D., Higgins, R., & Khovanova, N. (2019). Decision tree and random forest models for outcome prediction in antibody incompatible kidney transplantation. *Biomedical Signal Processing and Control*, *52*, 456–462.

Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding machine learning: From theory to algorithms*. Cambridge University Press.

Sherstinsky, A. (2020). Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network. *Physica D: Nonlinear Phenomena*, *404*, 132306.

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *ArXiv Preprint ArXiv:1409.1556*.

Sindhu Meena, K., & Suriya, S. (2020). A survey on supervised and unsupervised learning techniques. *Proceedings of International Conference on Artificial Intelligence, Smart Grid and Smart City Applications: AISGSC 2019*, 627–644.

Sperandei, S. (2014). Understanding logistic regression analysis. *Biochemia Medica*, *24*(1), 12–18.

Srikantamurthy, M. M., Rallabandi, V. P. S., Dudekula, D. B., Natarajan, S., & Park, J. (2023). Classification of benign and malignant subtypes of breast cancer histopathology imaging using hybrid CNN-LSTM based transfer learning. *BMC Medical Imaging*, *23*(1), 19. https://doi.org/10.1186/s12880-023-00964-0

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *J. Mach. Learn. Res.*, *15*(1), 1929–1958.

Stigler, S. M. (1971). Optimal experimental design for polynomial regression. *Journal of the American Statistical Association*, *66*(334), 311–318.

Sutskever, I., Martens, J., & Hinton, G. E. (2011). Generating text with recurrent neural networks. *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, 1017–1024.

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*, *27*.

Sykes, A. O. (1993). *An introduction to regression analysis*.

Szegedy, C., Ioffe, S., Vanhoucke, V., & Alemi, A. (n.d.). *Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning*.

Tripepi, G., Jager, K. J., Dekker, F. W., & Zoccali, C. (2008). Linear and logistic regression analysis. *Kidney International*, *73*(7), 806–810.

Turner, C. R., Fuggetta, A., Lavazza, L., & Wolf, A. L. (1999). A conceptual basis for feature engineering. *Journal of Systems and Software*, *49*(1), 3–15.

Wade, C., & Glynn, K. (2020). *Hands-On Gradient Boosting with XGBoost and scikit-learn: Perform accessible machine learning and extreme gradient boosting with Python*. Packt Publishing Ltd.

Wang, K., Xu, C., Li, G., Zhang, Y., Zheng, Y., & Sun, C. (2023). Combining convolutional neural networks and self-attention for fundus diseases identification. *Scientific Reports*, *13*(1). https://doi.org/10.1038/s41598-022-27358-6

Wang, W., & Sun, D. (2021). The improved AdaBoost algorithms for imbalanced data classification. *Information Sciences*, *563*, 358–374.

Wang, Z., Li, X., Duan, H., Su, Y., Zhang, X., & Guan, X. (2021). Medical image fusion based on convolutional neural networks and non-subsampled contourlet transform. *Expert Systems with Applications*, *171*, 114574. https://doi.org/10.1016/j.eswa.2021.114574

Weidman, S. (2019). *Deep learning from scratch: Building with python from first principles*. O'Reilly Media.

Weiss, K., Khoshgoftaar, T. M., & Wang, D. (2016). A survey of transfer learning. *Journal of Big Data*, *3*, 1–40.

Whang, S. E., Roh, Y., Song, H., & Lee, J.-G. (2023). Data collection and quality challenges in deep learning: a data-centric AI perspective. *The VLDB Journal*, *32*(4), 791–813. https://doi.org/10.1007/s00778-022-00775-9

Wickham, H. (2014). Tidy Data. *Journal of Statistical Software*, *59*(10). https://doi.org/10.18637/jss.v059.i10

Witten, I. H., Frank, E., Hall, M. A., Pal, C. J., & Data, M. (2005). Practical machine learning tools and techniques. *Data Mining*, *2*(4), 403–413.

Wu, D. J., Feng, T., Naehrig, M., & Lauter, K. (2015). Privately evaluating decision trees and random forests. *Cryptology EPrint Archive*.

Wu, J. (2012). *Advances in K-means clustering: a data mining thinking*. Springer Science & Business Media.

Wythoff, B. J. (1993). Backpropagation neural networks: a tutorial. *Chemometrics and Intelligent Laboratory Systems*, *18*(2), 115–155.

Xu, R., & Wunsch, D. (2005). Survey of clustering algorithms. *IEEE Transactions on Neural Networks*, *16*(3), 645–678.

Yang, F.-J. (2018). An implementation of naive bayes classifier. *2018 International Conference on Computational Science and Computational Intelligence (CSCI)*, 301–306.

Yao, X., Wang, X., Wang, S. H., & Zhang, Y. D. (2022). A comprehensive survey on convolutional neural network in medical image analysis. *Multimedia Tools and Applications*, *81*(29), 41361–41405. https://doi.org/10.1007/s11042-020-09634-7

Yegnanarayana, B. (2009). *Artificial neural networks*. PHI Learning Pvt. Ltd.

Ying, C., Qi-Guang, M., Jia-Chen, L., & Lin, G. (2013). Advance and prospects of AdaBoost algorithm. *Acta Automatica Sinica*, *39*(6), 745–758.

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into deep learning*. Cambridge University Press.

Zhang, Z. (2016). Derivation of backpropagation in convolutional neural network (cnn). *University of Tennessee, Knoxville, TN*, *22*(23), 4.

Zhang, Z., & Zhang, Z. (2018). Artificial neural network. *Multivariate Time Series Analysis in Climate and Environmental Research*, 1–35.

Zhao, Z. (2022). Skin Cancer Classification Based on Convolutional Neural Networks and Vision Transformers. *Journal of Physics: Conference Series*, *2405*(1), 012037. https://doi.org/10.1088/1742-6596/2405/1/012037

Zheng, A., & Casari, A. (2018). *Feature engineering for machine learning: principles and techniques for data scientists*. O'Reilly Media, Inc.

Zhou, X. (2018). Understanding the convolutional neural networks with gradient descent and backpropagation. *Journal of Physics: Conference Series*, *1004*, 012028.

Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., & He, Q. (2020). A comprehensive survey on transfer learning. *Proceedings of the IEEE*, *109*(1), 43–76.

Zoph, B., Vasudevan, V., Shlens, J., & Le, Q. V. (2017). *Learning Transferable Architectures for Scalable Image Recognition*.

Zou, J., Han, Y., & So, S.-S. (2009). Overview of artificial neural networks. *Artificial Neural Networks: Methods and Applications*, 14–22.